

SigmaDock and SigmaFlow

A Self-Contained Mathematical and Computational Monograph

From probability theory and Riemannian geometry to
 $SE(3)$ -equivariant diffusion and Riemannian flow matching
for protein–ligand docking

Technical reference document

companion to the SigmaFlow research project

August 16, 2026

Abstract

This monograph is a from-scratch, self-contained derivation and code-verified description of **SigmaDock**, an $SE(3)$ -equivariant diffusion model for protein–ligand pose generation, and **SigmaFlow**, its Riemannian flow-matching counterpart. It assumes only real analysis, linear algebra, elementary probability theory, and general mathematical maturity. Every other concept — Riemannian manifolds, Lie groups, Brownian motion on manifolds, score-based diffusion, flow matching, spherical harmonics, Wigner- D matrices, Clebsch–Gordan coefficients, and the Equiformer/EquiformerV2 architecture — is introduced from first principles, motivated, and derived in full. Every mathematical construction is, wherever it is used by SigmaDock or SigmaFlow, tied explicitly to the concrete implementation: file, class, function, tensor shape, and formula. The document distinguishes throughout between (i) idealised theory, (ii) what the SigmaDock method description states, (iii) what the SigmaDock reference implementation actually computes, and (iv) what SigmaFlow changes, and flags explicitly any point at which these four do not coincide or could not be verified with certainty from the available source.

Contents

How to read this document	12
Global notation	13
 I Preliminaries	 15
1 The Protein–Ligand Docking Problem	17
1.1 Proteins and ligands	17
1.2 Poses and degrees of freedom	18
1.3 What a docking model predicts, and what it does not	19
1.4 Evaluation: RMSD and structural validity	19
1.5 What you should now be able to do	20
 2 Probability and Generative Modelling Foundations	 21
2.1 Densities, conditional densities, and pushforward	21
2.2 Conditional expectation, and the one property we will use constantly	22
2.3 Score functions	23
2.4 Why the score suffices for sampling: Langevin dynamics	24
2.5 Kullback–Leibler divergence and maximum likelihood	25
2.6 Generative modelling as distribution transport	28
2.7 What you should now be able to do	29
 II Geometry	 30
3 Manifolds, Groups, and Rigid Motion	32
3.1 Manifolds and tangent spaces	32
3.1.1 Why a tangent space at all, and why one per point	33
3.1.2 Vector fields, and why a tangent field keeps you on the manifold	34
3.1.3 Measuring: metrics, lengths, and distance	35

3.1.4	Acceleration on a curved space: the covariant derivative	36
3.1.5	Geodesics	37
3.1.6	Replacing $x + v$ and $y - x$: the exponential and logarithmic maps . . .	38
3.1.7	The scaling property, and exp evaluated part-way	40
3.1.8	Where log stops being defined: injectivity radius and cut locus	41
3.2	Calculus on a manifold: volume, gradient, and divergence	42
3.2.1	Volume, and why a density needs a reference measure	42
3.2.2	The Riemannian gradient	43
3.2.3	The Riemannian divergence	44
3.3	Groups and Lie groups	45
3.4	The rotation group $SO(3)$	46
3.4.1	Definition and dimension	46
3.4.2	The Lie algebra $\mathfrak{so}(3)$ and the hat/vee maps	47
3.4.3	The exponential map: Rodrigues' formula	48
3.4.4	The logarithmic map, and its numerically dangerous points	49
3.4.5	$SO(3)$ as a metric space, and Brownian motion's invariant measure . .	51
3.5	Left and right trivialisation, resolved for this document	52
3.6	The pose group $SE(3)$ and the product manifold $SE(3)^N$	54
3.7	The torsion space $\mathbb{T}^k$	56
3.8	What you should now be able to do	57
4	Steerable Representations: Spherical Harmonics, Wigner Matrices, and Clebsch–Gordan Coefficients	59
4.1	The problem this chapter exists to solve	59
4.1.1	Rotating the whole system changes nothing physical	59
4.1.2	Two different kinds of “unchanged”	60
4.1.3	Why an ordinary network does not do this by itself	60
4.1.4	Why data augmentation is not a substitute	61
4.2	Group actions: how a group moves things	62
4.3	Invariance and equivariance, formally	63
4.4	Why representation theory, and not something simpler	65
4.5	Representations of $SO(3)$	66
4.6	Spherical harmonics	67
4.6.1	Explicit low-degree formulas, and the connection to familiar operations	69
4.7	Wigner- D matrices	69
4.8	The anatomy of a steerable feature	70

4.8.1	One feature, four indices	70
4.8.2	Why 16 channels of $\ell = 1$ is not a 48-dimensional vector	71
4.8.3	What the channel index is for	72
4.8.4	A concrete feature specification	72
4.9	Tensor products and Clebsch–Gordan coefficients	73
4.9.1	Starting from two ordinary vectors	73
4.9.2	The outer product and how it rotates	74
4.9.3	Decomposing $1 \otimes 1$ completely	75
4.9.4	The selection rule as a type system	78
4.9.5	Clebsch–Gordan coefficients are a change of basis	78
4.9.6	Reading (4.7) index by index	79
4.9.7	Why the result is equivariant	81
4.9.8	From the mathematical product to a network layer	82
4.9.9	Where this meets geometry: the edge direction	84
4.10	Consequences for equivariant linear maps and nonlinearities	85
4.10.1	The question	85
4.10.2	The condition a linear layer must satisfy	85
4.10.3	Three cases, worked by hand	86
4.10.4	Schur’s lemma	87
4.10.5	What the layer actually looks like	88
4.10.6	Bias terms	89
4.10.7	Why linear layers are not enough	89
4.10.8	Nonlinearity 1: scalars are unrestricted	90
4.10.9	Nonlinearity 2: norm nonlinearities	90
4.10.10	Nonlinearity 3: gating	91
4.10.11	The three admissible operations, and what each is for	92
4.10.12	A small block, end to end	92
4.10.13	Summary in one formula	93
4.11	What you should now be able to do	94

III Diffusion Models 95

5 From Random Walks to Stochastic Differential Equations 97

5.1	Start with something concrete: a discrete noising chain	97
5.2	Why the noise scales as $\sqrt{\Delta t}$	98

5.3	Brownian motion	99
5.3.1	Brownian motion is nowhere differentiable	99
5.4	What an SDE actually is	100
5.4.1	The Itô integral, at the level we need	100
5.5	Euler–Maruyama: the bridge in both directions	101
5.6	Drift and diffusion	102
5.7	Paths versus marginals	102
5.8	Fokker–Planck: how the population moves	103
5.9	DDPM is a VP-SDE in disguise	104
5.10	SMLD is a VE-SDE	105
5.11	Summary	106
6	Diffusion Models in $\mathbb{R}^d$	107
6.1	What the score is, before we learn it	107
6.2	Denoising score matching	108
6.3	The SDE perspective, and why a single noise level is not enough	109
6.3.1	Score, weighting, and the training objective in continuous time	110
6.3.2	Why running time backwards is not enough	111
6.3.3	The reverse-time SDE	112
6.4	The probability flow ODE, and the bridge to flow matching	112
6.4.1	How can a deterministic process match a stochastic one?	112
6.4.2	Reverse SDE versus probability flow ODE, side by side	114
6.5	What you should now be able to do	115
7	Diffusion Models on $SO(3)$	116
7.1	Brownian motion on a Riemannian manifold	116
7.2	The heat kernel on $SO(3)$: IGSO(3)	117
7.2.1	The score of the IGSO(3) kernel	119
7.2.2	Numerical implementation: truncation, tabulation, and caching	120
7.3	Riemannian denoising score matching and the reverse-time SDE on $SO(3)$	121
7.4	What you should now be able to do	122
IV	Architecture	123
8	From Sets to Geometry: Attention, Message Passing, and the Equivariance Obstruction	125
8.1	The network’s signature	125

8.2	Two structural requirements	125
8.3	Attention, and the permutation-equivariance half of the problem	127
8.4	Geometric graphs and message passing	128
8.4.1	An ordinary GNN, with no geometry at all	128
8.4.2	The question: where is the 3D structure?	128
8.4.3	Why absolute positions are the wrong input	128
8.4.4	Splitting an edge into radius and direction	129
8.4.5	The radial half: from a distance to a feature vector	130
8.4.6	The cutoff	131
8.4.7	The angular half: why spherical harmonics	131
8.4.8	Recombining radius and direction	132
8.5	The obstruction: invariant features cannot output a vector	134
8.6	A minimal repair, and why it is not enough	134
8.7	The steerable message	135
8.7.1	The equation	135
8.7.2	Two cases worth doing by hand	136
8.7.3	Aggregation over neighbours	137
8.7.4	Self-interaction and residual connections	138
8.7.5	The pipeline, end to end	139
8.7.6	A small worked example	139
8.7.7	What is fixed and what is learned	140
8.7.8	Why this deserves the name “steerable”	140
8.7.9	The question this leaves open	141
8.8	What you should now be able to do	141
9	Equiformer and EquiformerV2	143
9.1	From scalar messages to steerable messages	143
9.2	Embedding geometry: spherical harmonics on edges	144
9.3	The eSCN reduction: aligning every edge to its own frame	144
9.4	Why EquiformerV2 exists: the cost of full $SO(3)$ coupling	145
9.4.1	Counting the work in one message	145
9.4.2	The idea: use a frame in which the edge is trivial	146
9.5	$SO(2)$ -equivariant convolution: the actual EquiformerV2 message	146
9.6	Equivariant nonlinearity: gating and the separable S^2 activation	147
9.6.1	Resolving the apparent contradiction	148
9.6.2	What “separable” means here	150

9.7	The block, in the order the code executes it	152
9.8	Normalisation in EquiformerV2	153
9.9	Equivariant linear layers and normalisation	153
9.10	Equivariant attention weights	154
9.10.1	Ordinary attention, recalled	154
9.10.2	Where the invariant comes from: the $m = 0$ channels	155
9.10.3	Assembling the full attention block	157
9.11	Node embedding: turning chemistry into a steerable feature	157
9.12	Reading out the vector field	158
9.13	One block, end to end, with numbers	159
9.13.1	Step 1: the atoms, their features, and the basis	159
9.13.2	Step 2: edge geometry	160
9.13.3	Step 3: radial features are invariant scalars	160
9.13.4	Step 4: the edge frame	160
9.13.5	Step 5: reading the m sectors	161
9.13.6	Step 6: the $SO(2)$ convolution	161
9.13.7	Step 7: extracting the invariant, and the attention logit	162
9.13.8	Step 8: softmax over the incoming edges	162
9.13.9	Step 9: values, back-rotation, aggregation	162
9.13.10	Step 10: residual, and the pre-norm order	163
9.13.11	Step 11: why not just apply a ReLU	163
9.13.12	Step 12: the sanity check	163
9.13.13	Step 13: the toy against the real thing	164
9.13.14	Closing the arc: from this block to an atomwise vector field	164
9.14	EquiformerV1 vs. EquiformerV2: what actually changed, and what this code- base actually ships	165
9.15	Summary: architecture, hyperparameters, and parameter count	165
9.16	What you should now be able to do	166

V SigmaDock 167

10 SigmaDock: Full Reconstruction 169

10.1	Choosing the state space	169
10.2	The conformational manifold: what rigidity costs	171
10.3	FR3D: reducing fragment count	172
10.3.1	Rotatable-bond detection, as actually implemented	172

10.3.2	Dummy atoms and fragment boundaries	172
10.4	Triangulation: an invariant, soft geometric prior at fragment boundaries . . .	173
10.5	Symmetry: gauge invariance and stochastic $SE(3)$ -equivariance	174
10.6	The forward (noising) process	175
10.7	Input graph, featurisation, and backbone	176
10.7.1	Fragment reconstruction: the map φ	176
10.8	The Newton–Euler prediction head	177
10.8.1	From network output to a physical analogy	177
10.8.2	Net force, torque, and inertia: exact classical mechanics	178
10.8.3	Newton–Maruyama: one discretised step of rigid-body dynamics . . .	178
10.8.4	Dressing the raw output into a score: the two branches	179
10.9	The training loss	180
10.10	Sampling	181
10.11	Ranking generated poses	182
10.12	What you should now be able to do	182
VI	Flow Matching	183
11	Continuous Normalizing Flows and Flow Matching in $\mathbb{R}^d$	185
11.1	The generative transport problem	185
11.2	Flows: how individual samples move	187
11.3	The continuity equation	188
11.3.1	One dimension first: probability as a conserved fluid	188
11.3.2	The general equation, term by term	189
11.3.3	Two descriptions of one motion	191
11.3.4	A path does not determine its field	191
11.4	CNF likelihood: the instantaneous change of variables	192
11.4.1	Why this is not how CNFs are trained any more	193
11.5	The marginal vector field, and why we cannot write it down	194
11.6	Conditional probability paths	195
11.7	The marginalisation theorem	196
11.8	Conditional expectation as an L^2 projection	197
11.9	The conditional flow matching loss	199
11.9.1	The linear (conditional optimal transport) path	201
11.10	Three worked examples	203

11.10.1	Example 1: one source point, one data point	203
11.10.2	Example 2: Gaussian source, one data point	203
11.10.3	Example 3: two data points in two dimensions	204
11.11	Gaussian probability paths	206
11.12	Score and velocity	207
11.12.1	The conditional score of a Gaussian path	208
11.12.2	The conversion formula	208
11.12.3	The four objects, and the denoiser that connects them	210
11.13	Diffusion models are flow matching with a particular path	211
11.13.1	First, the time convention	211
11.13.2	A diffusion model’s marginal path is a Gaussian path	212
11.13.3	The probability flow ODE is the Gaussian path’s velocity	213
11.13.4	Same path, different learned object	214
11.14	What does the network predict?	214
11.14.1	Interconversion, and exactly when it is possible	215
11.14.2	The name collision: diffusion v is not flow velocity	216
11.15	Sampling: integrating the learned ODE	217
11.15.1	Higher-order solvers, and what a “step” costs	218
11.16	The path is a design choice	219
11.16.1	Two separate decisions	219
11.16.2	Straight versus curved trajectories	219
11.17	Diffusion and flow matching in one picture	220
11.18	What flow matching is not	221
11.19	Numerical verification of this chapter	222
11.20	Where $\mathbb{R}^d$ runs out	223
11.21	What you should now be able to do	223
12	Riemannian Flow Matching	225
12.1	What breaks, and what survives	225
12.1.1	Points are not vectors: the distinction that does all the work	226
12.2	Tangent-vector-valued dynamics	227
12.2.1	Curves, velocities, and what the ODE now means	228
12.3	Riemannian probability paths	228
12.4	The Riemannian continuity equation	228
12.4.1	Conservation, on a manifold	229
12.4.2	The same equation, seen in coordinates	229

12.5	Conditional geodesic paths and their velocity	230
12.5.1	The conditional vector field, in general	230
12.6	Marginalisation and the RCFM objective	232
12.6.1	Why averaging tangent vectors is legal here	232
12.6.2	The marginalisation theorem	232
12.6.3	The L^2 projection, at a fixed point	233
12.7	What actually changed	234
12.8	Worked example: the circle S^1	235
12.9	Worked example: the sphere S^2	236
12.10	Flow matching on $SO(3)$	236
12.10.1	The state space, recalled in one place	237
12.10.2	The geodesic interpolant	237
12.10.3	The velocity, derived	238
12.10.4	Intrinsic vector versus trivialised coordinates	238
12.10.5	The current-point form	239
12.10.6	A complete numerical case	240
12.10.7	The training objective on $SO(3)$	240
12.10.8	Sampling	241
12.10.9	The cut locus, and what it does and does not cost	241
12.11	Product manifolds: many rigid bodies at once	242
12.11.1	The state, its tangent space, and its metric	242
12.11.2	Everything factorises	243
12.12	Riemannian CNFs and likelihood	244
12.13	Riemannian diffusion versus Riemannian flow matching	245
12.14	The whole construction in one picture	245
12.15	What Riemannian flow matching is not	246
12.16	Numerical verification of this chapter	247
12.17	Bridge to SigmaFlow	249
12.18	What you should now be able to do	249

VII SigmaFlow 251

13 SigmaFlow: Full Reconstruction 253

13.1	What changes, and what does not	253
13.2	A time-convention warning, confirmed at the source level	254

13.3	Translation flow matching, exactly as implemented	254
13.4	Rotation flow matching, exactly as implemented	255
13.5	No output dressing: the entire diffusion-specific scalar machinery is gone . . .	256
13.6	The loss function	257
13.7	The full training step	257
13.8	Sampling	259
13.9	Likelihood and confidence: what is implemented, and what is not	259
13.10A	tested, and rejected, loss variant	260
14	The Output Heads, Audited	261
14.1	Locating the heads	261
14.2	The tensor entering the heads	262
14.3	Translation	262
14.4	Rotation	263
14.4.1	The two angular velocities, restated next to the code	263
14.4.2	The trace, operation by operation	264
14.4.3	$\mathbb{R}^3$ or $\mathfrak{so}(3)$?	264
14.4.4	Rotation or rotation rate?	264
14.4.5	The group action is conjugation, not left multiplication	265
14.5	Numerical verification	266
14.6	A defect found on the way	267
14.7	Summary tables	268
14.8	Open items	268
15	Backbone, Mechanics, and Generative Semantics	269
15.1	Layer A is byte-identical	269
15.2	Layer B: the mechanical reduction	270
15.3	Layer C, SigmaDock: a score model	270
15.4	Layer C, SigmaFlow: a velocity model	271
15.5	Score versus velocity	272
15.6	Numerical validation	272
15.6.1	Source distributions are not quite matched	273
15.6.2	An inherited weight that lost its justification	273
15.6.3	The log map is not exact	274
15.6.4	<code>sigma_min</code> is dead	274
15.7	The minimal-change table	274
15.8	Findings, categorised	274
15.9	Verdict	275

VIII	Implementation Reference and Synthesis	276
16	Numerical and Implementation Reference	278
16.1	Every clamp, epsilon, and numerical safeguard in one table	278
16.2	Masking, batching, and scatter conventions	279
16.3	Floating-point precision	280
16.4	Dimensional and consistency checks	280
16.5	Summary of open items	281
17	Closing Synthesis	282
17.1	The two models, on one page	282
17.2	Closing remarks	284

How to read this document

This document is organised as a linear course: later chapters depend on earlier ones, and no chapter uses a concept before that concept has been defined *in this document*. If you already know a topic (say, Lie groups, or diffusion models), you may skip the corresponding chapter, but you should still skim its **Notation** boxes, since notation is fixed globally and is not repeated in later chapters.

Every new mathematical construction is introduced in six steps, repeated as a pattern throughout: (1) *motivation* — why do we need this object at all; (2) *intuition* — an informal picture; (3) *formal definition*; (4) *derivation* of its key properties; (5) *a small worked example*; (6) *application* to SigmaDock or SigmaFlow. Results that are used later are stated as numbered theorems, propositions, or lemmas; all proofs that are not immediate are carried out in full, not sketched or deferred to a citation, *unless* the result is a classical one whose proof would require substantially more background than this document provides (e.g. existence/uniqueness for SDEs), in which case this is stated explicitly and a precise reference is given rather than an unstated leap.

Whenever a mathematical object is realised in the SigmaDock/SigmaFlow codebase, a grey **Code correspondence** box follows immediately, of the form

Code correspondence

Mathematical object: $v_t(x) \in T_x\mathcal{M}$

Implementation: `sigma_flow_generator.py::calc_vector_field`

Tensor shape: `[B x F, 3]`

Interpretation: translational velocity, one $\mathbb{R}^3$ vector per ligand fragment in the batch.

These boxes are the load-bearing content of the document in the sense that they are what makes the mathematics falsifiable against the actual program that runs on real data. Every box has been checked against the source files named in it, at the time of writing; file paths are given relative to the repository roots `SigmaDock/` (the read-only original diffusion reference) and `SigmaFlow_Development/` (the flow-matching conversion), or the checked-out reproduction clone `SigmaDock_Reproduction_JulianMueller/sigmadock/` where the two happen to coincide.

Caution 0.1 (On uncertainty). Some implementation details are genuinely ambiguous from static reading of the code (for instance, a numerical constant whose provenance is not commented, or a design choice for which no docstring states the intent). Wherever this occurs, the text says so explicitly, using the phrase “*not determinable from the code as written; interpretation:*” followed by the most plausible reading. Nothing in this document is invented to fill such a gap silently.

Global notation

Notation is introduced at first use and collected here for reference; this table is updated as new symbols are introduced and should be treated as living, not exhaustive at first read.

Symbol	Meaning
$\mathbb{R}^d$	real Euclidean space of dimension d
$\mathcal{M}$	smooth (Riemannian) manifold
$T_x\mathcal{M}$	tangent space of $\mathcal{M}$ at x
$T\mathcal{M}$	tangent bundle, $\bigsqcup_x T_x\mathcal{M}$
$g_x(\cdot, \cdot), \ \cdot\ _g$	Riemannian metric and induced norm at x
$\exp_x, \log_x$	Riemannian exponential map and its (local) inverse at x
$G, \mathfrak{g}$	a Lie group and its Lie algebra $\mathfrak{g} = T_e G$
$SO(3), \mathfrak{so}(3)$	the rotation group and its Lie algebra (skew-symmetric 3×3 matrices)
$SE(3)$	the group of rigid motions of $\mathbb{R}^3$
$(\cdot)^\wedge, (\cdot)^\vee$	hat and vee maps, $\mathbb{R}^3 \leftrightarrow \mathfrak{so}(3)$
$\mathcal{M}_N = SE(3)^N$	product pose manifold of N rigid fragments
$p_{\text{data}}, p_{\text{init}}$	data distribution and prior (source) distribution
$p_t, p_t(\cdot z)$	marginal / conditional probability path at time $t \in [0, 1]$
$u_t, u_t(\cdot z)$	marginal / conditional (velocity) vector field
u_t^θ, s_θ	learned vector field network / learned score network
$\nabla_x \log p(x)$	(Riemannian) score of a density p
W_t	standard Wiener process (Brownian motion in $\mathbb{R}^d$)
$B_t^\mathcal{M}$	Brownian motion on a Riemannian manifold $\mathcal{M}$
Y_m^ℓ	real spherical harmonic of degree ℓ , order m
$D^\ell(Q)$	Wigner- D matrix of degree ℓ for $Q \in SO(3)$
$C_{(\ell_1, m_1)(\ell_2, m_2)}^{(\ell, m)}$	Clebsch–Gordan coefficient
$\otimes_{\text{cg}}$	Clebsch–Gordan (steerable) tensor product
$\mathcal{N}(\mu, \Sigma)$	Gaussian/normal distribution with mean μ , covariance Σ
$\text{IGSO}(3)(R_0, \sigma^2)$	isotropic Gaussian distribution on $SO(3)$, centred at R_0 , scale σ
$\mathcal{U}(\mathcal{M})$	uniform (Haar, when $\mathcal{M}$ is a compact group) measure on $\mathcal{M}$
B, F, N, A	batch size, number of fragments, number of atoms/nodes, generic index
t	generation time, $t \in [0, 1]$, with convention $p_0 = p_{\text{init}}, p_1 = p_{\text{data}}$
$s = 1 - t$	noising time, used only when discussing the forward/noising process

Remark 0.2 (Time convention, fixed once and for all). Throughout this document, $t \in [0, 1]$ denotes *generation time*: $t = 0$ is the source/prior distribution (p_{init} , e.g. pure noise), and $t = 1$ is the data distribution (p_{data} , e.g. the true bound pose). All sampling procedures therefore integrate *forward* in t , from $t = 0$ to $t = 1$. Where a *noising* (forward diffusion) process is discussed, it runs in the reversed variable $s = 1 - t$, i.e. it starts at data ($s = 0$) and ends at noise ($s = 1$). This convention is fixed because the diffusion and flow-matching literatures disagree with each other (and sometimes with themselves) on which direction

is “forward”, and because the SigmaDock and SigmaFlow codebases must each be checked individually against it – this is done explicitly in Chapters 10 and 13.

Caution 0.3 (Symbols this document deliberately reuses). A document spanning probability, differential geometry, representation theory and two model implementations cannot give every object its own letter, and silently reusing one is how sign errors are born. The following collisions are unavoidable (each notation is standard in its own field) and are therefore stated here once, globally. In every case the two meanings are distinguished by *shape*, not by context alone.

Symbol	Meaning A	Meaning B
“score”	the <i>statistical</i> score $\nabla_x \log p(x)$, a vector field (Definition 2.10) — the thing diffusion models learn	a <i>docking</i> score: a scalar quality estimate used to rank poses (Section 1.3, Section 10.11). Unrelated.
s	noising time, $s = 1 - t$ (Remark 0.2); always a bare letter or a subscript, as in $Y_s, q_s, \beta(s)$	the learned score network s_θ ; always carries the subscript θ . Both occur in one formula, $s_\theta(Y_s, s)$.
λ	the relative weight of the rotational factor in a product metric, g^λ (Proposition 3.67); no argument	the loss weighting $\lambda(s)$ of denoising score matching ((6.7)); always written with its argument
φ	the fragment-reconstruction map $\varphi : SE(3)^m \rightarrow \mathbb{R}^{n \times 3}$ (Section 10.7.1); always applied to a pose	the angle parameter φ_t of the trigonometric variance-preserving schedule (Example 11.35); always subscripted by t , always a scalar
v	the flow-matching velocity of a path, v_θ or u_t	the diffusion “ v -prediction” target of Definition 11.49. These are not the same object ; Proposition 11.51 states exactly how close they get.
d	a dimension, as in $\mathbb{R}^d$	the Riemannian distance $d_g(x, y)$; always carries the subscript g

Symbols that are *not* reused, and can be trusted globally: p_t (a density), u_t (the true velocity field), u_θ / v_θ (the learned one), R, Q (rotations, with Q reserved for a *global* rotation acting on the whole system), $\omega^{\text{bd}} / \omega^{\text{sp}}$ (body/spatial angular velocity), ℓ, m (degree and order of a spherical harmonic), $T_x \mathcal{M}$ (a tangent space), ϕ_t (a flow map).

Part I

Preliminaries

Part I at a glance

What you will learn. What a protein–ligand pose is, what a docking model is asked to predict, and how a prediction is judged; then the probability vocabulary — densities, conditioning, conditional expectation, pushforward, KL divergence, the score — in which every generative model in this document is phrased.

Why it is needed. The rest of the document is a long answer to one question: *how do you sample from a distribution over poses?* Part I states the question precisely and assembles the language.

What depends on it. Everything. Section 2.2 in particular is load-bearing: the proofs that both denoising score matching and flow matching work (Theorems 6.4 and 11.25) are statements about conditional expectation and nothing else.

Chapter 1

The Protein–Ligand Docking Problem

Before any mathematics is introduced, we fix precisely what SigmaDock and SigmaFlow are trying to compute. This chapter is deliberately non-technical in the sense that it uses no probability theory or differential geometry yet; its purpose is to pin down the *objects* (proteins, ligands, poses, degrees of freedom) and the *task* (what must be predicted, and how success is measured) so that every later mathematical construction has an unambiguous referent.

1.1 Proteins and ligands

Definition 1.1 (Protein, for docking purposes). For the purposes of this document, a *protein* is a rigid, known three-dimensional structure: a finite set of atoms $\{(y_j, \zeta_j)\}_{j=1}^M$, where $y_j \in \mathbb{R}^3$ is the atom’s position and ζ_j collects its chemical identity (element, residue type, atomic partial charge if used). We do not model protein flexibility: y_j is held fixed throughout.

This is a modelling choice, not a physical fact — real proteins move — and it corresponds to the standard *rigid-receptor docking* setting used by essentially all methods discussed in this document, including SigmaDock. The *binding pocket* is the subset of protein atoms within some distance of the ligand; SigmaDock and SigmaFlow both assume the pocket (equivalently, a point defining its centre) is known in advance, so that the generative model only has to search over ligand poses *within* a neighbourhood of a given location, not over the protein surface.

Definition 1.2 (Ligand, for docking purposes). A *ligand* is a small molecule, represented as a molecular graph $\mathcal{G}^{2D} = (\mathcal{V}, \mathcal{B})$: atoms $\mathcal{V}$ with chemical labels (element, formal charge, hybridisation, aromaticity, chirality tag), and bonds $\mathcal{B} \subseteq \mathcal{V} \times \mathcal{V}$ with labels (bond order, aromaticity, whether the bond lies in a ring, stereochemistry). A *conformer* of the ligand is an assignment of 3D coordinates $x = (x_1, \dots, x_n) \in \mathbb{R}^{n \times 3}$, $n = |\mathcal{V}|$, to its atoms.

Crucially, the molecular graph $\mathcal{G}^{2D}$ is *given* (it is computed once, deterministically, from the ligand’s chemical structure, e.g. from a SMILES string), whereas the 3D coordinates x are what a docking model must predict. This is the sense in which docking is a *structure prediction* problem: the input already specifies which atoms exist and how they are chemically connected; the output is where they sit in space.

1.2 Poses and degrees of freedom

Definition 1.3 (Pose). A *pose* of a ligand with n atoms, relative to a fixed protein, is an assignment of 3D coordinates $x \in \mathbb{R}^{n \times 3}$ to its atoms, in the same coordinate frame as the protein.

Naively, a pose has $3n$ real degrees of freedom (three coordinates per atom). This naive count is almost never the right one to reason about, because it ignores that the ligand is a *chemical object*, not an arbitrary point cloud: bond lengths and bond angles are, to excellent approximation, fixed by chemistry (they fluctuate by hundredths of an Ångström around equilibrium values, an order of magnitude below the accuracy that any docking method targets), and only a small number of internal coordinates vary substantially. This observation is what licenses essentially every design decision reviewed in Chapter 10, so we make it precise now.

The internal geometry of a molecule can be decomposed into:

- **Bond lengths** $\|x_i - x_j\|$ for bonded pairs $(i, j) \in \mathcal{B}$: essentially fixed by chemistry (covalent bond lengths are determined, to first approximation, by the identities of the two bonded atoms and the bond order).
- **Bond angles** $\angle(x_i, x_j, x_k)$ for two bonds sharing atom j : also close to fixed, determined by the hybridisation of atom j .
- **Dihedral (torsion) angles** ϕ_{ijkl} , defined for four atoms i – j – k – l connected by three consecutive bonds, as the angle between the plane through (i, j, k) and the plane through (j, k, l) : these are, for single (non-ring) bonds, comparatively free to rotate, since rotation about a single bond changes no bond length and no bond angle.

A bond about which the dihedral angle can vary freely without breaking the molecule (i.e. a single, non-ring, non-terminal bond) is called a *rotatable bond*. If a ligand has k rotatable bonds, and bond lengths and angles are treated as exactly fixed, then the *internal* conformation of the ligand is fully described by k angles $\phi = (\phi_1, \dots, \phi_k) \in \mathbb{T}^k$, where $\mathbb{T} = \mathbb{R}/2\pi\mathbb{Z}$ is the circle (the space of angles); we define and use $\mathbb{T}^k$ formally in Section 3.7. The *external* pose of the (now internally fixed) ligand is then a single rigid motion — a rotation and a translation — applied to the whole molecule, contributing $3 + 3 = 6$ further degrees of freedom (formalised as an element of the group $SE(3)$ in Chapter 3).

Degrees of freedom of a ligand pose

$$\underbrace{3}_{\text{translation}} + \underbrace{3}_{\text{global rotation}} + \underbrace{k}_{\text{torsions}} = k + 6$$

against a naive count of $3n$, where typically $n \gg k$: for a drug-like ligand with $n \approx 30$ heavy atoms one might have $k \approx 5$, so the chemically-informed parameterisation has roughly 11 degrees of freedom against a naive 90.

This factorisation — translation, rotation, torsions — is the single most important structural fact about the docking problem, and essentially every design choice in SigmaDock (and hence SigmaFlow) can be read as a decision about *how literally to take it*. Section 10.1 works out three concrete choices (all-atom, torsional, fragment-based) and the consequences of each; the mathematics needed to state those consequences precisely (Riemannian volume forms, product measures) is developed in Chapter 3.

Remark 1.4 (What is *not* varied). The receptor is rigid (Definition 1.1). Bond lengths and bond angles are either held exactly fixed, or — as we will see is SigmaDock’s actual choice — allowed to fluctuate only implicitly, by resampling a fresh low-energy conformer rather than by being explicit generative degrees of freedom. No docking model discussed here generates bond lengths or angles directly.

1.3 What a docking model predicts, and what it does not

The docking literature distinguishes several related but distinct tasks, and conflating them is a common source of confusion when reading papers or comparing methods.

Pose generation. Given $\mathcal{G}_{\text{dock}}$ (ligand graph, protein structure, binding pocket), produce one or more candidate poses x . This is the task SigmaDock and SigmaFlow solve; it is a *generative* task (a probabilistic model is used to *sample* candidates), not a search or optimisation task, though it can be built on top of either.

Pose scoring. Given a candidate pose x , output a scalar estimate of its quality (e.g. predicted binding free energy, or a proxy such as a learned or physics-based score). This is used to *rank* multiple generated poses (see the ranking heuristic in Section 10.11), but it does not by itself produce new poses.

Ranking. Given a set of candidate poses (e.g. multiple i.i.d. draws from a generative model, each called a *seed*), order them by predicted quality so that a downstream user can inspect the top- k .

Confidence estimation. A calibrated estimate of *how likely a specific prediction is to be correct*, as opposed to a relative score used only for ranking among a fixed candidate set. Chapter 13 discusses to what extent SigmaFlow’s flow-matching formulation admits a principled confidence signal via the ODE log-likelihood (Section 13.9), and states explicitly which parts of this are implemented in the current codebase and which are conceptual only.

SigmaDock (Chapter 10) performs pose generation and a simple, non-learned ranking heuristic; it explicitly does *not* include a learned confidence model or post-hoc energy minimisation (Section 10.11), a deliberate design choice intended to isolate how much of its accuracy is attributable to the generative model itself, as opposed to a downstream correction step. SigmaFlow inherits this scope unless stated otherwise.

1.4 Evaluation: RMSD and structural validity

Two families of metrics are used throughout this document to talk about “how good” a predicted pose is.

Definition 1.5 (Root-mean-square deviation, RMSD). For a predicted pose $\hat{x} \in \mathbb{R}^{n \times 3}$ and a reference (experimentally determined, “ground truth”) pose $x^* \in \mathbb{R}^{n \times 3}$ of the same ligand, with atoms in the same order,

$$\text{RMSD}(\hat{x}, x^*) = \left(\frac{1}{n} \sum_{i=1}^n \|\hat{x}_i - x_i^*\|_2^2 \right)^{1/2}.$$

A pose is conventionally called a *success* (in the re-docking setting of this document) if $\text{RMSD}(\hat{x}, x^*) < 2 \text{ \AA}$ after optimal rigid alignment is *not* applied (i.e. RMSD is computed directly in the shared binding-site frame) — we return to this point, including the alternative of Kabsch-aligned RMSD, when we discuss the specific evaluation used for SigmaDock/SigmaFlow comparisons.

RMSD is a single number and, being an average, is insensitive to *local* chemical implausibility: a pose can have low RMSD to the reference while containing an impossibly short bond or an overlapping ring, if the error is averaged out over the rest of the molecule. This motivates a second, complementary family of metrics.

Definition 1.6 (PoseBusters-style validity checks, informal). A pose $\hat{x}$ is subjected to a battery of deterministic, chemistry-based checks — e.g. are all bond lengths within a tolerance of their expected values, are all bond angles chemically reasonable, is stereochemistry preserved, do any atoms sterically clash with each other or with the protein, does the ligand’s internal energy (under a cheap force field) exceed a threshold — each of which returns pass/fail. A pose *passes PoseBusters* if it passes every check in the battery.

We will use the phrase *PoseBusters pass rate* for the fraction of generated poses (over some fixed evaluation set) that pass every check, and we will occasionally need per-check pass rates (e.g. specifically the `bond_lengths` or `bond_angles` check) when discussing fragment-boundary artefacts; these are named explicitly whenever used.

Remark 1.7 (Why both metrics matter, and why neither suffices alone). RMSD measures *closeness to the correct answer*; PoseBusters measures *physical plausibility*, independent of whether the pose is close to any particular reference. A model can score well on one and poorly on the other: for instance, a model whose fragment-level rigid-body placement is accurate (low RMSD) can still produce chemically broken bonds precisely at the boundaries between rigid fragments if the relative placement of two fragments is even slightly off, since a rigid transformation exactly preserves lengths *within* a fragment but says nothing about the distance *between* fragments (see the discussion of triangulation in Section 10.4, and the analysis of transition-bond lengths at fragment boundaries in Chapter 13). PoseBusters, being a binary per-molecule pass/fail criterion, is additionally insensitive to *how far* a check is failed by, a limitation we discuss quantitatively when it becomes relevant.

This completes the non-mathematical scaffolding: we now know what a pose is, which of its coordinates are genuinely free, what a docking model is asked to produce, and how its output is judged. Chapter 2 turns to the probability theory required to make “a generative model for poses” a precise mathematical statement.

1.5 What you should now be able to do

- state what a pose is, and why $3n$ Cartesian coordinates are the wrong parameterisation for a rigid or piecewise-rigid ligand;
- distinguish the three tasks a “docking model” might perform — pose generation, pose scoring, and binding-affinity prediction — and say which one this document is about;
- explain why RMSD alone is an insufficient evaluation criterion, and what the PoseBusters checks add;
- recognise that “score” will mean something entirely different from Chapter 2 onward (Caution 0.3).

Chapter 2 now turns “generate a plausible pose” into “sample from a distribution”, which is the form the rest of the document can actually work with.

Chapter 2

Probability and Generative Modelling Foundations

This chapter collects the probabilistic vocabulary used everywhere in the rest of the document. None of it is specific to docking; it is the common language in which Chapters 6 and 11 will phrase diffusion and flow matching. Readers comfortable with densities, change-of-variables, KL divergence and maximum likelihood may skip to Section 2.6, returning here only if a later citation of a numbered result is unclear.

2.1 Densities, conditional densities, and pushforward

We work throughout with random variables taking values in $\mathbb{R}^d$ (this chapter) or, from Chapter 3 onward, in a Riemannian manifold $\mathcal{M}$; every definition below is stated for $\mathbb{R}^d$ and is repeated, without change to the underlying idea, for manifolds once the necessary geometric vocabulary (volume forms) is available.

Definition 2.1 (Density). A random variable X taking values in $\mathbb{R}^d$ has *density* $p : \mathbb{R}^d \rightarrow [0, \infty)$ with respect to Lebesgue measure if

$$\mathbb{P}(X \in A) = \int_A p(x) dx \quad \text{for every (measurable) } A \subseteq \mathbb{R}^d,$$

in particular $\int_{\mathbb{R}^d} p(x) dx = 1$.

Definition 2.2 (Conditional density). For a pair (X, Z) with joint density $p(x, z)$ and marginal $p(z) = \int p(x, z) dx > 0$, the *conditional density* of X given $Z = z$ is $p(x | z) = p(x, z)/p(z)$.

The identity that we will use over and over, sometimes without comment, is the following elementary but consequential rearrangement of Definition 2.2.

Proposition 2.3 (Marginalisation by conditioning). *If $Z \sim p_{\text{data}}$ and, given $Z = z$, $X \sim p(\cdot | z)$, then X marginally has density*

$$p(x) = \int p(x | z) p_{\text{data}}(z) dz.$$

Proof. Immediate from $p(x, z) = p(x | z)p_{\text{data}}(z)$ and integrating z out. □

Proposition 2.3 looks almost too simple to name, but it is the single mechanism by which both diffusion models (Chapter 6) and flow matching (Chapter 11) convert an *intractable* learning problem (regress against a marginal quantity that is only available as an integral over all of p_{data}) into a *tractable* one (regress against a *conditional* quantity that is available in closed form for one data point z at a time): both frameworks first construct a family $p_t(\cdot | z)$ that is easy to sample and differentiate, then use Proposition 2.3 to reason about the (unavailable) marginal p_t , and finally prove — this is the content of Theorems 6.4 and 11.25 below — that a regression loss built from the conditional quantity has the *same minimiser* as one built from the marginal quantity, sidestepping the need to ever evaluate the intractable integral.

2.2 Conditional expectation, and the one property we will use constantly

Definition 2.2 conditions a *density*. What later chapters need far more often is the conditional *expectation*: not the whole conditional distribution of Y given X , but its mean. It is worth introducing here, in the simplest possible setting, because the single most important theorem in this document — Lemma 11.23, which is why both denoising score matching and flow matching work at all — is a statement about it.

Definition 2.4 (Conditional expectation). Let (X, Y) be random variables with $\mathbb{E}\|Y\| < \infty$. The *conditional expectation of Y given X* , written $\mathbb{E}[Y | X]$, is the random variable obtained by evaluating

$$m(x) := \mathbb{E}[Y | X = x] = \int y p(y | x) dy$$

at X ; that is, $\mathbb{E}[Y | X] := m(X)$.

Two features of Definition 2.4 cause most of the confusion around it, and both are worth stating explicitly.

It is a random variable, not a number. $\mathbb{E}[Y]$ is a number; $\mathbb{E}[Y | X]$ is a *function* of X , hence random, because X is. Only once X is observed does it become a number. This is why expressions like $\mathbb{E}[\mathbb{E}[Y | X]]$ — an expectation of an expectation — are meaningful rather than redundant.

It is the best guess of Y from X . That is the content of Lemma 11.23: among all functions of X , the one minimising the expected squared error against Y is exactly $\mathbb{E}[Y | X]$. This document uses that fact as a *construction*: we deliberately regress a network on a noisy target so that least squares hands us its conditional mean, which is the quantity we actually wanted and could not compute directly.

Proposition 2.5 (The two rules we use). *Let (X, Y) be as above and let h be any function with $\mathbb{E}\|h(X)Y\| < \infty$.*

- (i) **Tower property (law of total expectation).** $\mathbb{E}[\mathbb{E}[Y | X]] = \mathbb{E}[Y]$.
- (ii) **Functions of X pass through.** $\mathbb{E}[h(X)Y | X] = h(X)\mathbb{E}[Y | X]$; in particular $\mathbb{E}[h(X) | X] = h(X)$, and $\mathbb{E}[Y - \mathbb{E}[Y | X] | X] = 0$.

Proof. (i) Write both sides as integrals against the joint density $p(x, y) = p(y | x)p(x)$:

$$\mathbb{E}[\mathbb{E}[Y | X]] = \int \left(\int y p(y | x) dy \right) p(x) dx = \int \int y p(y | x) p(x) dy dx = \int \int y p(x, y) dy dx = \mathbb{E}[Y],$$

exchanging the order of integration (Fubini, legitimate under the stated integrability).

(ii) Conditionally on $X = x$, the factor $h(x)$ is a constant and comes out of the inner integral: $\int h(x)yp(y | x)dy = h(x) \int yp(y | x)dy = h(x)m(x)$. Evaluating at X gives the claim. The two special cases follow by taking $Y \equiv 1$, and by linearity together with $\mathbb{E}[\mathbb{E}[Y | X] | X] = \mathbb{E}[Y | X]$ (which is the first special case applied to the function m). $\square$

Example 2.6 (Both rules, on a two-point example). Let $X \in \{0, 1\}$ be a fair coin flip and let $Y = X + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$ independent of X . Then $\mathbb{E}[Y | X = 0] = 0$ and $\mathbb{E}[Y | X = 1] = 1$, so $\mathbb{E}[Y | X] = X$ — a random variable, taking the values 0 and 1 with probability $\frac{1}{2}$ each.

Tower: $\mathbb{E}[\mathbb{E}[Y | X]] = \mathbb{E}[X] = \frac{1}{2}$, and directly $\mathbb{E}[Y] = \mathbb{E}[X] + \mathbb{E}[\varepsilon] = \frac{1}{2}$ $\checkmark$.

Pass-through: $\mathbb{E}[XY | X] = X \mathbb{E}[Y | X] = X^2$, whereas the *wrong* manipulation $\mathbb{E}[XY | X] \stackrel{?}{=} \mathbb{E}[X | X]\mathbb{E}[Y] = X/2$ gives a different answer. The difference is exactly that X is known once we condition on it, and Y is not.

Note also that $Y - \mathbb{E}[Y | X] = \varepsilon$, which has conditional mean zero — the residual carries no information that X could have predicted. That observation, in general form, is the entire proof of Lemma 11.23.

Definition 2.7 (Pushforward measure). Let X have density p on $\mathbb{R}^d$ and let $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^d$ be a (measurable) map. The *pushforward* of p under ϕ , written $\phi_{\#}p$, is the distribution of $Y = \phi(X)$:

$$\mathbb{P}(Y \in A) = \mathbb{P}(\phi(X) \in A) = \mathbb{P}(X \in \phi^{-1}(A)) \quad \text{for measurable } A.$$

When ϕ is a diffeomorphism (smooth, invertible, smooth inverse), the pushforward has an explicit density, given by the classical change-of-variables formula.

Theorem 2.8 (Change of variables). *Let X have density p_X and let ϕ be a diffeomorphism of an open subset of $\mathbb{R}^d$ onto its image. Then $Y = \phi(X)$ has density*

$$p_Y(y) = p_X(\phi^{-1}(y)) |\det D\phi^{-1}(y)| = \frac{p_X(\phi^{-1}(y))}{|\det D\phi(\phi^{-1}(y))|},$$

where $D\phi$ is the Jacobian matrix of ϕ .

We take Theorem 2.8 as known from multivariable calculus and do not reprove it; what matters for us is how it is used. In Section 11.2 we will apply it to a map ϕ that is itself the time- t flow of an ordinary differential equation, and derive from it the *continuity equation* that governs how a probability density evolves under such a flow — the central object of flow matching.

Example 2.9 (A one-dimensional sanity check). Let $X \sim \mathcal{N}(0, 1)$ and $\phi(x) = 2x$ (so $Y = 2X$). Then $\phi^{-1}(y) = y/2$, $D\phi(x) = 2$, and Theorem 2.8 gives $p_Y(y) = p_X(y/2)/2 = \frac{1}{2\sqrt{2\pi}}e^{-y^2/8}$, i.e. $Y \sim \mathcal{N}(0, 4)$, as expected from elementary Gaussian scaling.

2.3 Score functions

Definition 2.10 (Score). Let p be a density on $\mathbb{R}^d$ that is strictly positive and differentiable on an open set $U \subseteq \mathbb{R}^d$. For $x \in U$, the *score* of p at x is

$$\nabla_x \log p(x) = \frac{\nabla_x p(x)}{p(x)} \in \mathbb{R}^d,$$

the gradient (with respect to x) of the log-density.

Caution 2.11 (Where the score is defined). The score is a *vector field on U* , not a global object: it is undefined wherever p vanishes or fails to be differentiable. This is not pedantry. Every Gaussian used in Chapter 6 is positive on all of $\mathbb{R}^d$, so the issue never arises there; but p_{data} for molecular poses is supported on a lower-dimensional set (bond lengths are essentially fixed), so $\nabla_x \log p_{\text{data}}$ does not exist in the sense above. This is precisely why diffusion models learn $\nabla_x \log p_\sigma$ for the *smoothed* densities p_σ ($\sigma > 0$), which are positive everywhere, and never the score of p_{data} itself — see the discussion following Theorem 6.4.

The score is the direction in which $\log p$, and hence p itself, increases fastest; equivalently it is the vector field that would define gradient ascent on $\log p$. Two properties of the score matter disproportionately for what follows.

Remark 2.12 (The score does not need a normalising constant). If $p(x) = \tilde{p}(x)/Z$ for an unnormalised $\tilde{p}$ and a normalising constant $Z = \int \tilde{p}$, then $\nabla_x \log p(x) = \nabla_x \log \tilde{p}(x)$, since $\nabla_x \log Z = 0$. The score is therefore computable from an unnormalised density, which ordinary likelihood evaluation is not. This is one of the two reasons (the other being Theorem 2.16 below) that score-based methods can be trained without ever evaluating Z .

Remark 2.13 (The score of a Gaussian, used repeatedly). For $p(x) = \mathcal{N}(x \mid \mu, \sigma^2 I)$,

$$\log p(x) = -\frac{\|x - \mu\|^2}{2\sigma^2} + \text{const}, \quad \nabla_x \log p(x) = -\frac{x - \mu}{\sigma^2}.$$

This single formula is, after substitution of the appropriate μ, σ , the target of every Euclidean denoising-score-matching loss used in Chapter 6.

2.4 Why the score suffices for sampling: Langevin dynamics

It is not obvious *a priori* that knowing $\nabla_x \log p$ everywhere is enough to sample from p — after all, the score is a strictly weaker piece of information than p itself (Remark 2.12). The classical result that resolves this is Langevin dynamics, and understanding why it works is what motivates the entire diffusion-model construction of Chapter 6.

We first need Brownian motion in $\mathbb{R}^d$; a manifold version is given in Chapter 6 once the necessary geometry is available, but the flat case can be defined immediately.

Definition 2.14 (Standard Wiener process). A *standard Wiener process* (Brownian motion) $(W_\tau)_{\tau \geq 0}$ in $\mathbb{R}^d$ is a continuous-time stochastic process with $W_0 = 0$, independent increments, and $W_{\tau+h} - W_\tau \sim \mathcal{N}(0, hI_d)$ for all $\tau, h \geq 0$.

Definition 2.15 (Stochastic differential equation, informal). Given a drift $f : \mathbb{R}^d \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^d$ and a diffusion coefficient $g : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$, the SDE

$$dX_\tau = f(X_\tau, \tau) d\tau + g(\tau) dW_\tau$$

denotes, informally, the continuous-time limit of the discrete update $X_{\tau+h} = X_\tau + f(X_\tau, \tau)h + g(\tau)\sqrt{h}z$, $z \sim \mathcal{N}(0, I_d)$, as $h \rightarrow 0$. We use SDEs only as an organising language for such limits and as a source of update rules to discretise; we do not develop Itô calculus, and every fact about SDEs used below is either a discretisation recipe (which needs no further theory) or is quoted, with a precise reference, as a classical result.

Theorem 2.16 (Langevin dynamics; invariance of the target density). *Let $U \in C^1(\mathbb{R}^d)$ with ∇U Lipschitz and $Z = \int e^{-U} < \infty$, and set $p = e^{-U}/Z$. Then the overdamped Langevin SDE*

$$dX_\tau = -\nabla U(X_\tau) d\tau + \sqrt{2} dW_\tau$$

has a unique strong solution, and p is a stationary distribution: if $X_0 \sim p$ then $X_\tau \sim p$ for all $\tau \geq 0$. Because the diffusion coefficient is non-degenerate, p is moreover the only stationary probability distribution. Convergence $\text{Law}(X_\tau) \rightarrow p$ as $\tau \rightarrow \infty$ from an arbitrary initial distribution requires an additional confinement (Lyapunov) condition, e.g. $\langle \nabla U(x), x \rangle \geq a\|x\|^2 - b$ for some $a > 0$ outside a compact set; see Pavliotis, Stochastic Processes and Applications, 2014, Ch. 4, and Roberts–Tweedie (1996).

Caution 2.17 (Stationarity, uniqueness and convergence are three claims). They need different hypotheses and it is easy to conflate them. Lipschitz ∇U gives *existence and uniqueness of the solution*; $\int e^{-U} < \infty$ plus the Fokker–Planck computation gives *stationarity of p* ; non-degeneracy of the noise gives *uniqueness of the stationary distribution*; only the confinement condition gives *convergence from an arbitrary start*, and it also controls the rate. Nothing below depends on the rate, but the reader should not carry away the impression that integrability alone yields ergodicity.

We do not reprove Theorem 2.16 (it is a statement about the stationary distribution of a diffusion process via its Fokker–Planck equation, i.e. it requires exactly the machinery we introduce, in the special case needed for our models, in Section 6.3); we take it as a classical fact and note only the substitution that makes it useful here. Setting $U = -\log p_{\text{data}}$ gives $-\nabla U = \nabla_x \log p_{\text{data}}$, so Theorem 2.16 says that

$$dX_\tau = \nabla_x \log p_{\text{data}}(X_\tau) d\tau + \sqrt{2} dW_\tau \quad (2.1)$$

has p_{data} as its stationary distribution. Discretising (2.1) with step size ϵ gives the *unadjusted Langevin algorithm*,

$$x^{(k+1)} = x^{(k)} + \epsilon \nabla_x \log p_{\text{data}}(x^{(k)}) + \sqrt{2\epsilon} z^{(k)}, \quad z^{(k)} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, I_d), \quad (2.2)$$

which, run for enough steps k from an arbitrary $x^{(0)}$, produces an *approximate* sample from p_{data} using *only* the score, never the density itself. This is the precise sense in which “the score suffices for sampling”, and it is why score-based diffusion models train a score network rather than a density network: Section 6.2 shows how to fit that network without ever observing p_{data} in closed form.

Caution 2.18 (“Approximate” means biased, not merely unconverged). The word *approximate* in the previous sentence is doing real work, and it is worth being explicit about what it does and does not mean. For any fixed step size $\epsilon > 0$, the Markov chain (2.2) has a stationary distribution $p^{(\epsilon)}$ that is *not* p_{data} ; the discretisation error is $O(\epsilon)$ and does *not* shrink as $k \rightarrow \infty$. Running more steps removes the error from the initial condition, not the error from discretisation. Removing the latter requires either $\epsilon \rightarrow 0$ (slowly, jointly with $k \rightarrow \infty$) or a Metropolis–Hastings accept/reject correction, which turns ULA into MALA — but that correction needs pointwise evaluation of p_{data} up to a constant, which is exactly what we do not have. The bias is therefore intrinsic to the score-only setting, not an implementation shortcoming. This is one reason diffusion samplers in practice do not run Langevin dynamics at a single noise level, but anneal σ (Chapter 6).

2.5 Kullback–Leibler divergence and maximum likelihood

We record two further standard tools that recur when we discuss what a training objective is actually minimising.

Definition 2.19 (Kullback–Leibler divergence). Let p and q be densities on $\mathbb{R}^d$ with respect to Lebesgue measure. The *Kullback–Leibler divergence from q to p* is

$$\text{KL}(p \| q) = \int_{\{p>0\}} p(x) \log \frac{p(x)}{q(x)} dx = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right] \in [0, +\infty],$$

with the conventions $0 \log \frac{0}{0} = 0$ and $a \log \frac{a}{0} = +\infty$ for $a > 0$. In particular:

- the expectation is taken under $\mathbf{p}$, the *first* argument;
- $\text{KL}(p \| q) = +\infty$ whenever p puts mass where q does not, i.e. whenever p is *not* absolutely continuous with respect to q ;
- KL is always defined (possibly $+\infty$); it is finite only if $q(x) = 0 \Rightarrow p(x) = 0$ for Lebesgue-almost every x , and even then not automatically (the integral can still diverge).

The same formula with $\int \cdots dx$ replaced by $\sum_x$ defines KL for probability mass functions on a countable set.

Caution 2.20 (Absolute continuity is a conclusion, not a hypothesis). It is tempting to *assume* $p \ll q$ so that the integrand is always finite. That would hide the single most consequential feature of KL : it is $+\infty$ exactly when the second argument assigns zero probability to something the first argument considers possible. Concretely, if a model $q = p_\theta$ has support strictly smaller than p_{data} , then $\text{KL}(p_{\text{data}} \| p_\theta) = +\infty$ no matter how good the model looks elsewhere. This infinite penalty is the mechanism behind the mass-covering/mode-seeking dichotomy of Remark 2.26, and it is why models are trained on *smoothed* targets whose support is all of $\mathbb{R}^d$ (Caution 2.11).

Remark 2.21 (KL does not depend on the reference measure — differential entropy does). This point is easy to miss in $\mathbb{R}^d$ and becomes important from Chapter 3 onward, where we work on $SO(3)$ and there is no canonical “ dx ”.

KL is a functional of the two *measures* P, Q , not of the densities used to express them: if $p = dP/d\mu$ and $q = dQ/d\mu$ for *any* common reference measure μ dominating both, the value of $\int p \log(p/q) d\mu$ is the same. Changing μ rescales p and q by the same factor, which cancels inside the ratio p/q . Equivalently, KL is invariant under any diffeomorphic reparametrisation $x \mapsto \phi(x)$ applied to both arguments: the Jacobian factors from Theorem 2.8 cancel.

The quantity $-\mathbb{E}_p[\log p]$ (*differential entropy*) has *no* such invariance: it changes by $\mathbb{E}[\log |\det D\phi|]$ under reparametrisation and can be negative. So “ $\text{KL} = \text{cross-entropy} - \text{entropy}$ ” (Proposition 2.25) is an identity between three coordinate-*dependent* quantities whose combination happens to be coordinate-*independent*. When we later write KL on $SO(3)$ with respect to the Haar measure, the value does not depend on that choice; had we written an entropy, it would have.

Proposition 2.22 (Gibbs’ inequality). $\text{KL}(p \| q) \geq 0$, with equality if and only if $p = q$ Lebesgue-almost everywhere. In general $\text{KL}(p \| q) \neq \text{KL}(q \| p)$, so KL is not a metric (it also fails the triangle inequality).

Proof. If $p \not\ll q$ then $\text{KL}(p \| q) = +\infty > 0$ by Definition 2.19, so assume $p \ll q$. Write $S = \{p > 0\}$ and apply Jensen’s inequality to the strictly convex function $-\log$ under the probability measure $p dx$:

$$\text{KL}(p \| q) = \mathbb{E}_p \left[-\log \frac{q(x)}{p(x)} \right] \geq -\log \mathbb{E}_p \left[\frac{q(x)}{p(x)} \right] = -\log \int_S q(x) dx \geq -\log \int_{\mathbb{R}^d} q(x) dx = 0.$$

The last step is an *inequality*, not an equality: $\int_S q \leq \int q = 1$, with equality iff q vanishes outside S .

Equality in the whole chain forces both steps to be tight. Jensen is tight iff q/p is p -almost-surely equal to a constant c , and then $\int_S q = c \int_S p = c$; the second step is tight iff $\int_S q = 1$, i.e. $c = 1$. Hence $q = p$ on S and $q = 0$ off S , i.e. $p = q$ almost everywhere. Conversely $p = q$ a.e. gives $\text{KL} = 0$. $\square$

Caution 2.23 (A tempting but false shortcut in the proof). One frequently sees the chain above written with $\mathbb{E}_p[q/p] = \int q dx = 1$, i.e. with the second $\geq$ replaced by $=$. That step is valid only if $p > 0$ almost everywhere. In general $\mathbb{E}_p[q/p] = \int_{\{p>0\}} q dx$, which can be strictly less than 1.

Counterexample, satisfying $p \ll q$: let p be uniform on $[0, 1]$ and q uniform on $[0, 2]$. Then $\mathbb{E}_p[q/p] = \frac{1}{2}$, not 1, and the correct Jensen bound is $-\log \frac{1}{2} = \log 2$ — which happens to be the exact value $\text{KL}(p||q) = \log 2$. The false version of the chain would have produced the far weaker bound 0. The stated inequality $\text{KL} \geq 0$ survives, but the intermediate equality does not, and the equality case cannot be read off correctly from the flawed version.

Example 2.24 (The asymmetry, with numbers). For univariate centred Gaussians, $\text{KL}(\mathcal{N}(0, \sigma_1^2) || \mathcal{N}(0, \sigma_2^2)) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2}{2\sigma_2^2} - \frac{1}{2}$. With $\sigma_1 = 1, \sigma_2 = 2$ this is $\log 2 + \frac{1}{8} - \frac{1}{2} \approx 0.318$, whereas the reverse direction is $-\log 2 + 2 - \frac{1}{2} \approx 0.807$. The two differ by more than a factor of two, on the simplest possible pair of distributions.

Proposition 2.25 (KL = cross-entropy – entropy). *Define the differential entropy $H(p) = -\mathbb{E}_{x \sim p}[\log p(x)]$ and the cross-entropy $H(p, q) = -\mathbb{E}_{x \sim p}[\log q(x)]$. Whenever $H(p)$ is finite,*

$$\text{KL}(p || q) = H(p, q) - H(p).$$

Proof. Split the logarithm: $\mathbb{E}_p[\log(p/q)] = \mathbb{E}_p[\log p] - \mathbb{E}_p[\log q]$, which is $-H(p) + H(p, q)$. The split is legitimate only when the two terms are not both infinite of the same sign, which the finiteness of $H(p)$ guarantees. $\square$

Two consequences are used later. First, for a *fixed* p , minimising $\text{KL}(p||q)$ over q is the same optimisation as minimising the cross-entropy $H(p, q)$, because $H(p)$ is a constant — this is exactly the step taken in Proposition 2.28. Second, $H(p) \geq 0$ fails for continuous p : differential entropy can be negative (a Gaussian with $\sigma < 1/\sqrt{2\pi e}$ has $H < 0$), so no sign conclusion about KL may be drawn from this decomposition. The non-negativity of KL comes from Proposition 2.22, not from here.

Remark 2.26 (Forward versus reverse KL: which argument is which matters). Fix a data distribution p and a model family $\{q_\theta\}$. The two directions penalise different failures, and the difference follows directly from which distribution the expectation is taken under.

Forward $\text{KL}(p||q_\theta) = \mathbb{E}_p[\log(p/q_\theta)]$ averages under the *data*. Every region where p has mass is sampled, so q_θ is punished — infinitely, by Caution 2.20 — for assigning near-zero density anywhere p is positive. Regions where q_θ is large but p is nearly zero contribute almost nothing, because they are almost never sampled. Forward KL is therefore *mass-covering*: the optimum spreads q_θ over all modes of p , accepting density in between.

Reverse $\text{KL}(q_\theta||p) = \mathbb{E}_{q_\theta}[\log(q_\theta/p)]$ averages under the *model*. Regions the model avoids cost nothing regardless of p there, but the model is punished infinitely for putting mass where p has none. Reverse KL is therefore *mode-seeking*: it prefers to concentrate on one mode and represent it well.

Maximum likelihood — and hence everything in Chapters 6 and 11 that is justified by Proposition 2.28 — optimises the *forward* direction. Variational inference in the usual ELBO formulation optimises the *reverse* direction. This document uses only the forward direction; the distinction is recorded here because the reverse direction is what most readers will have met first, under the same name.

Definition 2.27 (Maximum likelihood). Given i.i.d. samples $x_1, \dots, x_n \sim p_{\text{data}}$ and a parametric family $\{p_\theta\}$, the *maximum likelihood estimator* is $\hat{\theta} = \arg \max_\theta \sum_{i=1}^n \log p_\theta(x_i)$.

Proposition 2.28 (MLE minimises forward KL to the data distribution). *Assume the differential entropy $H(p_{\text{data}})$ is finite, and that $\mathbb{E}_{p_{\text{data}}} |\log p_\theta(x)| < \infty$ for every θ . Then:*

- (i) *for each fixed θ , $\frac{1}{n} \sum_i \log p_\theta(x_i) \rightarrow \mathbb{E}_{x \sim p_{\text{data}}} [\log p_\theta(x)]$ almost surely as $n \rightarrow \infty$ (strong law of large numbers);*
- (ii) *as sets,*

$$\arg \max_\theta \mathbb{E}_{p_{\text{data}}} [\log p_\theta(x)] = \arg \min_\theta \text{KL}(p_{\text{data}} \| p_\theta).$$

Proof. (i) is the strong law applied to the i.i.d. variables $\log p_\theta(x_i)$, which are integrable by assumption. For (ii), by Proposition 2.25, $\text{KL}(p_{\text{data}} \| p_\theta) = -H(p_{\text{data}}) - \mathbb{E}_{p_{\text{data}}} [\log p_\theta(x)]$, and the first term is a finite constant independent of θ ; adding a constant does not change the set of minimisers, and minimising the negative of a function is maximising the function. $\square$

Caution 2.29 (What Proposition 2.28 does *not* say). Part (i) is pointwise in θ ; it does *not* by itself imply that $\hat{\theta}_n \rightarrow \arg \min_\theta \text{KL}(p_{\text{data}} \| p_\theta)$. Consistency of the estimator additionally requires uniform convergence over θ (e.g. a Glivenko–Cantelli condition) plus identifiability. We never rely on consistency in this document — only on the statement that the two *population* objectives in (ii) define the same optimisation — so we do not develop those conditions.

Note also the direction: it is the *forward* KL, with p_{data} in the first argument, that maximum likelihood minimises (Remark 2.26). Finally, if the model family cannot cover the support of p_{data} , every member has infinite KL and (ii) is vacuous even though the likelihood objective still has a well-defined maximiser; this is not a pathology of the proposition but a genuine degeneracy of the setting, and it is one more reason the constructions of Chapters 6 and 11 work with everywhere-positive smoothed distributions.

Proposition 2.28 is why “fit the model by maximum likelihood” and “fit the model to minimise (forward) KL divergence to the data” are the same instruction, a fact we use once, in Section 13.9, when we discuss to what extent a flow-matching model’s ODE admits an exact likelihood and what that would be useful for.

2.6 Generative modelling as distribution transport

We can now state, precisely, the problem that both diffusion models and flow matching solve, and why casting protein–ligand docking as an instance of it is the right move.

The generative modelling problem. Given i.i.d. samples $x_1, \dots, x_n \sim p_{\text{data}}$, produce a mechanism that generates new samples $\hat{x} \sim p_{\text{data}}$ (or a good approximation thereof), *without* an explicit closed-form, evaluable expression for p_{data} ever being available.

The docking instance of this problem, following Chapter 1, sets p_{data} to be the (conditional, given the protein pocket and ligand graph) distribution of experimentally observed bound poses — a distribution about which we have samples (crystal structures) but no formula.

Rejection sampling and classical Markov chain Monte Carlo both require pointwise evaluation of p_{data} up to a known constant, which we do not have; Section 2.4 showed a way around this for MCMC-style methods (only the score is needed), but that still leaves the problem of *learning* the score from samples alone (Section 6.2). The alternative pursued by both halves of this document is:

1. Fix a *tractable* distribution p_{init} (called the *source* or *prior*) that we can sample from directly — for docking, e.g. an isotropic Gaussian for translations and the uniform (Haar) distribution for rotations (both defined precisely in Chapter 3).
2. Learn a *transport mechanism* — a stochastic process (Chapter 6) or a deterministic flow (Chapter 11) — that turns a sample from p_{init} into a sample from p_{data} .

The entire technical content of Chapters 6 and 11 is two different, and ultimately closely related (Section 6.4), answers to step 2. Both answers share one structural device, which is worth isolating before either is developed in full: both construct a *probability path*, a time-indexed family of distributions interpolating between the two endpoints.

Definition 2.30 (Probability path). A *probability path* is a family of densities $(p_t)_{t \in [0,1]}$ on $\mathbb{R}^d$ (or, later, on a manifold $\mathcal{M}$) satisfying the global convention of Remark 0.2: $p_0 = p_{\text{init}}$ and $p_1 = p_{\text{data}}$.

Diffusion models (Chapter 6) construct (p_t) implicitly, as the law of a stochastic process at time t , and characterise it via its score, $\nabla_x \log p_t(x)$; sampling then simulates a stochastic differential equation. Flow matching (Chapter 11) constructs (p_t) explicitly, via a chosen interpolation between a source point and a data point, and characterises it via a vector field u_t satisfying the continuity equation of Theorem 11.10; sampling then integrates a deterministic ordinary differential equation. Section 6.4 proves that these are not two disjoint frameworks but two different parameterisations of exactly the same underlying object, a fact that will let us describe the conversion from SigmaDock to SigmaFlow (Chapter 13) as a single, controlled substitution rather than a change of model class.

2.7 What you should now be able to do

- state what a density is *with respect to a reference measure*, and why that qualification matters on a manifold (Remark 3.31);
- use the marginalisation identity Proposition 2.3 — the single mechanism by which every intractable marginal object in this document is traded for a tractable conditional one;
- manipulate conditional expectations with the tower property and the pass-through rule (Proposition 2.5);
- explain what the score $\nabla_x \log p$ is geometrically, and why knowing it suffices to sample (Langevin dynamics);
- say why KL divergence is reference-measure independent while differential entropy is not.

Everything from here on is one of two answers to the transport problem of Section 2.6: learn a score, or learn a velocity.

Part II

Geometry

Part II at a glance

What you will learn. Two things a pose lives in that $\mathbb{R}^d$ is not. First, the geometry of rotations: manifolds, tangent spaces, metrics, geodesics, $\exp/\log$, and the group $SO(3)$ concretely. Second, the representation theory of $SO(3)$ — spherical harmonics, Wigner- D matrices, Clebsch–Gordan products — which is what makes a *network* able to respect that geometry.

Why it is needed. A rotation cannot be added, averaged, or perturbed by a Gaussian in the naïve way (Section 3.1.1), so neither diffusion nor flow matching can be written down on poses until the replacements for $+$ and $-$ exist. And a network that ignores rotational symmetry has to learn it from data, which Section 4.1.4 shows is a bad trade.

What depends on it. Chapter 7 (Brownian motion on $SO(3)$), Chapters 8 and 9 (the architecture), and Chapter 12 (the geometric half of flow matching) each consume one half of this part.

Chapter 3

Manifolds, Groups, and Rigid Motion

Chapter 1 established that the natural coordinates of a ligand pose are not $3n$ free Cartesian numbers but a translation, a rotation, and (if fragments are used) a handful of torsion angles. None of translations, rotations, and torsion angles are elements of a vector space in the way that is needed for the calculus of Chapter 2 to apply directly: a rotation is not closed under addition (the sum of two rotation matrices is generally not a rotation matrix), and an angle is only defined modulo 2π . This chapter develops exactly the amount of differential geometry needed to do calculus, probability, and eventually diffusion and flow matching correctly on such spaces.

3.1 Manifolds and tangent spaces

Motivation. Ordinary multivariable calculus is calculus on $\mathbb{R}^n$: derivatives are linear maps $\mathbb{R}^n \rightarrow \mathbb{R}^m$, gradients are vectors in $\mathbb{R}^n$, and one can always add two points or scale a point by a real number. None of this is true, globally, on the set of rotation matrices or on a circle of angles. What *is* true is that these sets look like $\mathbb{R}^n$ *locally* — close enough together, two rotations differ by “a small rotation”, and this small difference behaves, to first order, exactly like a vector. Formalising “looks like $\mathbb{R}^n$ locally, glued together globally” is the definition of a manifold.

Intuition. Picture the surface of a sphere $S^2 \subset \mathbb{R}^3$. Around any point, a small enough patch of the sphere looks flat — this is why maps of small regions of the Earth can be drawn on flat paper with negligible distortion, even though the Earth as a whole cannot. The “flat approximation at a point” is the tangent *plane* at that point; the entire curved sphere is not itself a vector space (you cannot add two points on a sphere and expect the sum to be on the sphere), but each tangent plane is.

Definition 3.1 (Smooth manifold, informal). An n -dimensional *smooth manifold* $\mathcal{M}$ is a set that can be covered by patches (*charts*), each in smooth, invertible correspondence with an open subset of $\mathbb{R}^n$, such that on overlaps the two competing correspondences are related by a smooth map. (We will only ever need two concrete examples, $SO(3)$ and $\mathbb{T} = \mathbb{R}/2\pi\mathbb{Z}$, both realised concretely below as subsets of a Euclidean space; a fully general, chart-based treatment is standard differential geometry and is not needed beyond this informal level for anything in this document.)

Definition 3.2 (Tangent space, informal). For $x \in \mathcal{M}$, the *tangent space* $T_x\mathcal{M}$ is the vector space of all velocity vectors $\dot{\gamma}(0)$ of smooth curves $\gamma : (-\varepsilon, \varepsilon) \rightarrow \mathcal{M}$ with $\gamma(0) = x$. Concretely, if $\mathcal{M} \subset \mathbb{R}^N$ is realised as a level set (as both $SO(3)$ and $\mathbb{T}$ will be), $T_x\mathcal{M}$ is the set of directions in $\mathbb{R}^N$ tangent to $\mathcal{M}$ at x , and it *is* a genuine linear subspace of $\mathbb{R}^N$: sums and scalar multiples of tangent vectors are again tangent vectors, even though sums of points on $\mathcal{M}$ are generally not points on $\mathcal{M}$.

Example 3.3. Let $\mathcal{M} = S^1 = \{x \in \mathbb{R}^2 : \|x\| = 1\}$, the unit circle, and $x = (1, 0)$. A curve $\gamma(\tau) = (\cos \tau, \sin \tau)$ has $\gamma(0) = x$ and $\dot{\gamma}(0) = (0, 1)$. Every tangent curve at x has velocity a multiple of $(0, 1)$ (the direction perpendicular to x), so $T_x S^1 = \{(0, c) : c \in \mathbb{R}\} \cong \mathbb{R}$: a one-dimensional vector space, matching $\dim S^1 = 1$, even though S^1 itself is not a vector space (e.g. $(1, 0) + (1, 0) = (2, 0) \notin S^1$).

3.1.1 Why a tangent space at all, and why one per point

Before the formal machinery accumulates, it is worth being explicit about the problem being solved, because the answer determines the shape of everything that follows.

The problem. In $\mathbb{R}^n$ a single vector space does two jobs at once: it holds the *points* we care about, and it holds the *directions* we move in. That coincidence is what makes $x + v$ meaningful. On a curved space the two jobs come apart. A point of S^2 is a unit vector in $\mathbb{R}^3$; a “direction to move” at that point is also a vector in $\mathbb{R}^3$, but it is a *different kind* of object, and adding it to the point takes you off the sphere.

Why not just use one global set of directions? On S^2 , “north” is a perfectly good direction at the equator, but it is meaningless at the north pole. There is no way to choose, continuously and non-vanishingly, one direction at every point of the sphere — this is the hairy-ball theorem. So a single global space of directions does not exist; we are forced to attach a *separate* space of directions to each point. That family of spaces is the tangent bundle, and each member is a tangent space.

What a tangent vector is. The cleanest mental model, and the one Definition 3.2 formalises, is: a tangent vector at x is the *initial velocity of a curve through x* . This is worth dwelling on, because it explains why tangent vectors can be added even though points cannot. Two curves through x have two velocities; the sum of those velocities is again realisable as the velocity of some third curve through x . Nothing about this construction requires the *points* to be addable. Velocities live in a linear space; positions do not.

The same idea, three times.

- $\mathbb{R}^n$. $T_x\mathbb{R}^n \cong \mathbb{R}^n$ for every x , and all these copies are canonically the same space. This is exactly the degenerate case in which the distinction between points and directions is invisible — which is why Euclidean intuition transfers so badly.
- $S^2 \subset \mathbb{R}^3$. $T_x S^2 = \{v \in \mathbb{R}^3 : v \cdot x = 0\}$, the plane through the origin perpendicular to x . Two dimensions, matching $\dim S^2 = 2$. Different x give genuinely different planes: there is no canonical way to compare a tangent vector at the north pole with one at the equator.
- $SO(3)$. A point is a rotation matrix R ; a tangent vector at R turns out to be a matrix of the form $R\hat{\omega}$ with $\omega \in \mathbb{R}^3$ — three dimensions, matching $\dim SO(3) = 3$. Section 3.4 derives this. The extra structure that $SO(3)$ has and S^2 lacks — a group operation —

will let us identify all these tangent spaces with one another, which is the content of Definition 3.44.

Why this matters for SigmaFlow

A ligand pose contains, per rigid fragment, a rotation $R \in SO(3)$. The network must output “how this fragment should turn” — a direction of motion, not a new rotation. That direction is a tangent vector at R , and it is a 3-vector, while the point R is a 3×3 matrix with six constraints. Confusing the two is not a stylistic error: adding a 3×3 update matrix to R produces something that is not a rotation, and the pose it encodes is then not a rigid motion. Every appearance of `exp` and `log` in `so3_flow_matcher.py` exists to keep points and directions in their respective places.

3.1.2 Vector fields, and why a tangent field keeps you on the manifold

Before measuring anything, one more piece of vocabulary. A tangent space holds directions *at one point*. What a generative model needs is a rule assigning a direction to *every* point at once — and, crucially, assigning to each point a direction that is legal *there*.

Definition 3.4 (Tangent bundle). The *tangent bundle* of $\mathcal{M}$ is the disjoint union of all its tangent spaces,

$$T\mathcal{M} := \bigsqcup_{x \in \mathcal{M}} T_x\mathcal{M} = \{(x, v) : x \in \mathcal{M}, v \in T_x\mathcal{M}\}.$$

An element of $T\mathcal{M}$ is a *pair*: a point together with a direction at that point. The map $(x, v) \mapsto x$ is called the projection.

The pairing is the whole point. A tangent vector carries its base point with it, because “the vector $(0, 1)$ ” is meaningless on S^2 until one says *where* it is attached: it is tangent at the north pole and not tangent at the equator. In $\mathbb{R}^n$ the bundle is a product, $T\mathbb{R}^n = \mathbb{R}^n \times \mathbb{R}^n$, and the base point can be dropped without harm — which is exactly the degenerate coincidence Section 3.1.1 warned about.

Definition 3.5 (Vector field on a manifold). A *vector field* on $\mathcal{M}$ is a map u assigning to each $x \in \mathcal{M}$ a tangent vector $u(x) \in T_x\mathcal{M}$ — i.e. a map $u : \mathcal{M} \rightarrow T\mathcal{M}$ with $u(x)$ in the fibre over x (a *section* of the tangent bundle). A *time-dependent* vector field is a family u_t , $t \in [0, 1]$, of such maps.

Caution 3.6 (The constraint is the content). Read Definition 3.5 carefully: it is not “a map $\mathcal{M} \rightarrow \mathbb{R}^N$ ”. The requirement $u(x) \in T_x\mathcal{M}$ — and *not* merely $u(x) \in \mathbb{R}^N$ — is the entire difference between a vector field on a manifold and an arbitrary collection of arrows.

Concretely on $S^2 \subset \mathbb{R}^3$, where $T_x S^2 = \{v \in \mathbb{R}^3 : x^\top v = 0\}$: the assignment $u(x) = x$ (the outward radial field) is a perfectly good smooth map $S^2 \rightarrow \mathbb{R}^3$, but it is *not* a vector field on S^2 , because $x^\top x = 1 \neq 0$. Following it for any positive time leaves the sphere immediately. The assignment $u(x) = e_3 - (e_3^\top x)x$, by contrast, satisfies $x^\top u(x) = x^\top e_3 - (e_3^\top x)\|x\|^2 = 0$ and *is* a legal vector field: it is the “flow towards the north pole along meridians” field.

This is the constraint a Riemannian flow matching network has to satisfy at its output, and it is why Chapter 12 cannot simply reuse a network that emits an unconstrained element of $\mathbb{R}^N$.

Proposition 3.7 (A tangent field never leaves the manifold). *Let $\mathcal{M} \subset \mathbb{R}^N$ be a manifold given as a level set $\mathcal{M} = \{y \in \mathbb{R}^N : F(y) = 0\}$ for a smooth $F : \mathbb{R}^N \rightarrow \mathbb{R}^m$ whose differential has constant rank on $\mathcal{M}$, and let u_t be a time-dependent vector field on $\mathcal{M}$ in the sense of Definition 3.5, extended smoothly to a neighbourhood. Then the solution of $\dot{X}_t = u_t(X_t)$ with $X_0 \in \mathcal{M}$ satisfies $X_t \in \mathcal{M}$ for all t for which it exists.*

Proof. Consider the scalar-valued (in general $\mathbb{R}^m$ -valued) function $h(t) := F(X_t)$. By the chain rule,

$$\dot{h}(t) = DF(X_t) \dot{X}_t = DF(X_t) u_t(X_t).$$

Now, the tangent space of a level set at a point of it is exactly the kernel of the differential, $T_y \mathcal{M} = \ker DF(y)$ (this is the standard characterisation: a curve stays in the level set iff differentiating $F(\gamma(\tau)) = 0$ gives $DF \dot{\gamma} = 0$). Since $u_t(X_t) \in T_{X_t} \mathcal{M}$ by hypothesis, $DF(X_t) u_t(X_t) = 0$, so $\dot{h} \equiv 0$ and h is constant. As $h(0) = F(X_0) = 0$, we get $F(X_t) = 0$, i.e. $X_t \in \mathcal{M}$, for all t . $\square$

Proposition 3.7 is the reason a Riemannian ODE sampler needs no projection or renormalisation step *in exact arithmetic*: the constraint is preserved automatically, by the tangency of the field, not by any correction applied afterwards. (In floating point, and after discretisation with a finite step, it is preserved only approximately — which is precisely why $SO(3)$ integrators use $\exp$, whose output is *exactly* a rotation matrix for any step size, rather than a linear Euler step in the ambient $\mathbb{R}^{3 \times 3}$; Proposition 3.63 and Section 12.10 return to this.)

3.1.3 Measuring: metrics, lengths, and distance

Motivation, before any formula. So far $\mathcal{M}$ has directions but no ruler. That is genuinely not enough, and it is worth listing what is unavailable without one:

- **How long is a tangent vector?** Without an answer there is no notion of speed, hence no notion of “constant-speed” curve.
- **What angle do two tangent vectors make?** Without an answer there is no orthogonality, hence no notion of a vector being “perpendicular to the manifold”.
- **Which of two paths from x to y is shorter?** Without an answer there is no shortest path, hence no geodesic, hence no $\exp/\log$, hence no geodesic interpolant — and hence no Riemannian flow matching.
- **What does $\|u_\theta(x, t) - u_t(x)\|^2$ mean?** A squared error between two tangent vectors is a length, and lengths are exactly what is missing.

All four are supplied by one object: an inner product on each tangent space. Not on the manifold — on each tangent space separately, since that is where the vectors live.

Definition 3.8 (Riemannian metric). A *Riemannian metric* on $\mathcal{M}$ is a smoothly varying family of inner products $g_x : T_x \mathcal{M} \times T_x \mathcal{M} \rightarrow \mathbb{R}$ ($x \in \mathcal{M}$), inducing the norm $\|v\|_g = \sqrt{g_x(v, v)}$. The length of a smooth curve $\gamma : [0, 1] \rightarrow \mathcal{M}$ is $L(\gamma) = \int_0^1 \|\dot{\gamma}(\tau)\|_g d\tau$.

“Smoothly varying” means the coefficients of g_x depend smoothly on x ; “inner product” means, as usual, bilinear, symmetric, and positive definite ($g_x(v, v) > 0$ for $v \neq 0$). Positive definiteness is what makes $\|\cdot\|_g$ a norm and is used constantly without comment.

Notation 3.9 (Two norms, two subscripts). This document writes $\|v\|_g$, or $\|v\|_x$ when the base point needs emphasis, for the Riemannian norm of $v \in T_x\mathcal{M}$, and $\|\cdot\|_2$ for the ordinary Euclidean norm of a column of numbers. On $SO(3)$ these two happen to coincide under the identification $\mathfrak{so}(3) \cong \mathbb{R}^3$, by the deliberate normalisation of Proposition 3.58 — a convenience, not a general fact.

Where this is heading

The norm of Definition 3.8 is what will make the Riemannian flow matching loss well posed. In $\mathbb{R}^d$ the training objective was $\mathbb{E}\|u_\theta(x, t) - u_t(x)\|_2^2$ (Theorem 11.25). On a manifold both vectors lie in $T_x\mathcal{M}$, and the only sensible way to square their difference is with the inner product that lives there:

$$\mathbb{E}\|u_\theta(x, t) - u_t(x)\|_{g_x}^2.$$

Every ingredient of that expression — the tangency of both arguments, the subtraction inside one tangent space, the point-dependent norm — has been set up in this section and the last. Theorem 12.18 is where it is used.

From a metric on vectors to a distance between points. The metric measures tangent vectors. A distance between *points* is derived from it, not postulated:

$$d_g(x, y) := \inf\{L(\gamma) : \gamma \text{ a smooth curve with } \gamma(0) = x, \gamma(1) = y\}. \quad (3.1)$$

This is worth stating explicitly because the word “metric” is doing double duty in the literature: *Riemannian metric* g (an inner product on each tangent space) and *metric* d_g (a distance function on the manifold, in the sense of metric spaces) are different objects, and g determines d_g but not conversely in any elementary way.

Example 3.10 (Length on the sphere). Take $\mathcal{M} = S^2 \subset \mathbb{R}^3$ with the metric induced from $\mathbb{R}^3$ (i.e. $g_x(v, w) = v^\top w$ for $v, w \in T_x S^2$ — restrict the ambient dot product to the tangent plane). Let $\gamma(\tau) = (\cos \tau, \sin \tau, 0)$, the equator, for $\tau \in [0, \alpha]$. Then $\dot{\gamma}(\tau) = (-\sin \tau, \cos \tau, 0)$ has $\|\dot{\gamma}\|_g = 1$, so $L(\gamma) = \int_0^\alpha 1 d\tau = \alpha$: the length of an arc equals the angle it subtends, as it must for a unit sphere. And $d_g(x, y)$ for two points at angle $\alpha \leq \pi$ is exactly α — the great-circle distance.

3.1.4 Acceleration on a curved space: the covariant derivative

Defining a geodesic requires saying that a curve does not accelerate. That turns out to be subtler than it sounds, and since the phrase $\nabla_{\dot{\gamma}}\dot{\gamma} = 0$ is about to appear, we develop exactly enough of its meaning to read it — and no more. A complete theory of connections is standard differential geometry and is not needed here.

The problem. To differentiate a velocity we would like to write

$$\lim_{h \rightarrow 0} \frac{\dot{\gamma}(\tau + h) - \dot{\gamma}(\tau)}{h}.$$

But $\dot{\gamma}(\tau + h) \in T_{\gamma(\tau+h)}\mathcal{M}$ and $\dot{\gamma}(\tau) \in T_{\gamma(\tau)}\mathcal{M}$ are vectors in *different* tangent spaces (Section 3.1.1), and there is no canonical way to subtract them. Differentiating a vector field along a curve is therefore not automatically defined on a manifold; it requires extra structure, and that structure is a *connection*, written ∇ .

The resolution, for an embedded manifold. When $\mathcal{M} \subset \mathbb{R}^N$ carries the metric induced from the ambient space — which is the case for every manifold in this document — there is a completely concrete answer. Both vectors *are* elements of $\mathbb{R}^N$, so the ambient difference quotient makes sense; the result simply need not be tangent. So take it and project.

Definition 3.11 (Covariant derivative along a curve, for an embedded manifold). Let $\mathcal{M} \subset \mathbb{R}^N$ with the induced metric, let $\gamma : [0, 1] \rightarrow \mathcal{M}$ be a smooth curve, and let $V(\tau) \in T_{\gamma(\tau)}\mathcal{M}$ be a smooth vector field along it. Write $P_x : \mathbb{R}^N \rightarrow T_x\mathcal{M}$ for the orthogonal projection onto the tangent space at x . The *covariant derivative* of V along γ is

$$\frac{DV}{d\tau} := P_{\gamma(\tau)}\left(\frac{dV}{d\tau}\right),$$

the tangential part of the ordinary (ambient) derivative. We write $\nabla_{\dot{\gamma}}V$ for the same object.

That this coincides with the Levi-Civita connection of the induced metric — the unique connection that is compatible with the metric and torsion-free — is a standard theorem which we use but do not prove (see, e.g., do Carmo, *Riemannian Geometry*, 1992, Ch. 2, or Lee, *Introduction to Riemannian Manifolds*, 2nd ed., 2018, Ch. 5). What matters here is the reading:

The ambient acceleration of a curve on $\mathcal{M}$ splits into a part that turns within the manifold and a part that is spent merely on staying on it. The covariant derivative keeps the first and discards the second.

3.1.5 Geodesics

Definition 3.12 (Geodesic). A *geodesic* is a curve with zero covariant acceleration,

$$\nabla_{\dot{\gamma}}\dot{\gamma} = 0,$$

i.e. (by Definition 3.11) a curve whose ambient acceleration $\ddot{\gamma}$ is everywhere *orthogonal* to the manifold: all of it is used up staying on $\mathcal{M}$, none of it turns the curve within $\mathcal{M}$. Geodesics are the manifold’s straight lines.

Proposition 3.13 (Geodesics have constant speed). *If γ is a geodesic then $\|\dot{\gamma}(\tau)\|_g$ does not depend on τ .*

Proof. Differentiate the squared speed, using that the ambient metric is the constant Euclidean one so ordinary differentiation applies:

$$\frac{d}{d\tau}\|\dot{\gamma}\|_g^2 = \frac{d}{d\tau}\langle\dot{\gamma}, \dot{\gamma}\rangle = 2\langle\ddot{\gamma}, \dot{\gamma}\rangle = 0,$$

because $\ddot{\gamma}$ is orthogonal to $T_{\gamma}\mathcal{M}$ (Definition 3.12) while $\dot{\gamma}$ lies in it. □

Constant speed is thus a *consequence* of the geodesic equation, not an extra assumption — a small point that is easy to get backwards, since many texts introduce geodesics as “constant-speed locally shortest curves”.

Example 3.14 (Great circles on S^2). Let $a, b \in \mathbb{R}^3$ be orthonormal and set $\gamma(\tau) = \cos(\tau)a + \sin(\tau)b$, a unit-speed traversal of the great circle through a in direction b . Then $\ddot{\gamma}(\tau) = -\cos(\tau)a - \sin(\tau)b = -\gamma(\tau)$, which is parallel to the position vector, i.e. *normal* to $T_{\gamma(\tau)}S^2 = \gamma(\tau)^\perp$. So $\nabla_{\dot{\gamma}}\dot{\gamma} = 0$ and great circles are geodesics — the acceleration is entirely centripetal, purely the cost of not flying off the sphere.

Proposition 3.15 (One-parameter subgroups are the geodesics of $SO(3)$). *Equip $SO(3) \subset \mathbb{R}^{3 \times 3}$ with the metric induced from the ambient inner product $\langle A, B \rangle := \frac{1}{2} \text{tr}(A^\top B)$. Then for any $R_0 \in SO(3)$ and $\Xi \in \mathfrak{so}(3)$, the curve $R(\tau) = R_0 \exp(\tau \Xi)$ is a geodesic.*

Proof. First, the induced metric is the bi-invariant one of Proposition 3.51: for $R\Omega_1, R\Omega_2 \in T_R SO(3)$,

$$\langle R\Omega_1, R\Omega_2 \rangle = \frac{1}{2} \text{tr}(\Omega_1^\top R^\top R \Omega_2) = \frac{1}{2} \text{tr}(\Omega_1^\top \Omega_2) = g_I(\Omega_1, \Omega_2),$$

using $R^\top R = I$; the same computation with $\Omega_i R$ shows right invariance. So the ambient restriction *is* the metric this document uses, and Definition 3.11 applies.

Now differentiate twice. As in Proposition 3.62, $\dot{R} = R\Xi$, and differentiating once more, $\ddot{R} = \dot{R}\Xi = R\Xi^2$. We must show $R\Xi^2$ is orthogonal to $T_R SO(3) = \{R\Omega : \Omega \in \mathfrak{so}(3)\}$. For any $\Omega \in \mathfrak{so}(3)$,

$$\langle R\Xi^2, R\Omega \rangle = \frac{1}{2} \text{tr}((\Xi^2)^\top R^\top R \Omega) = \frac{1}{2} \text{tr}((\Xi^2)^\top \Omega) = \frac{1}{2} \text{tr}(\Xi^2 \Omega),$$

where the last step uses $(\Xi^2)^\top = (\Xi^\top)^2 = (-\Xi)^2 = \Xi^2$, i.e. Ξ^2 is *symmetric*. Finally, the trace of a symmetric matrix times a skew-symmetric matrix vanishes: with $S = S^\top$ and $\Omega^\top = -\Omega$,

$$\text{tr}(S\Omega) = \text{tr}((S\Omega)^\top) = \text{tr}(\Omega^\top S^\top) = -\text{tr}(\Omega S) = -\text{tr}(S\Omega) \implies \text{tr}(S\Omega) = 0,$$

using cyclicity of the trace. Hence $\ddot{R} \perp T_R SO(3)$ and $R(\tau)$ is a geodesic. $\square$

Proposition 3.15 supplies, by elementary matrix algebra, the fact that Proposition 3.51 previously imported as a standard result about bi-invariant metrics. It is worth having in full, because every $SO(3)$ construction in Chapters 7 and 12 rests on it.

Caution 3.16 (Geodesics are only *locally* shortest). “Geodesic” and “shortest path” are not synonyms, and the difference is not academic for us. Definition 3.12 is a purely *local* condition — it constrains the curve’s acceleration at each instant and says nothing about competing routes elsewhere.

On S^2 , the great circle through two points is a geodesic in *both* directions around the sphere, but only the shorter arc is length-minimising: the long way round satisfies $\nabla_{\dot{\gamma}} \dot{\gamma} = 0$ at every point and is nonetheless not the shortest path. On $SO(3)$ the same phenomenon occurs, with the transition at a rotation angle of π : continuing a geodesic past a half-turn keeps zero acceleration but stops being the shortest route.

This is exactly why $\log$ on $SO(3)$ is conventionally taken to return the rotation vector of *smallest* angle, $\|\omega\| \leq \pi$ (Theorem 3.52), and it is the geometric reason behind the numerical fragility near π that Chapter 16 documents: at exactly π there are two shortest geodesics, and the choice between them is discontinuous. Section 3.1.8 makes the general statement.

3.1.6 Replacing $x + v$ and $y - x$: the exponential and logarithmic maps

We now have directions and lengths. What we still lack is the single most used operation in all of Euclidean numerics: *take a point, take a direction, move*. In $\mathbb{R}^n$ this is $x \mapsto x + v$. On a manifold there is no “+”. The replacement is forced by the following observation: in $\mathbb{R}^n$, $x + tv$ is precisely the constant-speed straight line through x with initial velocity v — that is, the geodesic. So the right generalisation of “move from x in direction v ” is “follow the geodesic through x with initial velocity v , for unit time”.

Definition 3.17 (Exponential and logarithmic map). For $x \in \mathcal{M}$ and $v \in T_x\mathcal{M}$, let γ_v be the unique geodesic with $\gamma_v(0) = x$ and $\dot{\gamma}_v(0) = v$. The *exponential map* is

$$\exp_x : T_x\mathcal{M} \rightarrow \mathcal{M}, \quad \exp_x(v) := \gamma_v(1).$$

On a neighbourhood $U \ni x$ small enough that $\exp_x$ is a diffeomorphism onto its image, its inverse is the *logarithmic map* $\log_x : U \rightarrow T_x\mathcal{M}$. The largest radius for which this holds is the *injectivity radius* at x .

Caution 3.18 ($\exp_x$ and $\log_x$ are not globally as well behaved as $+$ and $-$). Three differences from the Euclidean case, each of which shows up later:

- (a) $\exp_x$ need not be injective. On $SO(3)$, rotating by θ and by $\theta + 2\pi$ about the same axis give the same element, so infinitely many tangent vectors map to the same point.
- (b) $\log_x(y)$ is therefore not unique in general; one *chooses* a branch, conventionally the shortest. The injectivity radius of $SO(3)$ with the standard bi-invariant metric is π , so this choice is unambiguous except exactly at antipodal rotations.
- (c) $\exp_x$ need not be surjective on a general manifold (it is on $SO(3)$, which is compact and connected).

Writing $\exp_x(v)$ “for $x + v$ ” is therefore a good mnemonic and a bad theorem. Whenever a statement in this document depends on invertibility, the hypothesis is stated.

Example 3.19 (Everything at once, on the circle). Take $\mathcal{M} = S^1$, $x = (1, 0)$, and identify $T_x S^1 \cong \mathbb{R}$ via $c \mapsto (0, c)$ as above. The geodesics are constant-speed traversals of the circle, so $\exp_x(c) = (\cos c, \sin c)$. Then: $\exp_x$ is surjective but not injective (c and $c + 2\pi$ agree); $\log_x(y)$ is the signed angle from x to y , unique if we demand $|\log_x(y)| < \pi$; and at $y = -x$ the two choices $\pm\pi$ are equally short — the injectivity radius is π , and the antipode is where the convention has to break a tie. Every one of these features recurs verbatim on $SO(3)$, with “angle” replaced by “rotation angle”.

The pair $(\exp_x, \log_x)$ is, throughout this document, the mechanism that replaces ordinary vector addition and subtraction: $\exp_x(v)$ plays the role of $x + v$, and $\log_x(y)$ plays the role of $y - x$, valid even though $\mathcal{M}$ has no addition of its own. Diffusion models (Chapter 6) will use $\exp_x$ to push a randomly sampled tangent-space perturbation onto the manifold; flow matching (Chapter 11) will use $\log_x$ to construct a geodesic interpolant between two points and $\exp_x$ to integrate the learned velocity field one step at a time.

Why this matters for SigmaFlow

The correspondence is exact and worth memorising, because it is what the sampler literally executes:

Euclidean translation channel	Rotational channel
$x_{t+dt} = x_t + dt v$	$R_{t+dt} = \exp_{R_t}(dt v) = R_t \exp(dt \hat{v})$
target $u = x_1 - x_0$	target $u = \log_{R_0}(R_1) = \log(R_0^\top R_1)^\vee$
interpolant $(1 - t)x_0 + tx_1$	interpolant $R_0 \exp(t \log(R_0^\top R_1))$

The right-hand column is `so3_flow_matcher.py`; the left-hand column is `r3_flow_matcher.py`. Chapter 13 verifies each line against the source.

3.1.7 The scaling property, and $\exp$ evaluated part-way

One property of $\exp_x$ is used so often, and so silently, that it deserves to be isolated: *scaling the tangent vector is the same as stopping the geodesic early*. It is what makes $\exp_{x_0}(t \log_{x_0}(x_1))$ a sensible definition of “the point t of the way from x_0 to x_1 ”, and it rests on a small lemma that will be needed again in Chapter 12.

Lemma 3.20 (Affine reparameterisation of a geodesic is a geodesic). *If γ is a geodesic and $\sigma(s) := \gamma(as + b)$ for constants $a, b \in \mathbb{R}$, then σ is a geodesic, with $\dot{\sigma}(s) = a \dot{\gamma}(as + b)$.*

Proof. By the chain rule $\dot{\sigma}(s) = a \dot{\gamma}(as + b)$ and $\ddot{\sigma}(s) = a^2 \ddot{\gamma}(as + b)$. Since $\ddot{\gamma}$ is orthogonal to the tangent space at each point (Definition 3.12), so is any scalar multiple of it, hence $\nabla_{\dot{\sigma}} \dot{\sigma} = 0$. $\square$

Note what Lemma 3.20 does *not* say: a *non*-affine reparameterisation — traversing the same set of points at varying speed — is in general *not* a geodesic, because Proposition 3.13 would fail. “Geodesic” constrains the parameterisation, not only the image.

Proposition 3.21 (Scaling property of the exponential map). *For $x \in \mathcal{M}$, $v \in T_x \mathcal{M}$ and $t \in \mathbb{R}$ for which both sides are defined,*

$$\gamma_v(t) = \exp_x(tv),$$

where γ_v is, as in Definition 3.17, the geodesic with $\gamma_v(0) = x$, $\dot{\gamma}_v(0) = v$. In words: following the geodesic with initial velocity v for time t lands in the same place as following the geodesic with initial velocity tv for one unit of time.

Proof. Fix t and set $\sigma(s) := \gamma_v(ts)$. By Lemma 3.20 (with $a = t$, $b = 0$), σ is a geodesic; it has $\sigma(0) = x$ and $\dot{\sigma}(0) = t \dot{\gamma}_v(0) = tv$. By uniqueness of geodesics with prescribed initial position and velocity, $\sigma = \gamma_{tv}$. Evaluating at $s = 1$ and using Definition 3.17, $\gamma_v(t) = \sigma(1) = \gamma_{tv}(1) = \exp_x(tv)$. $\square$

Corollary 3.22 (The geodesic interpolant). *If y lies in the domain where $\log_x$ is defined, the curve*

$$\tau \longmapsto \exp_x(\tau \log_x(y)), \quad \tau \in [0, 1],$$

is the constant-speed geodesic from x (at $\tau = 0$) to y (at $\tau = 1$).

Proof. At $\tau = 0$: $\exp_x(0) = \gamma_0(1) = x$. At $\tau = 1$: $\exp_x(\log_x(y)) = y$ by definition of $\log_x$ as the inverse of $\exp_x$. In between it is $\gamma_{\log_x(y)}(\tau)$ by Proposition 3.21, a geodesic, hence of constant speed by Proposition 3.13. Its speed is $\|\log_x(y)\|_g$, so its length is $\|\log_x(y)\|_g$; when the geodesic is minimising this equals $d_g(x, y)$, so that

$$\|\log_x(y)\|_g = d_g(x, y). \tag{3.2}$$

$\square$

Corollary 3.22 is the object that will replace $(1 - \tau)x + \tau y$, and (3.2) — the log map’s norm *is* the distance — is used repeatedly below.

3.1.8 Where log stops being defined: injectivity radius and cut locus

Caution 3.18 already recorded that $\exp_x$ is not globally invertible. We now name the objects that measure how far it *is* invertible, because Chapter 12 needs to state its hypotheses in these terms rather than case by case.

Definition 3.23 (Injectivity radius and cut locus). Fix $x \in \mathcal{M}$.

- The *injectivity radius* $\text{inj}(x)$ is the supremum of $r > 0$ such that $\exp_x$ restricted to the ball $\{v \in T_x \mathcal{M} : \|v\|_g < r\}$ is a diffeomorphism onto its image.
- The *cut locus* $\text{Cut}(x) \subset \mathcal{M}$ is the set of points y at which the geodesic from x *stops* being minimising: $y \in \text{Cut}(x)$ if some minimising geodesic from x reaching y fails to be minimising when extended past y , or if there is more than one minimising geodesic from x to y .

On $\mathcal{M} \setminus \text{Cut}(x)$, $\log_x$ is well defined, single-valued, and smooth.

Example 3.24 (The three cases we need). • S^1 . $\text{Cut}(x) = \{-x\}$, a single point; $\text{inj} = \pi$. Going clockwise or anticlockwise to the antipode costs the same, so $\log_x(-x)$ has two equally good values, $\pm\pi$.

- S^2 . $\text{Cut}(x) = \{-x\}$, again a single point, but now *infinitely many* minimising geodesics reach it — every meridian through the poles. $\text{inj} = \pi$.
- $SO(3)$. $\text{Cut}(I) = \{R \in SO(3) : \theta(R) = \pi\}$, the set of half-turns. $\text{inj} = \pi$ (with the metric of Proposition 3.51). By bi-invariance, $\text{Cut}(R_0) = \{R : \theta(R_0^\top R) = \pi\}$.

Caution 3.25 (On $SO(3)$ the cut locus is not a single point). The $SO(3)$ case in Example 3.24 differs from the two spheres in a way that matters practically. On S^2 the cut locus of a point is zero-dimensional (one antipode) inside a two-dimensional manifold. On $SO(3)$ it is a *two-dimensional surface* inside a three-dimensional manifold: a half-turn is specified by its axis, and axes form a two-parameter family (with n and $-n$ identified, so $\text{Cut}(I) \cong \mathbb{RP}^2$).

Both are sets of measure zero, so “the cut locus is hit with probability zero” is true in both cases. But a set of codimension one is approached far more often than an isolated point, and the density of the relative angle between two independent Haar-random rotations, $p(\theta) \propto 1 - \cos \theta$ (Proposition 3.59), is *largest* exactly at $\theta = \pi$. The cut locus of $SO(3)$ is therefore the most-visited neighbourhood in the whole group — the geometric reason behind the quantitative finding of Caution 3.57 that roughly one rotation pair in eleven lands within 8° of it.

The following elementary lemma is what makes the cut-locus hypothesis usable, and it is the one geometric fact Proposition 12.11’s proof will need.

Lemma 3.26 (A sub-arc of a minimising geodesic is minimising). *Let $\gamma : [0, 1] \rightarrow \mathcal{M}$ be a minimising geodesic from x to y , and let $0 \leq a \leq b \leq 1$. Then $\gamma|_{[a,b]}$ is a minimising geodesic from $\gamma(a)$ to $\gamma(b)$.*

Proof. It is a geodesic by Lemma 3.20 (restriction plus affine reparameterisation). Suppose it were not minimising, i.e. some curve σ from $\gamma(a)$ to $\gamma(b)$ had $L(\sigma) < L(\gamma|_{[a,b]})$. Then the concatenation $\gamma|_{[0,a]} * \sigma * \gamma|_{[b,1]}$ is a curve from x to y of length $L(\gamma|_{[0,a]}) + L(\sigma) + L(\gamma|_{[b,1]}) < L(\gamma)$, contradicting minimality of γ . $\square$

Example 3.27 (Exp and Log on S^2 , in closed form). Let $\mathcal{M} = S^2 \subset \mathbb{R}^3$ with the induced metric, $x \in S^2$, and $T_x S^2 = \{v : x^\top v = 0\}$.

Exponential map. For $v \in T_x S^2$, $v \neq 0$, write $\alpha = \|v\|_2$ and $\hat{v} = v/\alpha$. Since x and $\hat{v}$ are orthonormal, Example 3.14 says $\tau \mapsto \cos(\tau)x + \sin(\tau)\hat{v}$ is a unit-speed geodesic; it starts at x with velocity $\hat{v}$, so the geodesic with initial velocity v is $\gamma_v(\tau) = \cos(\alpha\tau)x + \sin(\alpha\tau)\hat{v}$ (constant speed α). Evaluating at $\tau = 1$,

$$\exp_x(v) = \cos(\|v\|)x + \sin(\|v\|)\frac{v}{\|v\|}, \quad \exp_x(0) = x. \quad (3.3)$$

Logarithmic map. Given $y \in S^2$ with $y \neq \pm x$, let $\alpha = \arccos(x^\top y) \in (0, \pi)$ — the great-circle distance, by Example 3.10 — and let $w = y - (x^\top y)x$ be the component of y orthogonal to x . Then

$$\log_x(y) = \alpha \frac{w}{\|w\|}. \quad (3.4)$$

Check. $\|w\|^2 = \|y\|^2 - (x^\top y)^2 = 1 - \cos^2 \alpha = \sin^2 \alpha$, so $\|w\| = \sin \alpha$ and $y = \cos \alpha x + \sin \alpha (w/\|w\|)$; comparing with (3.3) evaluated at $v = \alpha w/\|w\|$ gives $\exp_x(\log_x(y)) = y$ ✓. Also $\|\log_x(y)\| = \alpha = d_g(x, y)$, confirming (3.2), and $x^\top \log_x(y) = 0$, confirming $\log_x(y) \in T_x S^2$ as Caution 3.6 demands.

Numbers. With $x = (1, 0, 0)$ and $y = (0, 1, 0)$: $x^\top y = 0$ so $\alpha = \pi/2$; $w = y = (0, 1, 0)$, $\|w\| = 1$; hence $\log_x(y) = (0, \frac{\pi}{2}, 0)$, of norm $\pi/2$ — a quarter of the way round the sphere ✓. And $\exp_x((0, \frac{\pi}{2}, 0)) = \cos(\frac{\pi}{2})x + \sin(\frac{\pi}{2})(0, 1, 0) = (0, 1, 0) = y$ ✓.

Where it breaks. At $y = -x$ we get $\alpha = \pi$ but $w = 0$, so the direction $w/\|w\|$ is 0/0: every tangent direction gives a minimising geodesic to the antipode. That is $\text{Cut}(x)$ from Example 3.24, seen in the formula.

3.2 Calculus on a manifold: volume, gradient, and divergence

The last three tools are the ones needed to write down probability *densities* on a manifold and the equation they obey. They are used in Chapter 7 and, centrally, in Chapter 12.

3.2.1 Volume, and why a density needs a reference measure

In $\mathbb{R}^n$ one writes $\mathbb{P}(X \in A) = \int_A p(x) dx$ without comment, because there is an obvious background measure — Lebesgue — and everyone uses it. On a manifold there is no such default, and the choice has to be made explicit.

*A probability density is not a function on its own. It is a function **relative to a reference measure**: the factor by which probability is more concentrated at x than that reference measure is.*

Formally, p is a density of the probability measure $\mathbb{P}$ with respect to a reference measure μ if $\mathbb{P}(A) = \int_A p d\mu$ for all measurable A (the Radon–Nikodym derivative $p = d\mathbb{P}/d\mu$). Change μ and the same distribution has a different density. We need no more measure theory than this sentence; what we do need is a canonical μ , and the metric supplies one.

Definition 3.28 (Riemannian volume measure). In a coordinate chart $(x^1, \dots, x^n)$, write $g_{ij} = g(\partial_i, \partial_j)$ for the matrix of the metric in the coordinate basis, and $|g| = \det(g_{ij})$. The *Riemannian volume measure* is

$$d\text{vol}_g = \sqrt{|g|} dx^1 \cdots dx^n.$$

The $\sqrt{|g|}$ is not a convention; it is forced. A coordinate box of sides $dx^1, \dots, dx^n$ is, geometrically, the parallelepiped spanned by the vectors $\partial_1 dx^1, \dots, \partial_n dx^n$, and the volume of a parallelepiped spanned by vectors $v_1, \dots, v_n$ in an inner-product space is $\sqrt{\det(\langle v_i, v_j \rangle)}$ — the Gram determinant. Applying that with $\langle \partial_i, \partial_j \rangle = g_{ij}$ gives exactly $\sqrt{|g|} dx^1 \cdots dx^n$. In Euclidean coordinates $g_{ij} = \delta_{ij}$, $|g| = 1$, and we recover Lebesgue measure.

Example 3.29 (Polar coordinates, as a sanity check). On $\mathbb{R}^2$ in polar coordinates (r, θ) , an infinitesimal step dr has length dr but a step $d\theta$ has length $r d\theta$, so $g = \text{diag}(1, r^2)$ and $\sqrt{|g|} = r$. Hence $d\text{vol}_g = r dr d\theta$ — the familiar Jacobian of polar coordinates, recovered from the metric rather than memorised.

Notation 3.30 (Densities in this document). From here on, a density p on a manifold always means a density *with respect to* $d\text{vol}_g$, so that $\mathbb{P}(X \in A) = \int_A p(x) d\text{vol}_g(x)$ and $\int_{\mathcal{M}} p d\text{vol}_g = 1$. On $\mathbb{R}^d$ with the flat metric this is the usual convention and nothing changes.

Remark 3.31 (Why this bookkeeping is not pedantry). Two consequences are worth flagging, both of which recur later.

Differential entropy is not intrinsic. — $\int p \log p d\mu$ changes if μ changes: replacing μ by $c\mu$ shifts the entropy by $\log c$, and a non-constant change of reference measure changes it pointwise. So “the entropy of a distribution on $SO(3)$ ” is meaningless until the reference measure is named. *Kullback–Leibler divergence*, by contrast, is intrinsic: in $\text{KL}(P\|Q) = \int p \log(p/q) d\mu$ the reference measure cancels in the ratio, which is why Section 2.5 could treat it as a property of the two distributions alone.

On $SO(3)$ the natural reference measure is the one we already have. Proposition 3.32 below shows $d\text{vol}_g$ is Haar measure up to normalisation, so “uniform on $SO(3)$ ” and “constant density with respect to $d\text{vol}_g$ ” say the same thing.

Proposition 3.32 (Riemannian volume = Haar, on $SO(3)$). *With the bi-invariant metric of Proposition 3.51, the Riemannian volume measure of $SO(3)$ coincides with its Haar measure (Proposition 3.59) up to a positive constant factor.*

Proof. Bi-invariance means every left translation $L_Q : R \mapsto QR$ and every right translation $R \mapsto RQ$ is an isometry of $(SO(3), g)$. An isometry preserves g_{ij} in corresponding coordinates, hence preserves $\sqrt{|g|}$, hence preserves $d\text{vol}_g$ (Definition 3.28). So $d\text{vol}_g$ is a left- and right-invariant measure on the compact group $SO(3)$. Haar measure is the unique such measure up to a positive scalar (the classical fact quoted in Proposition 3.59), so $d\text{vol}_g = c \cdot d\mu_{\text{Haar}}$ for some $c > 0$. $\square$

3.2.2 The Riemannian gradient

Definition 3.33 (Riemannian gradient). For a smooth $f : \mathcal{M} \rightarrow \mathbb{R}$, the *Riemannian gradient* $\text{grad}_g f(x) \in T_x \mathcal{M}$ is the unique tangent vector with

$$g_x(\text{grad}_g f(x), v) = df_x(v) \quad \text{for every } v \in T_x \mathcal{M}, \quad (3.5)$$

where $df_x(v) = \left. \frac{d}{d\tau} \right|_{\tau=0} f(\gamma(\tau))$ for any curve γ with $\gamma(0) = x$, $\dot{\gamma}(0) = v$ — the directional derivative of f along v .

Existence and uniqueness are immediate from linear algebra: $v \mapsto df_x(v)$ is a linear functional on the finite-dimensional inner-product space $(T_x\mathcal{M}, g_x)$, and every such functional is represented by exactly one vector (Riesz). The content of (3.5) is that *the gradient depends on the metric*: the derivative df is metric-independent, but converting it into a *vector* requires an inner product, and a different metric gives a different gradient for the same f .

Example 3.34 (Euclidean and embedded cases). On $\mathbb{R}^n$ with $g = \langle \cdot, \cdot \rangle$, (3.5) reads $\langle \text{grad} f, v \rangle = \nabla f \cdot v$ for all v , so $\text{grad}_g f = \nabla f$, the ordinary gradient. On an embedded $\mathcal{M} \subset \mathbb{R}^N$ with the induced metric and any smooth extension $\bar{f}$ of f to a neighbourhood,

$$\text{grad}_g f(x) = P_{T_x\mathcal{M}}(\nabla \bar{f}(x)),$$

the tangential projection of the ambient gradient — because for tangent v , $df_x(v) = \nabla \bar{f}(x)^\top v = (P_{T_x\mathcal{M}} \nabla \bar{f}(x))^\top v$, the normal component contributing nothing.

When $f = \log p$ for a density p , $\text{grad}_g \log p$ is the *Riemannian score*, the object Chapter 7 learns; we note the definition here so that Chapter 12 can refer to it without re-deriving.

3.2.3 The Riemannian divergence

Intuition first. The divergence of a vector field measures how fast the field *spreads volume out*. Drop a small blob of dust at x and let every speck be carried by the flow of u . If the blob's volume grows, the divergence is positive there; if it shrinks, negative; if the flow merely stirs the dust around without changing volumes, zero. Precisely:

$$\left. \frac{d}{dt} \right|_{t=0} \text{vol}_g(\phi_t(A)) = \int_A \text{div}_g u \, d\text{vol}_g, \quad (3.6)$$

where ϕ_t is the flow of u (Definition 11.5) and A is any small region. Divergence is the *local rate of volume expansion, per unit volume*. Everything else is bookkeeping to express that in coordinates.

Definition 3.35 (Riemannian divergence). In a coordinate chart, writing $u = \sum_i u^i \partial_i$ for the components of u in the coordinate basis,

$$\text{div}_g u = \frac{1}{\sqrt{|g|}} \sum_{i=1}^n \partial_i (\sqrt{|g|} u^i). \quad (3.7)$$

Reading (3.7) symbol by symbol:

- u^i is the i -th *coordinate component* of the field — how fast the i -th coordinate is changing, not a physical speed;
- $\sqrt{|g|}$ converts coordinate volume into geometric volume (Definition 3.28), so $\sqrt{|g|} u^i$ is the geometric *flux* in the i -th coordinate direction;
- ∂_i of that flux, summed, is the net outflow measured in coordinate volume;
- dividing by $\sqrt{|g|}$ converts back to a rate per unit *geometric* volume, which is what (3.6) asks for.

So the formula is: convert to geometric flux, take the flat divergence, convert back. The two $\sqrt{|g|}$ factors are the currency conversion in and out.

Example 3.36 (The Euclidean special case). With $g_{ij} = \delta_{ij}$ we have $|g| = 1$, and (3.7) collapses to $\operatorname{div}_g u = \sum_i \partial_i u^i = \nabla \cdot u$, the ordinary divergence of (11.4). So div_g is a generalisation, not a replacement.

Example 3.37 (Polar coordinates, again as a check). With $\sqrt{|g|} = r$ from Example 3.29, (3.7) gives

$$\operatorname{div}_g u = \frac{1}{r} \partial_r (r u^r) + \partial_\theta u^\theta,$$

which, on substituting the physical components $U_r = u^r$, $U_\theta = r u^\theta$ (the components in an *orthonormal* frame), becomes $\frac{1}{r} \partial_r (r U_r) + \frac{1}{r} \partial_\theta U_\theta$ — exactly the polar divergence formula of ordinary vector calculus.

Two properties are used later and are immediate from (3.7).

Proposition 3.38 (Product rule and divergence theorem). *For a smooth scalar f and vector field V on $\mathcal{M}$,*

$$\operatorname{div}_g(fV) = f \operatorname{div}_g V + g(\operatorname{grad}_g f, V). \quad (3.8)$$

Moreover, for a compact region $A \subseteq \mathcal{M}$ with smooth boundary and outward unit normal ν ,

$$\int_A \operatorname{div}_g V \, d\operatorname{vol}_g = \int_{\partial A} g(V, \nu) \, dS, \quad (3.9)$$

and if $\mathcal{M}$ is compact without boundary (as $SO(3)$ is), then $\int_{\mathcal{M}} \operatorname{div}_g V \, d\operatorname{vol}_g = 0$.

Proof. For (3.8), expand (3.7):

$$\operatorname{div}_g(fV) = \frac{1}{\sqrt{|g|}} \partial_i (\sqrt{|g|} f V^i) = \frac{f}{\sqrt{|g|}} \partial_i (\sqrt{|g|} V^i) + V^i \partial_i f = f \operatorname{div}_g V + df(V),$$

and $df(V) = g(\operatorname{grad}_g f, V)$ by (3.5). Equation (3.9) is the Riemannian divergence (Gauss) theorem, the manifold generalisation of the classical one; we take it as standard (see, e.g., Lee, *Introduction to Riemannian Manifolds*, 2nd ed., 2018, Thm. 2.30). The closed-manifold statement is the case $\partial A = \emptyset$ with $A = \mathcal{M}$. $\square$

Example 3.39 (A divergence-free field on $SO(3)$). Fix $\xi \in \mathbb{R}^3$ and consider the *left-invariant* field $u(R) := R\hat{\xi}$, i.e. the field whose body-frame components are the same constant vector ξ at every R . Its flow is $\phi_t(R) = R \exp(t\hat{\xi})$ — right translation by a fixed group element — which is an isometry by bi-invariance, hence preserves $d\operatorname{vol}_g$ (Proposition 3.32’s argument). By (3.6), $\operatorname{vol}_g(\phi_t(A))$ is constant in t , so

$$\operatorname{div}_g u \equiv 0.$$

A constant body-frame angular velocity therefore transports probability around $SO(3)$ without concentrating or diluting it anywhere — the exact rotational analogue of a constant velocity field on $\mathbb{R}^d$. (The conditional field of Chapter 12 is *not* of this form, since its Ξ depends on the current point; this example is a calibration, not that computation.)

3.3 Groups and Lie groups

Motivation. A manifold alone tells us how to do calculus on a curved space; it says nothing about how to *compose* two elements (“first rotate by Q_1 , then by Q_2 ”). Rigid motions have exactly this compositional structure — applying two rotations in sequence is again a rotation — and capturing it precisely is the job of group theory.

Definition 3.40 (Group). A *group* is a set G with a binary operation $\circ : G \times G \rightarrow G$ such that: (i) *associativity*, $(g_1 \circ g_2) \circ g_3 = g_1 \circ (g_2 \circ g_3)$; (ii) there is an *identity* $e \in G$ with $e \circ g = g \circ e = g$ for all g ; (iii) every g has an *inverse* g^{-1} with $g \circ g^{-1} = g^{-1} \circ g = e$.

Example 3.41. $(\mathbb{R}^d, +)$ is a group (identity 0, inverse $-x$); it is commutative ($g_1 \circ g_2 = g_2 \circ g_1$), i.e. *abelian*. The set of invertible $n \times n$ matrices under matrix multiplication, $GL(n, \mathbb{R})$, is a (non-abelian, for $n \geq 2$) group.

Definition 3.42 (Lie group; restated). A *Lie group* is a group G that is simultaneously a smooth manifold, such that the group operations $\circ : G \times G \rightarrow G$ and $(\cdot)^{-1} : G \rightarrow G$ are smooth maps.

The point of requiring smoothness of the group operation is that it lets us transport tangent vectors from one point of the group to another using group multiplication itself — a device with no analogue on a generic manifold, and the reason Lie groups admit a canonical, group-wide notion of “the” tangent space, rather than merely a separate tangent space at every point.

Definition 3.43 (Lie algebra; restated). Let G be a Lie group with identity e . Its *Lie algebra* is $\mathfrak{g} := T_e G$, the tangent space at the identity.

Definition 3.44 (Left translation and left-invariant vector fields). For $g \in G$, *left translation by g* is the smooth map $L_g : G \rightarrow G$, $L_g(h) = g \circ h$. Its differential at the identity, $dL_g|_e : \mathfrak{g} = T_e G \rightarrow T_g G$, is a linear isomorphism (it has smooth inverse $dL_{g^{-1}}|_g$), so every $\xi \in \mathfrak{g}$ determines a tangent vector $dL_g|_e(\xi) \in T_g G$ at *every* point g simultaneously.

Definition 3.44 is the mechanism, promised above, by which a single vector space $\mathfrak{g}$ suffices to describe tangent vectors everywhere on G : any $\xi \in \mathfrak{g}$ can be moved to any point g by $v = dL_g|_e(\xi) \in T_g G$. This is called *left trivialisation*, and it is not the only choice — *right* translation $R_g(h) = h \circ g$ gives an equally valid but generally *different* identification, called *right trivialisation*. Which one a given piece of code or a given formula uses is a genuine convention, not a matter of taste, because $dL_g|_e(\xi)$ and $dR_g|_e(\xi)$ are different tangent vectors at g whenever G is non-abelian; Section 3.5 below makes this concrete for $SO(3)$, and Chapter 13 traces the convention actually used by the SigmaFlow implementation with care, because — as flagged already in the introduction to this document — the two codebases are not visibly consistent with each other or with the existing thesis draft on this exact point, and getting it wrong silently flips signs in the trained model without necessarily crashing anything.

3.4 The rotation group $SO(3)$

3.4.1 Definition and dimension

Definition 3.45 (Orthogonal and special orthogonal group).

$$O(3) = \{R \in \mathbb{R}^{3 \times 3} : R^\top R = I\}, \quad SO(3) = \{R \in O(3) : \det R = 1\}.$$

Every $R \in O(3)$ satisfies $\det(R^\top R) = \det(R)^2 = \det(I) = 1$, so $\det R = \pm 1$; $SO(3)$ is the subset with $\det R = +1$, and these are exactly the *orientation-preserving* orthogonal maps — rotations, as opposed to reflections ($\det R = -1$). $O(3)$ has two disjoint pieces (*connected components*), separated by the sign of the determinant, and $SO(3)$ is precisely the piece

containing the identity, i.e. the piece reachable from I by a continuous path within $O(3)$. Physically: a continuous, physical rotation of a rigid body can never turn it into its mirror image, which is why $SO(3)$, not $O(3)$, is the correct group for orientations of physical objects, and why the distinction resurfaces, non-optionally, when we discuss chirality of molecules in Remark 3.74.

Proposition 3.46 (Rotations preserve length and orientation). *For $R \in SO(3)$ and $x, y \in \mathbb{R}^3$: $\|Rx\| = \|x\|$, $\langle Rx, Ry \rangle = \langle x, y \rangle$, and $(Rx) \times (Ry) = R(x \times y)$.*

Proof. $\langle Rx, Ry \rangle = x^\top R^\top Ry = x^\top y = \langle x, y \rangle$ using $R^\top R = I$; taking $y = x$ gives $\|Rx\|^2 = \|x\|^2$. For the cross product identity, use the general fact $(Ax) \times (Ay) = \det(A) A^{-\top}(x \times y)$ for invertible A (a standard identity, provable by checking it on the standard basis and using multilinearity/antisymmetry of both sides), and specialise to $A = R$: $R^{-\top} = R$ (since $R^\top = R^{-1}$) and $\det R = 1$. $\square$

Proposition 3.47 (Dimension of $SO(3)$). *$SO(3)$ is a 3-dimensional manifold.*

Proof. The constraint $R^\top R = I$ is a system of equations in the 9 entries of R ; the matrix $R^\top R$ is automatically symmetric, so this constraint has only $\binom{3+1}{2} = 6$ independent scalar components (the entries on and above the diagonal of a 3×3 symmetric matrix). One checks (via the implicit function theorem, using that the differential of $R \mapsto R^\top R$ has full rank 6 at every $R \in O(3)$) that this cuts out a smooth $9 - 6 = 3$ -dimensional manifold, of which $SO(3)$ is one (open-and-closed, hence a full-dimensional) connected component; the extra condition $\det R = 1$ removes no further dimension, it only selects a component. We take the rank computation as a standard fact from differential topology and do not reprove it in full generality. $\square$

Three degrees of freedom is exactly the number we expect: an orientation in 3-dimensional space can be described by, for instance, two angles fixing an axis on the sphere S^2 (a 2-dimensional choice) and one further angle of rotation about that axis, and this count recurs throughout the rest of this chapter and Chapter 3 onward.

3.4.2 The Lie algebra $\mathfrak{so}(3)$ and the hat/vee maps

Proposition 3.48 ($\mathfrak{so}(3)$ is the skew-symmetric matrices). $T_I SO(3) = \{\Omega \in \mathbb{R}^{3 \times 3} : \Omega^\top = -\Omega\} =: \mathfrak{so}(3)$.

Proof. Let $\gamma(\tau)$ be a smooth curve in $SO(3)$ with $\gamma(0) = I$. Since $\gamma(\tau)^\top \gamma(\tau) = I$ for all τ , differentiating at $\tau = 0$ using the product rule gives $\dot{\gamma}(0)^\top I + I^\top \dot{\gamma}(0) = 0$, i.e. $\dot{\gamma}(0)^\top = -\dot{\gamma}(0)$: every tangent vector at I is skew-symmetric. Conversely, the space of skew-symmetric 3×3 matrices is 3-dimensional (three free entries, above the diagonal, with the rest determined by antisymmetry and a zero diagonal), matching Proposition 3.47, so this necessary condition is also sufficient (both sides are 3-dimensional subspaces, one containing the other, hence equal). $\square$

Definition 3.49 (Hat and vee operators; restated precisely).

$$(\cdot)^\wedge : \mathbb{R}^3 \rightarrow \mathfrak{so}(3), \quad \hat{\omega} = \begin{pmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{pmatrix}, \quad \omega = (\omega_1, \omega_2, \omega_3)^\top,$$

and $(\cdot)^\vee : \mathfrak{so}(3) \rightarrow \mathbb{R}^3$ is its (linear, bijective) inverse.

Proposition 3.50 (Hat map intertwines cross product and matrix commutator). *For $\omega, v \in \mathbb{R}^3$: $\widehat{\omega}v = \omega \times v$. Consequently, for $\omega_1, \omega_2 \in \mathbb{R}^3$, the matrix commutator satisfies $[\widehat{\omega}_1, \widehat{\omega}_2] := \widehat{\omega}_1\widehat{\omega}_2 - \widehat{\omega}_2\widehat{\omega}_1 = \widehat{\omega_1 \times \omega_2}$.*

Proof. Direct computation: $\widehat{\omega}v = (\omega_2v_3 - \omega_3v_2, \omega_3v_1 - \omega_1v_3, \omega_1v_2 - \omega_2v_1)^\top = \omega \times v$ by the definition of the cross product. The commutator identity follows by applying both sides to an arbitrary v and using the bilinearity and antisymmetry of $\times$ together with the vector identity $a \times (b \times v) - b \times (a \times v) = (a \times b) \times v$ (itself checkable directly, or via the BAC-CAB rule $a \times (b \times c) = b(a \cdot c) - c(a \cdot b)$). $\square$

Code correspondence

Mathematical object: $(\cdot)^\wedge : \mathbb{R}^3 \rightarrow \mathfrak{so}(3), (\cdot)^\vee : \mathfrak{so}(3) \rightarrow \mathbb{R}^3$

Implementation: `so3_utils.py::hat, vee` (identical in `SigmaDock/` and `SigmaFlow_Development/`)

Tensor shape: `[..., 3] ↔ [..., 3, 3]`

Interpretation: `hat` builds the skew-symmetric matrix by constructing only the strict upper triangle and then anti-symmetrising (`hat_v + -hat_v.transpose(-1,-2)`); `vee` inverts it by reading off $(-A_{23}, A_{13}, -A_{12})$ and only *prints* a warning (rather than raising) if the input is not skew-symmetric within a loose tolerance — this is not a hard safety check, and a caller that accidentally passes a non-skew matrix will not be stopped by `vee` itself.

3.4.3 The exponential map: Rodrigues' formula

Motivation. We now want an explicit, closed-form recipe for $\exp_I : \mathfrak{so}(3) \rightarrow SO(3)$ (Definition 3.17 specialised to $G = SO(3)$, $x = I$): given a small rotation $\widehat{\omega} \in \mathfrak{so}(3)$, what is the actual rotation matrix obtained by “following the corresponding geodesic for unit time”? On a Lie group, this question has a beautifully uniform answer.

Proposition 3.51 (Geodesics through the identity of a bi-invariant Lie group are one-parameter subgroups). *Equip $SO(3)$ with the metric $g_I(\Omega_1, \Omega_2) = \frac{1}{2} \text{tr}(\Omega_1^\top \Omega_2)$ at the identity, extended to every other point by left translation (Definition 3.44); one checks this metric is also invariant under right translation (bi-invariant), which is a special, convenient feature of compact groups such as $SO(3)$ and is what makes the following true. The geodesic through I with initial velocity $\widehat{\omega} \in \mathfrak{so}(3)$ is $\gamma(\tau) = \exp(\tau\widehat{\omega})$, the matrix exponential.*

We do not prove Proposition 3.51 in full generality (it is a standard fact about bi-invariant metrics on Lie groups, following from the observation that one-parameter subgroups $\tau \mapsto \exp(\tau\Omega)$ of a bi-invariant Lie group have constant speed and locally minimise length; see, e.g., do Carmo, *Riemannian Geometry*, 1992, Ch. 11); we use it only to justify that $\exp_I = \exp$, the ordinary matrix exponential $\exp(A) = \sum_{k=0}^{\infty} A^k/k!$, restricted to $\mathfrak{so}(3)$. What remains is to evaluate this infinite series in closed form.

Theorem 3.52 (Rodrigues' rotation formula). *For $\omega \in \mathbb{R}^3$ with $\theta = \|\omega\|$,*

$$\exp(\widehat{\omega}) = I + \frac{\sin \theta}{\theta} \widehat{\omega} + \frac{1 - \cos \theta}{\theta^2} \widehat{\omega}^2 \quad (\theta \neq 0),$$

and $\exp(\widehat{\omega}) = I$ when $\theta = 0$ (continuously extending the formula, since both fractions have removable singularities at $\theta = 0$). The resulting matrix is the rotation by angle θ about the axis ω/θ , in the right-hand sense.

Proof. Write $n = \omega/\theta$ (unit vector) so $\hat{\omega} = \theta\hat{n}$. The key algebraic fact is the following closed recursion for powers of $\hat{n}$, provable directly from Proposition 3.50 (with n a unit vector) via the BAC-CAB identity:

$$\hat{n}^2 = nn^\top - I, \quad \hat{n}^3 = \hat{n}\hat{n}^2 = \hat{n}(nn^\top - I) = -\hat{n},$$

using $\hat{n}n = n \times n = 0$ (so $\hat{n}nn^\top = 0$). Hence $\hat{n}^3 = -\hat{n}$, so every higher power reduces: $\hat{n}^4 = -\hat{n}^2$, $\hat{n}^5 = \hat{n}$, etc., with period 4 up to sign, i.e. odd powers are $\pm\hat{n}$ and even powers (beyond 0) are $\pm\hat{n}^2$. Substituting into the exponential series and grouping by parity,

$$\begin{aligned} \exp(\theta\hat{n}) &= I + \left(\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \dots\right)\hat{n} + \left(\frac{\theta^2}{2!} - \frac{\theta^4}{4!} + \frac{\theta^6}{6!} - \dots\right)\hat{n}^2 \\ &= I + \sin(\theta)\hat{n} + (1 - \cos(\theta))\hat{n}^2, \end{aligned}$$

recognising the Taylor series of $\sin$ and $1 - \cos$. Substituting back $\hat{n} = \hat{\omega}/\theta$ gives the claimed formula. That the result is a rotation by θ about axis n can be checked directly: $\exp(\theta\hat{n})n = n$ (since $\hat{n}n = 0$, so both correction terms vanish on n), so n is fixed, and restricted to the plane orthogonal to n the formula reduces (by direct computation using $\hat{n}^2v = -v$ for $v \perp n$) to the standard 2×2 rotation-by- θ matrix in an orthonormal basis of that plane. $\square$

Code correspondence

Mathematical object: $\exp : \mathfrak{so}(3) \rightarrow SO(3)$, $\text{Exp} : \mathbb{R}^3 \rightarrow SO(3)$

Implementation: `so3_utils.py::exp, Exp`

Tensor shape: $[\dots, 3, 3] \rightarrow [\dots, 3, 3]$ (`exp`); $[\dots, 3] \rightarrow [\dots, 3, 3]$ (`Exp`, defined as `exp(hat(v))`)

Interpretation: Neither codebase uses the closed-form Rodrigues formula of Theorem 3.52 directly. `exp` is implemented as a literal call to `torch.matrix_exp`, PyTorch’s generic (Padé-approximation-based) matrix exponential, with a CPU round-trip fallback on Apple-Silicon (MPS) backends where `torch.matrix_exp` is unsupported. This is mathematically equivalent to Theorem 3.52 (both compute the same matrix exponential; $\mathfrak{so}(3)$ ’s exponential series converges absolutely for every finite θ , so there is no issue of principle), but it is worth recording explicitly as a discrepancy between how this document derives the map and how the shipped code evaluates it: Theorem 3.52’s closed form is what makes the derivations of Section 3.4.4 and Chapters 6 and 11 tractable by hand, but the executed program never evaluates $\sin\theta/\theta$ directly at this step.

3.4.4 The logarithmic map, and its numerically dangerous points

Motivation. $\log : SO(3) \rightarrow \mathfrak{so}(3)$ (near I ; more on the domain of validity below) is the local inverse of $\exp$, and it is what lets us measure “how different are two rotations” as a single tangent vector, exactly the operation flow matching needs to build an interpolant between a source rotation and a target rotation (Section 12.10).

Theorem 3.53 (Logarithm of a rotation matrix). *For $R \in SO(3)$ with rotation angle $\theta = \arccos\left(\frac{\text{tr}(R)-1}{2}\right) \in [0, \pi]$ and $\theta \notin \{0, \pi\}$,*

$$\log(R) = \frac{\theta}{2\sin\theta} (R - R^\top) \in \mathfrak{so}(3).$$

Proof. By Theorem 3.52, $R = I + \sin\theta\hat{n} + (1 - \cos\theta)\hat{n}^2$ for the (unknown) axis n and angle θ of R . Since $\hat{n}$ is skew-symmetric and $\hat{n}^2 = nn^\top - I$ is symmetric, the skew-symmetric part

of R is exactly $\frac{1}{2}(R - R^\top) = \sin \theta \hat{n}$. Solving for $\hat{n}$ (valid whenever $\sin \theta \neq 0$, i.e. $\theta \notin \{0, \pi\}$) gives $\hat{n} = (R - R^\top)/(2 \sin \theta)$, and hence $\log(R) = \theta \hat{n} = \frac{\theta}{2 \sin \theta}(R - R^\top)$. The formula for θ itself follows by taking the trace of Rodrigues' formula: $\text{tr}(R) = 3 + 0 + (1 - \cos \theta) \text{tr}(\hat{n}^2) = 3 + (1 - \cos \theta)(-2) = 1 + 2 \cos \theta$, using $\text{tr}(\hat{n}^2) = \text{tr}(nn^\top - I) = \|n\|^2 - 3 = -2$; solving gives $\cos \theta = (\text{tr}(R) - 1)/2$. $\square$

The two excluded points $\theta = 0$ and $\theta = \pi$ are not edge cases that can be ignored: $\theta = 0$ (the identity rotation) is reached with probability zero under any continuous sampling scheme but is approached arbitrarily closely whenever two rotations being compared are nearly equal, which happens *routinely* near convergence of an iterative sampler; and $\theta = \pi$ (a half-turn) has positive probability density under the uniform distribution on $SO(3)$ derived in Proposition 3.59 below, so it *will* be sampled during training and inference at a non-negligible rate. Both require separate formulas.

Proposition 3.54 (Small-angle behaviour of $\log$). *As $\theta \rightarrow 0$,*

$$\log(R) = \frac{1}{2} \left(1 + \frac{\theta^2}{6} + O(\theta^4) \right) (R - R^\top).$$

Proof. Taylor-expand $\theta/(2 \sin \theta) = \frac{1}{2}(1 - \theta^2/6 + O(\theta^4))^{-1} = \frac{1}{2}(1 + \theta^2/6 + O(\theta^4))$, using $\sin \theta = \theta - \theta^3/6 + O(\theta^5)$. $\square$

Proposition 3.55 (Behaviour of $\log$ near $\theta = \pi$). *As $\theta \rightarrow \pi$, $R \rightarrow 2nn^\top - I$ for the rotation axis n , so $\frac{1}{2}(R + I) \rightarrow nn^\top$; consequently $|n_i| = \sqrt{(\frac{1}{2}(R + I))_{ii}}$, with relative signs fixed by comparing the off-diagonal entries of $\frac{1}{2}(R + I) = nn^\top$ against $n_i n_j$. The overall sign of n is genuinely undetermined: at $\theta = \pi$ exactly,*

$$\exp(\pi \hat{n}) = 2nn^\top - I = 2(-n)(-n)^\top - I = \exp(\pi \widehat{(-n)}),$$

so πn and $-\pi n$ are two distinct tangent vectors mapping to the same rotation. Both are legitimate values of $\log(R)$, and both have the same length π ; the map $\log$ is therefore two-valued exactly on the set of half-turns, which is the antipodal set of Caution 3.18(b).

Proof. At $\theta = \pi$: $\sin \theta = 0$, $\cos \theta = -1$, so Rodrigues' formula gives $R = I + 0 + 2\hat{n}^2 = I + 2(nn^\top - I) = 2nn^\top - I$, hence $\frac{1}{2}(R + I) = nn^\top$, a rank-one symmetric matrix with unit-norm columns along $\pm n$. Reading off $|n_i|$ from the diagonal and signs from the off-diagonal entries is then direct linear algebra on the outer product $nn^\top$. $\square$

Caution 3.56 (Do not read this as $\hat{n} = \widehat{-n}$). It is tempting to compress the last statement to “the sign does not matter because $\hat{n} = \widehat{-n}$ ”. That equation is *false*: the hat map is linear, so $\widehat{-n} = -\hat{n}$. What is true is the weaker and more specific statement above — that the two *exponentials* coincide at $\theta = \pi$. Away from π the sign of n matters and cannot be chosen freely.

Code correspondence

Mathematical object: $\log : SO(3) \rightarrow \mathfrak{so}(3)$ near a generic rotation; its numerically safe extension to $\theta \rightarrow 0$ and $\theta \rightarrow \pi$

Implementation: `so3_utils.py::rotation_vector_from_matrix` (called by `Log`, whose output is then hat-lifted by `log`)

Tensor shape: $[\dots, 3, 3] \rightarrow [\dots, 3]$ (vee-map convention: the vector form $v = \theta n$, not the $\mathfrak{so}(3)$ matrix)

Interpretation: the code computes *all three* regimes — generic (Theorem 3.53), near-zero (Proposition 3.54, using the exact Taylor truncation derived above), and near- π (Proposition 3.55, with sign recovered from whichever row of $\frac{1}{2}(R + I)$ has the largest norm) — for *every* input, and blends them with continuous 0/1 masks built from `torch.isclose` (tolerance $\text{atol} = 10^{-8}$ near $\theta = 0$, the much looser $\text{atol} = 10^{-2}$ near $\theta = \pi$), rather than a hard `if/else` branch. This is a deliberate autodiff-friendly pattern: computing all three branches unconditionally avoids NaN-producing gradients from a branch that was not selected but would otherwise still be backpropagated through, at the cost of some redundant computation.

Caution 3.57 (The angle-clamp bug, and why it mattered only for flow matching). The function that computes $\theta = \arccos(\frac{\text{tr}(R)-1}{2})$ (`Omega` in `so3_utils.py`) must clamp its argument to $[-1, 1]$ before calling `arccos`, since floating-point round-off can push $\frac{\text{tr}(R)-1}{2}$ fractionally outside this interval even for a valid $R \in SO(3)$. The original SigmaDock code clamps to $[-0.99, 0.99]$, i.e. $\theta \in [\arccos(0.99), \arccos(-0.99)] \approx [8.1^\circ, 172^\circ]$ — *any true angle beyond 172° is silently rounded down to 172°* . For diffusion this is close to harmless: θ enters only through the smooth score function of Section 7.3, degrading gracefully under a slightly wrong clamp. For flow matching it is not harmless: the geodesic interpolant of Section 12.10 needs $\log$ to be an *exact* inverse of $\exp$ (an exact round trip $\exp(\log(R)) = R$), which the clamp breaks for every pair whose relative angle leaves $[8.1^\circ, 171.9^\circ]$.

How often is that? *Not* $(180 - 172)/180 \approx 4.4\%$: the relative angle between two independent Haar-random rotations is not uniform on $[0, \pi]$ but has density $p(\theta) = (1 - \cos \theta)/\pi$ (Proposition 3.59), which puts most of its mass at large angles. Integrating,

$$\mathbb{P}(\theta > 171.9^\circ) = \frac{\pi - (\theta_0 - \sin \theta_0)}{\pi} \Big|_{\theta_0=171.9^\circ} \approx 9.0\%, \quad \mathbb{P}(\theta < 8.1^\circ) \approx 0.015\%.$$

So the damage is essentially one-sided: about **one rotation pair in eleven** is silently truncated at the upper end, while the lower end is irrelevant. (A 400,000-sample Monte Carlo check reproduces 9.01% and 0.018%, and en route confirms the median relative angle 132.4° and mean 126.6° quoted elsewhere in this document.) The frequently-quoted “ $\approx 9\%$ ” is therefore the right order of magnitude, but the uniform-angle reasoning that appears to produce it is wrong twice over — an incorrect density and a spurious doubling — and would give the wrong answer for any other clamp value. SigmaFlow’s `so3_utils.py` narrows the clamp to $[-1+10^{-7}, 1-10^{-7}]$, i.e. $\theta \in [0.026^\circ, 179.97^\circ]$, purely for `arccos`’s numerical domain, and documents the reason for the change directly in the source (`so3_flow_matcher.py`, inline comment). This document treats the fix as settled and uses the corrected clamp implicitly wherever θ is discussed.

3.4.5 $SO(3)$ as a metric space, and Brownian motion’s invariant measure

We record two facts about $SO(3)$ that are used repeatedly from Chapter 6 onward and are best derived once, here, in isolation from any generative-modelling context.

Proposition 3.58 (Bi-invariant Riemannian distance on $SO(3)$). *With the metric of Proposition 3.51, the Riemannian distance between $R_1, R_2 \in SO(3)$ is $d(R_1, R_2) = \|\log(R_1^\top R_2)\|_g = \theta$, the rotation angle of the relative rotation $R_1^\top R_2$.*

Proof sketch. By bi-invariance, $d(R_1, R_2) = d(I, R_1^\top R_2)$ (left-translate by $R_1^{-1} = R_1^\top$), and $d(I, R)$ is the length of the geodesic from I to R , which by Proposition 3.51 is $\exp(\tau\hat{\omega})$ for

the ω with $\exp(\hat{\omega}) = R$, $\|\omega\|$ minimal — this is exactly $\theta = \|\log(R)^\vee\|$ under the metric normalisation of Proposition 3.51 (chosen precisely so that the identification $\mathfrak{so}(3) \cong \mathbb{R}^3$ of Definition 3.49 is a metric isometry, i.e. so that $\|\hat{\omega}\|_g = \|\omega\|_2$ — a direct check: $g_I(\hat{\omega}, \hat{\omega}) = \frac{1}{2} \text{tr}(\hat{\omega}^\top \hat{\omega}) = \frac{1}{2} \text{tr}(-\hat{\omega}^2) = \|\omega\|_2^2$ by a short computation using Proposition 3.50). $\square$

Proposition 3.59 (The Haar measure on $SO(3)$, in axis–angle coordinates). *The measure on $SO(3)$ invariant under both left and right multiplication (the Haar measure, unique up to normalisation on a compact group; existence/uniqueness is a classical fact we do not reprove — see, e.g., Folland, A Course in Abstract Harmonic Analysis, 1995, Ch. 2), when $SO(3)$ is parameterised by axis $n \in S^2$ and angle $\theta \in [0, \pi]$ via $R = \exp(\theta \hat{n})$, has density, with respect to $d\theta \times (\text{uniform measure on } S^2)$, proportional to $1 - \cos \theta$.*

Proof sketch. This is the statement that the Riemannian volume form of Proposition 3.51, expressed in axis–angle (geodesic normal) coordinates, has Jacobian $\propto (1 - \cos \theta)$ relative to the flat product measure $d\theta \times d\Omega_{S^2}$; the computation is a standard but slightly involved exercise in Riemannian normal coordinates on a compact Lie group (the Jacobian of $\exp_I$ at $\theta \hat{n}$, for a bi-invariant metric, is $\prod_k \left(\frac{2 \sin(\lambda_k \theta / 2)}{\lambda_k \theta} \right)$ over the eigenvalues λ_k of $\text{ad}_{\hat{n}}$; for $\mathfrak{so}(3)$ these are $0, \pm i$, and evaluating the product reduces, after simplifying the complex-conjugate pair, to $(1 - \cos \theta) / (\theta^2 / 2) \cdot (\text{const})$ times the flat measure), which we do not carry out in full here and instead take as a standard fact (see, e.g., Chirikjian, *Stochastic Models, Information Theory, and Lie Groups*, Vol. 2, 2012, §10.2). We record the consequence that matters practically: to sample $R \sim \text{Haar}(SO(3))$, sample n uniformly on S^2 and, independently, sample θ from the density $\propto 1 - \cos \theta$ on $[0, \pi]$. $\square$

Code correspondence

Mathematical object: $R \sim \text{Haar}(SO(3))$

Implementation: `so3_utils.py::sample_uniform`

Tensor shape: returns `[N, 3, 3]`

Interpretation: builds a 1000-point grid on $[0, \pi]$, numerically integrates the density $\propto 1 - \cos \theta$ of Proposition 3.59 into a CDF, draws θ by inverse-CDF sampling via linear interpolation on this grid (`np.interp`), draws an axis by normalising a standard Gaussian vector in $\mathbb{R}^3$ (a standard, exact way to sample uniformly on S^2 : the direction of an isotropic Gaussian is rotation-invariant in distribution, since the Gaussian density $\propto e^{-\|x\|^2/2}$ depends on x only through $\|x\|$), and returns $\exp(\theta \hat{n})$. This sampling routine runs on `numpy`’s global random state, *not* through `torch`’s seeded generator — a genuine reproducibility subtlety: seeding `torch.manual_seed` alone does not fix the axes or angles drawn by this function.

3.5 Left and right trivialisation, resolved for this document

We can now make Definition 3.44’s warning fully concrete for $SO(3)$, since this is a point at which, as flagged in the introduction to this document, the existing thesis draft and the SigmaFlow source code do not visibly agree, and it must be resolved by direct derivation rather than by trusting either source.

Definition 3.60 (Body-frame and spatial-frame angular velocity). Let $R(\tau)$ be a smooth curve in $SO(3)$. Its *body-frame* angular velocity is $\omega^{\text{bd}}(\tau) \in \mathbb{R}^3$ with

$$\dot{R}(\tau) = R(\tau) \widehat{\omega^{\text{bd}}(\tau)}, \quad \text{equivalently} \quad \widehat{\omega^{\text{bd}}} = R^{-1} \dot{R};$$

its *spatial-frame* (or *world-frame*) angular velocity is $\omega^{\text{sp}}(\tau) \in \mathbb{R}^3$ with

$$\dot{R}(\tau) = \widehat{\omega^{\text{sp}}(\tau)} R(\tau), \quad \text{equivalently} \quad \widehat{\omega^{\text{sp}}} = \dot{R} R^{-1}.$$

In the language of Definition 3.44, ω^{bd} is the **left-trivialised** and ω^{sp} the **right-trivialised** velocity, because $\dot{R} = R\widehat{\omega}$ is exactly $dL_R|_e(\widehat{\omega})$ and $\dot{R} = \widehat{\omega}R$ is exactly $dR_R|_e(\widehat{\omega})$.

Caution 3.61 (The words “left” and “right” are a trap; “body” and “spatial” are not). Two independent naming conflicts meet here, and getting either backwards silently transposes a rotation.

First, some authors name the trivialisation after *which side of R the hat matrix sits on* rather than after which translation map produces it, which inverts the labels relative to Definition 3.60. The attribution fixed above is the standard Lie-theoretic one and is forced by Definition 3.44: left translation L_R acts as $S \mapsto RS$, so $dL_R|_e(\xi) = R\xi$, so a tangent vector expressed by left translation is $\dot{R} = R\widehat{\omega}$ — the *body* frame. An earlier draft of this document had these two labels interchanged; the formulas were unaffected, but the names were.

Second, the SigmaDock source comments use “left-invariant” as a synonym for *world* frame (Section 10.8), i.e. with the opposite attribution to the one fixed here. That comment is quoted verbatim where it occurs, and is *not* silently corrected — but the reader should know that the code’s label and this document’s label disagree, while the underlying object (which side of R the multiplication happens on) is unambiguous and is what every reconstruction in Chapters 10 and 13 is checked against.

Practical rule for reading the rest of this document: trust $\omega^{\text{bd}} = R^{-1}\dot{R}$ and $\omega^{\text{sp}} = \dot{R}R^{-1}$, and treat the words “left” and “right” as mnemonics only.

By definition, $\widehat{\omega^{\text{sp}}} = \dot{R}R^\top = \dot{R}R^{-1}$ and $\widehat{\omega^{\text{bd}}} = R^\top \dot{R} = R^{-1}\dot{R}$, and they are related by $\omega^{\text{sp}} = R\omega^{\text{bd}}$ (apply Proposition 3.50-style conjugation: $R\widehat{\omega^{\text{bd}}}R^\top = \widehat{R\omega^{\text{bd}}}$, a direct consequence of Proposition 3.46’s cross-product identity), i.e. the two differ by *rotating the vector itself by R* . They are equal only when R commutes with the relevant tangent direction, generally not the case.

Proposition 3.62 (The geodesic interpolant is naturally left-trivialised (body-frame)). *Let $R(\tau) = R_0 \exp(\tau\Xi)$ for fixed $R_0 \in SO(3)$, $\Xi \in \mathfrak{so}(3)$ (this is exactly the geodesic construction of Definition 12.20 below). Then $\dot{R}(\tau) = R(\tau)\Xi$, i.e. the curve has constant left-trivialised (body-frame) angular velocity $\omega^{\text{bd}} = \Xi^\vee$, in the sense of Definition 3.60.*

Proof. Differentiate $R(\tau) = R_0 \exp(\tau\Xi)$ directly: $\dot{R}(\tau) = R_0 \exp(\tau\Xi) \Xi = R(\tau) \Xi$, using that Ξ commutes with $\exp(\tau\Xi)$ (both are polynomials/power series in Ξ). This matches the first case of Definition 3.60 exactly, with $\omega^{\text{bd}}(\tau) \equiv \Xi^\vee$, constant in τ . $\square$

Proposition 3.63 (The corresponding one-step update is a right multiplication). *If a numerical scheme advances R by one Euler step using a body-frame (left-trivialised) velocity $v \in \mathbb{R}^3$ over a time increment dt , the update that remains exactly on $SO(3)$, consistent with Proposition 3.62, is $R_{\text{new}} = R \exp(dt\widehat{v})$ (multiplying the exponential on the right); the analogous world-frame (right-trivialised) update would instead be $R_{\text{new}} = \exp(dt\widehat{v})R$.*

Propositions 3.62 and 3.63 settle the question for this document: wherever SigmaFlow’s geodesic interpolant (Section 12.10) or its Euler integrator (Section 13.8) is discussed, the angular velocity supplied by, and returned to, the network is the **left-trivialised (body-frame)** angular velocity, consistent with the SigmaFlow source code’s own comment (`so3_flow_matcher.py`)

and with the update rule $R \mapsto R \exp(dt \hat{v})$ used by both `euler_step` and `expmap` in the actual implementation (both right-multiply the exponential onto R , so — once Definition 3.60’s labelling convention is fixed and applied uniformly — the two are in fact mutually consistent, and the apparent conflict flagged by the code-research pass resolves to a terminology mismatch between the source comment’s word choice and the existing thesis draft’s word choice, not a mathematical inconsistency in either). Chapter 10 performs the same check, independently, for SigmaDock’s diffusion construction, since diffusion’s Brownian-motion increments (Section 7.3) are built from `expmap/tangent_gaussian`, the same right-multiplying primitives, and the two chapters must therefore use the *same* trivialisation convention for their comparison in Chapter 13 to be meaningful.

Caution 3.64. This resolution is a derivation carried out for this document; it should not be taken to mean the original published SigmaDock description necessarily uses matching language, only that the code that actually runs is self-consistent under Definition 3.60. Where the distinction between ω^{sp} and ω^{bd} affects a formula materially (as opposed to being a book-keeping choice with no observable consequence, e.g. because a subsequent step immediately re-expresses the result in world frame), this document states which one is meant at the point of use, rather than relying on the reader to track it globally.

3.6 The pose group $SE(3)$ and the product manifold $SE(3)^N$

Definition 3.65 (Special Euclidean group). $SE(3)$ is the group of rigid motions of $\mathbb{R}^3$: as a set, $SE(3) \cong SO(3) \times \mathbb{R}^3$, with element $T = (R, x)$ acting on $y \in \mathbb{R}^3$ by $T \cdot y = Ry + x$, and group operation

$$(R_1, x_1) \circ (R_2, x_2) = (R_1 R_2, R_1 x_2 + x_1)$$

(composition of affine maps: apply T_2 then T_1).

Proposition 3.66. $SE(3)$ is a 6-dimensional Lie group, with identity $(I, 0)$ and inverse $(R, x)^{-1} = (R^\top, -R^\top x)$.

Proof. Immediate from Definition 3.65: $\dim SE(3) = \dim SO(3) + \dim \mathbb{R}^3 = 3 + 3 = 6$ as a manifold (a product of manifolds, Definition 3.1, has dimension the sum of the factors’ dimensions); associativity and the group axioms follow from associativity of composition of affine maps; smoothness of the group operations is immediate since Definition 3.65’s formula is polynomial in (R_1, x_1, R_2, x_2) . $\square$

Metric choice, and the weight nobody can avoid choosing. $SE(3)$ admits more than one natural Riemannian metric, and — unlike on $\mathbb{R}^3$ or on $SO(3)$ separately, where a normalisation constant is a harmless global scale — on a product the *relative* scale of the two factors is a real geometric decision. The reason is dimensional: a translation is measured in ångströms and a rotation in radians, and there is no canonical exchange rate between them. Any metric on $SE(3)$ must pick one.

We therefore write the general *product metric* with an explicit weight,

$$g_{(R,x)}^\lambda((\dot{R}_1, \dot{x}_1), (\dot{R}_2, \dot{x}_2)) = \langle \dot{x}_1, \dot{x}_2 \rangle_{\mathbb{R}^3} + \lambda \langle \dot{R}_1, \dot{R}_2 \rangle_{SO(3)}, \quad \lambda > 0, \quad (3.10)$$

so that, in the trivialised coordinates of Definition 3.60, a tangent vector $(v, \omega) \in \mathbb{R}^3 \times \mathbb{R}^3$ has squared norm

$$\|(v, \omega)\|_{g^\lambda}^2 = \|v\|_2^2 + \lambda \|\omega\|_2^2. \quad (3.11)$$

Dimensionally λ carries units of (length)² per (radian)²; choosing $\lambda = \ell^2$ for a characteristic length ℓ of the body being moved says “a rotation of one radian counts as much as a translation of ℓ ”, which is the sense in which the choice “reflects the physical size of the body”. We follow Yim et al. (2023, cited precisely in Chapter 13 where this choice is used) in taking $\lambda = 1$ as the default, but we keep λ visible because Proposition 3.67 below is the cleanest available explanation of what a rotational loss weight actually *is*.

Proposition 3.67 (What λ does, and what it does not do). *Let $\lambda > 0$ and consider $SE(3)$ with the metric (3.10). Then:*

- (i) *The **geodesics are the same curves** for every λ , and so are $\exp$, $\log$, and the geodesic interpolant of Corollary 3.22.*
- (ii) *The **Riemannian volume measure** changes only by the global constant factor $\lambda^{3/2}$, so normalised probability densities are unaffected.*
- (iii) *The **squared-error loss** does change: by (3.11),*

$$\|u_\theta - u\|_{g^\lambda}^2 = \underbrace{\|v_\theta - v\|_2^2}_{\text{translation}} + \lambda \underbrace{\|\omega_\theta - \omega\|_2^2}_{\text{rotation}},$$

i.e. a two-term loss with a relative weight λ on the rotational term is exactly the squared Riemannian norm of the product metric g^λ .

- (iv) *Nevertheless, the **exact minimiser over all measurable functions is independent of λ** : the two terms involve disjoint outputs, so each is minimised separately, and by Lemma 11.23 the optimum is the pair of conditional expectations regardless of the positive weight in front.*

Proof. (i) Scaling a metric by a positive constant does not change its geodesics: the length of every curve is multiplied by the same factor $\sqrt{\lambda}$, so the minimisation in (3.1), and the local condition of Definition 3.12, have the same solutions. For a product metric the geodesic equation decouples (see Proposition 3.69 below), so scaling one factor leaves both factors’ geodesics alone.

(ii) In coordinates adapted to the product, g^λ is block diagonal with blocks I_3 and $\lambda g_{SO(3)}$, so $\det g^\lambda = \lambda^3 \det g^1$ and $\sqrt{|g^\lambda|} = \lambda^{3/2} \sqrt{|g^1|}$, a constant multiple; by Definition 3.28 the volume measure is scaled by that constant, and a density normalised to integrate to 1 absorbs it.

(iii) is (3.11) written out.

(iv) The loss is $\mathbb{E}\|v_\theta - v\|^2 + \lambda \mathbb{E}\|\omega_\theta - \omega\|^2$; the first sum depends only on the network’s translational output and the second only on its rotational output, so an unconstrained minimisation splits into two independent problems, each solved by the corresponding conditional expectation (Lemma 11.23), with λ multiplying a term whose minimiser it cannot move. $\square$

Caution 3.68 (“Loss weight = metric choice” — true, and less powerful than it sounds). Proposition 3.67(iii) is a genuine and useful identification: a rotational loss weight is not an arbitrary knob bolted onto an otherwise geometric method, it *is* the statement that the pose manifold carries the metric g^λ . That reframing is worth having, because it says the weight belongs to the same family of modelling decisions as the choice of source distribution or interpolant, not to the same family as the learning rate.

But Proposition 3.67(iv) bounds how much can be concluded from it. At the population optimum, over an unconstrained function class, λ changes nothing at all: it neither moves the target field, nor the path, nor the minimiser. Its entire effect is on the *approximation*: with finite capacity, finite data, and a finite optimisation budget, a larger λ buys rotational

accuracy at the cost of translational accuracy. So λ is simultaneously “a metric” and “a hyperparameter”, and neither description licenses ignoring the other. Chapter 15 returns to this with the measured scales of the two terms in the actual SigmaFlow objective, where the two are *not* commensurable and the inherited weight is one of the audit’s open findings.

Proposition 3.69 ($SE(3)$ with the product metric is a Riemannian product). *With (3.10), the exponential and logarithmic maps, geodesics, and (once defined in Chapter 6) Brownian motion on $SE(3)$ all factorise componentwise:*

$$\exp_{(R,x)}(\dot{R}, \dot{x}) = (\exp_R(\dot{R}), x + \dot{x}), \quad \log_{(R,x)}(R', x') = (\log_R(R'), x' - x).$$

Proof sketch. A Riemannian product $(\mathcal{M}_1 \times \mathcal{M}_2, g_1 \oplus g_2)$ has geodesics exactly the curves (γ_1, γ_2) with γ_i a geodesic of $(\mathcal{M}_i, g_i)$ (since the geodesic equation, derived from the metric via the Euler–Lagrange equations for the length functional, decouples exactly when the metric does, as (3.10) does by construction), and $\mathbb{R}^3$ with the flat Euclidean metric has $\exp_x(v) = x + v$, $\log_x(y) = y - x$ trivially. Combining these two facts with $SO(3)$ ’s own maps (Sections 3.4.3 and 3.4.4) gives the claim. $\square$

Proposition 3.69 is, computationally, the single most consequential fact in this chapter: it licenses treating translation and rotation *completely independently* in every later construction (the conditional probability paths of Chapters 6 and 11, the network’s two output heads of Chapters 10 and 13), with no cross term anywhere, provided the metric (3.10) — rather than some other, coupled metric — is the one actually in force. Chapter 10 verifies, directly against the code, that this decoupled treatment is indeed what is implemented (no coupling term appears in either SE3Diffuser/SE3_FlowMatcher, both of which are pure componentwise dispatchers, per the code-research findings recorded there).

Definition 3.70 (The pose manifold of N rigid fragments). The state space of a docking model that represents a ligand as N independently posable rigid fragments (Section 10.1) is the product manifold

$$\mathcal{M}_N = SE(3)^N, \quad \mathbf{T} = (R^{(1)}, x^{(1)}, \dots, R^{(N)}, x^{(N)}),$$

again with the product metric, extending (3.10) fragmentwise.

By the same reasoning as Proposition 3.69 applied N times, every construction on $\mathcal{M}_N$ factorises over fragments *and* over rotation/translation within each fragment: the entire generative model, seen purely geometrically, is $2N$ independent copies of the two constructions this document develops in full generality — one Euclidean ($\mathbb{R}^3$, for each fragment’s translation) and one $SO(3)$ (for each fragment’s rotation) — coupled *only* through the neural network that predicts the vector field/score, never through the geometry itself. This is exactly the content of the “inter-fragment correlations enter only through the learned model” design point discussed in Section 10.1.

3.7 The torsion space $\mathbb{T}^k$

The remaining internal degree of freedom identified in Section 1.2, the dihedral angle, lives on a space we have not yet defined formally.

Definition 3.71 (Circle group and torus). $\mathbb{T} := \mathbb{R}/2\pi\mathbb{Z}$, the quotient of $\mathbb{R}$ by the equivalence $\phi \sim \phi + 2\pi m$ ($m \in \mathbb{Z}$), with group operation addition mod 2π ; equivalently, $\mathbb{T} \cong S^1 = \{(\cos \phi, \sin \phi) : \phi \in \mathbb{R}\} \subset \mathbb{R}^2$ via $\phi \mapsto (\cos \phi, \sin \phi)$, an isomorphism of Lie groups (the group law on the right is complex multiplication of unit-modulus complex numbers, matching addition of angles). $\mathbb{T}^k$ is the k -fold direct product, an abelian Lie group of dimension k .

Being abelian, $\mathbb{T}^k$'s Lie algebra is simply $\mathbb{R}^k$ with the identity map as exponential ($\exp(\phi) = \phi \bmod 2\pi$ in each coordinate), and the natural (bi-invariant, since the group is abelian and hence trivially so) metric is flat Euclidean — *locally*. The one genuinely new phenomenon $\mathbb{T}^k$ introduces, absent from $\mathbb{R}^k$, is that *differences* of angles are only defined up to the 2π -periodicity, and there are two equally valid representatives differing by a full turn; the convention used throughout is to take the *shortest* representative.

Definition 3.72 (Periodic (shortest-path) angular difference). For $\phi_1, \phi_2 \in \mathbb{T}$, the geodesic distance is $d_{\mathbb{T}}(\phi_1, \phi_2) = |((\phi_2 - \phi_1 + \pi) \bmod 2\pi) - \pi| \in [0, \pi]$, and the logarithmic map at ϕ_1 is the signed version, $\log_{\phi_1}(\phi_2) = ((\phi_2 - \phi_1 + \pi) \bmod 2\pi) - \pi \in (-\pi, \pi]$, the unique representative of $\phi_2 - \phi_1 \pmod{2\pi}$ lying in $(-\pi, \pi]$.

Remark 3.73. SigmaDock and SigmaFlow, as reconstructed in Chapters 10 and 13 from the codebase, do **not** generate torsion angles as an explicit degree of freedom at all: the fragment-based parameterisation of Section 10.1 treats each rigid fragment's *internal* torsions as fixed at their reference-conformer values (Section 10.2), and only the N copies of $SE(3)$ — never an explicit $\mathbb{T}^k$ factor — are generated. This chapter nonetheless develops $\mathbb{T}^k$ in full, for two reasons stated precisely and not merely asserted: first, understanding *why* SigmaDock avoids torsional generation requires the volume-form argument of Section 10.1, which itself requires comparing the torus construction against the fragment construction on equal footing; second, the torsion angle ϕ_{ijkl} of Section 1.2 is exactly the periodic dihedral variable this section formalises, and it reappears, undiffused and ungenerated but still present as a fixed input, throughout Chapters 10 and 13's discussion of what a rigid fragment template actually is.

Remark 3.74 (Parity and chirality). We noted in Definition 3.45 that $O(3) \supsetneq SO(3)$ additionally contains orientation-*reversing* maps (reflections, $\det = -1$). Molecules are, in general, chiral: a molecule and its mirror image can be distinct chemical species (different smell, different biological activity, etc.), so a docking model must not be invariant under reflection — a network that could not distinguish a ligand from its mirror image would place both equally well into a pocket that in reality binds only one of them. This is why Chapter 9 tracks a *parity* label alongside the degree ℓ once it introduces representations of $O(3)$ rather than only of $SO(3)$: a rotationally-equivariant-only architecture that additionally happened to be reflection-invariant would be a strictly worse inductive bias for this task, not a neutral simplification.

3.8 What you should now be able to do

- explain why a point and a tangent vector are different kinds of object, and what breaks if they are conflated;
- read the geodesic equation $\nabla_{\dot{\gamma}} \dot{\gamma} = 0$ as “all of the acceleration is spent staying on the manifold”, and derive constant speed from it;
- use $\exp_x$ and $\log_x$ as the replacements for $x + v$ and $y - x$, and say exactly where they stop being inverse to each other (cut locus, Definition 3.23);
- derive $T_R SO(3) = \{R\hat{\omega}\}$, Rodrigues' formula, and the matrix logarithm, and handle the two dangerous angles 0 and π ;

- distinguish body-frame from spatial-frame angular velocity (Definition 3.60) and say which one a given formula uses;
- write down the Riemannian volume measure, gradient and divergence, and check each against its Euclidean special case.

Chapter 4 now asks the complementary question: not how to do calculus on rotations, but how to build a *function* that respects them.

Chapter 4

Steerable Representations: Spherical Harmonics, Wigner Matrices, and Clebsch–Gordan Coefficients

Chapter 3 equipped us with $SO(3)$ itself; this chapter equips us with the machinery to build *features that transform predictably under $SO(3)$* — scalars, ordinary 3-vectors, and their higher-degree analogues — and to combine such features while preserving that predictability. This is not optional scaffolding for the neural network of Chapter 9: it *is* the architecture, in the precise sense that once the objects of this chapter are defined, the network’s layers will turn out to be almost forced, as Chapter 9 shows in detail. Everything here is representation theory of $SO(3)$, developed only as far as needed and always tied back to 3-dimensional physical intuition (directions, angular momentum) rather than presented abstractly first.

4.1 The problem this chapter exists to solve

Before any representation theory, we should be completely clear about the problem. It is a physical statement, not a mathematical one, and everything technical in this chapter is downstream of it.

4.1.1 Rotating the whole system changes nothing physical

Take a protein pocket with a ligand docked in it. Now pick up the entire assembly — protein *and* ligand together — and rotate it by some $Q \in SO(3)$ about the origin. Every atom moves: $x_i \mapsto Qx_i$ for all i , protein and ligand alike.

Has anything physical changed? No. The binding energy is the same. Every interatomic distance is the same. Whether the pose is correct is the same. We have merely chosen to look at the same physical configuration from a different angle. There is no privileged orientation of space; the laws of chemistry do not know which way is “up”.

That single observation constrains what our model is allowed to be, and the constraint has two distinct flavours depending on *what kind of thing* the model outputs.

4.1.2 Two different kinds of “unchanged”

Some quantities should not change at all. The binding affinity of a pose is a number. Rotate the system and you must get the same number. A quantity like this is called *invariant*.

Other quantities must change, in lockstep. Suppose the model outputs, for a ligand fragment, the direction it should be nudged in — a vector $v \in \mathbb{R}^3$. Now rotate the whole input by Q . The correct answer is no longer v : the fragment now sits in a rotated pocket, and the direction it should move in is the rotated direction Qv . If the model still returned v , it would be pointing the wrong way. A quantity like this is called *equivariant*: it must *co-vary* with the input.

The two are not competing options; they are two cases of one principle. The principle is: *the model must not depend on our arbitrary choice of coordinate frame*. Which case applies depends on whether the output is the kind of object that has a direction.

Quantity	Behaviour under a global rotation Q	Type
Atomic number, formal charge	unchanged	invariant
Interatomic distance $\ x_i - x_j\ $	unchanged	invariant
Binding affinity, energy	unchanged	invariant
RMSD to a reference pose	unchanged	invariant
Atom position x_i	$x_i \mapsto Qx_i$	equivariant
Relative vector $x_{ij} = x_j - x_i$	$x_{ij} \mapsto Qx_{ij}$	equivariant
Force on an atom	$F_i \mapsto QF_i$	equivariant
Translational velocity output v	$v \mapsto Qv$	equivariant
Angular velocity output ω	$\omega \mapsto Q\omega$	equivariant
Fragment orientation R	$R \mapsto QR$	equivariant

Table 4.1: Invariant and equivariant quantities in docking. Note that distance is invariant while the vector it is the length of is equivariant: taking the norm is precisely an operation that *converts* equivariant data into invariant data. This will turn out to be the fundamental move in every equivariant architecture.

Caution 4.1 (Invariance is not “better” than equivariance). A common first reaction is: “invariance sounds safer — can we not just make everything invariant?” No, and the reason is decisive for docking. A model whose *every* internal quantity is invariant can only ever output invariant quantities, because there is no way to manufacture a direction out of numbers that carry none. Such a model can predict a binding *affinity*. It cannot predict a binding *pose*: it has no mechanism to say “move that way”.

SigmaDock and SigmaFlow must output $\omega, v \in \mathbb{R}^3$ per fragment (Equation (8.1)). Those are equivariant objects. So the network is *required* to carry directional information internally, and the entire difficulty of the architecture is carrying it *correctly*.

4.1.3 Why an ordinary network does not do this by itself

Suppose we simply feed the raw coordinates $x_1, \dots, x_N \in \mathbb{R}^3$ into a multilayer perceptron. Concatenate them into one vector of length $3N$ and apply $h \mapsto \sigma(Wh + b)$.

Rotate the input. The new input is $Qx_1, \dots, Qx_N$. Is the output the rotated old output? Almost never. Two independent reasons:

1. **The linear map does not commute with the rotation.** For the first layer to be equivariant we would need $W(Q \oplus \dots \oplus Q)h = (Q \oplus \dots \oplus Q)Wh$ for *every* $Q \in SO(3)$, i.e. W must commute with the entire rotation group acting block-wise. Generic weight matrices do not; the set that does is a very thin subspace, and nothing in ordinary training pushes W into it.
2. **The nonlinearity destroys it even if the linear part survived.** Applying ReLU componentwise to a 3-vector is not compatible with rotation: $\text{ReLU}(Qv) \neq Q \text{ReLU}(v)$ in general. We return to this with an explicit counterexample in Example 4.14.

So equivariance is not something a generic architecture has and can lose. It is something a generic architecture *lacks* and must be built to have.

4.1.4 Why data augmentation is not a substitute

The obvious cheap fix is to train on many randomly rotated copies of each example, hoping the network learns the symmetry. This helps, and it is what many models do, but it is strictly weaker than building the symmetry in. Four reasons, in increasing order of importance for us:

1. **It only ever holds approximately.** Augmentation encourages equivariance on the data distribution; it guarantees nothing off it, and nothing exactly. An architecturally equivariant model satisfies $f(Q \cdot x) = Q \cdot f(x)$ for every Q and every x , exactly, at initialisation, before any training.
2. **It spends capacity.** The network must devote parameters to memorising “these $|SO(3)|$ -many inputs are the same input”. Those parameters are not available for chemistry. Equivalently: the effective size of the training set is reduced, because the model must relearn each example in every orientation.
3. **It is a 3-parameter group.** Augmenting over a 3-dimensional continuous group by sampling is far less effective than augmenting over, say, a discrete flip. Coverage is thin.
4. **The error mode is invisible and pose-shaped.** A non-equivariant docking model can produce a pose that is correct up to an unnoticed frame error — and a systematically mis-rotated pose is exactly the failure mode that is hardest to detect from aggregate loss curves and most damaging to RMSD. This document’s own development history contains an instance of precisely this class of bug (Section 13.4), found only by a targeted diagnostic.

Why this matters for SigmaFlow

SigmaFlow replaces SigmaDock’s generative engine and keeps its backbone. The backbone is exactly the machinery of this chapter and Chapter 9. So this material is *not* part of what the thesis changes — but it is part of what the thesis must not *break*. When we later verify that a modification preserves equivariance (Definition 8.1, and the per-quantity table in Section 13.4), the property being checked is the one defined below in Definition 4.9.

4.2 Group actions: how a group moves things

Section 3.3 defined a group as a set with a composition law. That is a statement about the group *in isolation*. What we actually need is the group *doing something* to another set — rotations acting on points, on molecules, on features. That is a group action.

The problem. We have $SO(3)$, and we have several different kinds of object: single points in $\mathbb{R}^3$; whole molecules, which are tuples of points; and later, abstract feature vectors inside a network. “Rotate it” means something different in each case. We need one piece of vocabulary that covers all of them and states precisely what the common structure is.

Definition 4.2 (Group action). Let G be a group and X a set. A (left) *action* of G on X is a map

$$\cdot : G \times X \rightarrow X, \quad (g, x) \mapsto g \cdot x,$$

satisfying two axioms:

(A1) **Identity.** $e \cdot x = x$ for all $x \in X$, where $e \in G$ is the identity.

(A2) **Compatibility.** $g_1 \cdot (g_2 \cdot x) = (g_1 g_2) \cdot x$ for all $g_1, g_2 \in G, x \in X$.

Both axioms say something one would want to be true and which is easy to violate by accident. (A1) says “doing nothing does nothing”. (A2) says “rotating by g_2 and then by g_1 is the same as rotating once by the composite $g_1 g_2$ ” — the action must respect the group’s own composition law, which is the whole reason we insisted G was a group in the first place.

Caution 4.3 ($g \cdot x$ need not be matrix multiplication). The notation $g \cdot x$ is deliberately neutral. In our first example it *is* a matrix times a vector. In the next one it is a matrix applied to each of N vectors. Later, when X is a space of *functions* on the sphere, the action will be $(Q \cdot f)(\hat{x}) = f(Q^{-1}\hat{x})$ — note the inverse, which is forced by (A2) and is a classic source of sign-and-direction errors (Caution 4.24). And when $X = SO(3)$ itself, the action is left multiplication $Q \cdot R = QR$, where the “point” being acted on is itself a rotation. Writing all of these as $g \cdot x$ is what lets one definition of equivariance cover all of them.

Example 4.4 (Rotations acting on the plane). $G = SO(2)$, $X = \mathbb{R}^2$, and $g \cdot x = R_\vartheta x$ with $R_\vartheta = \begin{pmatrix} \cos \vartheta & -\sin \vartheta \\ \sin \vartheta & \cos \vartheta \end{pmatrix}$. (A1): $R_0 = I$. (A2): $R_{\vartheta_1} R_{\vartheta_2} = R_{\vartheta_1 + \vartheta_2}$, so rotating twice is rotating by the sum — exactly the group law of $SO(2)$. This is the smallest example in which the axioms are not vacuous.

Example 4.5 (Rotations acting on $\mathbb{R}^3$). $G = SO(3)$, $X = \mathbb{R}^3$, $Q \cdot x = Qx$. Axioms are immediate from matrix associativity and $Ix = x$. This action is what the word “rotation” means operationally.

Example 4.6 (Rotations acting on a whole molecule). This is the one we actually use. Let $X = (\mathbb{R}^3)^N = \{(x_1, \dots, x_N)\}$, the set of all configurations of N atoms, and define

$$Q \cdot (x_1, \dots, x_N) := (Qx_1, \dots, Qx_N),$$

i.e. the *same* Q applied to every atom simultaneously. Checking the axioms is componentwise and immediate.

Two things are worth noticing. First, this is a genuinely different action from “rotate each atom by its own rotation” — the latter is an action of the much bigger group $SO(3)^N$, and is not a symmetry of chemistry (it would tear the molecule apart). The physical symmetry is the *diagonal* action of a single $SO(3)$. Second, the action does not change N , the atom types, or the bonds: it moves only positions. Those other data are along for the ride, which is exactly why they end up as $\ell = 0$ features later.

Remark 4.7 (Translations, and why we largely ignore them). The full symmetry of free space is $SE(3)$: rotations *and* translations. Translations are handled by a trick so standard it is easy to miss: the architecture never sees absolute positions, only *relative* vectors $x_{ij} = x_j - x_i$, which satisfy $(x_j + a) - (x_i + a) = x_j - x_i$ and are therefore translation-invariant automatically. Once positions enter only through differences, translation symmetry is exact and free, and the remaining, genuinely difficult symmetry is $SO(3)$. That is why this chapter is about $SO(3)$ and not $SE(3)$; the translational half is discharged by Section 8.4’s choice of inputs, and SigmaDock additionally centres each pocket at the origin (Section 10.1).

4.3 Invariance and equivariance, formally

We can now state the property we want a network to have. The subtlety worth slowing down for is that a function has *two* sets — a domain and a codomain — and the group acts on each of them, possibly differently.

Definition 4.8 (Invariance). Let G act on X . A function $f : X \rightarrow Y$ is *G -invariant* if

$$f(g \cdot x) = f(x) \quad \text{for all } g \in G, x \in X.$$

No action on Y is needed or used.

Definition 4.9 (Equivariance). Let G act on X *and* on Y (the two actions may be different; we write both as $\cdot$). A function $f : X \rightarrow Y$ is *G -equivariant* if

$$f(g \cdot x) = g \cdot f(x) \quad \text{for all } g \in G, x \in X.$$

Read the equation slowly, because both sides describe a two-step recipe and the content of the definition is that the two recipes agree:

Left-hand side	first <i>transform the input</i> , then apply the model
Right-hand side	first apply the model, then <i>transform the output</i>

Equivalently, the following square commutes:

$$\begin{array}{ccc} X & \xrightarrow{f} & Y \\ \downarrow g \cdot & & \downarrow g \cdot \\ X & \xrightarrow{f} & Y \end{array}$$

“Commutates” means: both paths from the top-left to the bottom-right give the same answer. Go right then down, or down then right — same result.

Remark 4.10 (Invariance is a special case of equivariance). Take the action on Y to be trivial, $g \cdot y := y$ for all g . Then Definition 4.9 reads $f(g \cdot x) = f(x)$, which is Definition 4.8. So there is really only one concept, applied with two different choices of what the group does to the output. This is worth internalising, because it is why the same representation-theoretic machinery handles both, and why $\ell = 0$ features (Example 4.19) are exactly the invariant ones.

Example 4.11 (Distance is invariant, the vector is equivariant). Let $X = (\mathbb{R}^3)^2$ with the diagonal action of Example 4.6.

- $f(x_1, x_2) = \|x_2 - x_1\| \in \mathbb{R}$. Then $f(Qx_1, Qx_2) = \|Qx_2 - Qx_1\| = \|Q(x_2 - x_1)\| = \|x_2 - x_1\| = f(x_1, x_2)$, using Proposition 3.46. **Invariant.**
- $f(x_1, x_2) = x_2 - x_1 \in \mathbb{R}^3$, with $Y = \mathbb{R}^3$ carrying the action $Q \cdot y = Qy$. Then $f(Qx_1, Qx_2) = Qx_2 - Qx_1 = Q(x_2 - x_1) = Q \cdot f(x_1, x_2)$. **Equivariant.**

The second computation used nothing but linearity of Q . That is the prototype of every equivariance proof in this document: push the group element through the formula and check it comes out the other side.

Example 4.12 (A function that is neither). Let $f(x_1, x_2) = (x_2 - x_1)_z$, the z -component of the relative vector. Rotating by a Q that tilts the z -axis changes this number, so f is not invariant; and $\mathbb{R}$ carries no rotation action making it equivariant either. Reading off a fixed Cartesian component is exactly the kind of innocuous-looking operation that silently breaks equivariance, and it is worth having a name for the failure: it *privileges a coordinate axis*, and there is no privileged axis.

Proposition 4.13 (Compositions inherit equivariance). *If $f : X \rightarrow Y$ and $h : Y \rightarrow Z$ are both G -equivariant, so is $h \circ f : X \rightarrow Z$. If f is equivariant and h is invariant, then $h \circ f$ is invariant.*

Proof. $(h \circ f)(g \cdot x) = h(f(g \cdot x)) = h(g \cdot f(x)) = g \cdot h(f(x))$, using equivariance of f then of h . For the second claim, replace the last step by invariance of h . $\square$

Example 4.14 (Componentwise ReLU is not equivariant). This is the counterexample promised in Section 4.1, and it is worth doing with actual numbers because it is the single most common way a well-intentioned architecture silently loses equivariance.

Let $f : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ be $f(v) = \text{ReLU}(v) = (\max(v_1, 0), \max(v_2, 0), \max(v_3, 0))$, with $SO(3)$ acting by $Q \cdot v = Qv$ on both sides. Take

$$v = \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix}, \quad Q = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (\text{a } +90^\circ \text{ rotation about the } z\text{-axis}).$$

Then $Qv = (1, 1, 0)^\top$, so

$$f(Q \cdot v) = \text{ReLU} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad Q \cdot f(v) = Q \text{ReLU} \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix} = Q \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}.$$

These differ. Equivariance fails, and it fails badly — the two answers are not even close.

Why it fails is more instructive than *that* it fails. ReLU acts on each Cartesian component separately, so it treats the coordinate axes as meaningful. But the decomposition of a vector into (v_1, v_2, v_3) is an artefact of the frame we chose; a rotation mixes those components, and any operation that treats them as independent is comparing things that are not comparable. This is the same failure mode as Example 4.11’s “read off the z -component”, in a different costume.

The repair, developed in Section 4.10, is to apply nonlinearities only to quantities that carry no direction — e.g. to $\|v\|$, which is invariant, and then rescale v by the result: $v \mapsto \sigma(\|v\|)v$ is equivariant, because $\sigma(\|Qv\|)Qv = \sigma(\|v\|)Qv = Q(\sigma(\|v\|)v)$.

Proposition 4.13 is small but it is the reason the whole enterprise is feasible: we do not have to verify equivariance of a 20-layer network as one monolithic claim. We verify it *once per*

layer type, and composition does the rest. Every architectural decision in Chapter 9 is, in the end, a decision about which layer types are on the permitted list.

Why this matters for SigmaFlow

Equation (8.1) says the network maps a pose to a pair (ω, v) of 3-vectors per fragment. In the language just introduced: the domain carries the diagonal $SO(3)$ action on atom positions, the codomain carries the action $(\omega, v) \mapsto (Q\omega, Qv)$, and the network must be equivariant between them. Every layer we are allowed to use is a layer for which we can prove that statement.

4.4 Why representation theory, and not something simpler

We now have the goal (Definition 4.9) but not yet the means. Here is the gap.

To say that a network layer is equivariant, we must be able to say how the group acts on its *inputs* and on its *outputs*. For the very first layer, the input is a set of positions, and we know the action: $x \mapsto Qx$. For the very last layer, the output is (ω, v) , and we know that action too. But in between sit *hidden features* — vectors of numbers with no a priori geometric meaning, produced by the previous layer. Nothing tells us how a rotation acts on those.

And yet we must decide, because Definition 4.9 is meaningless without an action on both sides. The resolution is to turn this from a discovery problem into a *design* problem:

We will not *find out* how hidden features transform. We will *declare* how they transform, choosing the rule as part of the architecture, and then build only layers that respect the declaration.

A choice of “how a vector space of features transforms under G ” is precisely what representation theory formalises. Two constraints narrow the choice almost completely:

1. **The rule must be linear.** If rotating the input scrambled features nonlinearly, no linear layer could ever commute with it. Linearity is what makes the bookkeeping tractable, and it costs nothing, because all the actions we actually need are linear.
2. **The rule must respect composition.** Applying the rule for Q_1 after the rule for Q_2 must equal the rule for Q_1Q_2 — otherwise it is not consistent with how rotations compose, i.e. with axiom (A2) of Definition 4.2.

A map assigning to each group element a linear transformation, compatibly with composition, is exactly Definition 4.15 below. Representation theory then answers the follow-up question that makes the whole design finite: *which such rules are there?* For $SO(3)$ the answer is remarkably clean — a single infinite list indexed by $\ell = 0, 1, 2, \dots$, from which every other rule is assembled (Theorem 4.18). Designing an equivariant architecture then reduces to choosing how many features of each type ℓ to carry, and using only operations that map declared types to declared types.

4.5 Representations of $SO(3)$

Motivation. A scalar (say, an atom’s partial charge) does not change when the whole molecule is rotated: it is the same number before and after. A 3-vector attached to an atom (say, the direction to its nearest neighbour) *does* change: rotating the molecule by Q rotates the vector by Q too. Both are examples of the same general phenomenon — a feature associated with a rotation-dependent transformation rule — and we now formalise the full family of such transformation rules that $SO(3)$ admits.

Definition 4.15 (Representation). A (finite-dimensional, real) *representation* of $SO(3)$ is a smooth map $D : SO(3) \rightarrow GL(V)$, for some real vector space V , satisfying $D(Q_1 Q_2) = D(Q_1)D(Q_2)$ for all $Q_1, Q_2 \in SO(3)$ (a group homomorphism into the invertible linear maps of V).

The homomorphism property is exactly what makes D a consistent transformation rule: rotating first by Q_2 then by Q_1 has the same effect on a feature as rotating once by the composite $Q_1 Q_2$, matching how physical rotations actually compose.

Example 4.16. $V = \mathbb{R}$, $D(Q) = 1$ for all Q (the *trivial* representation): this is how scalars transform. $V = \mathbb{R}^3$, $D(Q) = Q$ (the *standard* or *defining* representation): this is how ordinary vectors transform. Both are representations of $SO(3)$ in the sense of Definition 4.15; direct sums (block-diagonal concatenations) and tensor products of representations are again representations (immediate from the definition, checking the homomorphism property component/factor-wise).

Definition 4.17 (Irreducible representation). A representation D on V is *irreducible* (an *irrep*) if V has no proper, nonzero subspace $W \subsetneq V$ (other than $\{0\}$ and V itself) with $D(Q)W \subseteq W$ for every $Q \in SO(3)$.

Irreducibility is the representation-theoretic analogue of “cannot be decomposed further”: if V *did* have such an invariant subspace W , the action of $SO(3)$ on V would really just be two smaller, independent actions (on W and, in a suitable sense, on a complement), and nothing would be lost by treating them separately. The classification of all irreps of $SO(3)$ is the single structural fact this chapter is building towards, and it is exactly what tells the network designer of Chapter 9 which “types” of feature exist at all.

Theorem 4.18 (Classification of the irreps of $SO(3)$). *The (real, finite-dimensional) irreducible representations of $SO(3)$ are indexed by a non-negative integer $\ell = 0, 1, 2, \dots$, called the degree; the irrep of degree ℓ acts on a vector space $V_\ell \cong \mathbb{R}^{2\ell+1}$, and every finite-dimensional representation of $SO(3)$ decomposes as a direct sum of copies of these.*

We do not prove Theorem 4.18 from scratch (it is a classical result, most naturally proved via the representation theory of the associated complex Lie algebra $\mathfrak{so}(3, \mathbb{C}) \cong \mathfrak{su}(2)$ and its weight theory, requiring substantially more Lie-algebra machinery than the rest of this document uses; see, e.g., Hall, *Lie Groups, Lie Algebras, and Representations*, 2nd ed., 2015, Ch. 4). We instead *construct* the degree- ℓ irrep concretely and explicitly in Section 4.6, by exhibiting V_ℓ as a specific, checkable space of functions on the sphere and verifying irreducibility is unnecessary for our purposes once the construction is in hand — what we need is the transformation rule (4.1), the dimension count $2\ell + 1$, and the tensor-product decomposition of Theorem 4.34, all of which Sections 4.6 and 4.9 derive directly.

We record the two cases familiar from Section 2.6 onward once and for all in the new language.

Example 4.19 ($\ell = 0$ and $\ell = 1$). $V_0 \cong \mathbb{R}$ with $D^0(Q) = 1$: type-0 features are scalars/invariants. $V_1 \cong \mathbb{R}^3$ with $D^1(Q) = Q$ (the standard representation): type-1 features are ordinary 3-vectors. Every feature encountered so far in this document — atomic charges, distances (type 0); relative position vectors, the translational and rotational velocity outputs of (8.1) (type 1) — is one of these two cases; $\ell \geq 2$ is new content, introduced precisely because it lets the network represent *angular* patterns (e.g. “the neighbours lie in a ring around this axis”) that neither scalars nor single vectors can express, as Chapter 9 discusses concretely.

4.6 Spherical harmonics

Motivation. We want an explicit, computable realisation of each degree- ℓ irrep — not just the abstract knowledge that it exists and has dimension $2\ell + 1$, but an actual formula that can be evaluated on a direction $\hat{x} \in S^2$ and fed into a neural network. Spherical harmonics provide exactly this, and the cleanest way to see why they arise is by analogy with a construction the reader may already know: Fourier series on the circle.

Intuition: from the circle to the sphere. A periodic function on $\mathbb{T}$ (Definition 3.71) decomposes into Fourier modes $e^{im\phi}$, $m \in \mathbb{Z}$; the mode $e^{im\phi}$ transforms under rotation of the circle by α (i.e. $\phi \mapsto \phi + \alpha$) by an overall phase, $e^{im(\phi+\alpha)} = e^{im\alpha}e^{im\phi}$ — a 1-dimensional representation of the rotation group of the circle, $SO(2)$, one for each integer m . Spherical harmonics are the exact analogue for functions on the sphere S^2 and the rotation group $SO(3)$: instead of a single integer m indexing 1-dimensional pieces, we get a degree ℓ indexing $(2\ell + 1)$ -dimensional pieces (the “ m ” index survives too, now ranging over $-\ell, \dots, \ell$ within a fixed ℓ , and mixing among themselves under a rotation, rather than each being separately invariant as on the circle — exactly the passage from the abelian $SO(2)$ to the non-abelian $SO(3)$).

Definition 4.20 (Spherical harmonics via harmonic homogeneous polynomials). For $\ell \geq 0$, let $\mathcal{H}_\ell$ be the space of *harmonic* (satisfying Laplace’s equation $\Delta p = 0$) *homogeneous polynomials* of degree ℓ in three real variables (x_1, x_2, x_3) . The (real) *spherical harmonics of degree ℓ* are the restrictions of $\mathcal{H}_\ell$ to the unit sphere S^2 .

Proposition 4.21 (Dimension count). $\dim \mathcal{H}_\ell = 2\ell + 1$.

Proof. The space of *all* homogeneous degree- ℓ polynomials in 3 variables has dimension $\binom{\ell+2}{2}$ (the number of monomials $x_1^a x_2^b x_3^c$ with $a + b + c = \ell$). The Laplacian Δ maps this space onto the space of homogeneous degree- $(\ell - 2)$ polynomials (surjectivity is a short, standard computation: given a target degree- $(\ell - 2)$ polynomial q , one directly exhibits a degree- ℓ polynomial p with $\Delta p = q$, e.g. by induction on ℓ using that $\Delta(\|x\|^2 q) = 2(2\ell + 1)q + \|x\|^2 \Delta q$ for homogeneous q of degree $\ell - 2$, which for $\ell \geq 2$ can be solved for q in terms of a suitable preimage), and its kernel is exactly $\mathcal{H}_\ell$. By rank-nullity,

$$\dim \mathcal{H}_\ell = \binom{\ell+2}{2} - \binom{\ell}{2} = \frac{(\ell+2)(\ell+1)}{2} - \frac{\ell(\ell-1)}{2} = 2\ell + 1.$$

□

Example 4.22 ($\ell = 0, 1$). $\mathcal{H}_0 = \text{span}\{1\}$ (constants; $\Delta 1 = 0$ trivially), restricting to the constant function on S^2 , matching $\dim = 1 = 2(0) + 1$. $\mathcal{H}_1 = \text{span}\{x_1, x_2, x_3\}$ (linear functions are automatically harmonic, since Δ of a linear function is 0), restricting to the three coordinate functions on S^2 , matching $\dim = 3 = 2(1) + 1$; note that restricted to S^2 , these are literally the components of the unit vector $\hat{x}$ itself.

Proposition 4.23 (Transformation rule of spherical harmonics). *Let $Q \in SO(3)$ act on functions $f : S^2 \rightarrow \mathbb{R}$ by $(Q \cdot f)(\hat{x}) = f(Q^\top \hat{x})$ (the natural action: the rotated function evaluated at $\hat{x}$ equals the original function evaluated at the pre-rotation direction). Then this action preserves $\mathcal{H}_\ell$ (restricted to S^2) for each ℓ ; consequently, in any basis $Y_{-\ell}^\ell, \dots, Y_\ell^\ell$ of degree- ℓ spherical harmonics, there is a matrix $D^\ell(Q) \in \mathbb{R}^{(2\ell+1) \times (2\ell+1)}$ with*

$$\boxed{Y^\ell(Q\hat{x}) = D^\ell(Q)Y^\ell(\hat{x})} \quad \text{i.e.} \quad Y_m^\ell(Q\hat{x}) = \sum_{m'} D^\ell(Q)_{mm'} Y_{m'}^\ell(\hat{x}), \quad (4.1)$$

and D^ℓ is a representation of $SO(3)$ in the sense of Definition 4.15, of dimension $2\ell + 1$, matching Theorem 4.18's degree- ℓ irrep. In the standard orthonormal real basis, $D^\ell(Q)$ is an orthogonal matrix, so $D^\ell(Q)^{-1} = D^\ell(Q)^\top = D^\ell(Q^\top)$.

Caution 4.24 (Rotating the point versus rotating the function: the inverse). (4.1) is stated with Q applied to the *point* $\hat{x}$. One also frequently meets the companion statement with Q applied to the *function*. The two differ by an inverse, and conflating them is a standard and hard-to-see error. Substituting $\hat{x} \mapsto Q^\top \hat{x}$ into (4.1) gives

$$Y^\ell(\hat{x}) = D^\ell(Q)Y^\ell(Q^\top \hat{x}) \quad \Longleftrightarrow \quad Y^\ell(Q^\top \hat{x}) = D^\ell(Q)^{-1}Y^\ell(\hat{x}) = D^\ell(Q^\top)Y^\ell(\hat{x}),$$

not $D^\ell(Q)Y^\ell(\hat{x})$. A numerical check settles it immediately: for $\ell = 1$ with $Y^1(\hat{x}) = \hat{x}$, the claim $Y^1(Q^\top \hat{x}) = D^1(Q)Y^1(\hat{x})$ reads $Q^\top \hat{x} = Q\hat{x}$, which holds only for symmetric Q .

The reason the inverse appears is Caution 4.3: when a group acts on a space of *functions*, the action must be $(Q \cdot f)(\hat{x}) := f(Q^{-1}\hat{x})$. Without the inverse, axiom (A2) of Definition 4.2 fails — one would get $(Q_1 \cdot (Q_2 \cdot f)) = (Q_2 Q_1) \cdot f$, the composition in the wrong order. The inverse is what restores $(Q_1 Q_2) \cdot f = Q_1 \cdot (Q_2 \cdot f)$.

Convention fixed for this document: we always use the boxed form, $Y^\ell(Q\hat{x}) = D^\ell(Q)Y^\ell(\hat{x})$, i.e. D^ℓ is the matrix that implements a rotation *of the direction argument*. This is the convention of `e3nn` and hence of `EquiformerV2`, so no translation is needed when the code is read in Chapter 9. It is consistent with $D^1(Q) = Q$ (Example 4.19); a document using the opposite convention would have $D^1(Q) = Q^\top$ there, and every subsequent formula would carry a transpose.

Proof. If $p \in \mathcal{H}_\ell$ (a harmonic homogeneous degree- ℓ polynomial on $\mathbb{R}^3$), then $p \circ Q^\top$ (i.e. $x \mapsto p(Q^\top x)$) is again homogeneous of degree ℓ (composing with a linear map preserves homogeneity), and harmonic (the Laplacian is invariant under orthogonal changes of coordinates: $\Delta(p \circ Q^\top) = (\Delta p) \circ Q^\top$ for Q orthogonal, a standard fact provable by the chain rule and $Q^\top Q = I$), so $p \circ Q^\top \in \mathcal{H}_\ell$ too: the action preserves $\mathcal{H}_\ell$. Since the action is linear on the finite-dimensional space $\mathcal{H}_\ell$, expressing $Q \cdot Y_m^\ell$ in the chosen basis gives exactly a matrix $D^\ell(Q)$ as claimed, and $D^\ell(Q_1 Q_2) = D^\ell(Q_1)D^\ell(Q_2)$ follows because the action of Q on functions is itself a group action (i.e. $(Q_1 Q_2) \cdot f = Q_1 \cdot (Q_2 \cdot f)$, a direct check from the definition), which is exactly the homomorphism property required by Definition 4.15. $\square$

We do not additionally re-derive *irreducibility* of D^ℓ here (it follows from the classical theory cited after Theorem 4.18, and the identification $V_\ell \cong \mathcal{H}_\ell$ is the standard one); what Proposition 4.23 already gives us — an explicit, computable, correctly-transforming basis for every degree ℓ — is everything the rest of this document needs.

Code correspondence

Mathematical object: $Y^\ell : S^2 \rightarrow \mathbb{R}^{2\ell+1}$, degree- ℓ real spherical harmonics; the equivariant edge embedding $(\psi(d_{ij}), Y^\ell(\hat{x}_{ij}))_{\ell \leq L}$

Implementation: `net/edge_rot_mat.py` (per-edge frame alignment, see Chapter 9) together with the `e3nn` library’s spherical-harmonics routines, called from within the $SO(2)$ -convolution machinery of Section 9.10 rather than as an explicit, separately-visible tensor-product step (a structural point this document flags precisely, not glossed over, in Section 9.10)

Interpretation: in EquiformerV2, $L = \text{lmax} = 3$ by default (Chapter 9), so the code works with degrees $\ell \in \{0, 1, 2, 3\}$, i.e. steerable features of dimension $1 + 3 + 5 + 7 = 16$ per channel.

4.6.1 Explicit low-degree formulas, and the connection to familiar operations

We record the degree 0, 1, 2 spherical harmonics explicitly (up to the normalisation convention, which varies across sources and is immaterial to every argument in this document, since it only rescales each ℓ -block by a constant that a subsequent learned linear layer can absorb):

$$Y^0 = 1, \quad Y^1 = (x_1, x_2, x_3) = \hat{x}, \quad Y^2 \propto (x_1x_2, x_2x_3, 2x_3^2 - x_1^2 - x_2^2, x_3x_1, x_1^2 - x_2^2),$$

the last being (up to normalisation and a choice of basis) the five independent components of the traceless symmetric matrix $\hat{x}\hat{x}^\top - \frac{1}{3}I$, i.e. exactly a (traceless) quadrupole moment of the direction $\hat{x}$. This connects the abstract degree ℓ directly to physical/geometric intuition: $\ell = 0$ answers “is there something here” (an unsigned scalar); $\ell = 1$ answers “in what single direction” (a vector, capable of representing one preferred direction); $\ell = 2$ answers “along what *axis*, ignoring sign” (a nematic/quadrupolar pattern, e.g. distinguishing a planar ring of neighbours in the xy -plane from an isotropic distribution, something no single vector can encode, since the ring itself has no preferred in-plane direction and Y^1 averaged over the ring would cancel to zero while Y^2 would not); higher ℓ resolve correspondingly finer angular structure.

4.7 Wigner- D matrices

Proposition 4.23 already *is* the definition of the Wigner- D matrix: $D^\ell(Q)$ is, by construction, the matrix realising $SO(3)$ ’s action on degree- ℓ spherical harmonics in a chosen basis. We isolate the properties of D^ℓ that are used repeatedly.

Proposition 4.25 (Basic properties of Wigner- D matrices). *For every ℓ : (i) $D^\ell(I) = I_{2\ell+1}$; (ii) $D^\ell(Q_1Q_2) = D^\ell(Q_1)D^\ell(Q_2)$ (Proposition 4.23); (iii) $D^\ell(Q)$ is orthogonal, $D^\ell(Q)^\top D^\ell(Q) = I$ (since the action of Proposition 4.23 preserves the natural $L^2(S^2)$ inner product, under which a suitable choice of real spherical-harmonic basis is orthonormal); (iv) $D^0(Q) = 1$ for all Q (the trivial representation, as in Example 4.19); $D^1(Q) = Q$ itself, in the standard Cartesian basis $Y^1 = \hat{x}$ of Example 4.22.*

Proof. (i) and (ii) restate Proposition 4.23. (iii): the action $f \mapsto f \circ Q^\top$ preserves the rotation-invariant surface measure on S^2 , hence preserves the $L^2(S^2)$ inner product $\langle f, g \rangle = \int_{S^2} fg \, d\Omega$; choosing Y_m^ℓ orthonormal in this inner product (always possible by Gram–Schmidt within the finite-dimensional space $\mathcal{H}_\ell|_{S^2}$) makes $D^\ell(Q)$ an isometry of $\mathbb{R}^{2\ell+1}$ with the standard

inner product, i.e. orthogonal. (iv): $D^0(Q) = 1$ since $Y^0 \equiv 1$ is fixed pointwise by every rotation of its argument; $D^1(Q) = Q$ is immediate from Equation (4.1) with $Y^1(\hat{x}) = \hat{x}$: $Y^1(Q\hat{x}) = Q\hat{x} = QY^1(\hat{x})$. $\square$

Proposition 4.25(iv) is worth dwelling on: it says that $SO(3)$'s *defining* representation (rotation matrices themselves, which is how we have described $SO(3)$ throughout Chapter 3) is nothing but the degree-1 Wigner matrix. Every subsequent degree $\ell \geq 2$ is a genuinely new object, not reducible to ordinary matrix multiplication by a rotation matrix, and this is exactly why Chapter 9's architecture cannot simply reuse the familiar 3×3 rotation-matrix machinery once $\ell \geq 2$ features are involved — it needs the general $D^\ell(Q)$.

Code correspondence

Mathematical object: $D^\ell(Q) \in \mathbb{R}^{(2\ell+1) \times (2\ell+1)}$, the per-edge rotation applied to steerable features

Implementation: `net/wigner.py` (function assembling D^ℓ block-diagonally from Euler angles, via `e3nn`), called from `net/edge_rot_mat.py::RotationToWignerDMatrix`

Tensor shape: one D^ℓ per edge, per degree $\ell \leq L$, assembled into a single block-diagonal matrix of size $\sum_{\ell=0}^L (2\ell+1)$ acting on the full multi-degree feature at once

Interpretation: every edge in the molecular graph gets its own per-edge frame (Chapter 9, Section 9.3), built from the edge direction $\hat{x}_{ij}$; the corresponding Wigner- D matrix rotates a neighbour's steerable feature *into* that edge-aligned frame before any further processing, and the inverse rotates the result back — exactly the mechanism, detailed in Chapter 9, by which EquiformerV2 reduces full $SO(3)$ -equivariant processing to a computationally cheaper $SO(2)$ -equivariant one.

4.8 The anatomy of a steerable feature

We now have the list of available feature *types* ($\ell = 0, 1, 2, \dots$) and, for each, the matrix $D^\ell(Q)$ that says how it rotates. What we do not yet have is a picture of what a feature actually looks like inside a network. This section supplies it, because every shape in Chapter 9 and every tensor in the code is an instance of it.

4.8.1 One feature, four indices

The central object is

$$h_{i,c,m}^{(\ell)}$$

a single real number carrying four indices. Each index means something different, and — this is the point — *they behave completely differently under rotation*. Taking them one at a time:

Index	Range	Meaning	Behaviour under a rotation Q
i	$1, \dots, N$	node (atom)	unchanged: rotating the molecule does not renumber atoms
ℓ	$0, 1, \dots, L$	irrep type	unchanged: a type-1 feature stays type-1
c	$1, \dots, C_\ell$	channel	unchanged: channels never mix under rotation
m	$-\ell, \dots, +\ell$	component within the irrep	mixed: this is the only index a rotation touches

Everything about equivariant architectures follows from the last row. A rotation acts on exactly one of the four axes. Concretely, for fixed i , ℓ and c , the $(2\ell + 1)$ numbers

$$h_{i,c}^{(\ell)} := (h_{i,c,-\ell}^{(\ell)}, \dots, h_{i,c,+\ell}^{(\ell)})^\top \in \mathbb{R}^{2\ell+1}$$

form one *irreducible block*, and it transforms as

$$h_{i,c}^{(\ell)} \mapsto D^\ell(Q) h_{i,c}^{(\ell)} \quad \text{for every } i \text{ and every } c \text{ separately.} \quad (4.2)$$

The same matrix $D^\ell(Q)$ is applied to every channel, independently. No information passes between channels, between nodes, or between degrees when the molecule is rotated.

4.8.2 Why 16 channels of $\ell = 1$ is not a 48-dimensional vector

This is the distinction worth being pedantic about, because both objects contain the same count of real numbers and they are nonetheless mathematically different.

Suppose $C_1 = 16$ channels of type $\ell = 1$. The total number of stored reals is $16 \times (2 \cdot 1 + 1) = 48$. Now compare two candidate transformation rules for that block of 48 numbers:

A generic 48-vector	16 channels of $\ell = 1$
transforms by <i>some</i> 48×48 matrix	transforms by $I_{16} \otimes D^1(Q)$
$48^2 = 2304$ free entries	determined by the single 3×3 matrix Q
any two of the 48 slots may mix	only slots within one channel mix
no notion of “direction”	each channel <i>is</i> a direction in $\mathbb{R}^3$

The block-diagonal form is worth writing out. Stack the channels into one column $h_i^{(1)} \in \mathbb{R}^{48}$ in the order $(c = 1, m = -1), (c = 1, m = 0), (c = 1, m = +1), (c = 2, m = -1), \dots$. Then

$$h_i^{(1)} \mapsto (I_{16} \otimes D^1(Q)) h_i^{(1)} = \begin{pmatrix} D^1(Q) & & \\ & \ddots & \\ & & D^1(Q) \end{pmatrix} h_i^{(1)}, \quad (4.3)$$

16 identical 3×3 blocks down the diagonal and zeros elsewhere. The notation $I_C \otimes D^\ell(Q)$ is just a compact name for this block-diagonal matrix: the Kronecker product $A \otimes B$ replaces each entry a_{jk} of A by the block $a_{jk}B$, so $I_{16} \otimes D^1(Q)$ is 16 copies of $D^1(Q)$ on the diagonal. Read it as: “do nothing to the channel index, apply $D^1(Q)$ to the m index”.

Caution 4.26 (The ordering convention is a real choice). Whether the 48 numbers are laid out channel-major (c slowest, m fastest, as above, giving $I_C \otimes D^\ell$) or m -major (m slowest, c fastest, giving $D^\ell \otimes I_C$) is an implementation decision, not mathematics. Both describe the same transformation; the matrices differ by a permutation. Getting it wrong produces a tensor of the right *shape* that is silently wrong, which is why Chapter 9 records the actual memory layout used by EquiformerV2 rather than assuming one. Nothing in *this* chapter depends on the choice.

4.8.3 What the channel index is for

A natural objection: “if $\ell = 1$ means ‘a vector’, why do we need 16 of them? A point in space only has one direction to it.”

The answer is that a channel is not a coordinate; it is a *learned feature that happens to be directional*. Consider what a single atom might usefully want to represent:

- the direction towards the densest nearby region of protein,
- the direction along which its ring system extends,
- the direction in which a hydrogen-bond donor points,
- the direction it is currently being pushed by the flow field.

Each of these is a separate piece of information, and each is directional — so each needs its own $\ell = 1$ channel. Sixteen channels means the atom carries sixteen independent directional facts about itself. They are all 3-vectors, they all rotate the same way, and they mean different things.

Where the CNN analogy holds, and where it breaks. In a convolutional network, “16 channels” likewise means 16 independent learned quantities per spatial location, and a 1×1 convolution mixes them freely. That much carries over: a linear layer here may also mix the 16 channels freely, with a learned 16×16 (or $C_{\text{out}} \times 16$) matrix.

The analogy breaks in one specific way. In a CNN the channel index is the *only* feature index, so a linear layer may mix everything with everything. Here there is a second index, m , and mixing across it is forbidden — a layer that combined $h_{c=1,m=0}$ with $h_{c=2,m=+1}$ using arbitrary weights would not commute with $I_C \otimes D^\ell(Q)$ and would destroy equivariance. So:

Mix channels freely; never mix m by hand. Whatever happens to the m index must be done by an operation that representation theory certifies — which, as Section 4.9 shows, essentially means the Clebsch–Gordan product.

That sentence is the informal content of Schur’s lemma (Proposition 4.46), which we prove in Section 4.10 once the tensor-product machinery is in place.

4.8.4 A concrete feature specification

Architectures specify their feature content as a list of (multiplicity, degree) pairs, written in e3nn notation as e.g.

"32x0e + 16x1o + 8x2e".

Read: 32 channels of $\ell = 0$, 16 of $\ell = 1$, 8 of $\ell = 2$. (The trailing **e/o** is *parity*, relevant for $O(3)$ rather than $SO(3)$; we discuss it only where it matters, in Remark 3.74, and it plays no role in this chapter.) The total stored numbers per node are

$$32 \times 1 + 16 \times 3 + 8 \times 5 = 32 + 48 + 40 = 120,$$

and under a rotation these 120 numbers transform by the block-diagonal matrix

$$\underbrace{I_{32} \otimes D^0(Q)}_{32 \times 32, = I_{32}} \oplus \underbrace{I_{16} \otimes D^1(Q)}_{48 \times 48} \oplus \underbrace{I_8 \otimes D^2(Q)}_{40 \times 40}.$$

Note that the 32 scalar channels are genuinely untouched ($D^0(Q) = 1$), which is the precise sense in which $\ell = 0$ features are the invariant ones.

Why this matters for SigmaFlow

Two things you will need when reading code.

First, *shapes*. A node-feature tensor in EquiformerV2 is not $[N, F]$ for a flat feature count F ; it is indexed by node, channel and m , with the degrees stacked. When Chapter 9 reports a shape like $[N, (L+1)^2, C]$, the middle axis is the m -index of *all* degrees concatenated — because $\sum_{\ell=0}^L (2\ell+1) = (L+1)^2$. Recognising which axis is m and which is c is the difference between reading the code and guessing at it.

Second, *where the output comes from*. SigmaFlow’s network must emit $\omega, v \in \mathbb{R}^3$ per fragment. Those are $\ell = 1$ objects. So the final readout takes $\ell = 1$ channels and reduces them to one channel — it cannot manufacture them from $\ell = 0$ scalars, and it cannot read them off $\ell = 2$ features without a Clebsch–Gordan step. The architecture’s `lmax` must therefore be at least 1, and the reason is this paragraph, not a hyperparameter sweep.

4.9 Tensor products and Clebsch–Gordan coefficients

Motivation. Message passing (Section 8.4) needs to *combine* two features — e.g. a neighbour’s current steerable feature $h_j^{\ell_1}$ and the spherical-harmonic embedding $Y^{\ell_2}(\hat{x}_{ij})$ of the edge direction — into a new feature. For ordinary (type-0, scalar) features, “combine” just means multiply. For steerable features of possibly different, positive degree, an arbitrary bilinear combination will *not* in general transform correctly under rotation; we need the *specific* bilinear combinations that do.

4.9.1 Starting from two ordinary vectors

Before any ℓ ’s, take the most familiar case: two ordinary 3-vectors $u, v \in \mathbb{R}^3$ attached to an atom. Under a global rotation they transform as $u \mapsto Qu$ and $v \mapsto Qv$. We want to build something new out of them. The question is what is available.

Two products we already know. There are two standard ways to multiply vectors in $\mathbb{R}^3$, and both turn out to be equivariant — for reasons we can check in one line each.

Dot product. $u^\top v \in \mathbb{R}$, a single number. Under rotation,

$$(Qu)^\top (Qv) = u^\top Q^\top Qv = u^\top v,$$

using $Q^\top Q = I$. The output does not change: this is an **invariant**, i.e. an $\ell = 0$ output. One number.

Cross product. $u \times v \in \mathbb{R}^3$. Under rotation, by Proposition 3.46,

$$(Qu) \times (Qv) = Q(u \times v).$$

The output rotates like a vector: an $\ell = 1$ output. Three numbers.

The question that forces tensor products. We have made 1 number and 3 numbers out of a pair of vectors. But a pair of vectors is $3 + 3 = 6$ input numbers, and *all products* $u_i v_j$ number $3 \times 3 = 9$. Neither the dot product nor the cross product uses all nine; together they account for $1 + 3 = 4$. So:

Where are the other five?

Answering that question is the whole content of this section, and the answer — an object with 5 components, which is $2\ell + 1$ for $\ell = 2$ — is where higher-degree features come from. They are not an exotic addition to the theory; they are the part of an ordinary product of two vectors that dot and cross throw away.

4.9.2 The outer product and how it rotates

Definition 4.27 (Outer product). For $u, v \in \mathbb{R}^3$, their *outer product* is the 3×3 matrix

$$T = u \otimes v := uv^\top, \quad T_{jk} = u_j v_k.$$

As a list of numbers it has 9 components: every product of one component of u with one component of v . It retains strictly more information than $u^\top v$ or $u \times v$ — indeed, it retains everything about the pair that is bilinear.

Proposition 4.28 (Transformation of the outer product). *If $u \mapsto Qu$ and $v \mapsto Qv$, then*

$$T = uv^\top \mapsto (Qu)(Qv)^\top = Quv^\top Q^\top = QTQ^\top.$$

Proof. $(Qu)(Qv)^\top = Quv^\top Q^\top$, since $(Qv)^\top = v^\top Q^\top$. □

So the outer product carries the rotation on *both sides*: $T \mapsto QTQ^\top$. Written with indices, using the summation convention, $T'_{jk} = Q_{ja} Q_{kb} T_{ab}$ — two copies of Q , one per index. Viewing T instead as a single column of 9 numbers, this reads

$$\text{vec}(T) \mapsto (Q \otimes Q) \text{vec}(T),$$

where $Q \otimes Q$ is the 9×9 Kronecker product. That 9×9 matrix is what Definition 4.30 below calls the tensor product representation $D^1 \otimes D^1$.

Caution 4.29 (Three different “products”, kept apart).

Operation	Output	Dimension	Rotation rule
$u^\top v$ (dot)	scalar	1	unchanged
$u \times v$ (cross)	vector	3	$\mapsto Q(u \times v)$
$u \otimes v$ (outer/tensor)	matrix	9	$\mapsto Q(uv^\top)Q^\top$

The tensor product is the only one of the three that loses nothing; the other two are what you get after discarding part of it. That is the sense in which the tensor product is the *primitive* operation and dot/cross are derived.

Definition 4.30 (Tensor product representation). Given representations D^{ℓ_1} on V_{ℓ_1} and D^{ℓ_2} on V_{ℓ_2} , the *tensor product representation* acts on $V_{\ell_1} \otimes V_{\ell_2}$ by $(D^{\ell_1} \otimes D^{\ell_2})(Q)(u \otimes v) = D^{\ell_1}(Q)u \otimes D^{\ell_2}(Q)v$. Its dimension is $(2\ell_1 + 1)(2\ell_2 + 1)$.

Why this is a representation. The homomorphism property is inherited factor-wise: $(D^{\ell_1} \otimes D^{\ell_2})(Q_1 Q_2)(u \otimes v) = D^{\ell_1}(Q_1 Q_2)u \otimes D^{\ell_2}(Q_1 Q_2)v = D^{\ell_1}(Q_1)D^{\ell_1}(Q_2)u \otimes D^{\ell_2}(Q_1)D^{\ell_2}(Q_2)v$, which is $(D^{\ell_1} \otimes D^{\ell_2})(Q_1)$ applied to $(D^{\ell_1} \otimes D^{\ell_2})(Q_2)(u \otimes v)$. $\square$

4.9.3 Decomposing $1 \otimes 1$ completely

We now answer the “where are the other five?” question in full. The tool is a decomposition of 3×3 matrices that is pure linear algebra, and which turns out to be exactly the representation-theoretic decomposition.

Step 1: split any matrix into three pieces. For any $T \in \mathbb{R}^{3 \times 3}$ write

$$T = \underbrace{\frac{\text{tr } T}{3} I}_S + \underbrace{\frac{T - T^\top}{2}}_A + \underbrace{\left(\frac{T + T^\top}{2} - \frac{\text{tr } T}{3} I \right)}_Z. \quad (4.4)$$

That this sums back to T is immediate: the two halves $\frac{T - T^\top}{2} + \frac{T + T^\top}{2}$ already give T , and the $\frac{\text{tr } T}{3} I$ terms cancel. The three pieces are the *isotropic* part, the *antisymmetric* part, and the *symmetric traceless* part.

Step 2: count dimensions.

Piece	Constraint	Free parameters	Count
$S = \lambda I$	one scalar λ	λ	1
A , antisymmetric	$A^\top = -A$	strict upper triangle	3
Z , symmetric traceless	$Z^\top = Z$, $\text{tr } Z = 0$	$\frac{3 \cdot 4}{2} - 1$	5
			9

The symmetric 3×3 matrices form a space of dimension $\binom{3+1}{2} = 6$; imposing the single linear condition $\text{tr } Z = 0$ removes one, leaving $6 - 1 = 5$. And $1 + 3 + 5 = 9 = \dim(\mathbb{R}^{3 \times 3})$: the split (4.4) is a direct-sum decomposition of the whole space, with no overlap and nothing missing.

Step 3: each piece is closed under $T \mapsto QTQ^\top$. This is the crucial step — it is what makes the split *representation-theoretic* rather than merely algebraic.

Proposition 4.31 (The three pieces are invariant subspaces). *Let $T \mapsto T' = QTQ^\top$ for $Q \in SO(3)$. Then*

- (i) $\text{tr } T' = \text{tr } T$; hence the isotropic part maps to the isotropic part, and its single parameter is unchanged.
- (ii) If $A^\top = -A$ then $(QAQ^\top)^\top = -QAQ^\top$: antisymmetric matrices map to antisymmetric matrices.
- (iii) If $Z^\top = Z$ and $\text{tr } Z = 0$ then $QZQ^\top$ is again symmetric and traceless.

Proof. (i) $\text{tr}(QTQ^\top) = \text{tr}(TQ^\top Q) = \text{tr} T$ by cyclicity of the trace and $Q^\top Q = I$. (ii) $(QAT)^\top = QA^\top T^\top = -QAQ^\top$. (iii) symmetry is the same computation with $+$; tracelessness is (i). $\square$

So the 9-dimensional space splits into three subspaces of dimensions 1, 3, 5, each of which the rotation action maps into itself. By Definition 4.17, this says the 9-dimensional representation is *reducible* — and it exhibits the pieces explicitly.

Step 4: identify each piece as a known irrep. *The isotropic part is $\ell = 0$.* Its content is the single number $\text{tr} T$, and by (i) that number is unchanged by rotation. An invariant one-dimensional feature is precisely the trivial representation (Example 4.19), i.e. $\ell = 0$. For $T = uv^\top$ we have $\text{tr} T = \sum_j u_j v_j = u^\top v$: **the isotropic part is the dot product**, recovered exactly.

The antisymmetric part is $\ell = 1$. A 3×3 antisymmetric matrix is determined by three numbers, and Definition 3.49 gives the canonical identification: $A = \widehat{a}$ for a unique $a = A^\vee \in \mathbb{R}^3$. The question is whether a transforms as a vector. It does, and the reason is the conjugation identity already proved in Section 3.4.2:

$$Q\widehat{a}Q^\top = \widehat{Qa} \implies (QAT)^\vee = QA^\vee.$$

That is exactly the $\ell = 1$ transformation rule $a \mapsto Qa$. So the antisymmetric part *is* a vector in disguise, and the disguise is the hat map.

For $T = uv^\top$ we can be fully explicit. With our hat/vee convention (Definition 3.49),

$$(T - T^\top)^\vee = -(u \times v), \quad \text{so} \quad A^\vee = \left(\frac{T - T^\top}{2} \right)^\vee = -\frac{1}{2} (u \times v). \quad (4.5)$$

The antisymmetric part is the cross product, up to the factor $-\frac{1}{2}$.

Caution 4.32 (The sign in (4.5) is convention-dependent). The minus sign is not a typo and not universal: it is forced by the vee convention of Definition 3.49, under which $\widehat{\omega}v = \omega \times v$. Checking one component settles it: with $T = uv^\top$, the $(3, 2)$ entry of $T - T^\top$ is $u_3v_2 - u_2v_3 = -(u \times v)_1$, and $(\cdot)^\vee$ reads that entry off as the first component. Texts that define $T = vu^\top$, or that use the opposite vee sign, obtain $+\frac{1}{2}$. The *content* — that the antisymmetric part is the cross product up to a nonzero constant — is convention-independent; only the constant is not. Since every use in this document passes through a learned weight (Section 4.9.8), the constant is absorbed and never matters numerically; it matters only if one tries to compare formulas across sources.

The symmetric traceless part is $\ell = 2$. This is the five-dimensional piece we were missing. Three arguments identify it:

- (a) Its dimension is $5 = 2 \cdot 2 + 1$, matching $\ell = 2$ and no other irrep.
- (b) It is invariant under the action (Proposition 4.31(iii)), so it is *some* representation of dimension 5.
- (c) By Theorem 4.18, every representation is a sum of irreps; the only ways to build dimension 5 from $\{1, 3, 5, 7, \dots\}$ are 5, or $3 + 1 + 1$, or 1×5 . One checks it is not further reducible — equivalently, it contains no rotation-invariant direction — so it is the single irrep V_2 .

Intuitively, a symmetric traceless matrix encodes an *axis with a magnitude but no arrow*: its eigenvectors give preferred directions, but Z and the “flipped” configuration are not distinguished the way a vector distinguishes a from $-a$. This is exactly the kind of object

needed to express “the neighbours lie in a plane” or “this bond is aligned with that ring” — statements about orientation that are not statements about a direction. Physically, the moment-of-inertia tensor and the quadrupole moment are $\ell = 2$ objects for the same reason.

Conclusion. Putting the four steps together,

$$\underbrace{V_1 \otimes V_1}_{\dim 9} \cong \underbrace{V_0}_{\dim 1} \oplus \underbrace{V_1}_{\dim 3} \oplus \underbrace{V_2}_{\dim 5} \quad \text{i.e.} \quad 1 \otimes 1 = 0 \oplus 1 \oplus 2, \quad (4.6)$$

and the three summands are, concretely, the dot product, the cross product, and the symmetric traceless part. The five “missing” numbers from Section 4.9.1 are the $\ell = 2$ component.

Remark 4.33 (This is what “reducible” means, made concrete). Definition 4.17 defined irreducibility abstractly: no proper nonzero invariant subspace. We have now *exhibited* three such subspaces inside $V_1 \otimes V_1$. The tensor product representation is therefore reducible, and the Clebsch–Gordan decomposition is nothing more mysterious than the systematic version of the split (4.4): it names the invariant subspaces and supplies the change of basis onto them. Everything below is this example, generalised.

This is again a representation of $SO(3)$ (immediate check of the homomorphism property), but — for $\ell_1, \ell_2 \geq 1$ — it is generally *not* irreducible: $\dim(V_{\ell_1} \otimes V_{\ell_2}) = (2\ell_1 + 1)(2\ell_2 + 1)$, which for e.g. $\ell_1 = \ell_2 = 1$ is 9, not matching any single irrep dimension $2\ell + 1$ (odd numbers 1, 3, 5, 7, $\dots$, and $9 = 2(4) + 1$ is odd, but we will see explicitly that the 9-dimensional space splits into *three* pieces, of dimensions $1 + 3 + 5 = 9$, not one piece of dimension 9). By Theorem 4.18, every representation — including this one — decomposes as a direct sum of irreps; the following theorem says precisely which ones appear.

Theorem 4.34 (Clebsch–Gordan decomposition).

$$V_{\ell_1} \otimes V_{\ell_2} \cong \bigoplus_{\ell=|\ell_1-\ell_2|}^{\ell_1+\ell_2} V_{\ell}.$$

Proof sketch, and a dimension check. We do not derive Theorem 4.34 from first principles (the standard proof uses the weight/character theory of $\mathfrak{so}(3, \mathbb{C}) \cong \mathfrak{su}(2)$ cited after Theorem 4.18: the weights of $V_{\ell_1} \otimes V_{\ell_2}$ are all sums $m_1 + m_2$ with $m_i \in \{-\ell_i, \dots, \ell_i\}$, and grouping these sums by which range of consecutive integers they fill out recovers exactly the stated range of ℓ ; see Hall (2015), Thm. 4.44 for $\mathfrak{su}(2)$, transported to $SO(3)$ via the double cover). We record instead the dimension identity that this decomposition must, and does, satisfy, as a consistency check the reader can verify unaided:

$$\sum_{\ell=|\ell_1-\ell_2|}^{\ell_1+\ell_2} (2\ell + 1) = (2\ell_1 + 1)(2\ell_2 + 1),$$

provable by writing the left side as an arithmetic series (first term $2|\ell_1 - \ell_2| + 1$, last term $2(\ell_1 + \ell_2) + 1$, number of terms $\ell_1 + \ell_2 - |\ell_1 - \ell_2| + 1$, which for $\ell_1 \geq \ell_2$ equals $2\ell_2 + 1$) and simplifying; for $\ell_1 = \ell_2 = 1$ this gives $1 + 3 + 5 = 9 = 3 \times 3$, confirming the example above. $\square$

4.9.4 The selection rule as a type system

Theorem 4.34 deserves to be read not as a formula but as a *rule about what is allowed*. It says: if you combine a feature of type ℓ_1 with a feature of type ℓ_2 , the result can contain components of type ℓ only for

$$|\ell_1 - \ell_2| \leq \ell \leq \ell_1 + \ell_2,$$

and each such ℓ appears exactly once. Every other type is simply unavailable: there is no rotation-respecting bilinear map producing it.

$\ell_1 \otimes \ell_2$	allowed ℓ	dimension check	note
$0 \otimes 0$	0	$1 \cdot 1 = 1$	scalar times scalar
$0 \otimes \ell$	ℓ only	$1 \cdot (2\ell + 1) = 2\ell + 1$	scaling by an invariant
$1 \otimes 1$	0, 1, 2	$3 \cdot 3 = 1 + 3 + 5 = 9$	Section 4.9.3
$1 \otimes 2$	1, 2, 3	$3 \cdot 5 = 3 + 5 + 7 = 15$	
$2 \otimes 2$	0, 1, 2, 3, 4	$5 \cdot 5 = 1 + 3 + 5 + 7 + 9 = 25$	
$1 \otimes 3$	2, 3, 4	$3 \cdot 7 = 5 + 7 + 9 = 21$	note $\ell = 0, 1$ <i>not</i> allowed

The row $1 \otimes 3$ is instructive: the lower limit $|\ell_1 - \ell_2| = 2$ excludes $\ell = 0$ and $\ell = 1$ entirely. You cannot make a scalar out of a vector and an $\ell = 3$ feature. Not “it is hard”; there is no such bilinear map.

A useful way to think about it

Representation theory here behaves like a **type system** for geometric features. Each feature carries a type ℓ ; the tensor product is the only multiplication; and the selection rule is the typing rule stating which output types a given pair of input types can produce. A layer that claims to produce an ℓ outside the allowed range is not inaccurate — it is ill-typed, and cannot be equivariant.

Two consequences worth noting. *Composability*: because each allowed ℓ appears exactly once, there is no ambiguity about “which” $\ell = 1$ output is meant. *Reachability*: starting from $\ell = 0$ and $\ell = 1$ features only, repeated tensor products can reach every ℓ — e.g. $1 \otimes 1 \ni 2$, then $1 \otimes 2 \ni 3$, and so on — which is why an architecture with a finite $\mathbf{lmax}$ is a genuine modelling choice rather than a mathematical necessity.

4.9.5 Clebsch–Gordan coefficients are a change of basis

We have established *that* $V_{\ell_1} \otimes V_{\ell_2}$ splits. What remains is to write the splitting down in coordinates, and that is all the Clebsch–Gordan coefficients are. There is nothing else in them.

Two bases for one space. The space $V_{\ell_1} \otimes V_{\ell_2}$ has dimension $(2\ell_1 + 1)(2\ell_2 + 1)$, and we have two natural bases for it.

The product basis. Take a basis $\{e_{m_1}^{\ell_1}\}$ of V_{ℓ_1} and $\{e_{m_2}^{\ell_2}\}$ of V_{ℓ_2} and form all pairs:

$$e_{m_1}^{\ell_1} \otimes e_{m_2}^{\ell_2}, \quad m_1 = -\ell_1, \dots, \ell_1, \quad m_2 = -\ell_2, \dots, \ell_2.$$

This is the basis in which the raw product is easy to compute — coordinates are just the products $u_{m_1}v_{m_2}$ — but in which the decomposition is invisible.

The irreducible (coupled) basis. Alternatively, take a basis adapted to the decomposition Theorem 4.34: for each allowed ℓ , a basis $\{f_m^\ell\}$, $m = -\ell, \dots, \ell$, of the corresponding subspace. Counting, $\sum_\ell (2\ell + 1) = (2\ell_1 + 1)(2\ell_2 + 1)$: the same dimension, as it must be. This is the basis in which the decomposition is manifest but the product is not.

The coefficients are the transition matrix. Two bases of one finite-dimensional space are related by an invertible change-of-basis matrix. Its entries are the Clebsch–Gordan coefficients:

$$f_m^\ell = \sum_{m_1, m_2} C_{(\ell_1, m_1)(\ell_2, m_2)}^{(\ell, m)} e_{m_1}^{\ell_1} \otimes e_{m_2}^{\ell_2}.$$

That is the entire conceptual content. They are not a physical constant, not something learned, and not something the network chooses: they are the numerical entries of a change of basis, fixed once $SO(3)$ and the basis conventions are fixed.

Remark 4.35 (We have already computed one, by hand). The split (4.4) is a change of basis from the product basis of $V_1 \otimes V_1$ (the nine entries $T_{jk} = u_j v_k$) to a basis adapted to $V_0 \oplus V_1 \oplus V_2$ (one trace coordinate, three antisymmetric coordinates, five symmetric-traceless coordinates). The Clebsch–Gordan coefficients for $\ell_1 = \ell_2 = 1$ are precisely the 9×9 matrix implementing that split, written in the m -basis rather than the Cartesian jk -basis.

So the reader who followed Section 4.9.3 has already done a Clebsch–Gordan decomposition; the general theory adds systematic bookkeeping and a convention for the m -labels, not a new idea.

Definition 4.36 (Clebsch–Gordan coefficients and tensor product). The isomorphism of Theorem 4.34, written out in coordinates, defines real numbers $C_{(\ell_1, m_1)(\ell_2, m_2)}^{(\ell, m)}$ (the *Clebsch–Gordan coefficients*, uniquely determined up to an overall sign/phase convention per (ℓ_1, ℓ_2, ℓ) triple by orthonormality of the bases involved) such that, for $u \in V_{\ell_1}$, $v \in V_{\ell_2}$, the degree- ℓ component of their *Clebsch–Gordan tensor product* is

$$(u \otimes_{\text{cg}} v)_m^\ell = \sum_{m_1=-\ell_1}^{\ell_1} \sum_{m_2=-\ell_2}^{\ell_2} C_{(\ell_1, m_1)(\ell_2, m_2)}^{(\ell, m)} u_{m_1}^{\ell_1} v_{m_2}^{\ell_2}, \quad |\ell_1 - \ell_2| \leq \ell \leq \ell_1 + \ell_2. \quad (4.7)$$

4.9.6 Reading (4.7) index by index

The formula is short and every symbol in it matters. Unpacked:

Symbol	Lives in	Meaning
$u_{m_1}^{\ell_1}$	$\mathbb{R}$	one component of the first input; the input as a whole is $u^{\ell_1} \in \mathbb{R}^{2\ell_1+1}$
$y_{m_2}^{\ell_2}$	$\mathbb{R}$	one component of the second input, $y^{\ell_2} \in \mathbb{R}^{2\ell_2+1}$
m_1	$\{-\ell_1, \dots, \ell_1\}$	summed over: $2\ell_1 + 1$ values
m_2	$\{-\ell_2, \dots, \ell_2\}$	summed over: $2\ell_2 + 1$ values
ℓ	one allowed value	fixed , not summed: we compute one output type at a time
m	$\{-\ell, \dots, \ell\}$	free : the formula is evaluated once per m
$C_{(\ell_1, m_1)(\ell_2, m_2)}^{(\ell, m)}$	$\mathbb{R}$	a fixed number, known in advance

So the computation is: *for each* of the $2\ell + 1$ output slots m , sum over all $(2\ell_1 + 1)(2\ell_2 + 1)$ input pairs, weighting each product $u_{m_1} y_{m_2}$ by the corresponding coefficient. In shapes,

$$\underbrace{u^{\ell_1}}_{[2\ell_1+1]}, \quad \underbrace{y^{\ell_2}}_{[2\ell_2+1]} \xrightarrow{C^{(\ell)} \text{ of shape } [2\ell+1, 2\ell_1+1, 2\ell_2+1]} \underbrace{z^\ell}_{[2\ell+1]}.$$

It is a bilinear contraction: linear in u for fixed y , linear in y for fixed u , and the coefficient array is a fixed three-index tensor.

Two points that are easy to misread. First, ℓ does not appear on the right-hand side of (4.7) except inside C — choosing a different allowed ℓ means choosing a different coefficient array and producing a differently-sized output from the *same* inputs. Second, most of the coefficients are zero: $C_{(\ell_1, m_1)(\ell_2, m_2)}^{(\ell, m)}$ vanishes unless $m = m_1 + m_2$, so the double sum really has at most $\min(2\ell_1, 2\ell_2) + 1$ nonzero terms, not $(2\ell_1 + 1)(2\ell_2 + 1)$. That sparsity is what makes the operation affordable, and it is the structure EquiformerV2 exploits further (Section 9.5).

Example 4.37 (The simplest case: $0 \otimes \ell \rightarrow \ell$). Let $\ell_1 = 0$, so u^0 is a single number c and $m_1 = 0$ only. The selection rule permits $\ell = \ell_2$ alone. The coefficients reduce to $C_{(0,0)(\ell_2, m_2)}^{(\ell_2, m)} = \delta_{mm_2}$, and (4.7) collapses to

$$z_m^{\ell_2} = \sum_{m_2} \delta_{mm_2} c y_{m_2}^{\ell_2} = c y_m^{\ell_2}, \quad \text{i.e.} \quad z^{\ell_2} = c y^{\ell_2}.$$

Multiplying a steerable feature by an invariant scalar. Equivariance is obvious here — $D^{\ell_2}(Q)(cy) = c D^{\ell_2}(Q)y$ — and this is worth noticing because it is the mechanism by which *every* learned scalar weight in the network acts on higher-degree features.

Example 4.38 (The invariant output of $1 \otimes 1$, and why it is the dot product). Take $\ell_1 = \ell_2 = 1$ and $\ell = 0$. The output has $2 \cdot 0 + 1 = 1$ component, and $m = 0$ forces $m_2 = -m_1$ by the $m = m_1 + m_2$ rule, so the double sum has only three surviving terms:

$$z_0^0 = C_{(1,-1)(1,1)}^{(0,0)} u_{-1} y_1 + C_{(1,0)(1,0)}^{(0,0)} u_0 y_0 + C_{(1,1)(1,-1)}^{(0,0)} u_1 y_{-1}.$$

In the standard convention the three coefficients are $\frac{1}{\sqrt{3}}(1, -1, 1)$ up to an overall sign; in the *real* basis used by **e3nn**, where Y^1 is proportional to the Cartesian components, the same object becomes simply

$$z^0 \propto u^\top y,$$

the ordinary dot product. This is not a coincidence to be memorised: by Section 4.9.3 the $\ell = 0$ piece of $u \otimes y$ is the trace of $uy^\top$, which is $u^\top y$. The Clebsch–Gordan coefficients are the coordinates of that identification.

The proportionality constant depends on the normalisation convention (Caution 4.39) and is not worth tracking, because in a network the result is immediately multiplied by a learned weight.

Caution 4.39 (Normalisation, sign, and basis conventions). Three independent conventions affect the numerical values of the coefficients, and sources differ on all three:

- (i) **Real versus complex basis.** Physics texts almost always use complex spherical harmonics and complex C 's (Condon–Shortley phase); **e3nn**, Equiformer and hence SigmaDock use a *real* orthonormal basis. The two are related by a fixed unitary change of basis per ℓ . The decomposition (4.6) — which ℓ appear, and with what multiplicity — is identical either way; only the numbers differ.
- (ii) **Normalisation.** One may normalise so that the coupled basis is orthonormal, or so that the tensor product preserves feature variance (**e3nn** offers several **normalization** modes). This rescales each ℓ -block by a constant.
- (iii) **Overall sign per (ℓ_1, ℓ_2, ℓ) triple.** Fixed arbitrarily by a phase convention.

None of these affects anything in this document, for a reason worth stating plainly: every Clebsch–Gordan product in the architecture is immediately followed by a learned linear weight (Section 4.9.8), which absorbs any per- (ℓ_1, ℓ_2, ℓ) constant. Conventions matter when comparing formulas across papers, and when writing a custom kernel; they do not matter for understanding what the layer computes.

4.9.7 Why the result is equivariant

We now prove the property that justifies the whole construction. First the vocabulary.

Definition 4.40 (Intertwiner). Let D_V and D_W be representations of G on V and W . A linear map $\Phi : V \rightarrow W$ is an *intertwiner* (or *equivariant linear map*) if

$$\Phi(D_V(Q)x) = D_W(Q)\Phi(x) \quad \text{for all } Q \in G, x \in V,$$

equivalently $\Phi \circ D_V(Q) = D_W(Q) \circ \Phi$ as linear maps.

An intertwiner is exactly Definition 4.9 specialised to a *linear* map between two spaces each carrying a representation. The name says what it does: it interleaves the two group actions so that they agree.

Proposition 4.41 (The Clebsch–Gordan product is equivariant). *Write $\Phi_\ell : V_{\ell_1} \otimes V_{\ell_2} \rightarrow V_\ell$ for the linear map defined by the coefficients, i.e. $\Phi_\ell(u \otimes y)_m = \sum_{m_1, m_2} C_{(\ell_1, m_1)(\ell_2, m_2)}^{(\ell, m)} u_{m_1} y_{m_2}$. Then Φ_ℓ is an intertwiner between the tensor product representation and D^ℓ , and consequently, for all $Q \in SO(3)$,*

$$(D^{\ell_1}(Q)u) \otimes_{\text{cg}} (D^{\ell_2}(Q)y) = D^\ell(Q)(u \otimes_{\text{cg}} y)$$

in each degree- ℓ component.

Proof. By Theorem 4.34 there is an isomorphism of representations

$$\Psi : V_{\ell_1} \otimes V_{\ell_2} \xrightarrow{\cong} \bigoplus_{\ell'} V_{\ell'},$$

meaning Ψ is a linear bijection satisfying $\Psi \circ (D^{\ell_1} \otimes D^{\ell_2})(Q) = (\bigoplus_{\ell'} D^{\ell'}(Q)) \circ \Psi$ for every Q — this identity *is* what “isomorphism of representations” means, and it is the content of the decomposition, not an extra assumption.

Let $\pi_\ell : \bigoplus_{\ell'} V_{\ell'} \rightarrow V_\ell$ be the projection onto the ℓ -summand. A projection onto a summand commutes with a block-diagonal action by construction: $\pi_\ell \circ (\bigoplus_{\ell'} D^{\ell'}(Q)) = D^\ell(Q) \circ \pi_\ell$. Now $\Phi_\ell = \pi_\ell \circ \Psi$ by Definition 4.36 (the coefficients are exactly the matrix entries of this composite in the chosen bases), so for any $w \in V_{\ell_1} \otimes V_{\ell_2}$,

$$\Phi_\ell((D^{\ell_1} \otimes D^{\ell_2})(Q)w) = \pi_\ell \Psi((D^{\ell_1} \otimes D^{\ell_2})(Q)w) = \pi_\ell \left((\bigoplus_{\ell'} D^{\ell'}(Q)) \Psi w \right) = D^\ell(Q) \pi_\ell \Psi w = D^\ell(Q) \Phi_\ell(w).$$

Finally specialise to $w = u \otimes y$ and use $(D^{\ell_1} \otimes D^{\ell_2})(Q)(u \otimes y) = D^{\ell_1}(Q)u \otimes D^{\ell_2}(Q)y$ (Definition 4.30). $\square$

Remark 4.42 (Sanity check on the example we computed by hand). For $\ell_1 = \ell_2 = 1$ the proposition specialises to three statements we have already verified directly in Section 4.9.3: $\ell = 0$ gives $(Qu)^\top(Qy) = u^\top y$; $\ell = 1$ gives $(Qu) \times (Qy) = Q(u \times y)$; $\ell = 2$ gives that $Z \mapsto QZQ^\top$ preserves the symmetric traceless space (Proposition 4.31(iii)). So the general proof and the concrete example agree, as they must.

Proposition 4.41 is the single fact that makes (4.7) the “ $\times$ ” of equivariant deep learning: it is — up to the choice of basis fixing the specific numbers $C_{(\ell_1, m_1)(\ell_2, m_2)}^{(\ell, m)}$, a choice we do not need to make explicit anywhere in this document, since every use of (4.7) in Chapter 9 treats them as fixed, precomputed constants — the *unique* (up to scale, by Schur’s lemma, Proposition 4.46 below) bilinear map $V_{\ell_1} \times V_{\ell_2} \rightarrow V_\ell$ respecting the rotation action, and it is therefore the only mechanism by which two steerable features can be combined into a third without breaking equivariance.

4.9.8 From the mathematical product to a network layer

Everything so far is representation theory: given *one* feature of type ℓ_1 and *one* of type ℓ_2 , there is an essentially unique equivariant way to produce a type- ℓ output. A neural network layer is more than that, and it is worth being precise about which parts are fixed by mathematics and which are learned. There are three distinct objects, often all called “tensor product”:

Object	What it is	Free parameters
Tensor product $u \otimes y$	the raw bilinear object, $(2\ell_1 + 1)(2\ell_2 + 1)$ numbers	none
CG projection $\otimes_{\text{cg}}$	projection onto one allowed ℓ , (4.7)	none — fixed by symmetry
Tensor product <i>layer</i>	the above, applied across channels, with learned mixing	the weights W below

Adding channels. Suppose the first input has C_1 channels of type ℓ_1 , written $x_{c_1, m_1}^{(\ell_1)}$, and the second has C_2 channels of type ℓ_2 . For a chosen output type ℓ and C_{out} output channels,

the layer computes

$$z_{c,m}^{(\ell)} = \sum_{c_1=1}^{C_1} \sum_{c_2=1}^{C_2} W_{c,c_1,c_2}^{(\ell_1,\ell_2,\ell)} \underbrace{\sum_{m_1,m_2} C_{(\ell_1,m_1)(\ell_2,m_2)}^{(\ell,m)}}_{\text{the CG product of channel pair } (c_1,c_2)} x_{c_1,m_1}^{(\ell_1)} y_{c_2,m_2}^{(\ell_2)}. \quad (4.8)$$

Read from the inside out: for every pair of input channels, compute the Clebsch–Gordan product (a $(2\ell + 1)$ -vector); then take a learned linear combination of those $C_1 C_2$ vectors to produce each of the C_{out} output channels.

Note carefully where the two index groups sit. The learned weight W_{c,c_1,c_2} carries **no m -index**. It cannot: an m -dependent weight would mix components within an irrep by hand, which Section 4.8 already flagged as forbidden and which Proposition 4.46 will forbid formally. Conversely the coefficients C carry **no channel index**: they are the same for every channel pair. The two index sets are disjoint, and that separation is exactly what makes (4.8) equivariant.

Proposition 4.43 (The tensor product layer is equivariant). *For any weights W , the map defined by (4.8) satisfies $z_c^{(\ell)} \mapsto D^\ell(Q)z_c^{(\ell)}$ whenever $x_{c_1}^{(\ell_1)} \mapsto D^{\ell_1}(Q)x_{c_1}^{(\ell_1)}$ and $y_{c_2}^{(\ell_2)} \mapsto D^{\ell_2}(Q)y_{c_2}^{(\ell_2)}$ for all channels.*

Proof. Fix c . By Proposition 4.41 each inner bracket transforms by $D^\ell(Q)$. The outer sum is a fixed linear combination with m -independent coefficients, and $D^\ell(Q)$ is linear, so it passes through the sum: $\sum_{c_1 c_2} W_{c,c_1,c_2} D^\ell(Q)(\cdots) = D^\ell(Q) \sum_{c_1 c_2} W_{c,c_1,c_2}(\cdots)$. Note the proof works for *arbitrary* W — equivariance is structural, not a property the training has to preserve. $\square$

Proposition 4.43 is worth pausing on: it says the network is equivariant *at initialisation, for every weight setting, exactly*. Training cannot break it, numerical noise cannot erode it, and no augmentation is needed to encourage it. That is the payoff for all the machinery.

Geometry is fixed; channels are learned

The single most useful summary of this section:

Fixed by symmetry	Learned from data
which ℓ may appear ($ \ell_1 - \ell_2 \leq \ell \leq \ell_1 + \ell_2$)	how strongly each allowed ℓ contributes
how the m -components couple (the C 's)	how channels mix (the W 's)
that the result is equivariant	what the features mean

The network never learns *how to rotate*. It learns *what to compute*, within a space of operations all of which rotate correctly by construction.

One nuance to carry forward: this clean split is exact for the plain tensor-product layer above. EquiformerV2 reorganises the computation (Section 9.5) for efficiency, and in doing so the learned weights appear in a different place — but the principle survives, and Chapter 9 says exactly how.

4.9.9 Where this meets geometry: the edge direction

We can now see why this machinery is the right tool for a molecular graph, which is the point of the whole chapter.

Consider computing a message from atom j to atom i . Two ingredients are available:

- the neighbour’s current feature $h_j^{(\ell_1)}$ — *chemistry*: what kind of atom, in what environment, with what learned properties;
- the edge direction $\hat{x}_{ij} = x_{ij}/\|x_{ij}\| \in S^2$ — *geometry*: where the neighbour is.

The direction is not yet a steerable feature, but Section 4.6 made it one: $Y^{\ell_2}(\hat{x}_{ij}) \in \mathbb{R}^{2\ell_2+1}$ transforms by $D^{\ell_2}(Q)$ (Equation (4.1)). So both ingredients are steerable features, and the Clebsch–Gordan product is precisely the operation that combines them:

$$h_j^{(\ell_1)} \otimes_{\text{cg}} Y^{\ell_2}(\hat{x}_{ij}) \longrightarrow \text{message components of every allowed type } \ell.$$

Chemistry meets geometry, and the result is guaranteed to rotate correctly.

Schematically, the pipeline that Chapter 9 will build is:

$$\begin{array}{ccc} h_j^{(\ell_1)}, x_{ij} & \implies & \text{split } x_{ij} \text{ into } \|x_{ij}\| \text{ and } \hat{x}_{ij} \\ \Downarrow & & \\ Y^{\ell_2}(\hat{x}_{ij}) \text{ (angular, steerable)} & \text{ and } & \text{RBF}(\|x_{ij}\|) \text{ (radial, invariant)} \\ \Downarrow & & \\ \text{CG product, with weights modulated by the radial features} & & \\ \Downarrow & & \\ & \text{equivariant message } m_{ij}^{(\ell)} & \end{array}$$

Why the radius and the direction are handled separately — and why that separation is forced rather than convenient — is the subject of Section 9.2. For now the essential point is that the box in the middle is (4.8), and we have just built it.

Why this matters for SigmaFlow

Feature shapes. (4.8) is the operation whose tensor shapes you will meet in the code. Recognising which axis is c and which is m (Caution 4.26) is the difference between reading `so2_ops.py` and guessing.

Why tensor products at all. They are the *only* mechanism by which the network’s chemical features learn about the geometry of the pocket. Without them the architecture could pass around scalars and distances, and Proposition 8.15 shows that is provably insufficient to determine a pose.

Why Clebsch–Gordan specifically. Because the output type is not a design choice the network could get wrong: it is determined by symmetry. This is what makes the equivariance of SigmaDock and SigmaFlow a *theorem* about the architecture rather than a hope about the training.

Why none of this changes when diffusion becomes flow matching. Everything in this chapter concerns the *backbone* — how features are represented and combined. The generative engine decides what the network is asked to predict (a score, or a velocity) and how samples are produced (an SDE, or an ODE). It does not touch ℓ , m , the Clebsch–Gordan coefficients, or (4.8). That separation is precisely what makes SigmaFlow a controlled single-variable change relative to SigmaDock, and it is why this chapter is shared, unmodified, by both halves of the document.

Example 4.44 (Familiar special cases). $\ell_1 = \ell_2 = 1, \ell = 0$: the degree-0 (invariant) component of $u \otimes_{\text{cg}} v$ for $u, v \in \mathbb{R}^3$ is (up to normalisation) the ordinary dot product $\langle u, v \rangle$ — the unique way to combine two vectors into a rotation-invariant scalar, matching Proposition 8.15’s later discussion of what distance-only networks can express. $\ell_1 = \ell_2 = 1, \ell = 1$: the degree-1 component is (up to normalisation) the cross product $u \times v$ — matching Proposition 3.50’s appearance of the cross product as an $\mathfrak{so}(3)$ -valued (equivalently, since $\mathfrak{so}(3) \cong \mathbb{R}^3$, degree-1) object. $\ell_1 = 0$: tensoring with a scalar is just scalar multiplication, $(c \otimes_{\text{cg}} v)^\ell = cv$ (only $\ell = \ell_2$ appears, since $|\ell_1 - \ell_2| = \ell_1 + \ell_2 = \ell_2$ when $\ell_1 = 0$), recovering ordinary scaling of a steerable feature by an invariant weight — the operation that, per Proposition 4.46 below, is the *only* linear operation available on a single feature and hence the mechanism by which every learned scalar weight in Chapter 9’s network acts on higher-degree channels.

4.10 Consequences for equivariant linear maps and nonlinearities

4.10.1 The question

We have features organised as irreps. A standard neural network would now flatten everything into one long vector and apply $y = Wh + b$. Suppose a node carries

$$h^{(0)} \in \mathbb{R}^4, \quad h^{(1)} \in \mathbb{R}^{2 \times 3}, \quad h^{(2)} \in \mathbb{R}^{1 \times 5},$$

i.e. four scalar channels, two vector channels, one $\ell = 2$ channel: $4 + 6 + 5 = 15$ numbers in total. The naive layer would learn a 15×15 matrix, 225 parameters, and mix everything with everything.

Which of those 225 parameters may we actually use?

The answer, derived below, is: $16 + 4 + 1 = 21$. The remaining 204 are not merely unhelpful — switching any of them on breaks equivariance. This section explains where the number 21 comes from.

The intuition first. A weight matrix is a fixed array of numbers, chosen once and then held constant while the molecule may be presented in any orientation. Anything that a fixed array can “know” about direction is a lie: it was chosen in *some* frame, and there is no preferred frame. So a legal weight may combine quantities that carry no direction (channels of the same type), but it may not do anything that implicitly names a direction — and mixing an $\ell = 0$ with an $\ell = 1$ component, or one m -slot with another, does exactly that.

4.10.2 The condition a linear layer must satisfy

Make that precise. Let the input transform under a representation ρ_{in} and the output under ρ_{out} , and let W be the layer. Equivariance (Definition 4.9) demands $W(\rho_{\text{in}}(R)h) = \rho_{\text{out}}(R)(Wh)$ for all h , i.e. as an identity between matrices,

$$\boxed{W \rho_{\text{in}}(R) = \rho_{\text{out}}(R) W \quad \text{for every } R \in SO(3).} \quad (4.9)$$

Read the two sides:

$$\begin{array}{ll} W\rho_{\text{in}}(R)h & \text{rotate the input, then apply the layer} \\ \rho_{\text{out}}(R)Wh & \text{apply the layer, then rotate the output} \end{array}$$

Equation (4.9) says these agree for every rotation and every input. This is precisely Definition 4.40: **an equivariant linear layer is an intertwiner**, and the design problem is to describe the space of intertwiners — because that space *is* the space of admissible weight matrices.

Note what kind of constraint this is. It is not a soft preference; it is a system of linear equations on the entries of W , one for each R . Since $SO(3)$ is 3-dimensional there are infinitely many such equations, and the solution space is correspondingly small.

4.10.3 Three cases, worked by hand

Before invoking any general theorem, let us solve (4.9) in the three cases that matter.

Case 1: scalar $\rightarrow$ scalar. Here $\rho_{\text{in}} = \rho_{\text{out}} = D^0$, and $D^0(R) = 1$ for every R . So (4.9) reads $W \cdot 1 = 1 \cdot W$, which is *no constraint at all*. Any matrix works. With C_{in} scalar channels in and C_{out} out, the full $C_{\text{out}} \times C_{\text{in}}$ matrix is free. Scalars behave exactly like ordinary neural-network features, which is why $\ell = 0$ channels are the “easy” part of every equivariant architecture.

Case 2: vector $\rightarrow$ vector. Now $\rho_{\text{in}} = \rho_{\text{out}} = D^1$ and $D^1(R) = R$, so we must find every 3×3 matrix W with

$$WR = RW \quad \text{for every } R \in SO(3).$$

Claim: only $W = \lambda I$. Here is an elementary argument that does not require any representation theory.

Let $R_z(\vartheta)$ be rotation about the z -axis. Commuting with all of these forces W to preserve each eigenspace of R_z ; over $\mathbb{R}$ the concrete consequence is that W must map the z -axis to itself and act on the xy -plane as a scaled rotation. So

$$W = \begin{pmatrix} a & -b & 0 \\ b & a & 0 \\ 0 & 0 & c \end{pmatrix}.$$

Now impose commutation with rotation about the x -axis as well. By the same reasoning with the roles of the axes permuted, W must also have that form with respect to x , which forces $c = a$ and $b = 0$. Repeating with the y -axis leaves no freedom: $W = aI$.

The geometric reading is more memorable than the computation. W is a fixed linear map that must look the same from every viewpoint. A map that stretched more along one direction than another would let you *recover* that direction from W — and W is a constant, fixed before we knew the molecule’s orientation. The only maps with no preferred direction are uniform scalings.

Case 3: scalar $\rightarrow$ vector. Now $\rho_{\text{in}} = D^0$, $\rho_{\text{out}} = D^1$, so W is a 3×1 matrix, i.e. a fixed vector $w \in \mathbb{R}^3$, and (4.9) reads

$$w = w \cdot 1 = W\rho_{\text{in}}(R) = \rho_{\text{out}}(R)W = Rw \quad \text{for every } R \in SO(3).$$

A vector fixed by *every* rotation must be $w = 0$: any nonzero w has some rotation moving it. So the only equivariant linear map from scalars to vectors is the zero map.

Again the intuition is the point: a nonzero w would be a direction manufactured out of a number that carries none. It would single out an axis in space, and nothing in the input justifies that axis.

Caution 4.45 (This is why you cannot “just add a linear layer”). Case 3 generalises: there is no nonzero equivariant linear map between different degrees at all. A network therefore *cannot* create an $\ell = 1$ feature out of $\ell = 0$ features by any linear means, no matter how many layers are stacked. Since the output of SigmaFlow’s network is an $\ell = 1$ object (Equation (8.1)), directional information must enter from the geometry — and the only route is the tensor product of Section 4.9. This is the structural reason the architecture cannot be simplified into “scalars plus an MLP”.

4.10.4 Schur’s lemma

The three cases above are instances of one theorem. Informally:

Between irreducible representations there are almost no equivariant linear maps. If the two irreps are different, there are none but zero. If they are the same, there is only one, up to a scale factor.

That is a strong statement, and its strength is exactly what makes equivariant architecture design tractable: instead of searching a huge space of weight matrices, one is handed a very short list of options.

Proposition 4.46 (Schur’s lemma, and the form of equivariant linear maps). *Let $W : V_{\ell_1} \rightarrow V_{\ell_2}$ be linear and satisfy $W D^{\ell_1}(Q) = D^{\ell_2}(Q) W$ for all $Q \in SO(3)$ (an intertwiner). If $\ell_1 \neq \ell_2$, then $W = 0$. If $\ell_1 = \ell_2 = \ell$, then $W = cI$ for some scalar $c \in \mathbb{R}$.*

Proof. Case $\ell_1 \neq \ell_2$. Consider $\ker W \subseteq V_{\ell_1}$. If $x \in \ker W$ then $W D^{\ell_1}(Q)x = D^{\ell_2}(Q)Wx = 0$, so $D^{\ell_1}(Q)x \in \ker W$ too: the kernel is an invariant subspace. Since V_{ℓ_1} is irreducible, $\ker W$ is $\{0\}$ or all of V_{ℓ_1} . The same argument applied to $\text{im } W \subseteq V_{\ell_2}$ shows the image is $\{0\}$ or all of V_{ℓ_2} . If $W \neq 0$ then $\ker W = \{0\}$ and $\text{im } W = V_{\ell_2}$, i.e. W is an isomorphism — impossible when $\dim V_{\ell_1} = 2\ell_1 + 1 \neq 2\ell_2 + 1 = \dim V_{\ell_2}$. Hence $W = 0$.

Case $\ell_1 = \ell_2 = \ell$. Over $\mathbb{C}$ one argues that an eigenvalue λ of W exists and $W - \lambda I$ is again an intertwiner with nontrivial kernel, hence zero by the first part. Over $\mathbb{R}$ this step is not immediate, because a real matrix need not have a real eigenvalue, and in general the commutant of a real irrep is $\mathbb{R}$, $\mathbb{C}$ or $\mathbb{H}$ (real, complex or quaternionic type). The conclusion $W = cI$ with $c \in \mathbb{R}$ therefore requires knowing that the real irreps V_ℓ of $SO(3)$ are of **real type**, i.e. that their complexifications stay irreducible. That is true, and it is part of the classical classification cited after Theorem 4.18 (Fulton & Harris, *Representation Theory*, 1991, §1.2 and §3; Bröcker & tom Dieck, *Representations of Compact Lie Groups*, 1985, Ch. II). Given real type, the commutant is $\mathbb{R}I$ and the statement follows. $\square$

Caution 4.47 (The real-versus-complex step is not a formality). It is tempting to quote “Schur’s lemma: intertwiners between equivalent irreps are multiples of the identity” and move on. That sentence is true over $\mathbb{C}$ and *not* true over $\mathbb{R}$ in general. The standard counterexample is $SO(2)$ acting on $\mathbb{R}^2$ by rotation: this real representation is irreducible (no real line is preserved by all rotations), yet its commutant is two-dimensional — every

rotation matrix commutes with every other, so $W = \begin{pmatrix} a & -b \\ b & a \end{pmatrix}$ is an intertwiner for all a, b . That commutant is a copy of $\mathbb{C}$, not $\mathbb{R}$; $SO(2)$'s two-dimensional real irreps are of *complex* type.

So the conclusion we use genuinely depends on a property of $SO(3)$ and would fail for other groups. Since the whole architecture rests on it, it is worth checking rather than assuming. A direct numerical check settles it: solving the linear system (4.9) for $\ell_1 = \ell_2 = 1$ and for $\ell_1 = \ell_2 = 2$ yields a solution space of dimension exactly 1 in each case, spanned by the identity; for $\ell_1 = 1, \ell_2 = 2$ (and conversely) the solution space is $\{0\}$. Those are the four numbers 1, 1, 0, 0 that Proposition 4.46 predicts.

Proposition 4.48 (Consequence for multi-channel equivariant linear layers). *Let $h = \bigoplus_{\ell,c} h^{\ell,c}$ be a steerable feature with C_ℓ channels of each degree $\ell \leq L$ (Definition 4.54 below), and let $W : \bigoplus_{\ell,c} V_\ell \rightarrow \bigoplus_{\ell',c'} V_{\ell'}$ be linear and equivariant (commuting with the diagonal action $\bigoplus_{\ell,c} D^\ell(Q)$ on both sides). Then W acts as $h^{\ell,c} \mapsto \sum_{c'} W_{cc'}^\ell h^{\ell,c'}$ for scalar weights $W_{cc'}^\ell \in \mathbb{R}$: W mixes channels only within a fixed degree ℓ , never across degrees.*

Proof. Write W blockwise, indexed by (source degree, source channel) and (target degree, target channel); each block $W_{(\ell,c) \rightarrow (\ell',c')} : V_\ell \rightarrow V_{\ell'}$ is itself an intertwiner (since W commutes with the full diagonal action, and one may test equivariance against elements supported in a single source/target block by linearity), so Proposition 4.46 applies to each block separately: it vanishes unless $\ell = \ell'$, and is a scalar multiple of the identity when $\ell = \ell'$. Collecting the surviving (same-degree) blocks gives exactly the stated form. $\square$

4.10.5 What the layer actually looks like

Proposition 4.48 is best absorbed in coordinates. With C_{in} input channels and C_{out} output channels of degree ℓ , the layer is

$$y_{c,m}^{(\ell)} = \sum_{c'=1}^{C_{\text{in}}} A_{cc'}^{(\ell)} h_{c',m}^{(\ell)}, \quad A^{(\ell)} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}. \quad (4.10)$$

The index m appears on both sides untouched: it is a spectator. The learned matrix $A^{(\ell)}$ carries no m -index and acts identically on every one of the $2\ell + 1$ slots.

In Kronecker form, acting on the stacked vector,

$$W^{(\ell)} = A^{(\ell)} \otimes I_{2\ell+1}. \quad (4.11)$$

Compare this with how the *features* transform, $\rho^{(\ell)}(R) = I_C \otimes D^\ell(R)$ from (4.3). The two commute for a reason that is now visible at a glance:

$$(A \otimes I_{2\ell+1})(I_{C_{\text{in}}} \otimes D^\ell(R)) = A \otimes D^\ell(R) = (I_{C_{\text{out}}} \otimes D^\ell(R))(A \otimes I_{2\ell+1}),$$

using the mixed-product rule $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$. **Each factor acts on a different index and they simply pass through one another.** The learned part touches only c ; the rotation touches only m ; neither sees the other.

That single line is, in the end, the whole of Section 4.10.

Two axes, two owners

Axis	Owned by	May be manipulated by
channel c	the network	learned weights, freely
component m	the symmetry	only $D^\ell(R)$ and Clebsch–Gordan

The channel axis carries *learned feature semantics*: what the feature means. The m axis carries *geometric structure*: how the feature points. Keeping them separate is what equivariance amounts to in practice, and mixing them is the single most common way to break it.

4.10.6 Bias terms

A standard linear layer is $y = Wh + b$. We have discussed W ; what about b ?

The bias is a *constant*, added regardless of the input, so equivariance requires $\rho_{\text{out}}(R)b = b$ for every R : the bias must be a fixed point of the output representation.

- $\ell = 0$: $D^0(R) = 1$, so the condition reads $b = b$. Any bias is allowed. Scalar channels may have biases exactly as in an ordinary network.
- $\ell \geq 1$: the condition reads $D^\ell(R)b = b$ for all R , i.e. b is a rotation-invariant element of V_ℓ . Since V_ℓ is irreducible and nontrivial, the set of such fixed points is $\{0\}$ — otherwise the line $\mathbb{R}b$ would be a proper nonzero invariant subspace, contradicting irreducibility. Hence $b^{(\ell)} = 0$ is forced.

So:

$$b^{(0)} \text{ free,} \quad b^{(\ell)} = 0 \text{ for } \ell \geq 1.$$

The reasoning is the same as Case 3 of Section 4.10.3: a nonzero constant vector would name a direction, and there is no direction to name. Libraries enforce this by construction — `e3nn`’s linear layer accepts a bias only on scalar irreps — and a layer that appeared to have biases on $\ell \geq 1$ would be a bug, not a design choice.

4.10.7 Why linear layers are not enough

Everything so far is linear, and a composition of linear maps is linear. A network built only from (4.10) would be an (equivariant) linear function of its input, which is far too weak: it could not represent the dependence of a docking score on the shape of a pocket any more than a linear regression could.

So we need nonlinearities. And here the difficulty of the equivariant setting reappears in a new place:

An ordinary elementwise nonlinearity does not respect the irrep structure.

Example 4.14 already showed this with explicit numbers: for $v = (1, -1, 0)^\top$ and a 90° rotation about z , $\text{ReLU}(Rv) = (1, 1, 0)^\top$ while $R\text{ReLU}(v) = (0, 1, 0)^\top$. The reason bears

repeating, because it also tells us what *will* work: ReLU treats the three slots of an $\ell = 1$ feature as three independent numbers, but a rotation mixes them. Any operation that acts slot-by-slot is implicitly asserting that the slots have separate meaning, and they do not.

The repairs all follow one principle:

Do the nonlinear thing to something invariant; use the result to scale something equivariant.

We now examine the three standard realisations of that principle.

4.10.8 Nonlinearity 1: scalars are unrestricted

For $\ell = 0$ features there is no problem at all. Since $D^0(R) = 1$, an $\ell = 0$ feature s satisfies $s \mapsto s$ under rotation, so for any function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$,

$$\sigma(s) \mapsto \sigma(s).$$

The output is again invariant, hence again a legitimate $\ell = 0$ feature. ReLU, SiLU, GELU, layer normalisation, softmax over channels — all are available on scalar channels, unchanged from ordinary deep learning.

This is more useful than it sounds, because it means the network can perform arbitrary nonlinear computation, *provided* it does so on the invariant part of its state. The remaining two mechanisms are about transporting that nonlinearity back onto the higher- ℓ channels.

4.10.9 Nonlinearity 2: norm nonlinearities

For $\ell \geq 1$ we cannot touch the components. But we can compute an invariant from them.

Proposition 4.49 (The norm of an irrep feature is invariant). *If $D^\ell(R)$ is orthogonal — which it is in the standard real orthonormal basis, by Proposition 4.25 — then $\|D^\ell(R)v\| = \|v\|$ for every $v \in V_\ell$.*

Proof. $\|Dv\|^2 = v^\top D^\top Dv = v^\top v = \|v\|^2$. □

So $\|v\|$ is a scalar we are allowed to feed to any nonlinearity. Define

$$f(v) = \alpha(\|v\|) v, \tag{4.12}$$

for any function $\alpha : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$ (e.g. $\alpha(r) = \sigma(r)/(r + \varepsilon)$, which turns (4.12) into “rescale v to have length $\sigma(\|v\|)$ ”).

Proposition 4.50 (Norm nonlinearities are equivariant). $f(D^\ell(R)v) = D^\ell(R)f(v)$.

Proof.

$$f(D^\ell(R)v) = \alpha(\|D^\ell(R)v\|) D^\ell(R)v \stackrel{(a)}{=} \alpha(\|v\|) D^\ell(R)v \stackrel{(b)}{=} D^\ell(R)(\alpha(\|v\|) v) = D^\ell(R)f(v),$$

where (a) is Proposition 4.49 and (b) is linearity of $D^\ell(R)$ together with the fact that $\alpha(\|v\|)$ is a *scalar*, which therefore commutes with the matrix. □

Remark 4.51 (What norm nonlinearities cannot do). The direction of v is untouched: (4.12) can only rescale, never reorient. In particular $f(v)$ is always parallel to v (or antiparallel, if $\alpha < 0$). A network built only from linear layers and norm nonlinearities can therefore change how strongly a direction is expressed but cannot change *which* direction — reorientation must come from the tensor product. A second, practical drawback is the singularity at $v = 0$, which is why implementations carry an ε and why the derivative near the origin needs care.

4.10.10 Nonlinearity 3: gating

Gating is the variant Equiformer actually relies on, and it is a small generalisation of the previous idea: instead of deriving the scalar from v 's own norm, derive it from a *separate scalar channel*, which the network is free to compute however it likes.

The mechanism, in one line. Let $g \in \mathbb{R}$ be an invariant scalar and $v \in V_\ell$ a feature. Then $v \mapsto gv$ is equivariant:

$$g D^\ell(R)v = D^\ell(R)(gv),$$

because a scalar commutes with a matrix. That is the entire argument. Its force comes from where g is allowed to come from: *anywhere*, as long as it is invariant. In particular g may be the output of a deep, thoroughly nonlinear MLP acting on the node's scalar channels.

As a network block. A gated layer is organised as follows. Suppose we want C_ℓ output channels of degree ℓ . The preceding equivariant layer is asked to produce

- C_ℓ channels of degree ℓ : the features to be gated, $h_c^{(\ell)} \in \mathbb{R}^{2\ell+1}$;
- C_ℓ *extra* scalar channels: the gate pre-activations, $s_c \in \mathbb{R}$, one per output channel;
- plus whatever ordinary scalar channels the block also outputs.

Then

$$g_c = \sigma(s_c) \in \mathbb{R}, \quad h_{c,m}^{(\ell),\text{out}} = g_c h_{c,m}^{(\ell)}, \quad (4.13)$$

with σ any scalar nonlinearity (sigmoid, SiLU, ...). Shapes: $s \in \mathbb{R}^{C_\ell}$, $h^{(\ell)} \in \mathbb{R}^{C_\ell \times (2\ell+1)}$, and the product broadcasts g along the m -axis — again, m is a spectator.

Why this is strictly more expressive than a norm nonlinearity. In (4.12) the scaling factor is a function of that one feature's own magnitude. In (4.13) the gate can depend on *all* scalar information available at the node — atom type, local density, time embedding, anything — so the network can learn to suppress or amplify a directional channel based on chemical context rather than only on its own size. Both are equivariant; gating simply has a richer input.

Caution 4.52 (A gate is not an attention weight). Both are invariant scalars multiplying equivariant quantities, so they look alike in a formula. They occupy different roles:

	Gate	Attention weight
Indexed by	channel c (and node i)	<i>edge</i> (i, j)
Purpose	modulate feature channels	weight neighbours in a sum
Normalised?	no	yes, across j (softmax)
Appears in	the pointwise nonlinearity	the aggregation step

Keeping the two apart matters when reading Section 9.10, where both occur within a few lines of each other.

4.10.11 The three admissible operations, and what each is for

We can now state the design space completely. An equivariant network is assembled from exactly three kinds of operation, and each does something the others cannot:

Operation	Can do	Cannot do
Equivariant linear (4.10)	mix channels within one ℓ ; change channel count	create or mix degrees; introduce nonlinearity
Tensor product (4.8)	couple two degrees; <i>create</i> new degrees ℓ within the selection rule	act on a single feature alone
Gate / norm nonlinearity (4.13)	introduce nonlinearity; modulate by context	change ℓ ; reorient a feature

Each row is a genuine limitation, and together they are exhaustive: any equivariant layer is built from these. Read the middle column downwards and you get a recipe — couple geometry into features (tensor product), remix the resulting channels (linear), apply context-dependent nonlinearity (gate), repeat. That is, in outline, an Equiformer block.

4.10.12 A small block, end to end

Concretely, take a node carrying 4 scalar channels and 2 vector channels, and build a layer producing 8 scalars and 5 vectors.

Input. $h^{(0)} \in \mathbb{R}^4$, $h^{(1)} \in \mathbb{R}^{2 \times 3}$. Total numbers: $4 + 6 = 10$.

Equivariant linear. Two independent matrices,

$$A^{(0)} \in \mathbb{R}^{(8+5) \times 4}, \quad A^{(1)} \in \mathbb{R}^{5 \times 2}.$$

The $\ell = 0$ matrix produces 13 scalar outputs: 8 ordinary scalars plus 5 gate pre-activations, one for each intended vector channel. The $\ell = 1$ matrix produces 5 vector channels by mixing the 2 input vector channels. Parameter count: $13 \cdot 4 + 5 \cdot 2 = 52 + 10 = 62$, plus a bias $b^{(0)} \in \mathbb{R}^{13}$ — and *no* bias on the $\ell = 1$ part.

Note what is absent: there is no matrix from the 4 scalars to the 5 vectors, and none from the 2 vectors to the scalars. By Caution 4.45 those blocks are forced to zero. Of the $(8 \cdot 1 + 5 \cdot 3) \times (4 \cdot 1 + 2 \cdot 3) = 23 \times 10 = 230$ entries a naive dense layer would have, only 62 are admissible.

Gate. Split the 13 scalars into 8 outputs and 5 gates $s \in \mathbb{R}^5$; set $g = \sigma(s) \in \mathbb{R}^5$ and

$$h_{c,m}^{(1),\text{out}} = g_c h_{c,m}^{(1)}, \quad c = 1, \dots, 5, \quad m = 1, 2, 3.$$

Output. 8 scalars and 5 vectors, i.e. $8 + 15 = 23$ numbers, transforming under $I_8 \oplus (I_5 \otimes D^1(R))$.

What is still missing. Nothing in this block created an $\ell = 2$ feature, and nothing could have: linear layers preserve degree and gates preserve degree. Higher degrees enter only through the tensor product with the spherical harmonics of an edge — which is the message-passing step, and the subject of Section 9.1.

4.10.13 Summary in one formula

Collecting (4.3) and (4.11): if a node’s feature space is $\bigoplus_{\ell}(\mathbb{R}^{C_{\ell}} \otimes V_{\ell})$, then the rotation acts by

$$\rho(R) = \bigoplus_{\ell} (I_{C_{\ell}} \otimes D^{\ell}(R)),$$

and the most general equivariant linear layer is

$$W = \bigoplus_{\ell} (A^{(\ell)} \otimes I_{2\ell+1}), \quad A^{(\ell)} \in \mathbb{R}^{C_{\ell}^{\text{out}} \times C_{\ell}^{\text{in}}} \text{ arbitrary.}$$

Block-diagonal in ℓ ; free in c ; identity in m . Everything in this section is a consequence of those three words.

Remark 4.53 (A preview: parity, and $O(3)$ versus $SO(3)$). Implementations label irreps not by ℓ alone but by ℓ together with a *parity*, written $0\mathbf{e}$, $1\mathbf{o}$, $2\mathbf{e}$ and so on ($\mathbf{e}$ = even, $\mathbf{o}$ = odd). This extends the group from $SO(3)$ to $O(3)$ by including the inversion $x \mapsto -x$: an even feature is unchanged by inversion, an odd one flips sign. Ordinary position vectors are odd; angular momenta and cross products are even (they are *pseudovectors*).

Nothing in this chapter changes: parity refines the bookkeeping and adds a second selection rule (parities multiply), but the ℓ -structure, Schur’s lemma and the layer forms above are unaffected. We take parity up only where it actually bites — molecular chirality, where the distinction between $SO(3)$ and $O(3)$ is chemically real — in Remark 3.74.

Why this matters for SigmaFlow

Linear mixing. The backbone can learn arbitrary new combinations of same-type channels — new chemical semantics — without any risk to geometry. This is where most of the parameters live, and it is why the architecture can be wide without becoming fragile.

Gating. It is what makes the network nonlinear at all on its directional channels. Without it, all the $\ell \geq 1$ processing would be linear, and the model could not represent the strongly nonlinear dependence of a pose on its environment.

Tensor products remain essential. Section 4.10.11 makes precise why they cannot be dispensed with: they are the only operation that can create a degree that was not already present. Since the network’s output is $\ell = 1$ and its raw chemical inputs are $\ell = 0$, at least one tensor product with the geometry is unavoidable.

Independent of the generative engine. All three operations are statements about how features may be transformed. They say nothing about what the network is asked to predict. Replacing SigmaDock’s score target with SigmaFlow’s velocity target changes the loss and the sampler; it does not touch a single one of the constraints in this section. That is precisely why the backbone is shared, and why the SigmaFlow conversion could be confined to five files (Chapter 13).

Definition 4.54 (Steerable feature; restated). A *steerable feature* (of a network node or edge) is a direct sum $h = \bigoplus_{\ell=0}^L \bigoplus_{c=1}^{C_{\ell}} h^{\ell,c}$, $h^{\ell,c} \in V_{\ell} \cong \mathbb{R}^{2\ell+1}$, transforming under $Q \in SO(3)$

by $h \mapsto (\bigoplus_{\ell,c} D^\ell(Q))h$ — i.e. every one of the C_ℓ channels at degree ℓ transforms by the *same* $D^\ell(Q)$, independently of the others.

Proposition 4.48 is, in words, the entire content of “an equivariant linear layer has one learnable scalar per (source channel, target channel, shared degree) triple, and cannot move information between degrees”: every degree ℓ is processed by its *own*, otherwise completely ordinary, linear layer, exactly the pattern the `S03_LinearV2` code correspondence in Chapter 9 will confirm directly. Since linear maps can never mix degrees, and since pointwise nonlinearities (e.g. ReLU applied entrywise) do *not*, in general, commute with $D^\ell(Q)$ for $\ell \geq 1$ (a direction-dependent nonlinearity applied to a rotated vector is generally not the same as rotating the nonlinearly-transformed vector), the Clebsch–Gordan product (4.7) of this chapter is the *only* remaining mechanism, besides invariant-gated rescaling (Chapter 9’s gate nonlinearity), by which an equivariant network can create new degrees of information or genuinely nonlinearly combine two existing steerable features — and this is precisely why the tensor product, rather than being one architectural ingredient among many, is unavoidable, as promised at the end of Section 8.4.

4.11 What you should now be able to do

- distinguish invariance from equivariance, and say why a pose generator needs the second and not the first;
- explain what a representation of $SO(3)$ is, why the irreducible ones are indexed by a single integer ℓ , and what the Wigner- D matrix does;
- read a steerable feature’s four indices and say what each one is for;
- derive the Clebsch–Gordan selection rule $|\ell_1 - \ell_2| \leq \ell \leq \ell_1 + \ell_2$ and use it as a type system for what a layer may compute;
- state the three admissible operations of an equivariant network (linear map, tensor product, invariant-gated nonlinearity) and why nothing else is allowed.

This is the mathematics the network of Part IV is built from; Part III first supplies what that network is asked to predict.

Part III

Diffusion Models

Part III at a glance

What you will learn. How to turn noise into data by learning a *score*. Starting from a discrete random walk, we build stochastic differential equations, the Fokker–Planck equation, denoising score matching, the reverse-time SDE and the probability flow ODE — first in $\mathbb{R}^d$, then on $SO(3)$, where the Gaussian is replaced by the IGSO(3) heat kernel.

Why it is needed. SigmaDock is a diffusion model. To audit it — and to say precisely what SigmaFlow replaces — the diffusion machinery has to be derived, not quoted.

What depends on it. Chapter 10 assembles this into the shipped model. Chapter 11 then shows that its probability flow ODE is a flow matching model in disguise (Proposition 11.44), which is the hinge on which the whole SigmaDock/SigmaFlow comparison turns.

Chapter 5

From Random Walks to Stochastic Differential Equations

Every diffusion model in this document is a stochastic differential equation. The literature tends to introduce them in one line — “consider the SDE $dX_t = f(X_t, t) dt + g(t) dW_t$ ” — and move on. That line hides three genuinely non-obvious things: why the noise term carries $\sqrt{dt}$ rather than dt , what dW_t means when Brownian motion is nowhere differentiable, and why the whole expression is not really a differential equation at all.

This chapter builds the object from scratch, starting from something with no mystery in it: a discrete random walk. Nothing here is specific to generative modelling; it is the toolbox that Chapter 6 then uses. A reader comfortable with stochastic calculus can skip to Section 5.9, where the two concrete limits (DDPM $\rightarrow$ VP-SDE, SMLD $\rightarrow$ VE-SDE) are derived.

Remark 5.1 (Sources and how they differ). The route taken here is not the one in Holderrieth and Erives, *An Introduction to Flow Matching and Diffusion Models*, which defines an SDE directly through infinitesimal updates $X_{t+h} = X_t + hu_t(X_t) + \sigma_t(W_{t+h} - W_t) + hR_t(h)$ and never passes through a random-walk limit. That definition is cleaner, and this document adopts its *form* in Definition 5.5; but it presupposes that Brownian motion already exists and that one accepts the increment scaling. The construction below earns both. The concrete SDEs (VP, VE) and the reverse-time result follow Song et al., *Score-Based Generative Modeling through Stochastic Differential Equations*, ICLR 2021.

Two warnings about conventions, both of which will matter later:

- Song et al. (and hence SigmaDock) run $t = 0$ **data**, $t = T$ **noise**. Holderrieth–Erives (and hence SigmaFlow) run $t = 0$ **noise**, $t = 1$ **data**. Neither is more correct. This document states the convention at the top of every chapter that uses one.
- “Diffusion coefficient” means g in some texts and $g^2/2$ in others. Here it is always g , the factor multiplying dW_t .

5.1 Start with something concrete: a discrete noising chain

Take a data point $x_0 \in \mathbb{R}^d$ and corrupt it in n small steps. At each step, move a little in some systematic direction and add a little independent noise:

$$X_{k+1} = X_k + \mu(X_k, t_k) \Delta t + s_k \epsilon_k, \quad \epsilon_k \sim \mathcal{N}(0, I_d) \text{ i.i.d.}, \quad (5.1)$$

with $t_k = k\Delta t$ and $\Delta t = T/n$. This is a discrete-time Markov chain and there is nothing subtle about it: given X_k , the next state is a Gaussian with mean $X_k + \mu\Delta t$ and covariance $s_k^2 I$.

Two design choices are already visible. The systematic part is proportional to Δt — that is forced, because μ is meant to be a *rate*, and over a time interval Δt a rate contributes $\mu\Delta t$. The noise amplitude s_k is so far unspecified. Choosing it is the whole problem.

5.2 Why the noise scales as $\sqrt{\Delta t}$

We want (5.1) to have a sensible limit as $n \rightarrow \infty$: refining the time grid should refine our description of a *fixed* process, not change the process. So ask what the accumulated noise does under the natural guesses $s_k = \sigma \Delta t^\alpha$.

Set $\mu = 0$ and look only at the noise. After n steps the total perturbation is

$$S_n = \sigma \Delta t^\alpha \sum_{k=1}^n \epsilon_k. \quad (5.2)$$

The ϵ_k are independent with unit covariance, so variances add:

$$\text{Var}(S_n) = \sigma^2 \Delta t^{2\alpha} \cdot n = \sigma^2 \Delta t^{2\alpha} \cdot \frac{T}{\Delta t} = \sigma^2 T \Delta t^{2\alpha-1}. \quad (5.3)$$

Everything follows from the exponent $2\alpha - 1$:

α	s_k	$\text{Var}(S_n)$ as $\Delta t \rightarrow 0$	Verdict
1	$\sigma \Delta t$	$\sigma^2 T \Delta t \rightarrow 0$	noise vanishes; a plain ODE
$\frac{1}{2}$	$\sigma \sqrt{\Delta t}$	$\sigma^2 T$	finite, n-independent
0	σ	$\sigma^2 T / \Delta t \rightarrow \infty$	noise blows up

Only $\alpha = \frac{1}{2}$ survives. It is the unique exponent for which refining the grid neither destroys the randomness nor lets it explode.

Remark 5.2 (The counting argument in one sentence). Independent noise adds in *variance*, not in amplitude: n steps of size ε accumulate to $\sqrt{n}\varepsilon$, not $n\varepsilon$. Since n grows like $1/\Delta t$, an amplitude of $\sqrt{\Delta t}$ per step gives $\sqrt{1/\Delta t} \cdot \sqrt{\Delta t} = 1$. That is the entire content of $\sqrt{\Delta t}$, and it is the reason a random walk is fundamentally rougher than a smooth curve.

Measured (`audits/sde_limit_check.py`), variance of $\sum_k s_k \epsilon_k$ over $[0, 1]$, 20,000 samples:

n	$s_k \propto \Delta t$	$s_k \propto \sqrt{\Delta t}$	$s_k \propto 1$
10	0.099	0.995	9.9
100	0.010	1.005	100.5
1000	0.00100	0.996	995.6
10000	0.00010	1.015	10154.2

The middle column is the only one that converges, and it converges to $T = 1$, exactly as (5.3) predicts.

5.3 Brownian motion

Fix $\alpha = \frac{1}{2}$, $\sigma = 1$, $\mu = 0$, and define the accumulated noise up to time t :

$$W_t^{(n)} := \sum_{k: t_k < t} \sqrt{\Delta t} \epsilon_k. \quad (5.4)$$

Three properties hold for every n , and they characterise the limit.

1. $W_0^{(n)} = 0$. Empty sum.
2. **Independent increments.** For $s < t$, the increment $W_t^{(n)} - W_s^{(n)}$ uses only ϵ_k with $s \leq t_k < t$, and those are independent of the ones before s . Disjoint time intervals use disjoint noise.
3. **Gaussian increments with variance = elapsed time.** The increment is a sum of $\approx (t-s)/\Delta t$ independent $\mathcal{N}(0, \Delta t I)$ terms, hence Gaussian with covariance $\frac{t-s}{\Delta t} \cdot \Delta t I = (t-s)I$ — *independent of Δt* .

Property 3 is where the $\sqrt{\Delta t}$ pays off: the increment law does not depend on the discretisation at all. The limiting object is therefore already pinned down.

Definition 5.3 (Brownian motion / Wiener process). A *standard Brownian motion* on $\mathbb{R}^d$ is a stochastic process $(W_t)_{t \geq 0}$ with

1. $W_0 = 0$;
2. independent increments;
3. $W_t - W_s \sim \mathcal{N}(0, (t-s)I_d)$ for $s < t$;
4. continuous sample paths $t \mapsto W_t$.

That such a process exists is Wiener's theorem; we take it as given. What matters here is that Brownian motion is precisely the continuum limit of the $\sqrt{\Delta t}$ -scaled random walk (Donsker's invariance principle), and that it is the *unique* such limit — the central limit theorem means the limit does not depend on the distribution of ϵ_k , only on its mean and variance. Bernoulli ± 1 steps give the same Brownian motion as Gaussian steps.

5.3.1 Brownian motion is nowhere differentiable

This is the fact that makes the notation dW_t delicate, and it follows directly from property 3 without any hard analysis. Consider the difference quotient over a window h :

$$\frac{W_{t+h} - W_t}{h} \sim \mathcal{N}\left(0, \frac{h}{h^2} I\right) = \mathcal{N}\left(0, \frac{1}{h} I\right). \quad (5.5)$$

Its standard deviation is $h^{-1/2}$, which diverges as $h \rightarrow 0$. The difference quotient has no limit; it spreads out without bound. So $W'(t)$ does not exist — not at any t , almost surely.

Caution 5.4 (What dW_t is not). In an ordinary differential equation, $dx = f(x) dt$ abbreviates $\frac{dx}{dt} = f(x)$, and one may divide by dt . In

$$dX_t = f(X_t, t) dt + g(t) dW_t$$

one may **not**. There is no function $W'(t)$ such that $dW_t = W'(t) dt$. The expression is not an equation between derivatives; it is shorthand, and Definition 5.5 says for what.

5.4 What an SDE actually is

Since we cannot differentiate, we integrate. The honest object is an integral equation.

Definition 5.5 (SDE as an integral equation). The notation

$$dX_t = f(X_t, t) dt + g(t) dW_t, \quad X_0 = x_0 \quad (5.6)$$

is shorthand for the statement that the process (X_t) satisfies, for every t and almost surely,

$$X_t = x_0 + \underbrace{\int_0^t f(X_s, s) ds}_{\text{ordinary integral}} + \underbrace{\int_0^t g(s) dW_s}_{\text{Itô integral}}. \quad (5.7)$$

The first integral is an ordinary (pathwise Riemann) integral: for a fixed realisation, $s \mapsto f(X_s, s)$ is a continuous function and can be integrated as usual. The second needs a definition, because W has unbounded variation — one cannot integrate against it pathwise in the Riemann–Stieltjes sense.

5.4.1 The Itô integral, at the level we need

Define it as a limit of Riemann-type sums, but with one rule fixed in advance:

$$\int_0^t h(s) dW_s := \lim_{\Delta t \rightarrow 0} \sum_k h(t_k) (W_{t_{k+1}} - W_{t_k}), \quad (5.8)$$

evaluating the integrand at the **left endpoint** t_k of each subinterval.

For a Riemann integral of a continuous function the choice of evaluation point is irrelevant — left, right, or midpoint all converge to the same number. Here it is not. The increments are random and of size $\sqrt{\Delta t}$, and a right-endpoint rule would correlate $h(t_{k+1})$ with the increment $W_{t_{k+1}} - W_{t_k}$ it multiplies, producing a different limit. (The midpoint rule gives the *Stratonovich* integral.)

The left-endpoint rule is chosen because it makes the integrand *non-anticipating*: $h(t_k)$ is known before the increment happens. Two consequences follow immediately, and they are all we will use:

Proposition 5.6 (Mean and variance of an Itô integral). *For deterministic g ,*

$$\mathbb{E}\left[\int_0^t g(s) dW_s\right] = 0, \quad \text{Var}\left[\int_0^t g(s) dW_s\right] = \int_0^t g(s)^2 ds. \quad (5.9)$$

Proof. Each term of (5.8) has $\mathbb{E}[g(t_k)(W_{t_{k+1}} - W_{t_k})] = g(t_k) \cdot 0 = 0$. For the variance, cross terms vanish because increments over disjoint intervals are independent and mean-zero, so

$$\text{Var}\left[\sum_k g(t_k) \Delta W_k\right] = \sum_k g(t_k)^2 \text{Var}(\Delta W_k) = \sum_k g(t_k)^2 \Delta t \xrightarrow{\Delta t \rightarrow 0} \int_0^t g(s)^2 ds. \quad \square$$

The second identity is the *Itô isometry*. It converts a statement about a stochastic integral into an ordinary calculus statement, and it is the workhorse of every variance computation in this chapter. Note where the $\sqrt{\Delta t}$ went: $\text{Var}(\Delta W_k) = \Delta t$, so summing $n \sim 1/\Delta t$ terms gives something finite.

Remark 5.7 (Scope). We do not need Itô’s lemma, martingale theory, or the general theory of semimartingales anywhere in this document; every SDE we meet has deterministic, state-independent g , and every marginal we need is Gaussian and computable by Proposition 5.6 alone. Stratonovich calculus is mentioned only to explain why the evaluation point matters. Riemannian Brownian motion (Chapter 7) is the one place where more care is genuinely required, and it is treated there.

5.5 Euler–Maruyama: the bridge in both directions

Truncating (5.7) at one step of size h and using the left-endpoint rule gives

$$\boxed{X_{t+h} = X_t + f(X_t, t) h + g(t) \sqrt{h} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I_d).} \quad (5.10)$$

This is the **Euler–Maruyama** scheme, and it is worth reading it twice, once in each direction.

SDE $\rightarrow$ discretisation. Given the continuous object, this is how one simulates it. The single change from the deterministic Euler method is the extra $g\sqrt{h}\epsilon$ term. Note $\sqrt{h}$, not h : with h the noise would vanish in the limit, by Section 5.2.

Discretisation $\rightarrow$ SDE. Compare (5.10) with the discrete chain (5.1) we started from. They are the same equation, with $s_k = g(t_k)\sqrt{\Delta t}$. So an SDE is not a new kind of object to be taken on faith: it is exactly the $\Delta t \rightarrow 0$ limit of the diffusive random walk, and Euler–Maruyama walks back from the limit to the walk. Whenever an SDE in this document is unclear, the reliable move is to write down (5.10) and read off what one step does.

Example 5.8 (Ornstein–Uhlenbeck, worked in full). Take $d = 1$, $f(x, t) = -\theta x$ with $\theta > 0$, and constant $g = \sigma$:

$$dX_t = -\theta X_t dt + \sigma dW_t, \quad X_0 = x_0. \quad (5.11)$$

The drift pulls towards the origin at a rate proportional to the current displacement; the noise pushes outward at a constant rate. The two balance, and the process settles into a stationary distribution rather than either collapsing or escaping.

Solving it. Multiply by the integrating factor $e^{\theta t}$. In the integral form (5.7) one obtains

$$X_t = x_0 e^{-\theta t} + \sigma \int_0^t e^{-\theta(t-s)} dW_s.$$

The first term is deterministic. The second is an Itô integral with deterministic integrand, so by Proposition 5.6 it is Gaussian with mean 0 and variance

$$\sigma^2 \int_0^t e^{-2\theta(t-s)} ds = \frac{\sigma^2}{2\theta} (1 - e^{-2\theta t}).$$

Hence

$$X_t \sim \mathcal{N}\left(x_0 e^{-\theta t}, \frac{\sigma^2}{2\theta} (1 - e^{-2\theta t})\right) \xrightarrow{t \rightarrow \infty} \mathcal{N}\left(0, \frac{\sigma^2}{2\theta}\right). \quad (5.12)$$

The mean decays exponentially and the variance saturates: the process *forgets* x_0 . That is exactly the property a forward noising process needs, and the OU process is the prototype of every diffusion model’s forward pass.

Checking Euler–Maruyama against it. With $\theta = 1.5$, $\sigma = 0.8$, $x_0 = 2$, $T = 2$, (5.12) gives mean 0.0995741 and variance 0.2128045. The Euler–Maruyama recursion $X_{k+1} = (1 - \theta h)X_k + \sigma\sqrt{h}\epsilon$ is itself Gaussian, so its mean and variance can be computed exactly rather than sampled:

N	mean	error	error $\times N$	variance error
20	0.0775191	2.21×10^{-2}	0.441	1.75×10^{-2}
100	0.0951050	4.47×10^{-3}	0.447	3.29×10^{-3}
500	0.0986784	8.96×10^{-4}	0.448	6.50×10^{-4}
2500	0.0993949	1.79×10^{-4}	0.448	1.30×10^{-4}
12500	0.0995383	3.58×10^{-5}	0.448	2.59×10^{-5}

The fourth column is constant: the error falls exactly like $1/N$, i.e. Euler–Maruyama is first order in the weak sense (correct distributions to $O(h)$). Computing the discrete moments in closed form rather than by Monte Carlo matters here — with 40,000 samples the sampling error alone is $\approx 5 \times 10^{-3}$ and would hide the entire table.

5.6 Drift and diffusion

In $dX_t = f(X_t, t) dt + g(t) dW_t$:

- f is the **drift**: the systematic, predictable part. Over a short window it contributes $f \cdot h$, a displacement in a definite direction. It is what survives averaging: $\mathbb{E}[X_{t+h} - X_t | X_t] = f(X_t, t)h + O(h^2)$.
- g is the **diffusion coefficient**: the random, unpredictable part. It contributes $g\sqrt{h}\epsilon$, with zero mean and standard deviation $g\sqrt{h}$. It is what survives squaring: $\text{Var}[X_{t+h} - X_t | X_t] = g(t)^2 h + O(h^2)$.

Remark 5.9 (The apparent paradox of the two scales). For small h , $\sqrt{h} \gg h$. So locally the noise dominates: zoom in far enough on a sample path and it looks like pure Brownian motion, with the drift invisible. Yet over a fixed interval $[0, T]$ both contribute at the same order — $\sum fh \rightarrow \int f ds$ and $\text{Var}(\sum g\sqrt{h}\epsilon) \rightarrow \int g^2 ds$, both $O(T)$.

There is no contradiction. The drift accumulates *coherently*, n terms of size h summing to $nh = T$. The noise accumulates *incoherently*, n terms of size $\sqrt{h}$ summing not to $n\sqrt{h}$ but to $\sqrt{n}\sqrt{h} = \sqrt{T}$ in standard deviation. Locally the larger per-step term is the noise; globally the two balance, because cancellation eats most of the noise and none of the drift. This is the same distinction as between a biased and an unbiased random walk, and it is worth carrying around: it explains why a diffusion model can be simultaneously very noisy step-to-step and very well behaved end-to-end.

5.7 Paths versus marginals

An SDE describes two different things, and separating them is essential for everything that follows.

	Object	Question it answers
Path	$(X_t)_{t \in [0, T]}$, one random trajectory	Where does <i>this</i> sample go?
Marginal	$p_t = \text{Law}(X_t)$, a density per t	Where is the <i>population</i> at time t ?

The marginal p_t forgets everything about how a sample arrived: it is the histogram over infinitely many independent runs at a fixed time. Two completely different processes can have identical marginals — we will use exactly that fact twice, once for the reverse SDE and once for the probability flow ODE (Section 6.4), and it is the reason diffusion and flow matching can be trained toward the same p_t by different means.

Generative modelling only ever cares about the marginals: we want $p_T \approx p_{\text{data}}$. The paths are a means.

5.8 Fokker–Planck: how the population moves

So: if individual samples follow the SDE, what equation governs p_t ?

Start with the noise-free case $g = 0$, where the SDE is the ODE $\dot{X}_t = f(X_t, t)$. Probability mass is conserved — samples are neither created nor destroyed — and it is transported by the velocity field f . In one dimension, consider a small interval $[x, x + \delta]$. Mass flows in at the left at rate $p_t(x)f(x)$ and out at the right at rate $p_t(x + \delta)f(x + \delta)$; the density inside rises at the rate of the difference. Dividing by δ and letting $\delta \rightarrow 0$:

$$\partial_t p_t(x) = -\partial_x(p_t(x)f(x)), \quad \text{in general} \quad \partial_t p_t = -\nabla \cdot (p_t f). \quad (5.13)$$

This is the *continuity equation*, the same equation as for an incompressible fluid or an electric charge density, and it reappears as the foundation of flow matching in Chapter 11. The quantity $p_t f$ is the probability *flux*.

Now add noise. Independent Gaussian kicks spread mass symmetrically in every direction — they do not transport it anywhere on average, they smear it. The operator that describes local smearing is the Laplacian, and the resulting equation is:

Theorem 5.10 (Fokker–Planck equation). *Let (X_t) solve $dX_t = f(X_t, t) dt + g(t) dW_t$ with $X_0 \sim p_0$. Then the marginals $p_t = \text{Law}(X_t)$ satisfy*

$$\partial_t p_t(x) = \underbrace{-\nabla \cdot (p_t(x)f(x, t))}_{\text{transport}} + \underbrace{\frac{1}{2}g(t)^2 \Delta p_t(x)}_{\text{diffusion}}. \quad (5.14)$$

Two remarks make (5.14) memorable rather than magical.

First, the **factor** $\frac{1}{2}$ is the $\sqrt{\Delta t}$ scaling resurfacing. A one-step kick of standard deviation $g\sqrt{h}$ convolves p_t with $\mathcal{N}(0, g^2 h)$. Taylor-expanding a convolution with a mean-zero kernel of variance v gives $p + \frac{v}{2}\Delta p + O(v^2)$ — the first-order term vanishes by symmetry and the second-order term carries the $\frac{1}{2}$ from the Taylor coefficient. Substituting $v = g^2 h$ and dividing by h yields $\frac{1}{2}g^2 \Delta p$.

Second, the **heat equation is the special case** $f = 0$, g constant: $\partial_t p = \frac{1}{2}g^2 \Delta p$. Diffusion of probability and diffusion of heat are the same mathematics, which is why the term “diffusion model” is literal and not a metaphor.

Remark 5.11 (What Fokker–Planck buys us). It converts a statement about random trajectories into a deterministic PDE about densities. Every “these two processes have the same marginals” argument in this document is proved the same way: write down both Fokker–Planck equations and check they coincide. That is exactly how the reverse-time SDE (Theorem 6.8) and the probability flow ODE (Theorem 6.9) are established.

5.9 DDPM is a VP-SDE in disguise

We can now do the limit that connects the discrete diffusion-model literature to the continuous one. This section and the next are the payoff of the chapter.

DDPM (Ho et al., 2020; in the notation of Song et al., 2021, §2.2) defines a discrete Markov chain over N steps with a schedule $0 < \beta_1, \dots, \beta_N < 1$:

$$x_k = \sqrt{1 - \beta_k} x_{k-1} + \sqrt{\beta_k} \epsilon_k, \quad \epsilon_k \sim \mathcal{N}(0, I). \quad (5.15)$$

The factor $\sqrt{1 - \beta_k}$ on the signal is what makes this *variance preserving*: if $\text{Var}(x_{k-1}) = 1$ then $\text{Var}(x_k) = (1 - \beta_k) + \beta_k = 1$.

To take a continuum limit we must say how β_k shrinks as N grows. The only consistent choice — by the very scaling argument of Section 5.2, since the noise enters as $\sqrt{\beta_k}$ and must be $O(\sqrt{\Delta t})$ — is

$$\beta_k = \beta(t_k) \Delta t, \quad \Delta t = \frac{1}{N}, \quad t_k = k \Delta t, \quad (5.16)$$

for a fixed continuous schedule $\beta : [0, 1] \rightarrow \mathbb{R}_{>0}$.

Now expand. Since $\beta_k = O(\Delta t)$ is small,

$$\sqrt{1 - \beta_k} = 1 - \frac{1}{2}\beta_k + O(\beta_k^2) = 1 - \frac{1}{2}\beta(t_k)\Delta t + O(\Delta t^2). \quad (5.17)$$

Substituting into (5.15) and subtracting x_{k-1} :

$$x_k - x_{k-1} = (\sqrt{1 - \beta_k} - 1)x_{k-1} + \sqrt{\beta_k} \epsilon_k \quad (5.18)$$

$$= -\frac{1}{2}\beta(t_k) x_{k-1} \Delta t + \sqrt{\beta(t_k)} \sqrt{\Delta t} \epsilon_k + O(\Delta t^2). \quad (5.19)$$

Compare with Euler–Maruyama (5.10): the first term is $f \Delta t$ with $f(x, t) = -\frac{1}{2}\beta(t)x$, and the second is $g\sqrt{\Delta t}\epsilon$ with $g(t) = \sqrt{\beta(t)}$. Therefore

$$\boxed{dX_t = -\frac{1}{2}\beta(t) X_t dt + \sqrt{\beta(t)} dW_t} \quad (\text{VP-SDE}) \quad (5.20)$$

which is Song et al.’s variance-preserving SDE. Note it is an OU process (Example 5.8) with time-varying coefficients, so its marginals are available in closed form: with $\bar{\beta}(t) := \int_0^t \beta(s) ds$ and $\alpha_t := e^{-\bar{\beta}(t)/2}$,

$$X_t | X_0 \sim \mathcal{N}(\alpha_t X_0, (1 - \alpha_t^2)I). \quad (5.21)$$

This is precisely the formula SigmaDock’s R3Diffuser implements.

Remark 5.12 (Numerical confirmation, and one honest limitation). Both (5.15) and (5.21) are Gaussian, so the convergence can be checked exactly. With β linear from 0.1 to 20 and $x_0 = 1$, the limit gives mean $\alpha_1 x_0 = 0.0065716$:

N	discrete mean	error	error $\times N$
10	$\beta_k = 2.00 \geq 1$ —	undefined	
50	0.0024770	4.09×10^{-3}	0.205
200	0.0053662	1.21×10^{-3}	0.241
1000	0.0063211	2.51×10^{-4}	0.251
5000	0.0065211	5.05×10^{-5}	0.252
20000	0.0065590	1.26×10^{-5}	0.253

The last column is constant, confirming the $O(\Delta t)$ error promised by the Taylor expansion.

The first row is not a bug. The DDPM parametrisation (5.15) requires $\beta_k < 1$ for $\sqrt{1 - \beta_k}$ to be real, and with $\beta_{\max} = 20$ that forces $\Delta t < 1/20$, i.e. $N > 20$. The discrete formulation carries a step-size restriction that the SDE does not: (5.20) is perfectly well defined for any β . This is a small but genuine advantage of the continuous view, and a good illustration that the SDE is not merely a re-notation of the chain.

5.10 SMLD is a VE-SDE

The other family perturbs without shrinking the signal:

$$x_t = x_0 + \sigma(t) \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (5.22)$$

with σ increasing — typically geometric, $\sigma(t) = \sigma_{\min}(\sigma_{\max}/\sigma_{\min})^t$. Here the variance *grows* without bound, hence *variance exploding*.

Which SDE has these marginals? There is no drift to find: (5.22) has mean x_0 for all t , so $f = 0$. It remains to choose g so that the accumulated variance matches. By Proposition 5.6, a driftless SDE $dX_t = g(t) dW_t$ started at x_0 has

$$\text{Var}(X_t) = \int_0^t g(s)^2 ds. \quad (5.23)$$

We need this to equal $\sigma(t)^2 - \sigma(0)^2$. Differentiating both sides in t :

$$g(t)^2 = \frac{d}{dt} \sigma(t)^2 \quad \implies \quad \boxed{dX_t = \sqrt{\frac{d\sigma^2(t)}{dt}} dW_t} \quad (\text{VE-SDE}) \quad (5.24)$$

which is Song et al.’s variance-exploding SDE. The derivation is a one-line consequence of the Itô isometry, which is why Proposition 5.6 was worth proving.

For the geometric schedule, $\sigma^2(t) = \sigma_{\min}^2 r^{2t}$ with $r = \sigma_{\max}/\sigma_{\min}$, so $\frac{d\sigma^2}{dt} = 2\sigma(t)^2 \ln r$.

Remark 5.13 (Numerical confirmation). With $\sigma_{\min} = 0.01$, $\sigma_{\max} = 50$, the target accumulated variance is $\sigma(1)^2 - \sigma(0)^2 = 2499.9999$. Riemann sums $\sum_k g(t_k)^2 \Delta t$: 947.9 ($N = 10$), 2293.1 ($N = 100$), 2478.8 ($N = 1000$), 2497.9 ($N = 10^4$) — relative errors 6.2×10^{-1} , 8.3×10^{-2} , 8.5×10^{-3} , 8.5×10^{-4} , again $O(\Delta t)$. Convergence is slower in absolute terms than the VP case because the geometric schedule makes g^2 vary over four orders of magnitude across $[0, 1]$.

5.11 Summary

Question	Answer
Why $\sqrt{\Delta t}$?	Independent noise adds in variance; $n \sim 1/\Delta t$ steps of size $\sqrt{\Delta t}$ give $O(1)$. Any other exponent collapses or explodes (Section 5.2).
What is W_t ?	The limit of that walk: independent Gaussian increments with variance = elapsed time.
What is dW_t ?	Not a derivative — W is nowhere differentiable. Shorthand for an Itô integral (Definition 5.5).
Why left endpoints?	Keeps the integrand non-anticipating; gives the Itô isometry (Proposition 5.6).
How do I simulate?	Euler–Maruyama (5.10).
How does p_t move?	Fokker–Planck (5.14): transport + smoothing.
DDPM in continuous time?	VP-SDE (5.20), via $\sqrt{1-\beta} \approx 1 - \beta/2$.
SMLD in continuous time?	VE-SDE, via $g^2 = d\sigma^2/dt$.

What is still missing is the direction of travel. Everything above runs *forwards*, from data to noise, and destroys information. The next chapter asks how to run it backwards — and that is where the score appears.

Chapter 6

Diffusion Models in $\mathbb{R}^d$

We now build the first of the two generative-modelling machineries identified in Section 2.6: a way to construct a probability path $(p_t)_{t \in [0,1]}$ implicitly, as the law of a stochastic process, and to sample from $p_1 = p_{\text{data}}$ by simulating that process. Everything in this chapter is developed in $\mathbb{R}^d$ first; the manifold (in particular $SO(3)$) generalisation is Chapter 7, and SigmaDock’s actual use of both is reconstructed in full, with code correspondence, in Chapter 10.

6.1 What the score is, before we learn it

The score function

$$s_t(x) := \nabla_x \log p_t(x) \quad (6.1)$$

will be the central learned object of this chapter, so it is worth understanding it as a geometric object first, independently of any algorithm.

It is a direction, and it lives on the sample space. For each x , $s_t(x)$ is a vector in $\mathbb{R}^d$ — the direction in which the log-density increases fastest, with magnitude equal to the rate of that increase. It is a vector field on the same space the data lives in. Standing at x and asked “which way is it to somewhere more typical?”, $s_t(x)$ is the answer.

The normalising constant disappears. This is the property that makes the score usable at all. Write $p_t(x) = \tilde{p}_t(x)/Z$ with Z an unknown constant. Then

$$\nabla_x \log p_t(x) = \nabla_x [\log \tilde{p}_t(x) - \log Z] = \nabla_x \log \tilde{p}_t(x), \quad (6.2)$$

because $\log Z$ does not depend on x . Densities are hard precisely because $Z = \int \tilde{p}$ is an intractable integral; the score sidesteps it entirely. One can know the score everywhere and still not know a single value of p_t .

Example 6.1 (The score of a one-dimensional Gaussian). Let $p(x) = \mathcal{N}(x; \mu, \sigma^2)$. Then

$$\log p(x) = -\frac{(x - \mu)^2}{2\sigma^2} - \log(\sigma\sqrt{2\pi}), \quad s(x) = \frac{d}{dx} \log p(x) = -\frac{x - \mu}{\sigma^2}.$$

Read off three things:

- **It points home.** For $x > \mu$ the score is negative, for $x < \mu$ positive. It always points towards the mean.

- **It is a restoring force with spring constant $1/\sigma^2$.** The score of a Gaussian is exactly the force of a harmonic oscillator. Following it downhill in $-s$ is gradient descent on $\frac{1}{2}(x - \mu)^2/\sigma^2$.
- **It blows up as $\sigma \rightarrow 0$.** At fixed displacement $x - \mu$, $\|s\| \propto 1/\sigma^2$. A sharply concentrated distribution has an enormous score just outside its support.

The last point has a direct consequence for training. Under the noising kernel $p_t(\cdot | x_0) = \mathcal{N}(\alpha_t x_0, \sigma_t^2 I)$ the conditional score is

$$\nabla_x \log p_t(x | x_0) = -\frac{x - \alpha_t x_0}{\sigma_t^2}, \quad (6.3)$$

whose typical magnitude is $\sqrt{d}/\sigma_t$, diverging as $\sigma_t \rightarrow 0$. A plain mean-squared-error loss on the score would therefore be dominated entirely by the smallest-noise samples. Every practical implementation rescales — SigmaDock multiplies the network output by $1/\sigma_t$ and the loss by σ_t^2 , so that the network in fact regresses the *dimensionless* quantity $\sigma_t s_t$. That is not an arbitrary engineering choice but a direct response to (6.3).

Remark 6.2 (Why the score, and not the density?). Three reasons, in increasing order of importance:

1. It is normalisation-free, as above.
2. It is exactly what Langevin dynamics needs (Section 2.4): the update $x \leftarrow x + \frac{\eta}{2}s(x) + \sqrt{\eta}\epsilon$ requires no density values.
3. It is exactly the missing ingredient in the reverse-time dynamics (Section 6.3.2). This is the deep reason: the score is not chosen because it is convenient, it is forced by the structure of time reversal.

6.2 Denoising score matching

Section 2.4 showed that knowing $\nabla_x \log p_{\text{data}}$ would suffice for sampling, via Langevin dynamics, but left open how to *learn* this score from samples alone. The direct approach — fit a network $s_\theta(x) \approx \nabla_x \log p_{\text{data}}(x)$ by minimising $\mathbb{E}_{x \sim p_{\text{data}}} [\|s_\theta(x) - \nabla_x \log p_{\text{data}}(x)\|^2]$ — is circular: the target $\nabla_x \log p_{\text{data}}$ is exactly the unknown quantity. The resolution, exactly analogous in structure to Proposition 2.3’s role in Section 2.6, is to introduce a *noised* version of the data whose score *is* computable in closed form.

Definition 6.3 (Gaussian perturbation kernel). For $\sigma > 0$, let $p_\sigma(x | z) = \mathcal{N}(x | z, \sigma^2 I_d)$, and let $p_\sigma(x) = \int p_\sigma(x | z) p_{\text{data}}(z) dz$ be the corresponding *noised marginal* (Proposition 2.3, with $p_\sigma(\cdot | z)$ playing the role of the conditional density there).

By Remark 2.13, the *conditional* score is available in closed form:

$$\nabla_x \log p_\sigma(x | z) = -\frac{x - z}{\sigma^2}. \quad (6.4)$$

What we actually want, to run Langevin dynamics targeting p_σ rather than p_{data} directly (an approximation we return to and justify via the noise ladder / SDE construction of Section 6.3), is the *marginal* score $\nabla_x \log p_\sigma(x)$, which is *not* available in closed form, since it requires the (intractable) integral defining p_σ . The following theorem is the resolution, and its proof is the prototype for every “conditional loss has the same minimiser as marginal loss” argument in this document, including Theorem 11.25 in Chapter 11.

Theorem 6.4 (Denoising score matching). *Define*

$$\mathcal{L}_{\text{ESM}}(\theta) = \mathbb{E}_{x \sim p_\sigma} [\|s_\theta(x) - \nabla_x \log p_\sigma(x)\|^2] \quad (\text{explicit/marginal score matching}),$$

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{z \sim p_{\text{data}}, x \sim p_\sigma(\cdot | z)} [\|s_\theta(x) - \nabla_x \log p_\sigma(x | z)\|^2] \quad (\text{denoising score matching}). \quad (6.5)$$

Then $\mathcal{L}_{\text{ESM}}(\theta) = \mathcal{L}_{\text{DSM}}(\theta) + C$ with C independent of θ ; in particular the two losses have identical gradients in θ and the same minimisers.

Proof. Expand the square in $\mathcal{L}_{\text{ESM}}$ under the marginal $p_\sigma(x)$:

$$\mathcal{L}_{\text{ESM}}(\theta) = \mathbb{E}_{x \sim p_\sigma} [\|s_\theta(x)\|^2] - 2 \mathbb{E}_{x \sim p_\sigma} [\langle s_\theta(x), \nabla_x \log p_\sigma(x) \rangle] + \mathbb{E}_{x \sim p_\sigma} [\|\nabla_x \log p_\sigma(x)\|^2].$$

The third term does not depend on θ ; call it C_1 . The first term already matches $\mathcal{L}_{\text{DSM}}$'s first term, since $x \sim p_\sigma$ (marginally) is the same distribution as x obtained by first drawing $z \sim p_{\text{data}}$ then $x \sim p_\sigma(\cdot | z)$ (Proposition 2.3), so $\mathbb{E}_{x \sim p_\sigma} [\|s_\theta(x)\|^2] = \mathbb{E}_{z, x} [\|s_\theta(x)\|^2]$. It remains to handle the cross term. Using $p_\sigma(x) \nabla_x \log p_\sigma(x) = \nabla_x p_\sigma(x) = \int \nabla_x p_\sigma(x | z) p_{\text{data}}(z) dz = \int p_\sigma(x | z) \nabla_x \log p_\sigma(x | z) p_{\text{data}}(z) dz$ (differentiating under the integral sign, and using $\nabla_x p_\sigma(x | z) = p_\sigma(x | z) \nabla_x \log p_\sigma(x | z)$),

$$\begin{aligned} \mathbb{E}_{x \sim p_\sigma} [\langle s_\theta(x), \nabla_x \log p_\sigma(x) \rangle] &= \int \langle s_\theta(x), \nabla_x \log p_\sigma(x) \rangle p_\sigma(x) dx \\ &= \int \int \langle s_\theta(x), \nabla_x \log p_\sigma(x | z) \rangle p_\sigma(x | z) p_{\text{data}}(z) dz dx \\ &= \mathbb{E}_{z \sim p_{\text{data}}, x \sim p_\sigma(\cdot | z)} [\langle s_\theta(x), \nabla_x \log p_\sigma(x | z) \rangle], \end{aligned}$$

which is exactly $\mathcal{L}_{\text{DSM}}$'s cross term. Collecting terms, $\mathcal{L}_{\text{ESM}}(\theta) = \mathcal{L}_{\text{DSM}}(\theta) + C_1 - C_2$, where $C_2 = \mathbb{E}_{z, x} [\|\nabla_x \log p_\sigma(x | z)\|^2]$ is also θ -independent, giving $C = C_1 - C_2$. $\square$

The practical content of Theorem 6.4: to train a score network, draw a clean data point $z \sim p_{\text{data}}$ (a sample from the training set), draw a noised version $x = z + \sigma \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I_d)$ (sampling from $p_\sigma(\cdot | z)$ directly, rather than needing the marginal), and regress $s_\theta(x)$ against the *known*, closed-form target $-\varepsilon/\sigma$ (substituting $x - z = \sigma \varepsilon$ into (6.4)). No integral over p_{data} or p_σ is ever evaluated.

6.3 The SDE perspective, and why a single noise level is not enough

A single σ is not sufficient for high-quality sampling: for large σ , $p_\sigma \approx p_{\text{init}}$ is easy to sample from via Langevin dynamics (the perturbation dominates, and the resulting density is close to an isotropic Gaussian, which mixes quickly under Langevin dynamics) but far from p_{data} , and vice versa for small σ . Song and Ermon's original resolution (Song & Ermon, *Generative Modeling by Estimating Gradients of the Data Distribution*, NeurIPS 2019) trains a single, noise-conditional network $s_\theta(x, \sigma)$ across a decreasing ladder of noise levels $\sigma_1 > \dots > \sigma_N$ and runs annealed Langevin dynamics, decreasing σ over the course of sampling. Song et al. (*Score-Based Generative Modeling through Stochastic Differential Equations*, ICLR 2021) subsequently observed that this discrete ladder is the time-discretisation of a single, continuous-time stochastic process, and it is this continuous formulation, not the discrete ladder, that both SigmaDock's actual implementation and this document use throughout.

Write the *noising* process in the reversed time variable $s = 1 - t$ (Remark 0.2), so it runs from data ($s = 0$) to noise ($s = 1$):

$$dY_s = f(Y_s, s) ds + g(s) dW_s, \quad Y_0 \sim p_{\text{data}}, \quad (6.6)$$

with drift $f : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$, diffusion coefficient $g : [0, 1] \rightarrow \mathbb{R}_{\geq 0}$, and W a standard Wiener process (Definition 2.14). Write p_t for the law of Y_{1-t} (i.e. time is relabelled to our global generation-time convention, so p_0 is the law of Y_1 , the fully-noised end, and p_1 is the law of $Y_0 = p_{\text{data}}$).

Example 6.5 (Variance-exploding (VE) SDE). $f \equiv 0$, $g(s)^2 = \frac{d}{ds}\sigma(s)^2$ for an increasing schedule $\sigma(s)$. Then (6.6) integrates in closed form to $Y_s = Y_0 + \sigma(s)\varepsilon$, $\varepsilon \sim \mathcal{N}(0, I_d)$ (a direct check: this is a driftless SDE whose solution is Brownian motion time-changed by $\sigma(s)^2$, and its marginal variance at time s is exactly $\sigma(s)^2$ by Itô isometry, which we do not derive here but which matches the perturbation kernel of Definition 6.3 with $\sigma = \sigma(s)$) — exactly recovering the noise ladder of Section 6.2 as the time slices $\sigma(s_i) = \sigma_i$ of a single continuous process. This is the SDE family used, as Chapter 10 confirms directly against the code, by SigmaDock’s *rotational* diffusion process.

Example 6.6 (Variance-preserving (VP) SDE). $f(x, s) = -\frac{1}{2}\beta(s)x$, $g(s) = \sqrt{\beta(s)}$, for a noise-rate schedule $\beta(s) \geq 0$. Solving the resulting (linear, hence exactly solvable) SDE gives

$$Y_s = \alpha(s) Y_0 + \sqrt{1 - \alpha(s)^2} \varepsilon, \quad \alpha(s) = \exp\left(-\frac{1}{2} \int_0^s \beta(r) dr\right),$$

i.e. $p_s(\cdot | Y_0) = \mathcal{N}(\alpha(s)Y_0, (1 - \alpha(s)^2)I_d)$ — unlike the VE case, the *signal* Y_0 is itself attenuated, and the total variance $\alpha(s)^2 \cdot \text{Var}(Y_0) + (1 - \alpha(s)^2)$ stays bounded (indeed, if $\text{Var}(Y_0) = 1$, stays exactly at 1) as $s \rightarrow 1$, rather than exploding — hence the name. This is the SDE family used, as Chapter 10 again confirms directly, by SigmaDock’s *translational* diffusion process; the exact solution just derived is this document’s own re-derivation of the closed-form perturbation kernel that the code computes via R3Diffuser’s `integrated_beta_t`, not a claim quoted from elsewhere.

We record, once, the general fact that both examples instantiate, since it is exactly Definition 6.3’s Gaussian-perturbation-kernel construction generalised to an SDE with (possibly time- and state-dependent) linear drift.

Proposition 6.7 (Perturbation kernel of a linear SDE). *If $f(x, s) = -\frac{1}{2}\beta(s)x$ (including $\beta \equiv 0$, the VE case) and $g(s)^2 = \frac{d}{ds}\sigma(s)^2$ is otherwise free, the solution of (6.6) has Gaussian conditional law $p_s(\cdot | Y_0) = \mathcal{N}(\alpha(s)Y_0, \beta(s)^2 I_d)$ for schedules $\alpha(s), \beta(s)$ determined by f, g (linear SDEs are exactly solvable; we do not re-derive the general variation-of-constants formula here, having exhibited it explicitly for the two cases used by this document in Examples 6.5 and 6.6).*

6.3.1 Score, weighting, and the training objective in continuous time

Given a schedule as in Proposition 6.7, the conditional score is again available in closed form by Remark 2.13, and the continuous-time denoising score matching objective (obtained from the single-noise-level objective (6.5) by letting the fixed σ become the time-dependent $\beta(s)$ of Proposition 6.7 and averaging over $s \sim \mathcal{U}(0, 1)$ with weight $\lambda(s)$ — the literal continuous-time analogue of Theorem 6.4 applied at every noise level simultaneously) is

$$\mathcal{L}(\theta) = \mathbb{E}_{s \sim \mathcal{U}(0,1), Y_0 \sim p_{\text{data}}, Y_s \sim p_s(\cdot | Y_0)} \left[\lambda(s) \| s_{\theta}(Y_s, s) - \nabla_{Y_s} \log p_s(Y_s | Y_0) \|^2 \right], \quad (6.7)$$

for a positive weighting function $\lambda(s)$. The role of λ is purely practical, not conceptual: by Theorem 6.4 (applied noise-level by noise-level, which is valid since the theorem’s proof never uses anything special about a fixed σ , only that x is drawn from *some* fixed conditional given z), *any* positive $\lambda(s)$ gives a loss with the same minimiser, $s_\theta(x, s) = \nabla_x \log p_s(x)$ — but the target’s own scale, $\|\nabla_x \log p_s(x | z)\| = \|\varepsilon\|/\beta(s)$ in Proposition 6.7’s notation, varies enormously across s (it blows up as $\beta(s) \rightarrow 0$, i.e. as $s \rightarrow 0$, towards clean data), so an unweighted loss lets the large-noise (easy, large-target) terms dominate the gradient at the expense of the small-noise (hard, small-target) terms that determine fine sample detail. The standard choice, $\lambda(s) = \beta(s)^2$ (i.e. $\lambda(s) = 1/\mathbb{E}\|\nabla_x \log p_s(x | z)\|^2$ up to the constant d , since $\mathbb{E}\|\varepsilon/\beta(s)\|^2 = d/\beta(s)^2$ for $\varepsilon \sim \mathcal{N}(0, I_d)$), exactly cancels this scale variation, and reappears, generalised to $SO(3)$, as the `score_scaling` construction reconstructed precisely in Section 7.2.2.

6.3.2 Why running time backwards is not enough

Equation (6.6) destroys structure (runs from data to noise); generation needs the reverse. Before stating the theorem that provides it, it is worth seeing why the obvious idea fails — because the reason is exactly the reason the score appears, and the formula looks arbitrary otherwise.

The naive attempt. For an ODE $\dot{X}_t = f(X_t, t)$, reversing is trivial: integrate $\dot{X}_s = -f(X_s, \cdot)$ and the trajectory retraces itself exactly. Each path is reversible because it is deterministic. So one might hope that flipping the sign of dt in the SDE reverses it too.

It does not, and a single example shows why. Take pure Brownian motion, $f = 0$, $g = 1$, started at $X_0 = 0$. Flipping the sign of dt leaves $dX = dW$, which is the *same* process: a driftless random walk. Run it from $X_1 \sim \mathcal{N}(0, 1)$ and after one unit of time you get $\mathcal{N}(0, 2)$ — the variance keeps growing. Running the clock backwards did not undo the spreading; it produced more of it.

Why. The forward process is not path-reversible, because it is not deterministic. Many different starting points lead to the same x at time t , and the forward dynamics has forgotten which one it was. Reversal therefore cannot be a pathwise operation — it must be a statement about distributions, and it needs the information the forward process threw away.

That information is precisely *where the mass came from*. If at time t we are at x , the more likely origins are the ones in regions where p_t is large. Undoing a diffusion step therefore requires a systematic pull towards higher density, on top of the reversed drift. Quantitatively, the local direction of steepest increase of the log-density is

$$\nabla_x \log p_t(x),$$

and this is exactly the extra term that appears below.

A sanity check on the sign. Consider the Brownian example again with $p_t = \mathcal{N}(0, t)$, whose score is $\nabla \log p_t(x) = -x/t$. The reverse dynamics acquires a term $g^2 \nabla \log p_t = -x/t$: a restoring force pointing back towards the origin, whose strength grows as $t \rightarrow 0$. That is exactly what is needed to counteract the spreading, and it could not have come from the drift, which is zero. The score is not a technical device; it is the missing information, and it is missing for a structural reason.

6.3.3 The reverse-time SDE

The following classical theorem makes this precise, and it is what licenses training a score network and then *simulating backwards* to generate samples.

Theorem 6.8 (Anderson’s reverse-time SDE). *Let $(Y_s)_{s \in [0,1]}$ solve (6.6) with marginal laws $q_s = \text{Law}(Y_s)$. Under mild regularity conditions (existence and smoothness of q_s for all s , and suitable growth conditions on f, g ; see Anderson, Reverse-Time Diffusion Equation Models, Stochastic Processes and their Applications, 1982), the time-reversed process $\tilde{Y}_s := Y_{1-s}$ solves the SDE*

$$d\tilde{Y}_s = \left[-f(\tilde{Y}_s, 1-s) + g(1-s)^2 \nabla_x \log q_{1-s}(\tilde{Y}_s) \right] ds + g(1-s) d\bar{W}_s,$$

where $\bar{W}$ is a Wiener process for the reversed filtration (a technical point about which σ -algebra the increments are independent of, which does not affect how the SDE is simulated and which we do not elaborate on further).

We do not reprove Theorem 6.8 (it is a statement about how the Fokker–Planck equation governing q_s ’s time evolution transforms under time reversal, requiring Itô calculus we have not developed; see the cited reference, or Haussmann & Pardoux, *Time Reversal of Diffusions*, Annals of Probability, 1986, for the technically complete statement). We translate it into our generation-time convention. The reversed process $\tilde{Y}$ already runs *forward* in its own time argument, so no second reversal is needed — the translation is a pure renaming. Set

$$X_t := \tilde{Y}_t = Y_{1-t}, \quad \text{hence} \quad p_t = \text{Law}(X_t) = q_{1-t},$$

which gives $X_0 = Y_1 \sim p_{\text{init}}$ (the fully-noised end) and $X_1 = Y_0 \sim p_{\text{data}}$, as our convention demands. Renaming s to t in Theorem 6.8’s SDE and substituting $q_{1-t} = p_t$,

$$dX_t = \left[-f(X_t, 1-t) + g(1-t)^2 \nabla_x \log p_t(X_t) \right] dt + g(1-t) d\bar{W}_t, \quad X_0 \sim p_{\text{init}}, \quad (6.8)$$

integrated *forward* in t from 0 to 1. This is the SDE that generation simulates, and it requires exactly one previously-undetermined quantity: the score $\nabla_x \log p_t(x)$, precisely what (6.7) trains a network to approximate. Substituting $s_\theta \approx \nabla_x \log p_t$ into (6.8) and discretising by Euler–Maruyama (the direct generalisation of (2.2)’s update rule to a state- and time-dependent drift/diffusion) gives the sampling algorithm; Chapter 10 exhibits the exact discretisation SigmaDock’s code uses, which matches this recipe precisely for the translational (VP) component.

6.4 The probability flow ODE, and the bridge to flow matching

6.4.1 How can a deterministic process match a stochastic one?

The result below should look surprising at first, and it is worth pausing on the surprise rather than stepping over it. We are about to claim that a *deterministic* ODE — no randomness at all after the initial draw — produces the same distributions p_t at every time as a stochastic process whose paths visibly jitter.

The resolution is Section 5.7: *paths and marginals are different objects*. Nothing forces two processes with the same marginals to have anything else in common. The trajectories differ completely — SDE paths are nowhere differentiable, ODE paths are smooth; SDE paths starting from the same point diverge, ODE paths never cross. Only the histograms agree.

A picture in one dimension. Think of a cloud of dust spreading out. There are two ways to produce the same expanding cloud:

- let every speck jitter randomly (diffusion), or
- give every speck a definite outward velocity, faster the further out it already is (transport).

The first is the SDE, the second the ODE. A photograph at any fixed time cannot tell them apart; a video following one speck can.

Where the factor $\frac{1}{2}$ comes from. This is the one place a reader is likely to suspect a typo, so it is worth deriving. Fokker–Planck for the SDE reads

$$\partial_t p_t = -\nabla \cdot (p_t b) + \frac{1}{2} g^2 \Delta p_t,$$

whereas an ODE with velocity u_t obeys the continuity equation with *no* second-order term,

$$\partial_t p_t = -\nabla \cdot (p_t u_t).$$

To match them we must absorb the Laplacian into a divergence. The identity that does it is

$$\Delta p_t = \nabla \cdot (\nabla p_t) = \nabla \cdot \left(p_t \frac{\nabla p_t}{p_t} \right) = \nabla \cdot (p_t \nabla \log p_t), \quad (6.9)$$

using $\nabla \log p_t = \nabla p_t / p_t$. Substituting,

$$\partial_t p_t = -\nabla \cdot (p_t b) + \frac{1}{2} g^2 \nabla \cdot (p_t \nabla \log p_t) = -\nabla \cdot \left(p_t \left[b - \frac{1}{2} g^2 \nabla \log p_t \right] \right),$$

which is a continuity equation with velocity $u_t = b - \frac{1}{2} g^2 \nabla \log p_t$.

So the $\frac{1}{2}$ is inherited directly from the $\frac{1}{2}$ in Fokker–Planck, which in turn came from the second-order Taylor coefficient of a convolution (Theorem 5.10). The score enters with *half* the coefficient it has in the reverse SDE precisely because the reverse SDE keeps its own noise term, which supplies the other half. Equation (6.9) is the single algebraic step underlying the whole correspondence, and it is worth remembering on its own: *a Laplacian is a divergence of a probability flux with velocity $\nabla \log p$.*

Theorem 6.9 (Probability flow ODE). *Under the same conditions as Theorem 6.8, the deterministic ODE*

$$\frac{dX_t}{dt} = u_t(X_t) := -f(X_t, 1-t) + \frac{1}{2} g(1-t)^2 \nabla_x \log p_t(X_t) \quad (6.10)$$

has the same marginal laws $(p_t)_{t \in [0,1]}$ as (6.8) (and hence as the original forward process (6.6), relabelled).

Proof sketch. Both (6.8) and (6.10) are constructed so that their associated Fokker–Planck / continuity equations (the PDE governing how a density evolves under the dynamics; for an SDE $dX = b(X, t)dt + g(t)dW$, this is $\partial_t p_t = -\nabla \cdot (b p_t) + \frac{1}{2} g(t)^2 \Delta p_t$, and for an ODE $dX/dt = u_t(X)$ it is the driftless-diffusion special case $\partial_t p_t = -\nabla \cdot (u_t p_t)$, which is exactly Theorem 11.10 of Chapter 11, proved there in full) coincide: substituting $b = -f(\cdot, 1-t) + g(1-t)^2 \nabla \log p_t$ into the SDE’s Fokker–Planck equation and using $\nabla \cdot (p_t \nabla \log p_t) = \nabla \cdot (\nabla p_t) = \Delta p_t$ shows the diffusion term $\frac{1}{2} g(1-t)^2 \Delta p_t$ exactly cancels against half of the

drift’s contribution, leaving the ODE’s continuity equation with u_t as in (6.10). We do not carry out this cancellation in full generality here (it requires the Fokker–Planck equation, which Chapter 11 develops in the driftless case needed there), but note it is a standard, mechanical computation once Theorem 11.10 is available — see, e.g., Song et al. (2021, Appendix D.1) for the complete derivation in this SDE setting. $\square$

Theorem 6.9 is exactly the content of Section 6.4’s claim, made precise: *every* score-based diffusion model secretly defines a deterministic flow with the same marginals, whose velocity field (6.10) is an explicit, invertible function of the score. Chapter 11 develops the flow-matching framework as though starting from scratch, and Section 6.4 (there) proves the general form of this correspondence; what Theorem 6.9 establishes here is only the specific fact, needed to state Chapter 10’s Remark on “items requiring verification”, that SigmaDock’s own sampler *could* in principle be run as a deterministic ODE (setting the SDE’s injected noise to zero) with unchanged marginals — and Chapter 10 confirms, directly from the code, that this is in fact the *default* configuration SigmaDock ships with (`conf/sampling/base.yaml`’s default `noise_scale:0.0`), narrowing the “stochastic vs. deterministic” distinction between the two methods considerably.

6.4.2 Reverse SDE versus probability flow ODE, side by side

	Reverse SDE	Probability flow ODE
Stochastic?	yes, injects fresh noise	no, deterministic given X_0
Same marginals p_t ?	yes	yes
Same trajectories?	no	no — and not even the same <i>kind</i>
Paths	nowhere differentiable	smooth, never crossing
Needs the score?	yes	yes
Score coefficient	g^2	$\frac{1}{2}g^2$
Repeat from same X_1	different output each time	identical output
Exact likelihood?	no	yes, by the instantaneous change-of-variables formula
Discretisation error	$O(\sqrt{h})$ strong, $O(h)$ weak	$O(h^p)$ for a p -th order solver
Error self-correcting?	partly — noise re-mixes	no — errors accumulate

Three of these rows deserve a sentence.

Likelihoods. Because the ODE is a deterministic, invertible map, the change-of-variables formula applies and the exact log-likelihood of a sample can be computed by integrating $-\nabla \cdot u_t$ along the trajectory. The SDE offers no such thing. This is the main practical reason to prefer the ODE formulation even when sampling quality is equal.

Higher-order solvers. The ODE is a classical initial value problem, so Heun, Runge–Kutta and adaptive solvers apply directly and buy accuracy per function evaluation. For the SDE, high-order schemes exist but are much more delicate, because the $\sqrt{h}$ scaling caps the achievable order. This is why deterministic samplers dominate the few-step regime.

Self-correction. The injected noise in the SDE partially forgets accumulated error: if a step lands in a slightly wrong place, subsequent noise plus the score’s restoring pull can bring the sample back onto the data manifold. The ODE has no such mechanism, so a systematically biased velocity field — for instance, one whose equivariance is broken — integrates its bias coherently over every step. That asymmetry is worth keeping in mind when comparing a diffusion sampler against an ODE sampler that share a backbone.

Caution 6.10 (Deterministic $\neq$ better; it is a different trade). It is tempting to read the table as a case for the ODE. It is not. The ODE wins on likelihood, on solver technology, and on reproducibility; the SDE wins on robustness to an imperfect score network, which in practice is always what one has. Which dominates is an empirical question about a particular model, not a theoretical one.

6.5 What you should now be able to do

- explain why running an SDE backwards in time is not time reversal, and why the score is exactly the information the forward process destroyed;
- derive the denoising score matching objective and say why regressing on the *conditional* score has the same minimiser as the marginal one;
- write down the reverse-time SDE and the probability flow ODE, and account for the factor $\frac{1}{2}$ that distinguishes them;
- state what a diffusion model gains and loses by sampling deterministically (Caution 6.10).

Chapter 7 repeats the construction on $SO(3)$, where every Gaussian has to be replaced by a heat kernel.

Chapter 7

Diffusion Models on $SO(3)$

Chapter 6 developed diffusion entirely in $\mathbb{R}^d$; by Proposition 3.69, this already suffices for the *translational* component of a pose. This chapter develops the rotational analogue: what does “add Gaussian noise” mean on $SO(3)$, where addition is undefined, and what replaces the closed-form Gaussian kernel of Definition 6.3?

7.1 Brownian motion on a Riemannian manifold

Motivation. In $\mathbb{R}^d$, Brownian motion can be built either as the limit of a random walk with i.i.d. Gaussian increments, or as the unique (in law) continuous process with independent Gaussian increments of variance proportional to elapsed time. On a curved manifold, “increment” must be reinterpreted: an increment is a step in a *tangent space*, which must then be mapped back onto the manifold.

Definition 7.1 (Manifold Brownian motion, via a random-walk limit). Fix $x_0 \in \mathcal{M}$ and a step size $\Delta\tau > 0$. Define a discrete process $x_0, x_{\Delta\tau}, x_{2\Delta\tau}, \dots$ by

$$x_{(k+1)\Delta\tau} = \exp_{x_{k\Delta\tau}}(\sqrt{\Delta\tau} \xi_k), \quad \xi_k \stackrel{\text{iid}}{\sim} \mathcal{N}(0, I_{\dim \mathcal{M}}) \text{ in } T_{x_{k\Delta\tau}} \mathcal{M}$$

(a Gaussian tangent vector sampled in the tangent space at the current point, using an orthonormal-basis identification $T_x \mathcal{M} \cong \mathbb{R}^{\dim \mathcal{M}}$ via the metric g_x , then pushed back onto $\mathcal{M}$ by the exponential map). As $\Delta\tau \rightarrow 0$, this process converges (in an appropriate sense; we do not make this precise, taking it as the operational definition throughout this document, following the treatment standard in the geometric-diffusion-model literature, e.g. Leach et al., *Denoising Diffusion Probabilistic Models on $SO(3)$ for Rotational Alignment*, ICLR Workshop 2022, and De Bortoli et al., *Riemannian Score-Based Generative Modelling*, NeurIPS 2022) to a continuous-time process $B_\tau^\mathcal{M}$, *Brownian motion on $\mathcal{M}$* .

Code correspondence

Mathematical object: one step of $B_\tau^{SO(3)}$: sample a Gaussian tangent vector and push it onto the manifold

Implementation: `so3_utils.py::tangent_gaussian` (samples $\hat{\varepsilon}$, $\varepsilon \sim \mathcal{N}(0, I_3)$, and left-multiplies by R : returns $R\hat{\varepsilon}$, a left-trivialised (body-frame) — Section 3.5, note the tangent vector’s own components are drawn in the *same* $\mathbb{R}^3$ regardless of R , then transported by right-multiplication, i.e. this is exactly Proposition 3.63’s convention — Gaussian tangent vector at R) composed with `so3_utils.py::expmap` (the map $\exp_R$)

Tensor shape: [..., 3, 3] → [..., 3, 3]

Interpretation: `expmap`, per Section 3.4.3’s code correspondence, computes $\exp_R(v)$ via `torch.matrix_exp` after re-symmetrising the skew part for numerical stability, rather than Rodrigues’ closed form; this is the code-level realisation of one step of Definition 7.1’s random walk, used directly inside SigmaDock’s reverse-time sampler (Section 7.2.2).

Proposition 7.2 (Stationary distribution of $SO(3)$ Brownian motion). $B_\tau^{SO(3)}$ ’s law converges, as $\tau \rightarrow \infty$, to the Haar (uniform) measure of Proposition 3.59, independently of the starting point x_0 .

Proof sketch. Brownian motion on a compact Riemannian manifold is reversible with respect to, and converges to, the Riemannian volume measure — the manifold analogue of Theorem 2.16’s statement that $\mathbb{R}^d$ Langevin dynamics converges to $p \propto e^{-U}$, specialised to the driftless case $U \equiv 0$ (constant potential, i.e. pure diffusion with no preferred point), whose stationary measure is simply the volume measure itself; on $SO(3)$ this volume measure is (proportional to, hence equal to after normalisation) the Haar measure, since Haar measure *is* the (unique, up to normalisation) bi-invariant, and in particular the Riemannian-volume-induced, measure. We do not reprove the general convergence statement (it again requires the manifold Fokker–Planck / heat equation, whose generator is the Laplace–Beltrami operator, Definition 7.3 below), but note it is exactly why $p_{\text{init}} = \mathcal{U}(SO(3))$, not some other distribution, is the correct rotational prior for a diffusion model whose forward process runs Brownian motion for a long enough time — Section 10.6 confirms this is precisely what the code uses. $\square$

This is the promised structural point from Section 3.1: unlike $\mathbb{R}^d$ Brownian motion, whose stationary behaviour is “variance $\rightarrow \infty$ ”, a *compact* manifold’s Brownian motion has a genuine equilibrium distribution determined entirely by the manifold’s geometry, and $SO(3)$ ’s compactness (it is a closed, bounded subset of $\mathbb{R}^{3 \times 3}$) is what makes the variance-exploding schedule of Example 6.5 converge to something useful rather than literally exploding without bound, as it would in $\mathbb{R}^d$.

Definition 7.3 (Laplace–Beltrami operator, informal). The *Laplace–Beltrami operator* $\Delta_{\mathcal{M}}$ is the natural generalisation of the Euclidean Laplacian $\Delta = \sum_i \partial_i^2$ to a Riemannian manifold, defined so that it is the infinitesimal generator of Brownian motion on $\mathcal{M}$ (Definition 7.1) in exactly the sense that Δ is the generator of ordinary $\mathbb{R}^d$ Brownian motion: the transition density $k_\tau(x, y)$ of $B_\tau^{\mathcal{M}}$ (started at x) solves the *heat equation* $\partial_\tau k_\tau = \Delta_{\mathcal{M}} k_\tau$, exactly as the ordinary Gaussian kernel solves $\partial_\tau k_\tau = \Delta k_\tau$ in $\mathbb{R}^d$. We do not need $\Delta_{\mathcal{M}}$ ’s coordinate formula anywhere in this document; only the existence and role of the heat kernel it generates, constructed explicitly for $SO(3)$ in Section 7.2 below.

7.2 The heat kernel on $SO(3)$: IGSO(3)

Motivation. Definition 6.3’s Gaussian perturbation kernel $p_\sigma(x \mid z) = \mathcal{N}(x \mid z, \sigma^2 I)$ is, in $\mathbb{R}^d$ -specific language, exactly the *heat kernel*: the transition density of Brownian motion after time σ^2 . We now want its $SO(3)$ -analogue: the transition density $k_{\sigma^2}(R_0, R)$ of $SO(3)$ -Brownian motion (Definition 7.1) run for “time” σ^2 starting at R_0 .

Theorem 7.4 (The isotropic Gaussian distribution on $SO(3)$). *The heat kernel on $SO(3)$, called IGSO(3)(R_0, σ^2), depends on R_0, R only through the relative-rotation angle $\omega = \Omega(R_0^\top R)$*

of Theorem 3.53 (bi-invariance of the metric, hence of the Laplace–Beltrami operator and its heat kernel, under simultaneous left/right translation), and its density with respect to the Haar measure admits the spectral (Peter–Weyl) expansion

$$f_\sigma(\omega) = \sum_{\ell=0}^{\infty} (2\ell+1) \chi_\ell(\omega) \exp\left(-\frac{1}{2}\ell(\ell+1)\sigma^2\right), \quad \chi_\ell(\omega) = \frac{\sin((\ell + \frac{1}{2})\omega)}{\sin(\omega/2)}, \quad (7.1)$$

where χ_ℓ is the character of the degree- ℓ irrep of $SO(3)$ (Theorem 4.18).

Proof sketch. This is the Peter–Weyl expansion of the heat kernel of a compact Lie group: the heat kernel, viewed as a function on the group (using bi-invariance to reduce to a function of one variable via $k_{\sigma^2}(R_0, R) = k_{\sigma^2}(I, R_0^\top R) =: f_\sigma(\omega)$), decomposes in the orthogonal basis of *matrix coefficients* of the irreducible representations D^ℓ (the Peter–Weyl theorem: $L^2(G)$ for a compact group G decomposes as a direct sum, over irreps, of matrix-coefficient spaces), each an eigenfunction of the Laplace–Beltrami operator with eigenvalue $-\ell(\ell+1)$ (the Casimir eigenvalue of the degree- ℓ irrep of $SO(3)$, a standard fact from the representation theory cited after Theorem 4.18), and the coefficient $e^{-\ell(\ell+1)\sigma^2/2}$ is exactly the corresponding solution $e^{\lambda\tau}$ of the heat equation’s eigenvalue ODE with $\lambda = -\ell(\ell+1)/2$, $\tau = \sigma^2$; summing the trace of D^ℓ over the class-function structure of a bi-invariant kernel (i.e. summing each matrix-coefficient block’s diagonal, which is exactly the character χ_ℓ) and using the classical closed form for $SO(3)$ ’s degree- ℓ character, $\chi_\ell(\omega) = \sin((\ell + \frac{1}{2})\omega)/\sin(\omega/2)$ (itself derivable from the eigenvalues of $D^\ell(\exp(\theta\hat{n}))$, which are $\{e^{im\theta} : m = -\ell, \dots, \ell\}$ in a suitable complex basis, summing as a geometric series), gives (7.1). We take the Peter–Weyl decomposition and the Casimir eigenvalue computation as classical facts and do not reprove them (see Chirikjian, 2012, cited earlier, §10.3, for the complete derivation). $\square$

Remark 7.5 (The series must be truncated in practice). Equation (7.1) is an infinite sum; Section 7.2.2 below records exactly how the actual SigmaDock implementation truncates, tabulates, and interpolates it, since this is a genuine engineering component of the method, not an afterthought — and, as Section 12.10 in Chapter 11 shows, it is precisely this machinery that flow matching eliminates entirely.

Proposition 7.6 (The angle marginal density). *The density of $\omega = \Omega(R_0^\top R)$ alone (as opposed to the full density on $SO(3)$) is $\frac{1-\cos\omega}{\pi} f_\sigma(\omega)$: the IGSO(3) density f_σ times the same Haar-measure Jacobian $1 - \cos\omega$ that appeared in Proposition 3.59 (up to the same normalisation constant), since $\text{IGSO}(3)(R_0, \sigma^2)$, in the limit $\sigma \rightarrow \infty$, must reduce to the Haar measure of Proposition 3.59 by Proposition 7.2, and this is consistent with (7.1) since every term with $\ell \geq 1$ decays to 0 as $\sigma \rightarrow \infty$, leaving only the $\ell = 0$ term $f_\infty(\omega) = 1 \cdot \chi_0(\omega) \cdot 1 = 1$ (using $\chi_0(\omega) = \sin(\omega/2)/\sin(\omega/2) = 1$), recovering exactly Proposition 3.59’s density $\propto 1 - \cos\omega$.*

Example 7.7 (The two limits of IGSO(3), with numbers). Equation (7.1) is an infinite series of characters and gives very little intuition as written. Its two limits do, and both are checkable by hand from the formula.

Large σ : the uniform distribution. Every term with $\ell \geq 1$ carries $e^{-\ell(\ell+1)\sigma^2/2}$, which vanishes as σ grows; the $\ell = 0$ term is $1 \cdot \chi_0 \cdot 1 = 1$. So $f_\sigma \rightarrow 1$: the density *with respect to Haar* becomes constant, which is exactly what “uniform on $SO(3)$ ” means (Proposition 3.32). This is Proposition 7.2 falling out of the series in one line.

The interesting question is *how large* σ must be. Summing (7.1) numerically and measuring the worst deviation of f_σ from the uniform density 1 over $\omega \in (0, \pi]$:

σ	1.5	3.0	5.0
$\max_{\omega} f_{\sigma}(\omega) - 1 $	0.98	$1.1 \cdot 10^{-3}$	$1.3 \cdot 10^{-10}$

At $\sigma = 3$ the forward process has effectively reached equilibrium; at $\sigma = 1.5$ it has *not* — the density still deviates from uniform by order one. This is not an idle observation: $\sigma_{\max} = 1.5$ is the value SigmaDock’s rotational schedule actually uses, so its $t=0$ prior is a visibly non-uniform distribution that the sampler nevertheless treats as uniform. Section 15.6.1 measures the resulting discrepancy directly (mean relative angle 115.0° against Haar’s 126.5° , total variation 0.130) and records it as one of the two confounders between SigmaDock and SigmaFlow. The series limit above is the *reason* for that measurement.

Small σ : a Gaussian in the tangent space. In the other limit the kernel concentrates near $\omega = 0$, and there the manifold looks flat: $\text{IGSO}(3)(R_0, \sigma^2)$ becomes the pushforward under $\exp_{R_0}$ of an ordinary isotropic Gaussian $\mathcal{N}(0, \sigma^2 I_3)$ on the tangent space $T_{R_0}SO(3) \cong \mathbb{R}^3$. That is the mental picture to carry: *for small noise, rotating by IGSO(3) noise is rotating by a Gaussian rotation vector*.

The check is worth doing because it clears up a recurring confusion. If $v \sim \mathcal{N}(0, \sigma^2 I_3)$ then its *norm* $\omega = \|v\|$ has density $\propto \omega^2 e^{-\omega^2/2\sigma^2}$ — a χ -distribution with three degrees of freedom. Comparing against the exact angle marginal $\frac{1-\cos\omega}{\pi} f_{\sigma}(\omega)$ of Proposition 7.6:

σ	0.05	0.10	0.20
$\max_{\omega} p_{\text{IGSO}} - p_{\chi_3} $	$2.0 \cdot 10^{-3}$	$4.0 \cdot 10^{-3}$	$8.1 \cdot 10^{-3}$
peak density	11.7	5.9	2.9

— agreement to a few parts in 10^4 of the peak, degrading linearly in σ as the curvature of $SO(3)$ starts to matter.

The factor ω^2 in that density is geometry, not noise.

It is the Haar Jacobian $1 - \cos\omega \approx \omega^2/2$ of Proposition 3.59 — the fact that there are more rotations at angle ω than at angle 0 — and it appears even though the underlying noise is a perfectly ordinary isotropic Gaussian. A reader who reads the ω^2 as part of the noise model will conclude, wrongly, that IGSO(3) suppresses small rotations. It does not; the measure does. (Both tables verified in Section 12.16, check 11.)

7.2.1 The score of the IGSO(3) kernel

Proposition 7.8 (Riemannian score of IGSO(3)). *Under the right-trivialised (world-frame) identification of Definition 3.60 at R , the score of $p_{\sigma}(R | R_0) := \text{IGSO}(3)(R_0, \sigma^2)$ -density at R , as a tangent vector in $T_R SO(3) \cong \mathbb{R}^3$, is*

$$\nabla_R \log p_{\sigma}(R | R_0) = \frac{v}{\|v\|} \cdot \frac{\partial}{\partial \omega} \log f_{\sigma}(\omega) \Big|_{\omega=\|v\|}, \quad v = \log(R_0^{\top} R)^{\vee} \in \mathbb{R}^3. \quad (7.2)$$

Proof sketch. Since (Theorem 7.4) $p_{\sigma}(R | R_0) = f_{\sigma}(\omega)$ depends on R only through the scalar $\omega = \Omega(R_0^{\top} R)$, the chain rule gives $\nabla_R \log p_{\sigma} = f'_{\sigma}(\omega)/f_{\sigma}(\omega) \cdot \nabla_R \omega$ (the Riemannian gradient

of the composite scalar function). The gradient of the angle function $\omega(R) = \Omega(R_0^\top R)$, as a function on the manifold, points in the direction of the tangent vector that most rapidly increases ω — exactly the unit tangent vector along the geodesic away from R_0 , which by Proposition 3.58 is $v/\|v\| = \log(R_0^\top R)^\vee/\omega$ — with magnitude 1 (since ω itself is arc length along that geodesic, by Proposition 3.58 again, and the gradient of arc length along its own geodesic has unit norm, the direct manifold analogue of $\nabla_x \|x\| = x/\|x\|$ in $\mathbb{R}^d$). Combining gives (7.2). $\square$

Code correspondence

Mathematical object: $\nabla_R \log p_\sigma(R \mid R_0)$

Implementation: `so3_diffuser.py::S03Diffuser.score`

Tensor shape: `[..., 3, 3]` (world-frame element of $\mathfrak{so}(3)$, i.e. the code returns $R \cdot \hat{v} \cdot (\partial_\omega \log f_\sigma / \omega)$ rather than the bare $\mathbb{R}^3$ vector, matching the world-frame identification of Definition 3.60)

Interpretation: $\partial_\omega \log f_\sigma$ is *not* recomputed by differentiating the tabulated density table at score-evaluation time; it is evaluated *live* from the analytic term-by-term derivative of the truncated series (7.1) (function `d_log_f_d_omega`, Section 7.2.2), for the exact, continuous $\sigma(t)$ of the query, not a value snapped to the precomputed grid.

7.2.2 Numerical implementation: truncation, tabulation, and caching

We now record, precisely and only from the code (not reconstructed analytically), how SigmaDock evaluates the objects of the preceding two subsections in practice, since Remark 7.5 promised this level of detail and it is exactly the machinery Section 12.10 will show flow matching does not need.

Code correspondence

Mathematical object: the truncated series $f_\sigma(\omega) = \sum_{\ell=0}^{L-1} (2\ell+1) \chi_\ell(\omega) e^{-\ell(\ell+1)\sigma^2/2}$ of Theorem 7.4, and its analytic derivative $\partial_\omega f_\sigma, \partial_\omega \log f_\sigma$

Implementation: `so3_diffuser.py`, module-level functions `igso3_expansion` (the truncated sum, default $L = 1000$ terms, i.e. degrees $\ell = 0, \dots, 999$), `igso3_density` (multiplies in the angle-marginal Jacobian $1 - \cos \omega$ of Proposition 7.6), `d_f_igso3_d_omega` (the term-by-term derivative, via the quotient rule applied to each $\chi_\ell(\omega) = \sin((\ell + \frac{1}{2})\omega) / \sin(\omega/2)$), `d_log_f_d_omega`

Interpretation: `d_log_f_d_omega` additionally hard-branches (via boolean masking, not the soft `isclose`-blend pattern of Section 3.4.4) between the literal ratio $\partial_\omega f/f$ and the small-noise approximation $-\omega/\sigma^2$ whenever $\sigma^2 < 0.3$, since the literal ratio becomes numerically ill-conditioned at small σ (both numerator and denominator of the truncated series become extremely small simultaneously). We have not independently re-derived that $-\omega/\sigma^2$ is the correct $\sigma \rightarrow 0$ limit of $\partial_\omega \log f_\sigma$ from (7.1) (it is plausible — as $\sigma \rightarrow 0$, $\text{IGSO}(3)(R_0, \sigma^2)$ should concentrate at R_0 and locally resemble a flat Gaussian, whose log-density derivative in the radial direction is exactly $-\omega/\sigma^2$ by Remark 2.13’s formula specialised to one radial coordinate — but we record this as the code’s own documented claim, flagged per this document’s stated policy on uncertainty, not as a result independently verified here).

Code correspondence

Mathematical object: the full apparatus of caching, tabulation, and inverse-CDF sampling used at both training and inference time

Implementation: `so3_diffuser.py::S03Diffuser.__init__, sample_igso3_angle, sample, sample_ref`

Interpretation: at construction time, the diffuser builds a 1000×2000 grid: 1000 noise levels σ_i (linearly spaced in $t \in [0, 1]$, then mapped through the schedule $\sigma(t) = \log(te^{\sigma_{\max}} + (1-t)e^{\sigma_{\min}})$, the *logarithmic* schedule) $\times$ 2000 angles ω_j (linearly spaced in $(0, \pi]$, deliberately excluding $\omega = 0$ to avoid the $\sin(\omega/2) = 0$ singularity visible in (7.1)), and evaluates $f_{\sigma_i}(\omega_j)$ (via `igso3_density`, with `marginal=True`, i.e. including the Jacobian of Proposition 7.6) and $\partial_\omega \log f_{\sigma_i}(\omega_j)$ (via `d_log_f_d_omega`) at every grid point, once, and caches both tables to disk, together with a row-wise cumulative-sum CDF table (a Riemann-sum numerical integral of the density table, safe-normalised by its own final value clamped away from zero). The cache directory name encodes every hyperparameter of the grid (matching the `eps_1000_omega_2000_min_sigma..._schedule_logarithmic`-style paths visible in this project’s own training logs). *Sampling* an angle at a queried t (`sample_igso3_angle`) looks up the nearest tabulated σ_i row via `torch.bucketize`, draws $u \sim \text{Unif}(0, 1)$, locates u ’s bracketing interval in that row’s cached CDF, and *linearly interpolates* between the two bracketing grid angles — piecewise-linear inverse-CDF sampling on a fixed 2000-point grid, not exact analytic inversion. `sample_ref` (the prior sampler, i.e. $R \sim p_{\text{init}}$) calls `sample` at $t = 1$, the schedule’s maximum-noise end — an *approximation* to the exact Haar-uniform prior of Proposition 7.2, exact only in the limit $\sigma_{\max} \rightarrow \infty$, not literally the closed-form Haar sampler of Proposition 3.59 (that routine, `sample_uniform`, is a separate function, used instead by *SigmaFlow*’s prior, as recorded in Section 12.10). By contrast, the *score itself* (Proposition 7.8’s code correspondence, immediately above) is **not** read from this cached table at evaluation time — it is recomputed live via `d_log_f_d_omega` at the query’s exact, continuous (ω, σ) ; the cached `score_norms` table exists only to build the training loss-weight $\lambda(t)$ (`score_scaling`, matching (6.7)’s $\lambda(s)$ in this document’s notation), computed once at construction time as a weighted second moment, $\lambda_i^{-1/2} = \sqrt{\mathbb{E}_\omega[(\partial_\omega \log f_{\sigma_i})^2]/3}$ (the division by 3 normalising a full-vector squared-norm expectation down to a per-coordinate one), over the angle marginal at noise level σ_i .

7.3 Riemannian denoising score matching and the reverse-time SDE on $SO(3)$

With Proposition 7.8 in hand, the entire machinery of Chapter 6 transfers to $SO(3)$ by the single substitution “Euclidean gradient $\rightarrow$ Riemannian gradient”, exactly as Definition 3.8’s generalisation of $\mathbb{R}^d$ calculus to manifolds promised at the start of Chapter 3.

Theorem 7.9 (Riemannian denoising score matching). *Define, analogously to Theorem 6.4,*

$$\mathcal{L}_{\text{RDSM}}(\theta) = \mathbb{E}_{t, R_1 \sim p_{\text{data}}, R_t \sim p_t(\cdot | R_1)} \left[\lambda_t \| s_\theta(R_t, t) - \nabla_{R_t} \log p_t(R_t | R_1) \|_g^2 \right],$$

with $p_t(\cdot | R_1) = \text{IGSO}(3)(R_1, \sigma(1-t)^2)$ (Remark 0.2’s convention: R_1 is data, so the conditioning is on the $t = 1$ endpoint, and the noise level increases as t decreases, i.e. as we move away from data towards $t = 0$). Then, exactly as in Theorem 6.4’s proof (which uses nothing about the ambient space beyond the existence of the gradient/score and the marginalisation identity of Proposition 2.3, both of which hold verbatim on a Riemannian manifold with ∇_x replaced by the Riemannian gradient and dx by the Riemannian volume element), minimising $\mathcal{L}_{\text{RDSM}}$ has the same minimiser, $s_\theta(R, t) \approx \nabla_R \log p_t(R)$, as the (intractable) marginal Riemannian score matching loss.

We do not repeat the proof (it is, verbatim, the computation of Theorem 6.4, with every Euclidean integral replaced by an integral against the Riemannian volume measure, valid since none of the manipulations used — integration by parts under the gradient, Fubini, linearity of expectation — depend on the ambient space being flat).

Sampling then simulates the manifold reverse-time SDE, the direct $SO(3)$ -analogue of (6.8):

$$dR_t = g(1-t)^2 \nabla_R \log p_t(R_t) dt + g(1-t) d\bar{B}_t^{SO(3)}, \quad R_0 \sim p_{\text{init}}, \quad (7.3)$$

(no drift term f , since Example 6.5’s VE-type schedule, which SigmaDock’s rotational component uses per the code correspondence of Section 7.2.2, has $f \equiv 0$), discretised as a *geodesic random walk*: at each step, take one Definition 7.1-style Brownian increment, scaled and drifted by the (learned, substituted) score, and map it onto $SO(3)$ via $\exp_{R_t}$ — guaranteeing, by construction, that the discretised trajectory never leaves $SO(3)$, for any step size, unlike a naive Euclidean discretisation of a constrained system.

Code correspondence

Mathematical object: one Euler–Maruyama step of (7.3)

Implementation: `so3_diffuser.py::S03Diffuser.reverse`

Tensor shape: $[\dots, 3, 3] \rightarrow [\dots, 3, 3]$

Interpretation: computes `perturb = g_t^2 · score_t · dt + noise_scale · g_t · √dt · z` with $z = \text{tangent_gaussian}(R_t)$ (Section 7.1’s code correspondence) and $g_t = \text{diffusion_coef}(t)$, then returns $R_{t-dt} = \text{expmap}(R_t, \text{perturb})$ — exactly (7.3)’s Euler–Maruyama discretisation, with the manifold-respecting update $\exp_{R_t}(\cdot)$ in place of the naive Euclidean $R_t + (\cdot)$. An optional `noise_scale` parameter (default, per `conf/sampling/base.yaml`, exactly 0.0 — see Chapter 10) rescales the injected noise term z independently of the score-drift term, allowing the sampler to be run anywhere between fully stochastic (`noise_scale=1`) and fully deterministic (`noise_scale=0`, in which case (7.3)’s SDE degenerates to Theorem 6.9’s probability flow ODE, specialised to $SO(3)$ by the same cancellation argument, restated but not re-derived here).

This completes the diffusion side of the theory. Chapter 10 assembles Chapters 6 and 7 into the full SigmaDock model — combining the VP translational process of Example 6.6 and the VE rotational process of this chapter via the product-manifold structure of Proposition 3.69, and connecting every formula derived here to the exact network architecture (Chapter 9) that predicts the required scores.

7.4 What you should now be able to do

- say what Brownian motion on a Riemannian manifold is, and why its transition density is a heat kernel rather than a Gaussian;
- recognise IGSO(3) as that heat kernel, written as a series over the irreducible characters of Chapter 4;
- explain why the series must be truncated and tabulated, and what that costs in accuracy (Remark 7.5);
- write the Riemannian denoising score matching objective and the reverse-time SDE on $SO(3)$, and identify which pieces are geometry and which are the generative process.

This is the last ingredient of SigmaDock. It is also the piece that Chapter 12 deletes entirely: flow matching needs a geodesic, not a heat kernel.

Part IV

Architecture

Part IV at a glance

What you will learn. How to build a network whose output is a *vector field on a geometric graph*: message passing, why invariant features provably cannot emit a vector (Proposition 8.15), the steerable message that repairs it, and then EquiformerV2 in the exact form this project ships.

Why it is needed. Parts III and VI both reduce “generate a pose” to “learn a field”. Neither says what that learner looks like. Without this part, the score and the velocity are functions nobody can evaluate.

What depends on it. Chapters 10 and 13 both call this network, unchanged — Section 15.1 verifies the two trees are byte-identical here, which is what makes the diffusion-versus-flow-matching comparison controlled.

Chapter 8

From Sets to Geometry: Attention, Message Passing, and the Equivariance Obstruction

Chapters 6 and 7 reduced “generate a pose” to “learn a score/vector-field network”, but said nothing about what that network’s *input* looks like or how it must be built. This chapter develops that architecture from the ground up, assuming only that the reader knows what a matrix multiplication and a feed-forward layer are; by its end we will have derived, not merely stated, why an architecture built from scalars and distances alone *cannot* do the job, which is exactly the gap Chapter 9 closes.

8.1 The network’s signature

Consolidating Chapters 6 and 7: the object to be learned, whether called a score (s_θ) or (in Chapter 11) a velocity (u_θ), is a function

$$u_\theta : \mathcal{M}_N \times [0, 1] \longrightarrow T\mathcal{M}_N, \quad u_\theta(\mathbf{T}, t) = (\omega_\theta^{(i)}, v_\theta^{(i)})_{i=1}^N \in (\mathbb{R}^3 \times \mathbb{R}^3)^N, \quad (8.1)$$

where, by the left/right-trivialisation identification fixed in Section 3.5, $\omega_\theta^{(i)}$ is fragment i ’s rotational output and $v_\theta^{(i)}$ its translational output, each an element of $\mathbb{R}^3$ per fragment (never a full 3×3 matrix; the tangent-space identification of Chapter 3 is precisely what collapses “an infinitesimal rotation” to “three numbers”). The input $\mathbf{T} \in \mathcal{M}_N$ is not, however, presented to the network in this abstract group-element form; it is presented as the concrete Cartesian coordinates $\mathbf{x} = \varphi(\mathbf{T}) \in \mathbb{R}^{n \times 3}$ of every atom (protein and ligand) obtained by applying each fragment’s current rigid motion to its reference-conformer template (Section 10.7.1 makes this map φ precise), plus each atom’s discrete chemical identity, together with t . The network, therefore, is a map from an *unordered, geometric* collection of labelled points in $\mathbb{R}^3$ to a per-fragment vector-pair output, and the rest of this chapter and the next derive what such a map is allowed to look like.

8.2 Two structural requirements

Permutation equivariance. Atoms in $\mathbf{x}$ carry no canonical order; they are a *set* (indexed arbitrarily for storage), not a sequence. Reindexing the input must permute the corre-

sponding output identically and change nothing else.

$SE(3)$ -equivariance. The physical process being modelled (a ligand binding a pocket) does not depend on the pose of the laboratory reference frame. Applying a rigid motion to the entire input (protein and ligand together) must transform the predicted vector field exactly as Definition 8.1 below makes precise — *not* leave it unchanged, and *not* change it arbitrarily.

Definition 8.1 (Group action, invariance, equivariance). Let a group G act on a set (or manifold) $\mathcal{X}$ via maps $\Phi_Q : \mathcal{X} \rightarrow \mathcal{X}$, $Q \in G$, satisfying $\Phi_{Q_1 Q_2} = \Phi_{Q_1} \circ \Phi_{Q_2}$ (a group action, the natural generalisation of a linear representation, Definition 4.15, to a possibly nonlinear space). A function $\rho : \mathcal{X} \rightarrow \mathbb{R}$ is G -invariant if $\rho(\Phi_Q(x)) = \rho(x)$ for all Q, x . A function $u : \mathcal{X} \rightarrow T\mathcal{X}$ (with G also acting on the tangent bundle by the differential $d\Phi_Q$) is G -equivariant if $u(\Phi_Q(x)) = d\Phi_Q|_x u(x)$ for all Q, x : transform-then-evaluate equals evaluate-then-transform.

For the permutation group $G = S_n$ acting on $\mathbf{x} \in \mathbb{R}^{n \times 3}$ by reindexing, and for the rigid-motion group $SE(3)$ acting by $\Phi_{(Q,b)}(\mathbf{x})_i = Q\mathbf{x}_i + b$, Definition 8.1 specialises to exactly the two requirements stated informally above; for the rotational part of (8.1), using the world/body-frame bookkeeping fixed in Section 3.5. Under the action $R \mapsto QR$ on an *absolute* fragment orientation, a body-frame rotational output is left *unchanged*, since it is expressed relative to the fragment’s own, co-rotating frame.

Caution 8.2 (This is the textbook case, and it is *not* SigmaFlow’s). The invariance just stated holds for $R \mapsto QR$. SigmaFlow’s R_t is not an absolute orientation but a *delta* rotation measured against the reference structure `pos_0`, which rotates along with the scene; the group therefore acts by conjugation $R_t \mapsto QR_t Q^\top$, and the body-frame output is *not* invariant but transforms by conjugation too. This is derived from the code and measured in Section 14.4.5; the general lesson — “body-frame” fixes the relation to $\hat{R}$, not the transformation law — is stated in Caution 14.4. The network’s own output convention is discussed in Section 9.12.

Proposition 8.3 (Why equivariance, not merely invariance, is the right constraint). *Let G act on $\mathcal{M}$ by isometries (maps preserving the Riemannian metric g ; both the permutation action on a graph’s node set and the rigid-motion action on $\mathcal{M}_N$ satisfy this), let p_0 be G -invariant, and let u_θ be G -equivariant. Then every marginal p_t^θ of the flow $\frac{d}{dt}X_t = u_\theta(X_t, t)$, $X_0 \sim p_0$ (Chapter 11; the analogous statement for the diffusion SDEs of Chapters 6 and 7 holds by the same argument, stated precisely in Theorem 10.7), is G -invariant; in particular the generated data distribution p_1^θ is.*

Proof. Let $Y_t := \Phi_Q(X_t)$. Since Φ_Q is a fixed diffeomorphism, $\dot{Y}_t = d\Phi_Q|_{X_t} \dot{X}_t = d\Phi_Q|_{X_t} u_\theta(X_t, t) = u_\theta(\Phi_Q(X_t), t) = u_\theta(Y_t, t)$, using equivariance of u_θ in the last step: Y_t solves the *same* ODE as X_t . Since $Y_0 = \Phi_Q(X_0)$ has law $\Phi_{Q\#}p_0 = p_0$ (G -invariance of p_0), and solutions of an ODE are uniquely determined by their initial condition (under standard regularity, e.g. Lipschitz u_θ ; a classical existence/uniqueness fact for ODEs we take as given), Y_t and X_t have the *same* law for every t . But $Y_t = \Phi_Q(X_t)$ by definition, so $\text{Law}(X_t) = \text{Law}(\Phi_Q(X_t)) = \Phi_{Q\#}\text{Law}(X_t)$: $p_t^\theta := \text{Law}(X_t)$ is G -invariant. $\square$

This is the reason equivariance is not a stylistic preference: it is a *hard, exact* guarantee — for *every* value of the learned parameters θ , at *every* point during training, not only at convergence — that the generative model respects the symmetries of the physical problem, obtained purely from an architectural constraint rather than needing to be learned from data (data augmentation could *approximately* teach a non-equivariant network the same symmetry,

at the cost of a strictly larger effective hypothesis space to search and no exact guarantee at any finite amount of training).

8.3 Attention, and the permutation-equivariance half of the problem

Definition 8.4 (Scaled dot-product self-attention; restated). For tokens $X = (x_1, \dots, x_n)^\top \in \mathbb{R}^{n \times d}$ and projections $W_Q, W_K \in \mathbb{R}^{d \times d_k}$, $W_V \in \mathbb{R}^{d \times d_v}$, set $Q = XW_Q$, $K = XW_K$, $V = XW_V$ and

$$\text{Attn}(X) = AV, \quad A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{n \times n}$$

(row-wise softmax, so $A_{ij} \geq 0$, $\sum_j A_{ij} = 1$).

Row i of $\text{Attn}(X)$ is a convex combination $\sum_j A_{ij} v_j$ of value vectors, weighted by the learned compatibility $\langle q_i, k_j \rangle$ between tokens i and j ; the $1/\sqrt{d_k}$ scaling keeps logits at unit scale for i.i.d. unit-variance q, k entries, preventing early softmax saturation. Multiple parallel *heads* (independent W_Q, W_K, W_V triples, concatenated and mixed by a further W_O) and a pre-normalised residual block,

$$X' = X + \text{Attn}(\text{LN}(X)), \quad X'' = X' + \text{MLP}(\text{LN}(X')), \quad (8.2)$$

complete the ordinary Transformer layer.

Proposition 8.5 (Permutation equivariance of attention). *For a permutation matrix $P \in \mathbb{R}^{n \times n}$: $\text{Attn}(PX) = P \text{Attn}(X)$, and (8.2) inherits this property.*

Proof. $Q, K, V \mapsto PQ, PK, PV$ (the projections act on the feature axis, so permuting rows of X permutes rows of Q, K, V identically). Hence $QK^\top \mapsto PQK^\top P^\top$; since row-wise softmax acts identically on each row regardless of row order, $A \mapsto PAP^\top$. Then $AV \mapsto PAP^\top PV = PAV$ using $P^\top P = I$. LN and MLP act identically, tokenwise, on every row, hence commute with P . $\square$

Attention thus solves permutation equivariance exactly *because* it treats its input as an (index-labelled but otherwise symmetric) set: the formula (8.4) never references the token index i except through x_i itself. Two further observations connect attention to the constructions of the next section: first, (8.4) *is* message passing (Definition 8.14 below) on the *complete* graph, with learned edge weight A_{ij} ; second, its $\mathcal{O}(n^2 d)$ cost, from computing $QK^\top$ densely, is why in practice attention (and the equivariant version developed below) is restricted to a sparse graph — a radius or k -nearest-neighbour graph — rather than the complete graph, recovering tractability while keeping the learned, data-dependent weighting.

What (8.4) does *not* provide is any use of geometry: $\langle q_i, k_j \rangle$ depends on the tokens' *features*, never on where in $\mathbb{R}^3$ the corresponding atoms sit. Concatenating raw coordinates $x_i \in \mathbb{R}^3$ into the feature vector would inject geometry but destroy $SE(3)$ -equivariance immediately (the resulting logits, and hence A , would change under a global rotation, since nothing in (8.4) constrains how a rotated coordinate enters the dot product). Geometry must instead enter through *relationships between atoms* — the subject of the next section.

8.4 Geometric graphs and message passing

8.4.1 An ordinary GNN, with no geometry at all

Start from the version with no symmetry considerations whatsoever, so that we can see exactly what has to be added.

A graph neural network keeps a feature vector $h_i \in \mathbb{R}^d$ at every node and updates it by looking at neighbours. One round of *message passing* is two steps:

$$m_{ij} = M(h_i, h_j, e_{ij}) \quad \text{compute a message along each edge,} \quad (8.3)$$

$$h'_i = U\left(h_i, \sum_{j \in \mathcal{N}(i)} m_{ij}\right) \quad \text{aggregate and update.} \quad (8.4)$$

Here M and U are learned functions (typically MLPs), e_{ij} is an optional edge feature, and $\mathcal{N}(i)$ is the set of neighbours of i . Stacking T such rounds lets information travel T hops through the graph.

For a molecule the identification is immediate:

Graph object	Molecular meaning
node i	atom
h_i	learned chemical state: element, charge, hybridisation, ...
edge (i, j)	a pair of atoms considered to interact
e_{ij}	bond type, or nothing
$x_i \in \mathbb{R}^3$	the atom's position

Everything in (8.3)–(8.4) is standard deep learning. No representation theory has appeared, and none is needed — because no geometry has appeared either.

8.4.2 The question: where is the 3D structure?

Look at what the update actually consumes: h_i, h_j, e_{ij} . Atom types and bond types. The positions x_i appear nowhere.

The consequence is severe. Two conformers of the same molecule — the same atoms, the same bonds, folded into completely different shapes — have *identical* graphs in the sense of (8.3). A plain GNN cannot tell them apart, in principle, no matter how it is trained. For docking this is fatal: the entire task is to distinguish spatial arrangements.

So positions must enter. The question is how, and that question is delicate for a reason we can now state precisely:

Geometry must enter in a form that carries *real spatial information* but does not smuggle in a *choice of coordinate frame*.

8.4.3 Why absolute positions are the wrong input

The naive fix is to append x_i to h_i and carry on. It fails on the first symmetry we listed in Section 4.1.

Translate the whole system by $a \in \mathbb{R}^3$, so $x_i \mapsto x_i + a$ for every atom. Nothing physical changed — the molecule is the same molecule, we moved the origin. But every input to the network changed, so every output changes too. The network would have to learn, from data, that all of $\mathbb{R}^3$ worth of translations are irrelevant. That is the augmentation problem of Section 4.1 in its purest form, and now over a *non-compact* group, where sampling coverage is hopeless.

The fix, and why it works. Use differences instead of positions:

$$\boxed{x_{ij} := x_j - x_i \in \mathbb{R}^3} \quad (8.5)$$

Proposition 8.6 (The edge vector is translation-invariant and rotation-equivariant). *Under a global rigid motion $x \mapsto Qx + a$ with $Q \in SO(3)$, $a \in \mathbb{R}^3$:*

$$x_{ij} \mapsto (Qx_j + a) - (Qx_i + a) = Q(x_j - x_i) = Qx_{ij}.$$

Proof. The two copies of a cancel; Q factors out by linearity. $\square$

Read the conclusion carefully, because it contains two different statements:

- the translation a has **disappeared** — x_{ij} is translation-*invariant*, and translation symmetry is now exact and free, requiring no further work anywhere in the architecture;
- the rotation Q has **survived** — x_{ij} is rotation-*equivariant*, an $\ell = 1$ object, and rotation symmetry is therefore still a live constraint.

This asymmetry is why the rest of this document is about $SO(3)$ and barely mentions translations again.

Caution 8.7 (Edge orientation, fixed once). We use $x_{ij} = x_j - x_i$ throughout: the vector *from i to j* , i.e. “where is my neighbour, seen from me”. The message m_{ij} correspondingly flows from j into i .

The opposite convention $x_j - x_i \rightsquigarrow x_i - x_j$ is equally defensible and used by some sources, and the choice is *not* harmless: $x_{ji} = -x_{ij}$, and by Proposition 8.8 below the spherical harmonics satisfy $Y^\ell(-\hat{u}) = (-1)^\ell Y^\ell(\hat{u})$. So reversing the edge flips the sign of every *odd- ℓ* angular feature while leaving even ones alone. A model built with one convention and evaluated with the other is not merely mismatched — it is systematically wrong on exactly the $\ell = 1$ channels that carry the directional output.

Proposition 8.8 (Parity of the spherical harmonics). *For $\hat{u} \in S^2$ and every degree ℓ , $Y^\ell(-\hat{u}) = (-1)^\ell Y^\ell(\hat{u})$.*

Proof. By Definition 4.20, Y^ℓ is the restriction to S^2 of a homogeneous polynomial of degree ℓ , and a homogeneous polynomial of degree ℓ satisfies $p(-x) = (-1)^\ell p(x)$. $\square$

8.4.4 Splitting an edge into radius and direction

We now have one geometric object per edge, $x_{ij} \in \mathbb{R}^3$. The next move is the one that organises the entire architecture: *split it*.

$$x_{ij} = \underbrace{d_{ij}}_{\text{how far}} \cdot \underbrace{\hat{x}_{ij}}_{\text{which way}}, \quad d_{ij} = \|x_{ij}\| \in \mathbb{R}_{>0}, \quad \hat{x}_{ij} = \frac{x_{ij}}{d_{ij}} \in S^2. \quad (8.6)$$

Why split at all? Because the two halves have *different symmetry types*, and therefore must be processed by different machinery:

Piece	Lives in	Under Q	Consequence
d_{ij}	$\mathbb{R}_{>0}$	unchanged	$\ell = 0$: ordinary neural operations are legal
$\hat{x}_{ij}$	S^2	$\mapsto Q\hat{x}_{ij}$	equivariant: needs steerable treatment

The invariance of the distance is Proposition 3.46: $\|Qx_{ij}\| = \|x_{ij}\|$. That one line is what licenses everything in Section 8.4.5.

Caution 8.9 (The split degenerates at $d_{ij} = 0$). $\hat{x}_{ij}$ is undefined when $x_i = x_j$. In a molecular graph two distinct atoms never coincide, so this is not a modelling problem — but it is a numerical one, because $\hat{x}_{ij}$ and its derivatives blow up as $d_{ij} \rightarrow 0$, and self-edges $i = j$ would trigger it exactly. Implementations therefore exclude self-loops from the radius graph, or add an ε to the denominator. Chapter 16 records what SigmaDock actually does.

The organising principle

Not everything in an equivariant network needs representation theory. *Invariant scalars may be processed by ordinary neural operations* — MLPs, ReLU, layer norm, anything. Representation theory is needed only where directional information lives. The radial/angular split is the device that separates the two, so that each can be handled with the cheapest sufficient tool. Nearly every design decision in Chapter 9 is an instance of this principle.

8.4.5 The radial half: from a distance to a feature vector

Since d_{ij} is invariant, any function of it is invariant, and we may use a plain MLP. So why do molecular models almost always embed the distance first, into a vector $\phi(d_{ij}) \in \mathbb{R}^K$, instead of feeding the single number 3.71 directly?

This is an architectural choice, not a requirement. The reasons it is usually made:

1. **Locality of response.** A network often needs to react differently to 1.4 Å (a covalent bond), 2.8 Å (a hydrogen bond) and 5 Å (weak contact). A single scalar input forces an MLP to carve these regimes out of one monotone coordinate; a basis of localised bumps hands it the regimes directly, and each output weight can specialise.
2. **Conditioning.** Raw distances live on a scale of a few Ångström; basis outputs are $O(1)$ and bounded, which behaves better under standard initialisation.
3. **Smoothness control.** The basis width sets how fast the model *can* vary with distance, acting as an architectural prior against fitting sharp, spurious distance dependence.
4. **A natural place for the cutoff.** Multiplying the basis by an envelope (Section 8.4.6) makes every downstream quantity go smoothly to zero at the graph boundary.

Definition 8.10 (Radial basis embedding). A *radial basis* is a map $\phi : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^K$, $\phi(d) = (\phi_1(d), \dots, \phi_K(d))$. Two common choices:

$$\text{Gaussian: } \phi_k(d) = \exp(-\gamma(d - \mu_k)^2), \quad \text{Bessel: } \phi_k(d) = \sqrt{\frac{2}{r_c}} \frac{\sin(k\pi d/r_c)}{d},$$

with centres $\mu_1 < \dots < \mu_K$ spanning the interaction range, width γ , and cutoff r_c .

Shapes. Input $d_{ij} \in \mathbb{R}$ (one number per edge); output $\phi(d_{ij}) \in \mathbb{R}^K$. For a graph with E edges the batched tensor is $[E, K]$.

Invariance. Each ϕ_k is a function of an invariant, hence invariant: $\phi_k(\|Qx_{ij}\|) = \phi_k(\|x_{ij}\|)$. The whole vector $\phi(d_{ij})$ is therefore a collection of K independent $\ell = 0$ features, and may be fed to an ordinary MLP, $\psi(d_{ij}) = \text{MLP}(\phi(d_{ij}))$, whose output is again invariant.

Example 8.11 (Three Gaussian centres). Take $K = 3$, centres $\mu = (1.5, 3.0, 4.5) \text{ \AA}$ and $\gamma = 1$. Then

$$\phi(1.6) \approx (0.99, 0.13, 0.0003), \quad \phi(3.1) \approx (0.10, 0.99, 0.14), \quad \phi(4.4) \approx (0.0003, 0.14, 0.99).$$

Each distance activates mainly one coordinate, and neighbouring coordinates overlap smoothly so that the representation is continuous in d . A downstream linear layer can now assign an independent weight to “short”, “medium” and “long” contacts — which is exactly what one would have had to *learn* to extract from the raw number.

8.4.6 The cutoff

Molecular graphs are built with a radius cutoff, $\mathcal{E} = \{(i, j) : d_{ij} < r_c\}$. Three reasons: *physics* (interactions decay, and beyond a few Ångström the influence is negligible), *cost* (a complete graph on N atoms has N^2 edges; a radius graph has $O(N)$ for fixed density), and *node degree* (bounded neighbourhoods keep the aggregation well-scaled).

But a hard cutoff introduces a discontinuity: an atom drifting across r_c makes its message appear or vanish abruptly. That is undesirable for two concrete reasons.

First, the model’s output becomes discontinuous in the atom positions. For a *generative* model this is worse than cosmetic: the sampler integrates a vector field along a trajectory, and a field with jump discontinuities is not something an ODE solver can integrate reliably.

Second, gradients are ill-behaved near the boundary.

The remedy is an envelope $\chi : [0, r_c] \rightarrow [0, 1]$ with $\chi(r_c) = 0$ (and usually $\chi'(r_c) = 0$), multiplied into the radial features: $\psi(d)\chi(d)$. A common choice is the polynomial envelope $\chi(d) = 1 - 6u^5 + 15u^4 - 10u^3$ with $u = d/r_c$, which satisfies $\chi(0) = 1$, $\chi(r_c) = 0$ and $\chi'(0) = \chi'(r_c) = 0$. The message then fades to zero smoothly as a neighbour leaves the cutoff, and nothing jumps.

8.4.7 The angular half: why spherical harmonics

Now the harder piece: $\hat{x}_{ij} \in S^2$.

We cannot simply pass the three components of $\hat{x}_{ij}$ through an MLP as if they were scalars — that is precisely the error Example 4.14 rules out. We need to turn a direction into features whose transformation law we know.

We already have the tool. Section 4.6 constructed, for each degree ℓ , a map $Y^\ell : S^2 \rightarrow \mathbb{R}^{2\ell+1}$ with

$$Y^\ell(Q\hat{u}) = D^\ell(Q) Y^\ell(\hat{u})$$

(Equation (4.1), in this document’s fixed convention). Applying it to the edge direction gives

$$Y^\ell(\hat{x}_{ij}) \in \mathbb{R}^{2\ell+1}, \quad Y^\ell(\hat{x}_{ij}) \mapsto D^\ell(Q) Y^\ell(\hat{x}_{ij}) \quad \text{under a global rotation,}$$

because $\hat{x}_{ij} \mapsto Q\hat{x}_{ij}$ by Proposition 8.6.

The point of spherical harmonics, stated plainly

They are not merely a convenient basis for functions on a sphere. They are the map that converts a raw geometric direction — an object the network cannot use — into features of *exactly the types* $\ell = 0, 1, 2, \dots$ that the network’s algebra is built around. Before Y^ℓ , the edge direction is data of unknown type. After Y^ℓ , it is a steerable feature that can enter a Clebsch–Gordan product like any other.

What the different degrees carry. For a *single* direction $\hat{u}$:

- $\ell = 0$: Y^0 is constant. It carries *no* directional information at all — the same value for every edge.
- $\ell = 1$: $Y^1(\hat{u}) \propto \hat{u}$. First-order direction: which way.
- $\ell = 2$: quadratic in the components. It distinguishes $\hat{u}$ from $-\hat{u}$ not at all ($(-1)^2 = +1$, Proposition 8.8) but resolves *axis* information — alignment with a line rather than a ray.
- higher ℓ : finer angular resolution, in the same sense that higher Fourier modes resolve finer detail on a circle.

The useful mental model is angular resolution. Truncating at $L = \ell_{\max}$ is like low-pass filtering the angular dependence: the network can represent angular patterns up to that frequency and no finer. Raising L increases expressive power, but the cost grows quickly — the number of components per node is $\sum_{\ell \leq L} (2\ell + 1) = (L + 1)^2$, and the number of Clebsch–Gordan paths grows faster still. Whether a larger L helps is an empirical question and depends on how much genuine angular structure the task has; it is not a free improvement. EquiformerV2’s design is in large part a response to this cost (Section 9.5).

8.4.8 Recombining radius and direction

We now have, per edge, an invariant radial vector $\psi(d_{ij}) \in \mathbb{R}^K$ and a family of steerable angular features $Y^\ell(\hat{x}_{ij}) \in \mathbb{R}^{2\ell+1}$. The natural way to put them back together is

$$e_{ij}^{(\ell)} = f^{(\ell)}(d_{ij}) Y^\ell(\hat{x}_{ij}) \in \mathbb{R}^{2\ell+1}, \tag{8.7}$$

where $f^{(\ell)}$ is any learned scalar function of the distance (in practice a channel of $\text{MLP}(\phi(d_{ij}))$).

Why this is legal: $f^{(\ell)}(d_{ij})$ is an invariant *scalar*, and a scalar commutes with $D^\ell(Q)$. So

$$f^{(\ell)}(d_{ij}) D^\ell(Q) Y^\ell(\hat{x}_{ij}) = D^\ell(Q) \left(f^{(\ell)}(d_{ij}) Y^\ell(\hat{x}_{ij}) \right),$$

i.e. $e_{ij}^{(\ell)}$ is again a degree- ℓ feature. This is the gating argument of Section 4.10.10 reappearing in a geometric costume: an invariant modulating an equivariant.

Remark 8.12 (The split is not a trick; it is the geometry of $\mathbb{R}^3$). Writing a function of a 3-vector as “radial part times angular part” is exactly what separation of variables in spherical coordinates does: $f(x) = \sum_{\ell m} f_{\ell m}(r) Y_m^\ell(\hat{x})$, with the spherical harmonics as the angular basis. Equation (8.7) is one term of such an expansion with a learned radial coefficient. So the architecture is not imposing an arbitrary factorisation on the data — it is using the factorisation that the rotation group itself induces on functions of $\mathbb{R}^3$. We do not develop the full harmonic analysis; the point is only that the split is natural rather than expedient.

Definition 8.13 (Geometric graph; restated). A *geometric graph* is $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \{h_i\}, \{x_i\})$: nodes $i \in \mathcal{V}$ with $SE(3)$ -invariant features $h_i \in \mathbb{R}^d$ (atom type, charge, fragment/entity type, time embedding of t) and positions $x_i \in \mathbb{R}^3$; edges $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. Write $x_{ij} = x_j - x_i$, $d_{ij} = \|x_{ij}\|$; we take $\mathcal{E}$ to be a radius graph, $\{(i, j) : d_{ij} < r_{\text{cut}}\}$, for some fixed or per-edge-type cutoff r_{cut} .

Definition 8.14 (Message passing layer; restated). A message passing layer computes

$$m_{ij} = \phi_m(h_i, h_j, e_{ij}), \quad m_i = \bigoplus_{j \in \mathcal{N}(i)} m_{ij}, \quad h'_i = \phi_u(h_i, m_i), \quad (8.8)$$

for learned maps ϕ_m, ϕ_u , edge attributes e_{ij} , and a permutation-invariant aggregator $\bigoplus$ (sum or mean over j).

Since $\bigoplus$ depends only on the *multiset* $\{m_{ij}\}$, not on any ordering of $\mathcal{N}(i)$, (8.8) is permutation equivariant by the same argument as Proposition 8.5 (indeed Definition 8.4 is the special case $\mathcal{N}(i) = \mathcal{V}$ for all i , $\bigoplus$ a learned convex combination). Geometry enters through e_{ij} ; the invariant (and hence, per Section 8.5 below, safe-but-limited) choice expands the distance in a radial basis,

$$e_{ij} = (\psi_1(d_{ij}), \dots, \psi_K(d_{ij})), \quad \psi_k(d) = \exp(-\gamma(d - \mu_k)^2) \chi(d), \quad (8.9)$$

with basis centres μ_k and a smooth cutoff envelope χ vanishing (along with enough derivatives to avoid a kink) at $d = r_{\text{cut}}$: an edge that is about to leave the radius graph as atoms move contributes a message that continuously shrinks to zero *before* it is dropped from $\mathcal{E}$ entirely, rather than discontinuously disappearing — without this, the learned vector field would be discontinuous in $\mathbf{x}$ exactly at the cutoff radius, with predictable consequences for the stability of the ODE/SDE integrators of Chapters 6 and 11.

Code correspondence

Mathematical object: the radial basis $\psi_k(d)\chi(d)$ of (8.9)

Implementation: `net/encoder.py::BesselBasis, PolynomialCutoff`

Interpretation: the actual basis used is the (MACE-style) Bessel form $\psi_k(d) = \sqrt{2/r_{\text{max}}} \sin(k\pi d_{\text{scaled}}/r_{\text{max}})/(d_{\text{scaled}} + \varepsilon)$ (trainable frequencies $k\pi/r_{\text{max}}$, not fixed Gaussian centres as in (8.9) — this document presents the Gaussian form first purely because it is the more elementary starting point, and records the Bessel form actually used as a documented substitution, mathematically serving an identical role: an orthogonal-ish family of radial basis functions), multiplied by a degree-8 polynomial envelope

$$\chi(d) = \left[1 - \frac{(p+1)(p+2)}{2} \left(\frac{d}{r_{\text{max}}} \right)^p + p(p+2) \left(\frac{d}{r_{\text{max}}} \right)^{p+1} - \frac{p(p+1)}{2} \left(\frac{d}{r_{\text{max}}} \right)^{p+2} \right] \mathbf{1}[d < r_{\text{max}}], \quad p = 8,$$

constructed exactly so that χ and its first two derivatives vanish at $d = r_{\text{max}}$ (a standard DimeNet/MACE-style envelope), confirming the smooth-cutoff argument above at the level of the actual, exact formula used. **Exact cutoff radii used**, per `oracle.py`: protein–ligand interaction edges (`inter_complex`) and ligand–ligand cross-fragment edges

(inter_fragments), both 4.0 Å; Ca–Ca protein-virtual edges, 18 Å; ligand-virtual-protein-virtual edges, 15 Å — these are distinct from, and should not be confused with, the separate pocket-definition cutoffs (`data.py: pocket_distance_cutoff= 6.0, pocket_virtual_cutoff= 15.0`) governing which protein atoms are extracted into the graph *at all*, upstream of any of the above edge-construction cutoffs.

8.5 The obstruction: invariant features cannot output a vector

We can now state, precisely, why (8.8) as given is insufficient for (8.1)’s task.

Proposition 8.15 (Invariant features cannot produce an equivariant vector output). *Suppose every node feature h_i (initial and, by induction through (8.8), at every subsequent layer, since ϕ_m, ϕ_u are ordinary functions of $SE(3)$ -invariant inputs h_i, h_j, e_{ij}) is $SE(3)$ -invariant, and a readout $v_i = \rho(h_i) \in \mathbb{R}^3$ is defined as a function of h_i alone. If v_i is required to be $SE(3)$ -equivariant in the sense of (8.10) below, then $v_i \equiv 0$.*

$$v_i(\Phi_{(Q,b)}(\mathbf{x})) = Q v_i(\mathbf{x}) \quad \forall (Q, b) \in SE(3). \quad (8.10)$$

Proof. By hypothesis, $h_i(\Phi_{(Q,b)}(\mathbf{x})) = h_i(\mathbf{x})$ for every (Q, b) (invariance), so $v_i(\Phi_{(Q,b)}(\mathbf{x})) = \rho(h_i(\Phi_{(Q,b)}(\mathbf{x}))) = \rho(h_i(\mathbf{x})) = v_i(\mathbf{x})$: the left side of (8.10) equals $v_i(\mathbf{x})$ identically, so (8.10) forces $Q v_i(\mathbf{x}) = v_i(\mathbf{x})$ for every $Q \in SO(3)$. The only vector in $\mathbb{R}^3$ fixed by every rotation is 0 (if $v \neq 0$, take Q a rotation by 180° about an axis perpendicular to v : $Qv = -v \neq v$). $\square$

This is the obstruction promised at the end of Chapter 3’s motivation: a purely distance-based (invariant-feature) message passing network can predict *any scalar* (an energy, an affinity, a confidence score) but literally cannot, for any choice of ϕ_m, ϕ_u, ρ , output the translational or rotational velocity (8.1) requires. Invariance is too strong a symmetry constraint for a vector-valued task; equivariance (a strictly weaker requirement, since invariance is the special case “equivariant with respect to the trivial representation”, Example 4.19) is needed instead, and providing it is the entire content of Chapter 9.

8.6 A minimal repair, and why it is not enough

Before building the full steerable machinery, it is instructive to see the cheapest possible fix, since its specific limitations motivate exactly the representation-theoretic apparatus of Chapter 4.

Definition 8.16 (EGNN-style equivariant readout).

$$v_i = \sum_{j \in \mathcal{N}(i)} \phi_v(h_i, h_j, d_{ij}) x_{ij}, \quad \phi_v : \text{scalar-valued, built from invariant inputs only.}$$

Proposition 8.17. *Definition 8.16’s v_i satisfies (8.10).*

Proof. Under $\mathbf{x} \mapsto Q\mathbf{x} + b$: $x_{ij} \mapsto Qx_{ij}$ (translations cancel in a difference of two points; the term “+ b ” subtracted from itself), and $d_{ij} = \|x_{ij}\|$ is unchanged (Proposition 3.46), so $\phi_v(h_i, h_j, d_{ij})$, depending only on invariants, is unchanged too. Hence $v_i \mapsto \sum_j \phi_v(\dots) Qx_{ij} = Q \sum_j \phi_v(\dots) x_{ij} = Qv_i$. $\square$

This construction — known as EGNN (E(n)-Equivariant Graph Neural Networks; Satorras et al., ICML 2021) — is genuinely equivariant and genuinely cheap, but its expressive limitation is structural, not merely empirical, and worth stating precisely rather than only qualitatively. Definition 8.16’s v_i is a *scalar-weighted sum of degree-1 vectors*: in the language of Chapter 4, it uses only the $\ell_1 = 0$ (scalar weight ϕ_v) $\otimes_{\text{cg}}$ $\ell_2 = 1$ (vector x_{ij}) $\rightarrow \ell = 1$ case of Example 4.44, and *no other Clebsch–Gordan path at all*. Two concrete consequences follow directly from Proposition 4.48’s degree-mixing constraint and Theorem 4.34’s decomposition rule, not from vague “expressivity” intuition:

1. **Angular patterns among three or more neighbours are invisible.** A ring of neighbours arranged symmetrically around node i , all at the same distance and evenly spaced in angle, produces $\sum_j x_{ij} \approx 0$ by symmetry (the vectors cancel), so Definition 8.16’s readout is insensitive to the ring’s orientation, even though that orientation is physically meaningful (e.g. the plane of an aromatic ring relative to a binding pocket wall); Example 4.22’s explicit degree-2 harmonic $\propto 2x_3^2 - x_1^2 - x_2^2$ (a quadrupole) does *not* cancel under the same symmetric averaging, and this is precisely the sense in which $\ell \geq 2$ features carry information that $\ell \leq 1$ features cannot.
2. **Axial (rotation-vector) outputs get the wrong reflection parity unless tracked explicitly.** $\omega_\theta^{(i)} \in \mathfrak{so}(3) \cong \mathbb{R}^3$ is an *axial* vector (Remark 3.74): under a reflection, $\hat{\omega} \mapsto -\det(Q) Q \hat{\omega} Q^\top$ for $Q \in O(3)$ picks up an extra sign relative to an ordinary (*polar*) vector like x_{ij} under the same reflection, a fact provable directly from Proposition 3.50’s cross-product identity together with the general transformation rule $(Ax) \times (Ay) = \det(A) A^{-\top}(x \times y)$ quoted in the proof of Proposition 3.46, now with $A = Q \in O(3) \setminus SO(3)$ so $\det Q = -1$. A construction of exactly the form of Definition 8.16, built only from polar vectors x_{ij} , assigns $\omega_\theta^{(i)}$ the *polar* parity by default, which is simply the wrong object unless the reflection sign is tracked and corrected elsewhere — an easy silent error, and the reason Chapter 9 (and, per Remark 3.74, the discussion of molecular chirality) carries a parity label alongside ℓ .

What is needed, and what Chapter 4 already built, is: features of every angular degree ℓ , not only 0 and 1; and a way to combine two such features, of *any* degrees, into a third, without leaving the class of equivariant objects — exactly the Clebsch–Gordan tensor product (4.7). We assemble that now.

8.7 The steerable message

Everything is in place. We have, per edge, an invariant radial part and a steerable angular part; per node, steerable features; and, from Section 4.9, the one legal way to multiply two steerable objects. Assembling them gives the central equation of equivariant message passing.

8.7.1 The equation

$$m_{ij}^{(\ell)} = \sum_{\ell_1, \ell_2} w^{(\ell_1, \ell_2, \ell)}(d_{ij}) \left(h_j^{(\ell_1)} \otimes_{\text{cg}} Y^{\ell_2}(\hat{x}_{ij}) \right)^{(\ell)} \quad (8.11)$$

where the sum runs over pairs (ℓ_1, ℓ_2) for which ℓ is allowed, i.e. $|\ell_1 - \ell_2| \leq \ell \leq \ell_1 + \ell_2$ (Section 4.9.4).

Taken slowly, piece by piece:

$h_j^{(\ell_1)}$ — **what the neighbour knows.** The current state of atom j : a steerable feature of degree ℓ_1 , living in $\mathbb{R}^{2\ell_1+1}$ per channel. It encodes chemistry — element, environment, whatever the previous layers accumulated — with no reference to where j sits relative to i .

$Y^{\ell_2}(\hat{x}_{ij})$ — **where the neighbour is.** The edge direction, converted by Section 8.4.7 into a degree- ℓ_2 feature in $\mathbb{R}^{2\ell_2+1}$. It encodes geometry, with no reference to what kind of atom j is.

$\otimes_{\text{cg}}$ — **the only legal way to multiply them.** Chemistry and geometry are features of different degrees; the Clebsch–Gordan product is the unique (up to scale) bilinear map combining them equivariantly, and the superscript (ℓ) selects one allowed output degree. This is where the two kinds of information actually meet.

$w^{(\ell_1, \ell_2, \ell)}(d_{ij})$ — **how much, at this distance.** A learned scalar, computed from the radial features $\phi(d_{ij})$ of Section 8.4.5 by an ordinary MLP. Two questions about it deserve answers.

Why may it be an ordinary scalar? Because d_{ij} is invariant (Proposition 3.46), so w is invariant, and an invariant scalar multiplying an equivariant feature preserves equivariance (Section 4.10.10). No representation theory is needed for w itself.

Why should it depend on distance at all? Because the strength of an interaction does. A covalent neighbour at 1.4 Å and a loose contact at 5 Å should not contribute equally, and the network should be able to learn a different distance profile for each coupling path $(\ell_1, \ell_2) \rightarrow \ell$ — e.g. to let a short-range path dominate the $\ell = 2$ output while a long-range one dominates $\ell = 0$. Making w a function of d_{ij} , with a separate output channel per path, is what grants that freedom.

$m_{ij}^{(\ell)}$ — **the result.** A degree- ℓ feature in $\mathbb{R}^{2\ell+1}$, attached to the edge, ready to be delivered to node i .

What the message says

A plain GNN message says:

“Atom j has these properties.”

A steerable message says:

“Atom j has these properties, *and it is over there, this far away.*”

The difficulty is entirely in the words “over there”. A direction cannot be appended as three ordinary numbers without breaking rotation equivariance. Passing it through Y^{ℓ_2} converts it into an object of known type, and the Clebsch–Gordan product then couples it to the chemistry in the one way that keeps the whole message co-rotating with the molecule.

8.7.2 Two cases worth doing by hand

Example 8.18 (Scalar node feature: $0 \otimes 1 \rightarrow 1$). Suppose j carries only a scalar, $h_j^{(0)} \in \mathbb{R}$, and we use $\ell_2 = 1$. The selection rule allows only $\ell = 1$, and by Example 4.37 the Clebsch–

Gordan product with a scalar is ordinary multiplication:

$$m_{ij}^{(1)} = w(d_{ij}) h_j^{(0)} Y^1(\hat{x}_{ij}) \propto \underbrace{w(d_{ij}) h_j^{(0)}}_{\text{a number}} \cdot \underbrace{\hat{x}_{ij}}_{\text{a direction}}.$$

Interpretation: *a scalar at j becomes a vector at i , pointing along the edge, with magnitude set by the scalar and the distance*. If the scalar channel has learned to encode “how strongly I should push”, the message is literally a push in the direction of the neighbour.

This is worth dwelling on because it is the minimal mechanism by which directional information is created from non-directional information — which, by Caution 4.45, no linear layer could do. The direction did not come from the features; it came from the geometry, and the tensor product is what let them meet. It is also, structurally, exactly the EGNN readout of Definition 8.16: EGNN is the special case of (8.11) with $\ell_1 = 0$, $\ell_2 = 1$ and nothing else.

Example 8.19 (Vector node feature: $1 \otimes 1 \rightarrow 0 \oplus 1 \oplus 2$). Now let j carry a vector feature $h_j^{(1)} \in \mathbb{R}^3$, still with $\ell_2 = 1$. Now *three* output degrees are available, and by Section 4.9.3 we know exactly what each is:

Output	Concretely	What it detects
$\ell = 0$	$\propto h_j^{(1)} \cdot \hat{x}_{ij}$	how much j ’s vector points <i>along</i> the edge
$\ell = 1$	$\propto h_j^{(1)} \times \hat{x}_{ij}$	the component <i>perpendicular</i> to the edge
$\ell = 2$	sym. traceless part	the <i>alignment</i> of j ’s vector with the edge axis

A single pair of inputs produces three qualitatively different, and simultaneously available, geometric readings. The $\ell = 0$ output is an invariant that node i can feed to any MLP; the $\ell = 1$ output is a new direction; the $\ell = 2$ output is orientation information that neither of the others can express.

This is the payoff for Section 4.9.3. The decomposition $1 \otimes 1 = 0 \oplus 1 \oplus 2$ was, at the time, an exercise in splitting a 3×3 matrix. Here it is a statement about what a molecule’s geometry can tell a network.

8.7.3 Aggregation over neighbours

Each edge produces a message; node i must combine them:

$$m_i^{(\ell)} = \sum_{j \in \mathcal{N}(i)} m_{ij}^{(\ell)}. \quad (8.12)$$

Proposition 8.20 (Sum aggregation is equivariant). *If each $m_{ij}^{(\ell)} \mapsto D^\ell(Q) m_{ij}^{(\ell)}$, then $m_i^{(\ell)} \mapsto D^\ell(Q) m_i^{(\ell)}$.*

Proof.

$$\sum_j D^\ell(Q) m_{ij}^{(\ell)} = D^\ell(Q) \sum_j m_{ij}^{(\ell)} = D^\ell(Q) m_i^{(\ell)},$$

by linearity of $D^\ell(Q)$. □

The proof is one line, but the hypothesis is what matters: *all summands must live in the same representation space*. Adding an $\ell = 1$ message to an $\ell = 2$ message is not merely bad practice — the sum has no transformation law at all. This is why implementations keep degrees in separate buffers and sum within each.

Note also what is *not* required: nothing about $\mathcal{N}(i)$, its size, or the order of summation.

Caution 8.21 (Two different symmetries, easily conflated). Equation (8.12) is doing two independent jobs.

Spatial symmetry ($SO(3)$). Handled by Proposition 8.20: the sum commutes with $D^\ell(Q)$ because $D^\ell(Q)$ is linear.

Permutation symmetry (S_n). The neighbours of i form a *set*; they have no canonical order, and the answer must not depend on one. A sum is permutation-invariant because addition is commutative.

These are different groups acting on different things — rotations act on $\mathbb{R}^3$, permutations act on node labels — and a construction can respect one and violate the other. Concatenating neighbour messages in index order, for instance, is perfectly rotation-equivariant and badly permutation-dependent. Sum (or mean, or max over invariants) is the standard way to secure the second; Proposition 8.5 treats it in its own right.

8.7.4 Self-interaction and residual connections

A node should not forget itself. The update combines the aggregated message with the node’s own state:

$$h_i^{(\ell),\text{new}} = W_{\text{self}}^{(\ell)} h_i^{(\ell)} + m_i^{(\ell)}, \quad (8.13)$$

where $W_{\text{self}}^{(\ell)}$ is an equivariant linear layer (Equation (4.10)), i.e. channel mixing within degree ℓ .

Equivariance is immediate and worth stating as a general principle:

Proposition 8.22 (Sums of equivariant maps of the same type are equivariant). *If f, g both map into V_ℓ and both satisfy $f(Q \cdot x) = D^\ell(Q)f(x)$, then so does $f + g$. In particular, with $g = \text{id}$, the residual connection $h \mapsto h + f(h)$ is equivariant whenever f is and f ’s output has the same degree as h .*

Proof. $(f + g)(Q \cdot x) = D^\ell(Q)f(x) + D^\ell(Q)g(x) = D^\ell(Q)((f + g)(x))$. □

The residual case is what makes deep equivariant transformers possible: every block may be written $h \mapsto h + \text{Block}(h)$, and Proposition 8.22 guarantees the skip connection costs nothing in symmetry — *provided* the block’s output degrees match its input degrees, which is why architectures fix an irrep specification and keep it constant across blocks.

Remark 8.23 (Normalisation: a preview). Deep networks need normalisation, and the naive version is unavailable here: computing a mean and variance across all m -components of an $\ell \geq 1$ feature and subtracting the mean would treat the components as interchangeable numbers — the same error as componentwise ReLU. Equivariant normalisation must instead act on invariants (e.g. rescale each irrep block by a function of its norm, leaving direction untouched, in the spirit of Section 4.10.9). EquiformerV2 has a specific construction; we defer it to Chapter 9 rather than sketch it here.

8.7.5 The pipeline, end to end

Collecting everything:

$$\begin{aligned}
x_i, x_j \in \mathbb{R}^3 &\implies x_{ij} = x_j - x_i \in \mathbb{R}^3 \quad (\text{translation-invariant, Proposition 8.6}) \\
&\Downarrow \\
x_{ij} = d_{ij} \hat{x}_{ij} &\quad d_{ij} \in \mathbb{R}_{>0} \ (\ell = 0), \quad \hat{x}_{ij} \in S^2 \text{ (equivariant)} \\
&\Downarrow \\
\phi(d_{ij}) \in \mathbb{R}^K &\text{ (radial, invariant)} \quad Y^{\ell_2}(\hat{x}_{ij}) \in \mathbb{R}^{2\ell_2+1} \text{ (angular, steerable)} \\
&\Downarrow \\
h_j^{(\ell_1)} \otimes_{\text{cg}} Y^{\ell_2}(\hat{x}_{ij}) &\quad \text{allowed } \ell \in \{|\ell_1 - \ell_2|, \dots, \ell_1 + \ell_2\} \\
&\Downarrow \\
m_{ij}^{(\ell)} = \sum_{\ell_1 \ell_2} w^{(\ell_1, \ell_2, \ell)}(d_{ij}) (\dots)^{(\ell)} &\in \mathbb{R}^{2\ell+1} \\
&\Downarrow \\
m_i^{(\ell)} = \sum_{j \in \mathcal{N}(i)} m_{ij}^{(\ell)} &\quad (\text{Proposition 8.20}) \\
&\Downarrow \\
h_i^{(\ell), \text{new}} = W_{\text{self}}^{(\ell)} h_i^{(\ell)} + m_i^{(\ell)} &
\end{aligned}$$

8.7.6 A small worked example

Three atoms i, j, k ; features 1 channel of $\ell = 0$ and 1 channel of $\ell = 1$ per node; $L = \ell_{\max} = 1$; $K = 3$ radial basis functions.

Object	Shape (per edge / node)	Type
x_i	[3]	position (not a feature)
$x_{ij} = x_j - x_i$	[3]	$\ell = 1$
$d_{ij} = \ x_{ij}\ $	[1]	$\ell = 0$
$\hat{x}_{ij}$	[3]	on S^2
$\phi(d_{ij})$	[3]	$\ell = 0$ (three basis values)
$Y^0(\hat{x}_{ij})$	[1]	$\ell = 0$ (constant)
$Y^1(\hat{x}_{ij})$	[3]	$\ell = 1$
$h_j^{(0)}$	[1, 1] ($c \times m$)	$\ell = 0$
$h_j^{(1)}$	[1, 3]	$\ell = 1$

Which coupling paths exist? With inputs of degree $\ell_1 \in \{0, 1\}$, edge harmonics of degree $\ell_2 \in \{0, 1\}$, and outputs truncated at $\ell \leq 1$:

(ℓ_1, ℓ_2)	allowed ℓ	kept ($\ell \leq 1$)
(0, 0)	0	0
(0, 1)	1	1
(1, 0)	1	1
(1, 1)	0, 1, 2	0, 1 (the $\ell = 2$ path is discarded by the truncation)

So five surviving paths, each with its own learned radial weight $w^{(\ell_1, \ell_2, \ell)}(d_{ij})$ — five scalar functions of distance, i.e. a radial MLP $\mathbb{R}^3 \rightarrow \mathbb{R}^5$.

Messages. For edge $i \leftarrow j$:

$$m_{ij}^{(0)} = w_{(0,0,0)} h_j^{(0)} Y^0 + w_{(1,1,0)} (h_j^{(1)} \cdot \hat{x}_{ij}) \in \mathbb{R}^1,$$

$$m_{ij}^{(1)} = w_{(0,1,1)} h_j^{(0)} Y^1(\hat{x}_{ij}) + w_{(1,0,1)} h_j^{(1)} + w_{(1,1,1)} (h_j^{(1)} \times \hat{x}_{ij}) \in \mathbb{R}^3,$$

suppressing normalisation constants. Every term on the right of the first line is a scalar; every term on the second is a 3-vector. Nothing else could appear.

Aggregation and update. $m_i^{(\ell)} = m_{ij}^{(\ell)} + m_{ik}^{(\ell)}$, then (8.13). Output shapes are unchanged: $[1, 1]$ and $[1, 3]$ per node, ready for the next block.

Sanity check. Rotate all three positions by Q and translate by a . Then d_{ij} , ϕ , and all w 's are unchanged; $\hat{x}_{ij} \mapsto Q\hat{x}_{ij}$; $Y^1 \mapsto QY^1$; so $m_i^{(0)}$ is unchanged and $m_i^{(1)} \mapsto Qm_i^{(1)}$ — exactly as required. (Simulating this numerically on three random atoms reproduces the identity to machine precision, $\sim 10^{-16}$; it is a worthwhile five-line check when implementing.)

8.7.7 What is fixed and what is learned

Fixed by geometry and symmetry	Learned from data
$x_{ij} = x_j - x_i$	the radial MLP producing $w(d)$
$d_{ij}, \hat{x}_{ij}$	channel-mixing matrices $A^{(\ell)}$
the spherical harmonics Y^ℓ	self-interaction weights
which coupling paths exist	how strongly each path is used
the Clebsch–Gordan coefficients	gate pre-activations
all transformation laws	(later) attention weights

Note where the boundary falls. The network never learns *which* couplings are permitted — symmetry decides that — but it does learn how much to use each permitted one, and at what distances. That is a much smaller, much better-posed learning problem than “figure out three-dimensional geometry from data”.

8.7.8 Why this deserves the name “steerable”

The word is not decoration. A feature is *steerable* when its behaviour under the group is known in advance and given by an explicit matrix: rotate the input, and the feature transforms by $D^\ell(Q)$ — one can “steer” it without recomputing it from scratch. Every quantity in (8.11) is of this kind, and every operation used (spherical harmonics, Clebsch–Gordan product, scalar multiplication by an invariant, summation, equivariant linear layers) maps steerable inputs to steerable outputs with a known law.

That closure property is what makes the equivariance of the whole network a *theorem* obtained by induction over layers, rather than a property to be measured after training.

8.7.9 The question this leaves open

Look again at (8.12). Every neighbour contributes with the same structural weight; the only way a neighbour can matter more is through $w(d_{ij})$, which sees *distance alone*. Two neighbours at the same distance are treated identically regardless of what they are or what the rest of the environment looks like.

But chemistry is not like that. A hydrogen-bond donor at 3 Å and a hydrophobic carbon at 3 Å should not contribute equally, and how much each matters may depend on the rest of the pocket.

What if the model could learn, per edge and in context, how much each neighbour should contribute?

That is attention, and it is where Chapter 9 begins. Note the constraint it must satisfy, already derivable from what we have: an attention weight multiplies an equivariant message, so by Section 4.10.10 it must be an *invariant* scalar. Everything in Equiformer’s attention mechanism follows from getting an expressive, context-dependent, invariant number out of a pair of steerable features.

Why this matters for SigmaFlow

Relative geometry. Docking depends on where the ligand sits *relative* to the pocket, never on the global frame. Building the architecture on x_{ij} rather than x_i makes translation symmetry exact and free, and reduces the whole symmetry problem to $SO(3)$.

The radial/angular split. Distance and direction have different symmetry types, so they get different machinery: an ordinary MLP for the invariant half, spherical harmonics for the equivariant half. Recognising this split is what makes `gaussian_rbf.py` and `so3.py` look like two halves of one idea rather than two unrelated files.

Tensor-product messages. They are how chemical features learn about geometry at all. Proposition 8.15 proves a scalar-only network cannot output a pose; (8.11) is the repair.

Aggregation. Each atom integrates geometrically structured information from its neighbourhood, with spatial equivariance and permutation invariance secured by different arguments (Caution 8.21) — both of which must hold.

Independent of the generative engine. Nothing in this section mentions scores, velocities, noise schedules or ODEs. It is entirely about how a network may represent and propagate geometric information. Swapping SigmaDock’s diffusion for SigmaFlow’s flow matching changes what the final $\ell = 1$ readout *means* and how it is trained — not one line of the message construction above. That is the precise sense in which the backbone is shared.

Chapter 9 now assembles this into a complete, code-verified architecture.

8.8 What you should now be able to do

- state the two symmetry requirements a pose network must satisfy, and why data augmentation is not a substitute for either;
- explain why absolute atom positions are the wrong network input and edge vectors are the right one;

- prove that a network whose internal features are all invariant cannot emit a vector (Proposition 8.15), and see why the obvious repair is insufficient;
- read the steerable message equation term by term and say which parts are learned and which are fixed by geometry.

Chapter 9 now replaces this idealised message with the one EquiformerV2 actually computes, and explains what each deviation buys.

Chapter 9

Equiformer and EquiformerV2

This chapter is deliberately the slowest and most granular in the document, per the stated goal of enabling a reader to open the actual network source (`net/` in `SigmaFlow_Development/src/sigmadock/`) and follow it line by line afterward. We assume from the reader only Chapters 3 and 4 (manifolds, $SO(3)$, spherical harmonics, Wigner- D , Clebsch–Gordan) and Chapter 8 (attention, message passing, the invariance obstruction); nothing else from earlier chapters is needed to follow the architecture itself, though its role as the score/vector-field network of Chapters 6 and 7 is what it is *for*.

The architecture implemented is EquiformerV2 (Liao et al., *EquiformerV2: Improved Equivariant Transformer for Scaling to Higher-Degree Representations*, ICLR 2024), building on the original Equiformer (Liao & Smidt, ICLR 2023) and the eSCN reduction (Passaro & Zitnick, *Reducing $SO(3)$ Convolutions to $SO(2)$ for Efficient Equivariant GNNs*, ICML 2023). We derive the ideas in the order that makes them motivated, not necessarily the historical order, and flag explicitly, in Section 9.14, the one place where the actual shipped configuration is a genuine *hybrid* of the two papers’ recipes rather than a pure implementation of either.

9.1 From scalar messages to steerable messages

Motivation. Definition 8.14’s message $m_{ij} = \phi_m(h_i, h_j, e_{ij})$ was, implicitly, scalar-valued (or, at best, per Definition 8.16, a scalar times a fixed direction vector). We now let *every* object in sight — node features, messages, edge embeddings — be a full steerable feature in the sense of Definition 4.54: a direct sum over degrees $\ell = 0, \dots, L$ (EquiformerV2’s default, confirmed directly from the constructor defaults in `net/model.py`, is $L = \text{lmax} = 3$), each degree carrying some number C_ℓ of channels.

Definition 9.1 (Steerable node/edge feature, restated for the network). A node i ’s feature is $h_i = \bigoplus_{\ell=0}^L \bigoplus_{c=1}^{C_\ell} h_i^{\ell,c} \in \bigoplus_{\ell} (\mathbb{R}^{2\ell+1})^{C_\ell}$, transforming, under a rotation Q of the whole scene, by $h_i \mapsto (\bigoplus_{\ell,c} D^\ell(Q)) h_i$ (Definition 4.54). EquiformerV2’s default channel counts are uniform across degree, $C_\ell \equiv C = \text{sphere_channels} = 256$ (constructor default, `net/model.py`).

The embedding step (Section 9.11 below) initialises every node’s $\ell = 0$ channels from ordinary categorical/continuous input features (Section 8.5’s invariant quantities are exactly the correct content for a type-0 channel) and leaves every $\ell \geq 1$ channel at exactly zero; the network’s job, across its layers, is to populate the higher-degree channels with genuinely

angular information extracted from the geometry, and finally to read a 3-vector out of the $\ell = 1$ channels as the network’s geometric output (Section 9.12).

9.2 Embedding geometry: spherical harmonics on edges

Motivation. Chapter 4 constructed, for each degree ℓ , an explicit map $Y^\ell : S^2 \rightarrow \mathbb{R}^{2\ell+1}$ satisfying the equivariance law (4.1). This is exactly the tool needed to lift a *geometric* quantity — the direction $\hat{x}_{ij} = x_{ij}/d_{ij}$ of an edge — into a *steerable feature* that can be combined, via the Clebsch–Gordan product, with a node’s existing steerable feature.

Definition 9.2 (Equivariant edge embedding). Each edge (i, j) is embedded as the pair $(\psi(d_{ij})\chi(d_{ij}), Y^\ell(\hat{x}_{ij})_{\ell \leq L})$: an invariant radial part (Equation (8.9), code-verified form recorded in Section 8.4’s code correspondence) and an equivariant angular part, one $(2\ell + 1)$ -dimensional block per degree up to L .

Recall from Example 4.22 that $Y^0 \equiv \text{const}$ and (up to normalisation) $Y^1(\hat{x}) = \hat{x}$; substituting $L = 0$ or $L = 1$ into Definition 9.2 exactly recovers Section 8.4’s distance-only GNN or Definition 8.16’s EGNN readout respectively: the steerable-feature construction is a strict generalisation, not a different framework, of everything built so far.

9.3 The eSCN reduction: aligning every edge to its own frame

Motivation, and the problem eSCN solves. A naive implementation of steerable message passing would, at every edge, form a full Clebsch–Gordan tensor product (4.7) between the neighbour’s feature h_j (up to degree L) and the edge’s spherical harmonic $Y^{\ell'}(\hat{x}_{ij})$ (up to degree L), for every pair of degrees (ℓ, ℓ') — a computation whose cost grows steeply with L , since the number of $(\ell_1, \ell_2) \rightarrow \ell$ paths in Theorem 4.34 grows roughly cubically in L (each pair (ℓ_1, ℓ_2) contributes up to $\min(2\ell_1, 2\ell_2) + 1$ output degrees). EquiformerV2’s central algorithmic contribution, inherited from eSCN, is an exact reformulation that avoids ever explicitly instantiating a Clebsch–Gordan coefficient table.

Proposition 9.3 (Rotating an edge onto a fixed axis reduces $SO(3)$ to $SO(2)$). *Let $\hat{x}_{ij} \in S^2$ be an edge direction, and let $Q_{ij} \in SO(3)$ be any rotation with $Q_{ij}\hat{x}_{ij} = e_2$ (the second standard basis vector; such Q_{ij} always exists, and is unique up to an arbitrary further rotation about e_2 itself, since $\{Q : Qe_2 = e_2\} \cong SO(2)$). Working in the rotated frame (applying $D^\ell(Q_{ij})$ to every steerable feature, per Equation (4.1)), the edge direction itself becomes the fixed vector e_2 , and every subsequent operation that only needs to be equivariant with respect to rotations fixing the edge direction — rather than all of $SO(3)$ — may therefore use the much smaller symmetry group $SO(2)$ (rotations about e_2) instead of the full $SO(3)$, since $\hat{x}_{ij}$ itself no longer transforms (it is a fixed constant, e_2 , in this frame) and only the other steerable quantities (the rotated node features) need to respect the residual $SO(2)$ symmetry about the now-fixed axis.*

Justification, not a formal theorem. This is a change-of-frame argument, not a new mathematical fact requiring proof beyond Equation (4.1) itself: given any $SO(3)$ -equivariant operation F acting on (steerable feature, edge direction) pairs, define $\tilde{F}(h, e_2) := D^\ell(Q_{ij})F(D^\ell(Q_{ij})^{-1}h, \hat{x}_{ij})$; by construction $\tilde{F}$ agrees with F after transporting through the fixed rotation Q_{ij} , and since its second argument is now always the fixed vector e_2 , $\tilde{F}$ need only respect rotations fixing

e_2 (the residual $SO(2)$) to guarantee F itself was $SO(3)$ -equivariant — the constraint on the *first* argument’s transformation law has been reduced from a full $SO(3)$ representation to an $SO(2)$ representation, at the cost of applying $D^\ell(Q_{ij})$ and its inverse once per edge. $\square$

This is the eSCN insight stated precisely: *per-edge*, rotate into a frame where the edge direction is a fixed axis, do all the expensive feature-mixing work under the much cheaper $SO(2)$ -equivariance constraint (an abelian group, whose irreps are 1-dimensional — ordinary complex, equivalently 2×2 -real, numbers indexed by a single integer m , rather than $SO(3)$ ’s $(2\ell + 1)$ -dimensional irreps requiring the full Clebsch–Gordan machinery), then rotate back.

Code correspondence

Mathematical object: the per-edge rotation Q_{ij} of Proposition 9.3 and its Wigner- D realisation

Implementation: `net/edge_rot_mat.py::init_edge_rot_mat` (builds the orthonormal 3×3 frame: first axis = $\hat{x}_{ij}$; second axis chosen, per-edge, as the least-aligned of several candidate reference vectors, to avoid a near-degenerate cross product when constructing the third axis; near-zero-length edges, $\|x_{ij}\| < 10^{-5}$, fall back to a fixed default direction, logging a warning) composed with `net/wigner.py` (assembling the full block-diagonal Wigner- D matrix, via `e3nn`’s `o3.wigner_D` applied to the Euler angles of Q_{ij} , extracted via `o3.xyz_to_angles`)

Tensor shape: one $Q_{ij} \in \mathbb{R}^{3 \times 3}$, and one block-diagonal Wigner- D matrix of size $\sum_{\ell=0}^L (2\ell + 1) = \sum_{\ell=0}^3 (2\ell + 1) = 16$ (for the default $L = 3$), *per edge*

Interpretation: every neighbour’s steerable feature is rotated into its (edge-specific!) aligned frame *before* the message computation of Section 9.5 below, and the resulting message is rotated back *afterward*, via `S03_Embedding._rotate/_rotate_inv` (`transformer_block.py`) — i.e. this per-edge alignment happens *inside* every attention block, not once globally, since a different edge has a different direction and hence a different aligned frame.

9.4 Why EquiformerV2 exists: the cost of full $SO(3)$ coupling

Equiformer (V1) is the architecture assembled in Sections 8.7 and 9.10: steerable features, Clebsch–Gordan messages, invariant attention weights, gating, residual blocks. It is correct and it works. What it is not is cheap, and understanding *where* the cost sits is the whole motivation for V2.

9.4.1 Counting the work in one message

The message (8.11) evaluates, for each edge and each allowed triple (ℓ_1, ℓ_2, ℓ) ,

$$z_m^{(\ell)} = \sum_{m_1=-\ell_1}^{\ell_1} \sum_{m_2=-\ell_2}^{\ell_2} C_{(\ell_1, m_1)(\ell_2, m_2)}^{(\ell, m)} x_{m_1}^{(\ell_1)} y_{m_2}^{(\ell_2)}.$$

Three separate multiplicities pile up:

1. **Over m -indices.** Even exploiting the sparsity $m = m_1 + m_2$, each output component still sums over $O(\min(\ell_1, \ell_2))$ terms, and there are $2\ell + 1$ output components.

2. **Over coupling paths.** With degrees up to L , the number of admissible triples (ℓ_1, ℓ_2, ℓ) with all three $\leq L$ grows roughly like L^3 . Every one of them is a separate contraction with its own coefficient array.
3. **Over channel pairs.** (4.8) sums over $C_1 \times C_2$ channel pairs per path.

The product of the three is what makes raising L expensive. Doubling L does not double the cost; it multiplies it by something closer to 8 from the path count alone, before the per-path work is counted. This is the concrete reason V1 implementations are usually run at $L = 1$ or $L = 2$: not because higher degrees are useless, but because they were unaffordable.

The question V2 answers: can the coupling be computed in a frame where most of it is unnecessary?

9.4.2 The idea: use a frame in which the edge is trivial

Here is the observation. In (8.11) the second factor is not an arbitrary feature — it is $Y^{\ell_2}(\hat{x}_{ij})$, the spherical harmonic of *one specific direction*. And a spherical harmonic evaluated at a direction aligned with the reference axis is almost entirely zero: all but its $m = 0$ component vanish.

So if we first rotate into a frame where the edge *is* the reference axis, the sum over m_2 collapses to the single term $m_2 = 0$. Combined with $m = m_1 + m_2$, this forces $m = m_1$: **the double sum degenerates into a diagonal map in m** . That is the entire trick, and everything else — the rotation matrices, the $SO(2)$ language, the $m = 0$ attention channels — is bookkeeping around it.

Proposition 9.4 (Cost of the coupling before and after alignment). *Fix a path (ℓ_1, ℓ_2, ℓ) and one channel pair. In a general frame, the contraction costs $O((2\ell + 1) \cdot \min(2\ell_1, 2\ell_2))$ multiply-adds. In the edge-aligned frame, where only $m_2 = 0$ survives, it costs $O(2\ell + 1)$: one multiplication per output component. The saving per path is therefore a factor of order $\min(\ell_1, \ell_2)$.*

Caution 9.5 (The rotation is not free, and that is the real trade). Rotating into the edge frame means applying $D^\ell(Q_{ij})$ to every steerable feature on that edge — a dense $(2\ell + 1) \times (2\ell + 1)$ matrix-vector product per degree, per channel, *per edge*, and again to rotate back. That is real work, and for small L it can exceed what it saves.

The reason it pays at larger L is that the rotation cost grows like $\sum_\ell (2\ell + 1)^2 = O(L^3)$ per channel, while the *number of coupling paths* it eliminates work from grows like $O(L^3)$ *as well*, each path having its own m -sum. In other words: the rotation is paid once per edge and amortised over all paths and channels. This is why EquiformerV2 can run at $L = 4$ – 6 where V1 was confined to $L \leq 2$, and it is also why the reduction would be pointless for a single path.

9.5 $SO(2)$ -equivariant convolution: the actual EquiformerV2 message

What an $SO(2)$ -equivariant linear map looks like. By the same Schur’s-lemma argument as Proposition 4.46, but for the *abelian* group $SO(2)$ (whose irreps, unlike $SO(3)$ ’s,

are complex 1-dimensional, indexed by an integer $m \in \mathbb{Z}$, with Q_α (rotation by angle α) acting on the degree- m irrep by multiplication by $e^{im\alpha}$: an $SO(2)$ -equivariant linear map on the m -indexed components of a steerable feature, once expressed in the aligned frame of Proposition 9.3, may mix channels *within* a fixed order $|m|$ (matching real and imaginary/paired components together) but not across different values of $|m|$, and (since a real degree- ℓ spherical-harmonic component of order m and one of order $-m$ transform as a genuine, non-collapsible complex-conjugate pair under $SO(2)$ for $m \neq 0$) the map on this pair is exactly a 2×2 real matrix realising multiplication by a complex number, while the single $m = 0$ component per degree is an ordinary real scalar with no complex structure.

Definition 9.6 ($SO(2)$ convolution, informal). Given a steerable feature h (in the aligned frame), an $SO(2)$ -equivariant linear layer computes, for each order $m = 0, 1, \dots, \text{mmax}$: an ordinary real-linear map on the (concatenated, across degrees $\ell \geq |m|$) $m = 0$ components; and, for $m \geq 1$, a complex-linear map $(x_r, x_i) \mapsto (W_1 x_r - W_2 x_i, W_1 x_i + W_2 x_r)$ on the (real, imaginary) parts of the m -order components, with W_1, W_2 learned real matrices mixing channels within that order. All weights (W_1, W_2 , and the $m = 0$ map’s weights) are generated from an ordinary MLP applied to *invariant* scalars only (the radial+chemistry edge embedding), per edge, per edge type.

Code correspondence

Mathematical object: the $SO(2)$ -equivariant message layer of Definition 9.6

Implementation: `net/so2_ops.py::S02_Convolution, S02_m_Convolution`

Interpretation: this is, precisely, the departure from a naive Clebsch–Gordan-tensor-product message $m_{ij} = W_{ij}(h_j \otimes_{\text{cg}} Y(\hat{x}_{ij}))$ that a first-principles-only reading of Chapter 4 might suggest as the message formula, and it is worth being explicit that the *executed code does not form this Clebsch–Gordan product at all*: after the edge-frame rotation of Section 9.3, the edge direction is the fixed vector e_2 , so mixing a node’s steerable feature with the (now-fixed, hence not needing to be explicitly multiplied in via CG coefficients) edge direction reduces, by Proposition 9.3, to an $SO(2)$ -equivariant linear operation on the node feature alone, and it is *this* operation, not a tensor product, that `S02_Convolution` implements. Both are, by Proposition 9.3’s justification, mathematically legitimate realisations of an equivariant message; the eSCN reduction is a genuine *algorithmic* improvement (removing the CG-path combinatorics entirely, replacing them with ordinary/complex linear layers), not merely a reparameterisation of the same computation, and this is the precise, verifiable sense in which EquiformerV2 “improves on” the original Equiformer’s direct tensor-product message construction. The per-edge radial MLPs generating W_1, W_2 have their **bias terms removed** specifically for the transient (protein–ligand interaction) edge types — confirmed directly from the architecture description this monograph is auditing, and consistent with Section 8.4’s smooth-cutoff argument: with a bias, a message would not vanish even when the radial features $\psi(d)\chi(d)$ do, at $d \rightarrow r_{\text{cut}}$, breaking the continuity property the envelope χ was introduced to guarantee.

9.6 Equivariant nonlinearity: gating and the separable S^2 activation

Why an ordinary nonlinearity breaks equivariance. By Proposition 4.46, the *only* linear equivariant operations are per-degree scalar rescalings; a network built from linear layers alone is itself linear (up to the invariant-scalar-weight structure) and severely limited

in expressive power, exactly as an ordinary neural network with no nonlinearity would be. But an entrywise nonlinearity σ applied directly to a degree- $\ell \geq 1$ feature, $h \mapsto \sigma(h)$, does not commute with $D^\ell(Q)$ in general (a direction-dependent nonlinear reshaping of a vector is generally not the same before and after rotating the vector), so it is not equivariant, and cannot be used directly.

Definition 9.7 (Gate nonlinearity). For a steerable feature channel $h^{\ell,c}$ ($\ell \geq 1$) and an associated, separately-computed *invariant* (type-0) scalar $g^{\ell,c}$,

$$h^{\ell,c} \mapsto \sigma(g^{\ell,c}) \cdot h^{\ell,c},$$

for an ordinary scalar nonlinearity σ (e.g. a sigmoid or SiLU).

Proposition 9.8. *Definition 9.7’s map is equivariant.*

Proof. Under a rotation, $g^{\ell,c} \mapsto g^{\ell,c}$ (invariant, by construction — it is produced by a separate, invariant-only computation path) and $h^{\ell,c} \mapsto D^\ell(Q)h^{\ell,c}$, so $\sigma(g^{\ell,c})h^{\ell,c} \mapsto \sigma(g^{\ell,c})D^\ell(Q)h^{\ell,c} = D^\ell(Q)(\sigma(g^{\ell,c})h^{\ell,c})$ using linearity of $D^\ell(Q)$ and that $\sigma(g^{\ell,c})$, an unchanged scalar, commutes past the matrix. $\square$

Geometrically: the gate rescales a steerable feature’s *magnitude* nonlinearly (via the invariant scale $\sigma(g)$) while leaving its *direction* (its transformation behaviour) completely untouched — exactly the one degree of freedom (an overall scalar multiplier) that Proposition 4.46 says is available on a single channel without violating equivariance, now made nonlinear in the *scalar* channel that controls it.

Code correspondence

Mathematical object: the equivariant nonlinearity actually used by default

Implementation: `net/activation.py::SeparableS2Activation` (config default: `use_gate_act=False, use_sep_s2_act=True`)

Interpretation: EquiformerV2’s default nonlinearity is *not* literally Definition 9.7 but a closely related, more expressive construction: the steerable feature is projected onto a discrete grid of points on S^2 (via `e3nn`’s `ToS2Grid`, resolution set by `L/mmax`), an *ordinary*, pointwise nonlinearity is applied at each grid point (legitimate precisely because evaluating a steerable function at a rotated grid point equals evaluating the un-rotated function at the inverse-rotated point — the grid-sampling analogue of Proposition 4.23’s transformation law, applied to function values rather than coefficients), and the result is projected back (`FromS2Grid`) into spherical-harmonic coefficients. An additional, separate scalar “gating” path (extra invariant channels produced alongside the main $SO(2)$ -convolution output) modulates this activation further, combining both mechanisms. The equivariance of this construction is *not* exact once discretised; the next subsection derives where it holds, where it fails, and by how much.

9.6.1 Resolving the apparent contradiction

Example 4.14 proved that applying ReLU componentwise to an $\ell = 1$ feature breaks equivariance. The activation just described applies SiLU pointwise and claims to be fine. Both cannot be true without a distinction, and that distinction is the most important idea in this subsection.

Two different spaces. The coefficients $x_m^{(\ell)}$ and the function they represent are different objects:

$$\underbrace{\{x_m^{(\ell)}\}_{\ell \leq L, |m| \leq \ell}}_{\text{coefficients}} \longleftrightarrow \underbrace{f(\omega) = \sum_{\ell, m} x_m^{(\ell)} Y_m^\ell(\omega)}_{\text{a function } f: S^2 \rightarrow \mathbb{R}}.$$

The m -components are *Fourier coefficients* of f in the spherical-harmonic basis. Applying σ to a coefficient is like applying it to a Fourier coefficient of an audio signal: meaningless, because the index is a bookkeeping label. Applying σ to $f(\omega)$ is like applying it to the waveform: natural.

Why pointwise on the sphere is exactly equivariant. A rotation acts on functions by reparametrising the argument, $(Q \cdot f)(\omega) = f(Q^{-1}\omega)$ (Caution 4.3 — note the inverse). Then for any $\sigma : \mathbb{R} \rightarrow \mathbb{R}$,

$$(\sigma \circ (Q \cdot f))(\omega) = \sigma(f(Q^{-1}\omega)) = (\sigma \circ f)(Q^{-1}\omega) = (Q \cdot (\sigma \circ f))(\omega).$$

The middle step is just the definition of composition. *A pointwise operation cannot tell which point it is standing at*, so it commutes with any relabelling of points — and a rotation is exactly a relabelling. That is why the operation is legal in function space and illegal in coefficient space: there, a rotation is not a relabelling but a genuine mixing.

Why the discretised version is only approximate.

1. **Coefficients** $\rightarrow$ **grid**. Exact: evaluating a band-limited function at fixed points.
2. **Pointwise** σ . Exact in function space — *but* $\sigma(f)$ is no longer band-limited. A nonlinearity creates arbitrarily high angular frequencies.
3. **Grid** $\rightarrow$ **coefficients**. The projection onto degrees $\leq L$ is computed by quadrature on a *finite, fixed* grid. Since $\sigma(f)$ has energy above the band limit, the quadrature aliases; and because the grid is fixed in space while f rotates, the aliasing error depends on the orientation. *That orientation dependence is the loss of equivariance.*

In the continuum the operation *is* exactly equivariant: the space of degree- $\leq L$ functions is rotation-invariant, and orthogonal projection onto a rotation-invariant subspace commutes with rotation. The error is entirely an artefact of discretisation.

Remark 9.9 (How large is the error in this implementation?). Measurable, and not negligible. Feeding random coefficients through the shipped activations and comparing $\text{act}(D^\ell(Q)x)$ with $D^\ell(Q) \text{act}(x)$ over random $Q \in SO(3)$:

L	S2Activation	SeparableS2Activation	round trip <i>without</i> σ
1	1.3×10^{-2}	1.5×10^{-2}	7×10^{-16}
2	8.1×10^{-2}	2.1×10^{-1}	8×10^{-16}
3	6.3×10^{-1}	3.9×10^{-1}	1×10^{-15}

Two conclusions. First, the grid transform pair itself is exact to machine precision (last column) — the transforms are not the problem, the nonlinearity’s out-of-band energy is. Second, the deviation grows sharply with L , and SigmaDock runs at $L = 3$ with `use_sep_s2_act=True`, i.e. in the bottom row.

Caveat. These are measured on *random Gaussian* coefficients, the worst case: equal energy in every degree maximises out-of-band leakage. A trained network’s features are dominated

by low degrees, so the effective deviation in a real forward pass is smaller, possibly much smaller. We have not measured that, and it would require a trained checkpoint. What the table does establish is that this layer is **not exactly equivariant by construction**, and that its deviation grows with L .

Remark 9.10 (A second, separate truncation: $m_{\max} < \ell_{\max}$). SigmaDock sets `l_max_list=(3,)` but `m_max_list=(2,)`. These are different knobs: $L = 3$ says features carry degrees up to 3; $m_{\max} = 2$ says only components with $|m| \leq 2$ are retained, so for $\ell = 3$ the $m = \pm 3$ components are discarded outright.

This is a deliberate efficiency measure in the eSCN scheme — the $SO(2)$ blocks are indexed by m , so capping m caps the number of blocks.

Where exactly it acts is worth pinning down, because the code makes it easy to misread. `S03_Rotation.rotate` selects only the *rows* of the Wigner matrix with $|m| \leq m_{\max}$, so the truncated components are never computed in the edge frame; `rotate_inv` selects the corresponding *columns* and maps back into the full $(L+1)^2$ layout, with the source comment “Assume mmax = lmax when rotating back”. So the global-frame output is full-length; it is the *rank* that is reduced.

Crucially, this does **not** break equivariance (Proposition 9.11) — it restricts capacity. Worth knowing when reasoning about what angular patterns the model can represent; the answer is not simply “everything up to degree 3”, but the failure mode is a blind spot, not a frame error.

9.6.2 What “separable” means here

`SeparableS2Activation` is not a different mathematical idea; it is a split by degree:

- the $\ell = 0$ channels get an *ordinary* SiLU applied directly — legal without any grid, because scalars are invariant (Section 4.10.8), and exact;
- the $\ell \geq 1$ channels go through the grid round trip;
- the outputs are concatenated, the separately-computed scalar part *replacing* the grid-computed $\ell = 0$ output.

The saving is modest — one degree out of $(L+1)^2$ components — but the scalar path is both cheaper and exactly equivariant, so there is no reason to route it through the grid. The name refers to separating the invariant channels from the rest, not to a factorisation of the sphere.

Exact versus approximate equivariance: the ledger

The answer differs by operation, so it is worth keeping a running list.

Operation	Equivariance	Basis
Edge vector, distance, $\hat{x}$	exact	algebra
Spherical harmonics Y^ℓ	exact	Equation (4.1)
Clebsch–Gordan product	exact	Proposition 4.41
Equivariant linear / channel mixing	exact	Proposition 4.46
Edge rotation and its inverse	exact	D^ℓ is a representation
Attention weights from $m = 0$	exact	Proposition 9.15
Sum aggregation, residual	exact	linearity
RMS normalisation	exact (measured 5×10^{-8})	Section 9.8
$m_{\max} < L$ truncation	exact (measured 2×10^{-6})	Proposition 9.11
S^2 activation (grid round trip)	approximate	quadrature aliasing

Honest summary: every operation is exactly equivariant *except* the nonlinearity. “Equivariant by construction” is true of the architecture’s design and true of its arithmetic everywhere but the S^2 activation, whose deviation grows with L .

Proposition 9.11 (The m -truncation preserves equivariance but loses information). *Let $\Pi_{m_{\max}}$ discard all components with $|m| > m_{\max}$, applied in the edge-aligned frame, and let $T_{ij} = D(Q_{ij})^{-1} \Pi_{m_{\max}} D(Q_{ij})$ be the composite “rotate in, truncate, rotate back”. Then T_{ij} is exactly $SO(3)$ -equivariant: under a global rotation Q , both the features and the edge frame transform consistently, and T commutes with the action. However T_{ij} is a projection of rank $\sum_{\ell \leq L} \min(2\ell + 1, 2m_{\max} + 1)$, strictly less than $(L + 1)^2$ when $m_{\max} < L$.*

Argument. Under $x \mapsto Qx$ the edge direction becomes $Q\hat{x}_{ij}$, so the frame rotation satisfies $Q'_{ij} = Q_{ij}Q^{-1}$ up to the residual $SO(2)$ about the alignment axis. Hence $D(Q'_{ij})D(Q) = D(Q_{ij})$: the edge-frame representation of the feature is unchanged by a global rotation. Truncation in that frame is therefore applied to the same object either way, and rotating back reintroduces exactly the transformation $D(Q)$. The residual $SO(2)$ gauge freedom does not spoil this because rotations about the alignment axis act *diagonally in m* (Proposition 9.15), so $\Pi_{m_{\max}}$ commutes with them. $\square$

Remark 9.12 (What the truncation costs, measured). For SigmaDock’s $L = 3$, $m_{\max} = 2$: the composite T_{ij} has rank **14** out of 16. The two discarded dimensions are exactly the $m = \pm 3$ components of $\ell = 3$ in the edge frame. In the global frame nothing is zeroed — the output has all 16 components populated — but it is constrained to a 14-dimensional, edge-dependent subspace.

Equivariance error, measured over 300 random rotations: 2×10^{-6} with truncation versus 4×10^{-7} without, both at float32 round-off. So this is an **expressiveness restriction**, **not a symmetry violation** — a distinction worth making precisely, because the two have completely different consequences: the first costs model capacity, the second would cost correctness.

9.7 The block, in the order the code executes it

The theory so far names the ingredients. This section fixes their order, read from `net/transformer_block.py::T` rather than assumed from the Transformer literature.

The order is strict pre-norm, twice.

$$\begin{aligned} h &\leftarrow h + \text{Drop}\left(\text{Attn}(\text{Norm}_1(h))\right) \\ h &\leftarrow h + \text{Drop}\left(\text{FFN}(\text{Norm}_2(h))\right) \end{aligned} \tag{9.1}$$

i.e. normalisation *inside* each residual branch, never on the skip path. Two normalisation layers per block, two residual additions, and the dropout (`GraphDropPath` plus `EquivariantDropoutArraySpherical`) applied to the branch output before the addition.

Step	Class / call	Acts on	Equivariance
1	save h as <code>x_res</code>	—	—
2	<code>self.norm_1</code>	$[N, (L+1)^2, C]$	exact
3	<code>self.ga</code> (graph attention)	node + edge	exact except S^2 act
4	<code>drop_path, proj_drop</code>	branch only	exact (scaling)
5	$h += \text{x_res}$	—	exact
6	save h as <code>x_res</code>	—	—
7	<code>self.norm_2</code>	$[N, (L+1)^2, C]$	exact
8	<code>self.ffn</code>	node only	exact except S^2 act
9	<code>drop_path, proj_drop</code>	branch only	exact
10	<code>ffn_shortcut</code> on <code>x_res</code> <i>if channels change</i>	skip path	exact
11	$h += \text{x_res}$	—	exact

Step 10 is the one detail that departs from a textbook Transformer: when the feed-forward sub-block changes the channel count, the residual path cannot simply be added, so an equivariant linear layer (`S03_LinearV2`) is applied to the *skip* tensor first. It is still a channel-mixing map of the form $A^{(\ell)} \otimes I_{2\ell+1}$, so Proposition 8.22 still applies.

Why pre-norm. With post-norm, the residual stream passes through a normalisation at every block, and gradients through a deep stack are attenuated. Pre-norm leaves the skip path untouched, so there is a clean identity path from input to output — the standard reason modern Transformers use it. Nothing about the choice is equivariance-related; both orders would be equivariant.

Edge orientation, fixed from the code. In `S02EquivariantGraphAttention.forward`:

$$\text{edge_index}[0] = \text{source } j, \quad \text{edge_index}[1] = \text{target } i,$$

read off from `x_source._expand_edge(edge_index[0])` and `x_target._expand_edge(edge_index[1])`. Consistently, aggregation is `x_message._reduce_edge(edge_index[1], ...)`, and the attention normalisation is

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(s_{ik})},$$

i.e. `softmax(alpha, edge_index[1])` — per *target* node, over its incoming edges. This matches the convention $x_{ij} = x_j - x_i$ fixed in Caution 8.7: the message flows $j \rightarrow i$ and the softmax normalises what arrives at i .

9.8 Normalisation in EquiformerV2

Remark 8.23 flagged that ordinary LayerNorm is unavailable here. The reason, stated precisely: LayerNorm computes a mean and variance across a feature axis and subtracts the mean. For $\ell \geq 1$, the quantity $\frac{1}{2\ell+1} \sum_m x_m$ is *not* invariant — it is a particular linear functional of the components, and a rotation mixes them, so subtracting it introduces a frame-dependent shift. That is the same error as componentwise ReLU, in a different guise.

What the shipped default (`norm_type="rms_norm_sh"`, class `EquivariantRMSNORMArraySphericalHarmonicsV2`) does instead is exactly what the theory of Section 4.10 permits, and it is worth walking through because every choice in it is forced:

1. **Centering only on $\ell = 0$, and only across channels.** The mean is taken over the *channel* axis of the $\ell = 0$ slice alone (`feature.narrow(1,0,1).mean(dim=2)`). Scalars are invariant, so subtracting any function of them is safe; the $\ell \geq 1$ block is never centred. This is Section 4.10.6's bias rule applied to a running statistic.
2. **The scale is built from an invariant.** The normaliser is $\sum_m x_m^2$ summed over the m axis (then averaged over channels), which is invariant because $D^\ell(Q)$ is orthogonal (Proposition 4.49). Dividing an equivariant feature by an invariant scalar preserves equivariance.
3. **Affine scale is one weight per $(\ell, \text{channel})$, broadcast across m .** The learned tensor has shape $[1, L+1, C]$ and is expanded along the m axis by an index map. This is precisely the $A^{(\ell)} \otimes I_{2\ell+1}$ form of (4.11), restricted to a diagonal A .
4. **Affine bias only on $\ell = 0$.** Added exclusively to the $\ell = 0$ slice, as Section 4.10.6 requires.

Proposition 9.13 (The RMS normalisation is equivariant). *With $n(x) = (\overline{\sum_m x_m^2} + \epsilon)^{-1/2}$ invariant and $w_c^{(\ell)}$ independent of m , the map $x_{c,m}^{(\ell)} \mapsto w_c^{(\ell)} n(x) x_{c,m}^{(\ell)}$ (plus a bias on $\ell = 0$) satisfies $x \mapsto D^\ell(Q)x \Rightarrow \text{out} \mapsto D^\ell(Q)\text{out}$.*

Proof. $n(D^\ell(Q)x) = n(x)$ since $D^\ell(Q)$ is orthogonal and n depends on x only through $\sum_m x_m^2$. The remaining factors $w_c^{(\ell)}$ and $n(x)$ are scalars, which commute with $D^\ell(Q)$. The $\ell = 0$ bias is added to a block on which $D^0(Q) = 1$. $\square$

Verified numerically: at $L = 3$ with the shipped class, the relative deviation between $\text{norm}(Dx)$ and $D \text{norm}(x)$ over 300 random rotations is 5×10^{-8} — i.e. float32 round-off in the Wigner matrices, not a structural error. The contrast with the S^2 activation's 3.9×10^{-1} at the same L is instructive: one of these layers is exactly equivariant and the other is not, and the difference is entirely whether a discretisation was involved.

9.9 Equivariant linear layers and normalisation

Code correspondence

Mathematical object: the general equivariant linear map of Proposition 4.48

Implementation: `net/so3.py::S03_LinearV2`

Tensor shape: weight tensor `[lmax+1, out_features, in_features]`

Interpretation: implements exactly $h^{\ell,c} \mapsto \sum_{c'} W_{cc'}^\ell h^{\ell,c'}$ via `einsum("bmi,moi->bmo",`

...) with the weight broadcast identically across the $2\ell + 1$ components m of a given degree ℓ (one weight matrix *per degree*, shared across all m within it, exactly matching Proposition 4.46’s statement that an intertwiner is a scalar multiple of the identity *on* V_ℓ , generalised here to multiple channels per degree via Proposition 4.48); and a bias term is added **only to the $\ell = 0$ slice** of the output (`out[:,0:1,:] += bias`) — the code’s own structural enforcement of the fact, provable directly from Proposition 4.46 (an additive constant is a map from the trivial representation, $\ell = 0$, into the target; by Schur’s lemma such a map can only land in the $\ell = 0$ component of the target without violating equivariance, since a nonzero constant vector in a degree- $\ell \geq 1$ channel would not itself be fixed by every rotation, contradicting $D^\ell(Q) \cdot \text{bias} = \text{bias}$ required for a translation-equivariant *additive* bias), that higher-degree channels cannot carry an additive bias at all.

Code correspondence

Mathematical object: equivariant normalisation

Implementation: `net/layer_norm.py` (`norm_type` default `"rms_norm_sh"`)

Interpretation: normalises each degree- ℓ , channel- c block $h^{\ell,c}$ by (an RMS-style estimate of) its own *norm* $\|h^{\ell,c}\|_2$ — an invariant quantity by Proposition 3.46’s orthogonality of $D^\ell(Q)$, since $\|D^\ell(Q)v\| = \|v\|$ for orthogonal $D^\ell(Q)$ (Proposition 4.25(iii)) — rather than by any coordinate-wise statistic, which is what makes the normalisation equivariant (dividing every component of $h^{\ell,c}$ by the same invariant scalar $\|h^{\ell,c}\|$ commutes with $D^\ell(Q)$ by the same scalar-multiplication argument as Definition 9.7’s gate). The exact formula implemented (beyond “RMS of the norm, per spherical-harmonic degree”) was not read in the code-research pass underlying this document and is flagged, per this document’s stated policy, as not independently confirmed beyond its name and role.

9.10 Equivariant attention weights

9.10.1 Ordinary attention, recalled

Start from the standard Transformer, with no geometry. Given token features $h_1, \dots, h_n \in \mathbb{R}^d$, three learned matrices produce

$$q_i = W_Q h_i \text{ (query),} \quad k_j = W_K h_j \text{ (key),} \quad v_j = W_V h_j \text{ (value),}$$

and the update is

$$a_{ij} = \text{softmax}_j \left(\frac{q_i^\top k_j}{\sqrt{d}} \right), \quad h'_i = \sum_j a_{ij} v_j. \quad (9.2)$$

The roles, in words: the *query* is what token i is looking for; the *key* is what token j advertises; their inner product is a *logit* measuring how well they match; the *softmax* turns the logits for a fixed i into non-negative weights summing to one; and the *value* is the content actually transported. The whole construction is a learned, content-dependent weighted average.

Now the question that organises this entire section:

Which parts of (9.2) survive when h is a steerable feature rather than a plain vector?

The aggregation $\sum_j a_{ij} v_j$ survives verbatim, provided the v_j all carry the same degree (Proposition 8.20). What cannot survive unexamined is the logit: $q_i^\top k_j$ is an inner product of two

feature vectors, and if those vectors carry $\ell \geq 1$ components then the naive inner product across *all* components mixes m -slots in a way that must be checked, not assumed. And the weight a_{ij} then multiplies an equivariant object, which constrains what kind of number it may be.

The constraint attention weights must satisfy. We now assemble messages and weights into the full attention block, and the first thing to determine is what kind of object the attention weight a_{ij} (Definition 8.4’s analogue) is allowed to be.

Proposition 9.14 (Attention weights must be invariant scalars). *Let $h'_i = h_i + \sum_{j \in \mathcal{N}(i)} a_{ij} m_{ij}$, with each message m_{ij} steerable of a fixed degree ℓ . If the a_{ij} are $SE(3)$ -invariant scalars, this update is equivariant. If a_{ij} were instead permitted to be an arbitrary equivariant (not merely invariant) quantity of matching degree, the sum $\sum_j a_{ij} m_{ij}$ would generally fail to commute with $D^\ell(Q)$, since a_{ij} itself would then also transform under Q , and the product of two degree- ℓ -transforming quantities is not, in general, again degree- ℓ (Theorem 4.34).*

Proof. If a_{ij} is invariant and $m_{ij} \mapsto D^\ell(Q)m_{ij}$: $\sum_j a_{ij} m_{ij} \mapsto \sum_j a_{ij} D^\ell(Q)m_{ij} = D^\ell(Q) \sum_j a_{ij} m_{ij}$, using linearity of $D^\ell(Q)$ and that scalar a_{ij} commutes past it. Adding $h_i \mapsto D^\ell(Q)h_i$ (assuming h_i is also degree- ℓ ; otherwise, apply the same argument degree-by-degree in a multi-degree sum) preserves equivariance of h'_i . If a_{ij} instead transformed under Q (e.g. if it were itself a component of a degree- $\ell' > 0$ feature), the product $a_{ij} m_{ij}$ would transform by a Clebsch–Gordan combination of degrees ℓ' and ℓ , generally landing (partially) outside degree ℓ , breaking the claimed degree- ℓ equivariance of the sum. $\square$

This forces, structurally, exactly the design choice Chapter 8 already used implicitly for ordinary attention (Definition 8.4’s softmax output is a scalar) and is why an equivariant network cannot simply reuse a dot-product attention score computed *directly* between two full multi-degree steerable features (which would not, in general, be invariant, unless the specific $\ell \rightarrow 0$ Clebsch–Gordan path of Example 4.44 is used deliberately) without first projecting onto an invariant quantity.

9.10.2 Where the invariant comes from: the $m = 0$ channels

Proposition 9.14 says the logit must be invariant. It does not say where such an invariant is supposed to come from. The answer EquiformerV2 uses is elegant, and it is the reason the edge-alignment trick of Proposition 9.3 pays for itself twice.

Recall the setup: the edge has been rotated onto a fixed axis, so the residual symmetry is only $SO(2)$ — rotations about that axis. Now ask: *which components of a steerable feature are unchanged by that residual rotation?*

Proposition 9.15 (The $m = 0$ components are the $SO(2)$ -invariants). *Let $\hat{n}$ be the alignment axis and let $R_{\hat{n}}(\vartheta)$ denote rotation by ϑ about it. In the real spherical-harmonic basis adapted to $\hat{n}$, the matrix $D^\ell(R_{\hat{n}}(\vartheta))$ is block-diagonal with*

- a 1×1 identity block on the $m = 0$ component, and
- 2×2 rotation blocks $\begin{pmatrix} \cos m\vartheta & -\sin m\vartheta \\ \sin m\vartheta & \cos m\vartheta \end{pmatrix}$ on each pair $(m, -m)$ with $m \geq 1$.

Consequently the $m = 0$ component of every degree ℓ is invariant, and these are (for ϑ generic) the only invariants.

This is exactly the statement that m labels the angular momentum *about the chosen axis*: $m = 0$ means “no angular dependence about this axis”, hence unchanged when you spin around it.

Remark 9.16 (Verified against the shipped Wigner matrices). This is checkable directly with the code’s own `S03_Rotation`. Building D^ℓ for a rotation about the alignment axis and asking which basis vectors it fixes returns, for $L = 2$, exactly the flat indices $\{0, 2, 6\}$ — which in the (ℓ, m) ordering $\ell = 0, 1, 2$, $m = -\ell.. \ell$ are $(0, 0)$, $(1, 0)$, $(2, 0)$: one per degree, precisely the $m = 0$ slots.

The same test for a rotation about a *different* axis returns a different, non- $m=0$ set. So the choice of alignment axis is not cosmetic: it is chosen so that the residual $SO(2)$ acts diagonally in the stored basis, which is what makes both the $SO(2)$ convolution and the attention logit cheap.

The two payoffs of edge alignment

Rotating the edge onto a fixed axis buys two different things at once, and it is worth separating them:

1. Cheaper coupling. The full Clebsch–Gordan product couples every (m_1, m_2) pair. After alignment, the edge harmonic has only its $m = 0$ component non-zero, and the $m = m_1 + m_2$ selection rule collapses the double sum to a single index. This is the $SO(3) \rightarrow SO(2)$ reduction of Section 9.5.

2. A free supply of invariants. The $m = 0$ channels are automatically invariant under the residual symmetry (Proposition 9.15), so they can be fed to *any* ordinary neural operation — a `Linear`, a `LayerNorm`, a nonlinearity — without further thought. EquiformerV2 exploits this by asking the $SO(2)$ convolution for a few *extra* $m = 0$ output channels beyond what the message needs, and using them as the attention logits.

So the attention mechanism is not bolted on: it reuses invariants that the efficiency trick had already produced.

Caution 9.17 (Equiformer’s logit is not a $QK^\top$ dot product). Having recalled (9.2), it would be natural to expect the implementation to form queries and keys and take their inner product. It does not. Reading `net/transformer_block.py`, the logit is

$$\alpha_{ij,\text{head}} = \langle a_{\text{head}}, \text{SmoothLeakyReLU}(\text{LN}(z_{ij,\text{head}})) \rangle,$$

where z_{ij} are the extra $m = 0$ channels for that edge and a_{head} is a learned vector, one per head. That is the **GATv2** form (a learned projection of a jointly-computed edge representation), not scaled dot-product attention: there is no separate W_Q and W_K , and no $1/\sqrt{d}$.

The distinction matters when reading the source and when comparing to the Transformer literature; it does not affect the equivariance argument, which only requires that the logit be invariant — and it is, because z_{ij} consists of $m = 0$ channels.

Code correspondence

Mathematical object: the attention weight a_{ij} of Proposition 9.14

Implementation: `net/transformer_block.py::S02EquivariantGraphAttention.forward`

Interpretation: a_{ij} is computed **only** from the extra $\ell = 0, m = 0$ scalar output channels of the first $SO(2)$ -convolution (Section 9.5) applied to the edge message — i.e. the network allocates a few dedicated invariant “gating” channels alongside its main

steerable output, specifically for this purpose — via a $\text{LN} \rightarrow \text{SmoothLeakyReLU} \rightarrow$ learned per-head dot product, then a standard `torch_geometric.utils.softmax` keyed on the *target* node index (`edge_index[1]`), i.e. a per-target-node softmax over its incoming edges, exactly Definition 8.4’s row-wise softmax specialised to a sparse graph. This is, precisely, the architectural enforcement of Proposition 9.14: there is structurally no other path in the code by which a non-scalar, degree- ≥ 1 quantity could enter the attention weight.

9.10.3 Assembling the full attention block

Proposition 9.18 (The full block is equivariant). *Let messages m_{ij} be constructed (via Sections 9.3 and 9.5) to be equivariant, and a_{ij} invariant scalars (Proposition 9.14). Then $h'_i = h_i + \sum_{j \in \mathcal{N}(i)} a_{ij} m_{ij}$ is equivariant, degree-by-degree.*

This is exactly Proposition 9.14’s proof, restated as the conclusion for the assembled block. In code, the full per-layer pipeline (`TransBlockV2`, `net/transformer_block.py`) is: (1) concatenate source and target node features; (2) rotate into the per-edge frame (Section 9.3); (3) first $SO(2)$ -convolution (Section 9.5), also producing the invariant gating channels used for attention weights; (4) equivariant nonlinearity (Section 9.6); (5) second $SO(2)$ -convolution; (6) compute a_{ij} from the gating channels (Section 9.10); (7) rotate messages back out of the edge frame; (8) scatter-sum $a_{ij}m_{ij}$ into target nodes (an `index_add_` along `edge_index[1]`, the permutation-equivariant aggregator of Definition 8.14); (9) final equivariant linear projection (`S03_LinearV2`, Section 9.9); embedded, as in Equation (8.2), in a pre-normalised residual structure alongside a per-node equivariant feed-forward block.

9.11 Node embedding: turning chemistry into a steerable feature

Before any message passing, every node needs an initial steerable feature (Section 9.1); by Section 8.5’s argument in reverse, only the $\ell = 0$ (invariant) channels can be populated directly from ordinary categorical/continuous chemistry, since nothing in the raw per-atom input (an atom type, a formal charge, a time value) carries any directional information to seed a higher-degree channel with.

Code correspondence

Mathematical object: the initial (type-0-only) node embedding

Implementation: `net/encoder.py::AtomDiffusionEncoder`, called once per node-entity type from `net/model.py::compute_node_embedding`

Interpretation: for each of the (default) **ten** categorical atom features (`atom_feature_dims = (54, 5, 3, 4, 4, 7, 2, 2, 4, 3)`, `config.py` — cardinalities only; the exact chemical identity of each of the ten slots, e.g. which one is atomic number versus hybridisation, was not independently confirmed against the preprocessing code in the pass underlying this document and is left as an open item, not asserted), a separate `nn.Embedding` table (with one extra row for an explicit “unknown/out-of-range” category) produces a vector; these ten vectors are combined not by concatenation-then-MLP but — in the configuration actually used, `linear_aggregate=False` at every call site — by **summing them and dividing by $\sqrt{10}$** , a variance-preserving sum (if the ten embeddings were independent with unit variance per coordinate, the sum’s variance would

be $10\times$ larger, and dividing by $\sqrt{10}$ restores unit variance — the same normalisation principle as the $1/\sqrt{d_k}$ scaling of Definition 8.4’s attention logits). Continuous auxiliary features (e.g. protein residue/ESM embeddings, where used) are instead passed through a small MLP and *added*. The time embedding t_{emb} — a **sinusoidal** embedding (default `t_emb_dim=32`, `t_emb_scale=10000.0`; the exact functional form was not independently re-derived from the code in this pass, but the name and parameters strongly suggest the standard Transformer construction $(\sin(t/10000^{2k/d}), \cos(t/10000^{2k/d}))_k$, flagged here as plausible but not code-confirmed) — is concatenated (not summed) onto the aggregated categorical embedding and passed through one further linear layer (**time_projector**). **Ligand-fragment virtual nodes** (one per fragment, per Section 10.7 below) do *not* get an independent embedding at all: their $\ell = 0$ feature is the *mean* (a **scatter**, `reduce="mean"`) of the already-computed embeddings of every real atom belonging to that fragment, propagated along dedicated virtual-to-atom (**ligand_v2a**) edges — i.e. a fragment’s virtual node is, by construction, exactly $SE(3)$ -invariant-feature-wise a summary of its members, before any message passing layer has even run.

9.12 Reading out the vector field

After `num_layers = 6` (default) blocks, each atom carries a full steerable feature $h_i = \bigoplus_{\ell=0}^3 \bigoplus_{c=1}^{256} h_i^{\ell,c}$. The network’s geometric output, per (8.1), is a single 3-vector per *atom* (not yet per fragment; the reduction to per-fragment force/torque via Newton–Euler mechanics is a separate step, reconstructed precisely in Chapter 10), read off from the $\ell = 1$ channels by one further equivariant linear layer.

Code correspondence

Mathematical object: the per-atom pseudo-force readout

Implementation: `net/model.py::EquiformerV2.forward`, final `force_block` (an `S02EquivariantGraphAttention` with a single output channel), then `forces = force_block_output.embedding.narrow(1, 1, 3)`

Tensor shape: `[N_atoms, 3]`

Interpretation: `narrow(1, 1, 3)` slices the flattened $(L+1)^2 = 16$ -length coefficient axis starting at index 1, length 3 — i.e. it selects exactly the three $\ell = 1$ ($m = -1, 0, 1$) coefficients, discarding the $\ell = 0$ coefficient (index 0) and every $\ell \geq 2$ coefficient (indices 4 and up) entirely. By Proposition 4.25(iv), this $\mathbb{R}^3$ -valued output transforms under a global rotation Q exactly as $D^1(Q) = Q$, i.e. as an ordinary polar 3-vector in the *world frame* in which the network’s input coordinates were expressed — this is precisely the “atom-wise pseudo-force in the world frame” object that Chapter 10’s Newton–Euler reduction (net force, torque, per fragment) consumes, and it is only at that later, non-network step that a fragment’s rotational output is converted into the *body-frame* (left-trivialised, Section 3.5) angular quantity $\omega_\theta^{(i)} \in \mathfrak{so}(3)$ that (8.1) ultimately requires — the network itself, note carefully, never outputs a body-frame quantity directly; the conversion is a geometric post-processing step external to `EquiformerV2` proper, and Chapter 10 traces it precisely.

9.13 One block, end to end, with numbers

Everything in this chapter so far has been described. This section *computes* it: one simplified attention block, three atoms, two edges, every intermediate number written out. The aim is that a reader who has followed the pieces — edge alignment, $SO(2)$ convolution, $m = 0$ invariants, attention, back-rotation, residual — can see them meet.

Caution 9.19 (What is exact here and what is simplified). This is a toy block, and the boundary between “exact EquiformerV2 operation” and “didactic simplification” must not be blurred.

Exact, not simplified:

- the edge-frame construction and the claim $Q_{ij}\hat{x}_{ij}$ = the alignment axis (Proposition 9.3);
- the identification of the $m = 0$ component with the component *along* that axis, and of the $m = \pm 1$ pair with the transverse part (Proposition 9.15 at $\ell = 1$);
- the constraint structure of the $SO(2)$ convolution: a free real matrix on the $m = 0$ sector, a commutant-constrained one on $|m| = 1$, and no mixing between sectors;
- the fact that for $\ell = 1$ **the Wigner- D matrix in a Cartesian basis is the rotation matrix**, $D^{(1)}(Q) = Q$. This is what lets the whole example be done by hand, and it is an identity, not an approximation;
- softmax over incoming edges, invariant weights, equivariant values, back-rotation, residual — all structurally as shipped.

Didactic simplifications, marked wherever they occur:

- $L = 1$ instead of the configured $L_{\max}$; one attention head; one scalar and two $\ell = 1$ channels instead of the production widths;
- two Gaussian radial basis functions instead of the Bessel basis with polynomial cutoff;
- the $|m| = 1$ weights are taken to be pure real scalars; a general $SO(2)$ -equivariant map may additionally contain a 90° -rotation part (see Remark 9.21 below);
- the attention logit is a single fixed read-out vector composed with tanh, not the shipped MLP;
- layer normalisation, the feed-forward sub-block and the S^2 activation are discussed but not evaluated numerically (Section 9.13.11).

Every number below was produced by `audits/equiformer_toy_block.py` and is reproducible; none was computed by hand and copied.

9.13.1 Step 1: the atoms, their features, and the basis

Three atoms, a target i and two neighbours j, k :

$$x_i = (0, 0, 0), \quad x_j = (1, 0, 0), \quad x_k = (0, 2, 0).$$

Each node carries one scalar ($\ell = 0$) channel and two vector ($\ell = 1$) channels:

$$s_i = 1, \quad s_j = 2, \quad s_k = -1,$$

$$h_i^{(1)} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad h_j^{(1)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad h_k^{(1)} = \begin{pmatrix} 0 & 2 & 0 \\ 0 & 1 & 0 \end{pmatrix},$$

one *row* per channel, each row a vector in $\mathbb{R}^3$.

Notation 9.20 (The basis, stated plainly). The $\ell = 1$ features above are written in an ordinary Cartesian basis (x, y, z) . This is legitimate and exact, for one reason: the real spherical harmonics of degree 1 are, up to a common normalisation, the three Cartesian coordinate functions, so the degree-1 block of a Wigner- D matrix in this basis is the rotation matrix itself. Nothing is being approximated.

Two conventions must nevertheless be named, because a reader comparing against `e3nn` or against the shipped tensors will otherwise be confused:

- `e3nn` stores the degree-1 block in the order $(m = -1, m = 0, m = +1)$, which in Cartesian terms is (y, z, x) — so the *middle* slot is $m = 0$. This is a permutation of the three numbers we write, not different information.
- Consequently, “align the edge onto the second basis vector” — what the shipped `init_edge_rot_mat` does — means “align it onto the slot that carries $m = 0$ ”. Below we build frames whose *third* Cartesian axis is the edge direction and call that slot $m = 0$; this is the same statement with a different index label.

9.13.2 Step 2: edge geometry

$$\begin{aligned} x_{ij} &= x_j - x_i = (1, 0, 0), & r_{ij} &= 1, & \hat{x}_{ij} &= (1, 0, 0), \\ x_{ik} &= x_k - x_i = (0, 2, 0), & r_{ik} &= 2, & \hat{x}_{ik} &= (0, 1, 0). \end{aligned}$$

9.13.3 Step 3: radial features are invariant scalars

Take (didactic simplification) two Gaussian radial basis functions with centres $\mu = (0.5, 1.5)$:

$$\phi(r) = (e^{-(r-0.5)^2}, e^{-(r-1.5)^2}), \quad \phi(1) = (0.7788, 0.7788), \quad \phi(2) = (0.1054, 0.7788).$$

These depend on r only. A global rotation changes neither r_{ij} nor r_{ik} , so ϕ is *invariant* — it is exactly the “radial half” of Section 8.4’s edge decomposition, and it may therefore be fed to any ordinary, unconstrained neural network. All the directional information sits in $\hat{x}_{ij}$, which is handled separately.

9.13.4 Step 4: the edge frame

For each edge build an orthonormal frame $(\hat{u}, \hat{v}, \hat{n})$ with $\hat{n} = \hat{x}$, and let Q be the matrix whose *rows* are those three vectors, so that $Q\hat{x} = (0, 0, 1)$: the edge points along the third axis, the $m = 0$ slot.

For edge $j \rightarrow i$ ($\hat{x}_{ij} = (1, 0, 0)$) the script returns

$$Q_{ij} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & -1 & 0 \\ 1 & 0 & 0 \end{pmatrix}, \quad Q_{ij}\hat{x}_{ij} = (0, 0, 1) \checkmark, \quad \det Q_{ij} = +1.$$

For edge $k \rightarrow i$ ($\hat{x}_{ik} = (0, 1, 0)$),

$$Q_{ik} = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \quad Q_{ik}\hat{x}_{ik} = (0, 0, 1) \checkmark.$$

Both happen to be signed permutation matrices, which keeps the arithmetic below transparent.

Now rotate the neighbours' $\ell = 1$ features into their edge frames, using $D^{(1)}(Q) = Q$:

$$h_j^{(1)} \mapsto h_j^{(1)} Q_{ij}^\top = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}, \quad h_k^{(1)} \mapsto h_k^{(1)} Q_{ik}^\top = \begin{pmatrix} 0 & 0 & 2 \\ 0 & 0 & 1 \end{pmatrix}.$$

Read these off geometrically, since this is the whole point of the frame:

- j 's channel 1 was $(1, 0, 0)$, which points *exactly along* the edge $j \rightarrow i$; in the edge frame it is $(0, 0, 1)$ — all of it in the $m = 0$ slot, nothing transverse.
- j 's channel 2 was $(0, 0, 1)$, *perpendicular* to that edge; in the edge frame it is $(1, 0, 0)$ — purely transverse, $m = 0$ component zero.
- k 's two channels both lie along $\hat{x}_{ik} = (0, 1, 0)$, so both are purely $m = 0$, with magnitudes 2 and 1.

9.13.5 Step 5: reading the m sectors

In the edge frame, for $\ell = 1$:

Slot	Geometric meaning	Behaviour under a spin by ϑ about the edge
$m = 0$ (third)	component <i>along</i> the edge	unchanged
$m = \pm 1$ (first two)	the transverse plane	rotated by ϑ

This is Proposition 9.15 at $\ell = 1$, and at $\ell = 1$ it is simply the statement that spinning about an axis fixes that axis and rotates the plane perpendicular to it. For higher ℓ the $(m, -m)$ pair rotates by $m\vartheta$ instead, which is where the general proposition earns its keep.

9.13.6 Step 6: the $SO(2)$ convolution

Now the linear map. Because the residual symmetry group is only $SO(2)$, the constraint on the weights is far weaker than full $SO(3)$ -equivariance would impose — and it is different in the two sectors:

- **$m = 0$ sector.** These components are invariant, so *any* real matrix is admissible. Here it mixes the two channels: $W_0 = \begin{pmatrix} 2 & 0 \\ 0 & 0.5 \end{pmatrix}$.
- **$|m| = 1$ sector.** A map must commute with the 2×2 rotation acting on the pair. Here we take $W_1 = \begin{pmatrix} 1 & 0 \\ 0 & 0.5 \end{pmatrix}$, applied *with the same weights to both partners of the pair* — that identical treatment is exactly the commutation requirement.
- **The two sectors never mix.**

Applying this:

$$z_{ij} = \begin{pmatrix} 0 & 0 & 2 \\ 0.5 & 0 & 0 \end{pmatrix}, \quad z_{ik} = \begin{pmatrix} 0 & 0 & 4 \\ 0 & 0 & 0.5 \end{pmatrix}.$$

Check one entry by hand: for edge ij the $m = 0$ column is $(1, 0)^\top$ (channel 1 has 1, channel 2 has 0), and $W_0(1, 0)^\top = (2, 0)^\top$ — the first column of z_{ij} 's third slot. For the transverse part, edge ij 's first transverse column is $(0, 1)^\top$ and $W_1(0, 1)^\top = (0, 0.5)^\top$.

Remark 9.21 (Why this is cheaper, and what the general $|m| = 1$ block looks like). A full $SO(3)$ tensor product would couple every (m_1, m_2) pair through Clebsch–Gordan coefficients (Section 9.4 counts the cost). After alignment, the edge harmonic has only its $m = 0$ component non-zero, the selection rule $m = m_1 + m_2$ collapses the double sum to a single index, and the work splits into independent per- m blocks — here, two of them.

One honest caveat on the simplification: the commutant of the 2×2 rotation in the real 2×2 matrices is not the real scalars alone but $\{aI + bJ\}$ with $J = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$, i.e. the complex numbers. The shipped convolution uses that full complex freedom. Taking $b = 0$ here keeps the arithmetic readable and changes nothing structural.

9.13.7 Step 7: extracting the invariant, and the attention logit

The attention weight must be an invariant scalar (Proposition 9.14). Section 9.10.2 says where to get one: read the $m = 0$ slots.

$$\text{edge } ij : \quad m=0 \text{ components } (2, 0), \quad \text{edge } ik : \quad (4, 0.5).$$

Note that these already *differ*, and differ for a geometric reason: j 's first channel points along its edge and k 's points along its own edge with twice the magnitude. Concatenating with the (invariant) radial features scaled by the neighbour's scalar,

$$\phi(1) s_j = (1.5576, 1.5576), \quad \phi(2) s_k = (-0.1054, -0.7788),$$

and applying a fixed read-out vector $a = (0.9, -0.4, 0.6, 0.3)$ after a $\tanh$ (didactic simplification; the shipped model uses an MLP):

$$s_{ij} = a^\top \tanh(2, 0, 1.5576, 1.5576) = \mathbf{1.6912}, \quad s_{ik} = a^\top \tanh(4, 0.5, -0.1054, -0.7788) = \mathbf{0.4559}.$$

9.13.8 Step 8: softmax over the incoming edges

The softmax runs over the edges *entering the target node i* — not over channels, not over the whole graph:

$$\alpha_{ij} = \frac{e^{1.6912}}{e^{1.6912} + e^{0.4559}} = \mathbf{0.7747}, \quad \alpha_{ik} = \frac{e^{0.4559}}{e^{1.6912} + e^{0.4559}} = \mathbf{0.2253}, \quad \alpha_{ij} + \alpha_{ik} = 1.$$

Neighbour j dominates, and one can say why: it is closer (so its radial features are larger) and it carries a positive scalar, whereas k 's scalar is negative.

9.13.9 Step 9: values, back-rotation, aggregation

The values z_{ij}, z_{ik} were computed *in their own edge frames* and cannot be added there — the two frames are different. Rotate each back with $D^{(1)}(Q)^{-1} = Q^\top$:

$$z_{ij} Q_{ij} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 0 & 0.5 \end{pmatrix}, \quad z_{ik} Q_{ik} = \begin{pmatrix} 0 & 4 & 0 \\ 0 & 0.5 & 0 \end{pmatrix}.$$

Geometrically: z_{ij} 's channel 1 was 2 units along the $m = 0$ axis, and that axis *is* $\hat{x}_{ij} = (1, 0, 0)$, so it comes back as $(2, 0, 0)$. Likewise k 's messages come back along $(0, 1, 0)$.

Now aggregate with the attention weights:

$$m_i^{(1)} = \alpha_{ij} z_{ij} Q_{ij} + \alpha_{ik} z_{ik} Q_{ik} = \begin{pmatrix} 1.5495 & 0.9011 & 0 \\ 0 & 0.1126 & 0.3874 \end{pmatrix}.$$

The attention weights are invariant scalars; the values are equivariant vectors. A scalar times an equivariant vector is equivariant, and a sum of equivariant vectors is equivariant — so the aggregate is too. That is the entire reason attention is allowed here at all.

9.13.10 Step 10: residual, and the pre-norm order

Closing the block in the order the code executes it (Section 9.7, pre-norm):

$$h \mapsto h + \text{Attn}(\text{Norm}(h)), \quad \text{then} \quad h \mapsto h + \text{FFN}(\text{Norm}(h)).$$

Taking the normalisation as the identity for this example, the first half gives

$$h_i^{(1)} + m_i^{(1)} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} + \begin{pmatrix} 1.5495 & 0.9011 & 0 \\ 0 & 0.1126 & 0.3874 \end{pmatrix} = \begin{pmatrix} \mathbf{1.5495} & \mathbf{0.9011} & \mathbf{1} \\ \mathbf{0} & \mathbf{1.1126} & \mathbf{0.3874} \end{pmatrix}.$$

Node i 's channel 1, which started pointing purely along z , now points mostly along x — it has been pulled towards its dominant neighbour.

9.13.11 Step 11: why not just apply a ReLU

The block still needs a nonlinearity, and the obvious one is illegal. Take node i 's updated channel 1, $(1.5495, 0.9011, 1)$, and apply ReLU componentwise: nothing changes, since all three entries are positive. Now rotate the whole system by 180° about z first: the vector becomes $(-1.5495, -0.9011, 1)$, and componentwise ReLU returns $(0, 0, 1)$ — which, rotated back, is $(0, 0, 1)$, not the original $(1.5495, 0.9011, 1)$. Transform-then-activate and activate-then-transform disagree, so componentwise ReLU on $\ell \geq 1$ coefficients is not equivariant.

The three admissible repairs were established in Section 4.10.11: unrestricted nonlinearities on scalars, norm nonlinearities, and invariant gating; EquiformerV2 additionally uses the separable S^2 activation of Section 9.6, which evaluates a pointwise nonlinearity on a spherical grid rather than on coefficients. We do not evaluate it numerically here — it needs a quadrature grid, and the equivariance check below is deliberately run on the exact part of the block only.

9.13.12 Step 12: the sanity check

A worked example that only illustrates is worth less than one that can fail. Two structural claims were tested numerically on this exact block (`audits/equiformer_toy_block.py`):

Claim	Method	Residual
$F(Qx, Qh) = QF(x, h)$: the block is $SO(3)$ -equivariant	200 random rotations	$1.6 \cdot 10^{-15}$
The output does not depend on the arbitrary transverse axis used to build each edge frame	200 random reference vectors	$1.7 \cdot 10^{-16}$

The second is the more interesting one, and it is the check that shows the $SO(2)$ machinery is doing real work. The frame $(\hat{u}, \hat{v}, \hat{n})$ is not canonical: only $\hat{n}$ is fixed by the geometry, and $\hat{u}$ is chosen arbitrarily (the shipped code picks it from a list of candidate reference vectors).

If any step of the block were sensitive to that choice, the model’s output would depend on an implementation detail. It is not, because the $|m| = 1$ weights treat both transverse partners identically and the attention logit reads only $m = 0$.

Remark 9.22 (The check can fail, and does). To confirm the test is not vacuous, the script also runs it with the $SO(2)$ constraint deliberately broken — the two transverse partners given *different* weights, which is exactly the illegal operation. The spread across reference vectors then jumps from 10^{-16} to **0.85**. So the passing result above is evidence, not a tautology.

9.13.13 Step 13: the toy against the real thing

This example	Real EquiformerV2, as shipped
1 attention head $L = 1$	H heads, concatenated configured $L_{\max}$ ($= 3$ by default), 16 coefficients per channel
1 scalar +2 vector channels 2 edges, written out	production channel widths (Section 9.15) all graph edges within the cutoff radius, batched
2 Gaussian radial functions explicit 2×2 matrices tanh read-out vector identity normalisation no S^2 activation one block	Bessel basis with polynomial cutoff batched tensor contractions on GPU MLP on invariant features equivariant layer norm (Section 9.8) separable S^2 activation on a quadrature grid 6 blocks, stacked with residuals

Every row on the left is a size, not a structure. The sequence — align, convolve under $SO(2)$, read $m = 0$, attend, rotate back, aggregate, residual — is the same on both sides.

9.13.14 Closing the arc: from this block to an atomwise vector field

Stack blocks like the one above and each atom accumulates a steerable feature that has seen progressively more of its neighbourhood: after one block, its immediate neighbours; after n , everything within n edges. The $\ell = 0$ part of that feature is an invariant description of the local chemistry; the $\ell = 1$ part is a genuinely directional one, and it is directional in the only way an equivariant network permits — built from edge directions through operations that carry their transformation law with them.

The read-out of Section 9.12 then takes the $\ell = 1$ block of the final feature and projects it to a single vector per atom,

$$f_i \in \mathbb{R}^3,$$

which is the network’s entire geometric output. Chapter 10 consumes it as a score and Chapter 13 as a velocity, but the network does not know which: it emits an equivariant vector per atom, and what that vector *means* is decided outside it. This example ends where that discussion begins.

9.14 EquiformerV1 vs. EquiformerV2: what actually changed, and what this codebase actually ships

We can now state the V1-versus-V2 comparison precisely, having derived every ingredient involved, rather than merely asserting a version number.

Original Equiformer (V1). Messages are formed by an explicit Clebsch–Gordan tensor product (4.7) between a neighbour’s steerable feature and the edge’s spherical harmonic embedding (Definition 9.2), with per-path learned weights; no per-edge frame alignment is used. Attention weights are computed from invariant edge/node scalars via an ordinary MLP $\rightarrow$ dot product $\rightarrow$ softmax pipeline, structurally identical to Definition 8.4’s original Transformer recipe, merely applied to steerable rather than plain features.

EquiformerV2 (eSCN-based). Messages are formed via the per-edge frame alignment of Section 9.3 and the $SO(2)$ -convolution of Section 9.5, *replacing* the explicit Clebsch–Gordan tensor product with an algorithmically cheaper, exactly equivalent (by Proposition 9.3’s justification) construction. V2’s own paper additionally proposes an alternative attention-weight mechanism, `use_s2_act_attn`, computing attention directly from grid-sampled (S^2 -activation-based) features rather than V1’s separate scalar-dot-product path.

Caution 9.23 (This codebase implements a hybrid, not pure V2). The constructor of `EquiformerV2` in this codebase sets `use_s2_act_attn: bool = False` and enforces this with an explicit assertion (`assert not self.use_s2_act_attn`, with an inline comment recording that this path is unused). That is: the message mechanism is EquiformerV2’s eSCN-based $SO(2)$ -convolution (Section 9.5) throughout, but the *attention-weight* computation follows the **original Equiformer(V1) recipe** — separate invariant scalar channels, MLP, per-head dot product, softmax (Section 9.10) — rather than V2’s own alternative `use_s2_act_attn=True` path. This is a genuine, verifiable, non-cosmetic architectural fact about the shipped model, not a matter of degree or approximation, and this document states it explicitly rather than describing the backbone as “EquiformerV2” without qualification, as a less careful account might.

9.15 Summary: architecture, hyperparameters, and parameter count

Hyperparameter	Default value (constructor, <code>net/model.py</code>)
Maximum degree L (<code>lmax</code>)	3
Maximum order M (<code>mmax</code> , $SO(2)$ -conv truncation)	2
Number of resolutions	1 (single (L, M) pair)
Channels per degree (<code>sphere_channels</code>)	256
Number of attention layers (<code>num_layers</code>)	6
Attention heads (<code>num_heads</code>)	4
Edge/radial embedding dim	64 / 32 basis functions
Polynomial cutoff order	8
Time embedding	sinusoidal, dim 32, scale 10^4
Normalisation	<code>rms_norm_sh</code> (per-degree RMS norm)
Radial cutoff function	Bessel

Total learnable parameters, as reported in every training log inspected for this document: **14,979,377**.

Remark 9.24 (What this chapter has, and has not, independently verified). Several fine-grained implementation details — the exact RMS-norm formula, the exact sinusoidal time-embedding formula, the internals of the `S2Activation/GateActivation` classes as distinguished from `SeparableS2Activation`, and the precise semantics of each of the ten categorical atom features — were, per this document’s stated policy, flagged rather than guessed at in the code-research pass underlying this chapter, because the corresponding source files (`net/layer_norm.py`, `net/timestep_embedder.py`, `net/activation.py` in full, `chem/processing.py`) were not read to completion. Everything else in this chapter — the edge-frame mechanism, the $SO(2)$ -convolution message construction, the attention-weight computation and its V1/V2 hybrid status, the equivariant linear layer’s degree-block structure, the node embedding’s aggregation formula, and the final $\ell = 1$ readout — was confirmed directly against the source.

9.16 What you should now be able to do

- explain the cost problem that motivates EquiformerV2, and how aligning each edge to its own frame reduces a full $SO(3)$ tensor product to an $SO(2)$ convolution;
- say why that reduction is exact rather than approximate, and where the genuine approximation in the architecture actually sits;
- distinguish the three equivariant nonlinearities the architecture uses and the constraint each one respects;
- trace a feature through one block in the order the code executes it, and read off the final $\ell = 1$ output that both generative models consume.

Part V now has every ingredient: geometry, a generative process, and a network.

Part V

SigmaDock

Part V at a glance

What you will learn. The complete diffusion model: the fragment state space $SE(3)^m$, the forward noising process, the Newton–Euler prediction head, the training loss and the sampler — each traced to a theorem from Parts II–IV or flagged as a design decision with no derivation behind it.

Why it is needed. This is the system SigmaFlow modifies. A “minimally invasive replacement” is only meaningful once one knows exactly what is being replaced.

What depends on it. Part VI builds the alternative generative process; Part VII substitutes it and audits the result component by component against this part.

Chapter 10

SigmaDock: Full Reconstruction

Every mathematical ingredient is now available: Chapters 3 and 4 give the geometry, Chapters 6 and 7 give the generative-modelling machinery, and Chapters 8 and 9 give the network. This chapter assembles them into SigmaDock exactly as implemented, tracing every design decision back to either a theorem already proved or a specific line of code, and marking clearly the few points that remain genuinely open (not independently re-derivable from a static reading of the code alone).

10.1 Choosing the state space

Section 1.2 identified three coordinate systems in which a pose could be represented: all $3n$ Cartesian coordinates, the $(k+6)$ -dimensional torsional parameterisation $(SE(3) \times \mathbb{T}^k)$, or a fragment-based parameterisation $(SE(3)^m)$. SigmaDock’s central architectural decision is to use the third, and the argument for doing so is a statement about the induced *measure*, not merely about dimensionality — worth deriving carefully, since it is what licenses the product-manifold construction (Proposition 3.69) underlying every diffusion/flow-matching formula in this document.

Definition 10.1 (Pullback volume form). Let $\Omega : \mathcal{Y} \rightarrow \mathbb{R}^{n \times 3}$ be a smooth parameterisation map from a parameter space $\mathcal{Y}$ into Cartesian coordinates, with Jacobian $J_\Omega = \partial\Omega/\partial y$ and Gram matrix $G_\Omega = J_\Omega^\top J_\Omega$. The *pullback volume form* induced on $\mathcal{Y}$ is $d\mu_\Omega(y) = \sqrt{\det G_\Omega(y)} dy$ — the Riemannian volume form of the metric on $\mathcal{Y}$ obtained by declaring Ω a local isometry onto its image with the ambient Euclidean metric.

Theorem 10.2 (Torsional entanglement). *Let $\psi : SE(3) \times \mathbb{T}^k \rightarrow \mathbb{R}^{n \times 3}$ be the torsional parameterisation of a ligand with k rotatable bonds (global rigid motion composed with dihedral-angle-driven internal reconstruction), and let $\varphi : SE(3)^m \rightarrow \mathbb{R}^{n \times 3}$ be the fragment parameterisation with m pairwise-disjoint rigid fragments. Then, generically (for a molecular topology in which some rotatable bond’s rotation displaces atoms that a different rotatable bond also displaces — true for essentially every branched or chain topology with more than one rotatable bond), G_ψ is non-diagonal and $d\mu_\psi$ is not a product measure over the individual torsion coordinates. By contrast, J_φ is block-diagonal (one block per fragment, since moving fragment i ’s rigid motion changes no coordinate belonging to fragment $j \neq i$), so $\det G_\varphi = \prod_{i=1}^m \det(J_i^\top J_i)$ and $d\mu_\varphi$ is a product measure over fragments.*

Proof. Fragment case. By definition of φ , atom a ’s position depends only on the rigid motion of the fragment containing a : $\partial\varphi_a/\partial(\text{fragment } j\text{’s coordinates}) = 0$ whenever $a \notin \text{fragment } j$.

Hence J_φ , with rows grouped by atom and columns grouped by fragment, is block-diagonal (a given row has nonzero entries only in the column-block of its own fragment), and for a block-diagonal matrix $J = \bigoplus_i J_i$, $J^\top J = \bigoplus_i J_i^\top J_i$ is also block-diagonal, with $\det$ of a block-diagonal matrix the product of the blocks’ determinants.

Torsional case. Let ϕ_i, ϕ_j be two rotatable-bond angles whose associated atom-displacement fields overlap (an atom a moved by rotating about bond i is also moved by rotating about bond j — true whenever the two bonds lie on a common path through the molecular graph with atom a downstream of both, which occurs for any chain or branched topology with ≥ 2 non-independent rotatable bonds). Then $(G_\psi)_{ij} = \sum_a (\partial_{\phi_i} \psi_a) \cdot (\partial_{\phi_j} \psi_a)$ is generically nonzero, since it is a sum, over the atoms moved by *both* torsions, of dot products of two generically-non-orthogonal displacement vectors — there is no structural reason for this sum to vanish identically across all molecular geometries, and it generically does not. Since $(G_\psi)_{ij} \neq 0$ for some $i \neq j$, G_ψ is not diagonal, and $\det G_\psi$ does not factor as a product over the individual coordinates ϕ_i (a non-diagonal Gram matrix’s determinant is not, in general, the product of its diagonal entries), so $d\mu_\psi(\phi)$ cannot be written as $\prod_i d\mu_i(\phi_i)$ for single-coordinate measures $d\mu_i$. $\square$

The mechanism, made concrete: the Jacobian column for torsion ϕ_i is the Cartesian velocity field generated by rotating everything *downstream* of bond i about that bond’s axis, and in a chain or branched molecule this field moves *many* atoms, with the sets of atoms moved by different torsions typically overlapping. Three *practical* consequences follow directly from Theorem 10.2, each of which would separately motivate avoiding the torsional parameterisation for a diffusion or flow-matching model specifically (as opposed to, e.g., a search-based method, for which the measure’s factorisation is less directly load-bearing):

1. **Independent conditional noise does not stay independent.** A forward noising kernel that factorises as a product over torsion coordinates — exactly the kind of kernel Chapter 6 builds, one independent Gaussian/heat-kernel perturbation per coordinate — maps, under ψ , to a correlated, anisotropic perturbation of the Cartesian positions the network actually observes (since ψ ’s Jacobian mixes the torsion coordinates together non-trivially, per Theorem 10.2). The network must then implicitly learn to invert this metric-induced coupling, an unnecessarily hard learning problem compared to Proposition 3.69’s clean, coupling-free product structure on $SE(3)^m$.
2. **Global and internal motion do not decouple.** A torsional increment generically shifts the fragment’s centre of mass ($\sum_a m_a \partial_{\phi_i} \psi_a \neq 0$ in general, since the atoms downstream of bond i do not, in general, have their displacements sum to zero) and induces a net torque, so the $\mathbb{T}^k$ and $SE(3)$ factors of the torsional parameterisation are not, geometrically, independent either — any diffusion or flow constructed to treat them as independent (again, the natural, simple construction) is therefore only approximately correct, exactly (an infinitesimal-step) in the limit, not exactly at any finite step size.
3. **The dihedral definition itself has a gauge ambiguity.** A dihedral angle ϕ_{ABCD} does not, by itself, specify *which side* of the bond BC is held fixed while the other rotates; a change $\Delta\phi$ can be realised by rotating the A -side, the D -side, or any split between the two, and these give physically different (though dihedral-angle-equal) intermediate trajectories. Any concrete implementation must fix a convention, and a learned score/vector-field network must be trained consistently with whichever convention sampling uses — a consistency requirement with no natural enforcement mechanism, unlike the fragment parameterisation, where no such choice ever arises (each fragment simply *is* a specified rigid body).

Fragmentation dissolves all three simultaneously, *because* of Theorem 10.2’s block-diagonal Jacobian: moving one fragment’s rigid motion changes no other fragment’s coordinates at all, so independent per-fragment noise *stays* independent under φ , global and (now nonexistent, at the fragment level) internal motion trivially coincide, and there is no gauge ambiguity in specifying a rigid motion. The price is dimensional: $6m$ degrees of freedom against the torsional parameterisation’s $k + 6$, and $m > k + 1 - m$ generically (fragmenting at every rotatable bond, without merging, gives $\hat{m} = k + 1$ fragments, so $6\hat{m} = 6k + 6$ against $k + 6$ — *more* parameters than the torsional route, not fewer), which is exactly the deficit Section 10.3 below addresses.

Caution 10.3 (Status of this argument). Theorem 10.2 and its practical consequences are this document’s own reconstruction and elaboration of the reasoning given in the thesis draft this monograph audits (`Texte/main.tex`); we have verified the mathematical content — the block-diagonal-Jacobian argument for fragments, and the generic-non-diagonality argument for torsions — independently, by direct derivation, and found it sound, but we have *not* located a specific published SigmaDock source stating the theorem in this exact form, and it is presented here as a rigorous, self-contained argument for a design choice the codebase clearly makes (fragment-based, not torsional, generation), rather than as a verbatim quotation from a paper.

10.2 The conformational manifold: what rigidity costs

Treating each fragment as perfectly rigid is itself an approximation, and its cost should be quantified, not merely assumed acceptable. In vacuum, the accessible conformations of a ligand (fixed molecular topology) concentrate, under a Boltzmann distribution at physiological temperature, near a manifold $\mathcal{M}_c = \{x_c : g(x_c) \approx 0\}$ where g ’s components softly encode bond lengths ($\|r_{AB}\| - d_0$, d_0 the equilibrium bond length) and bond angles ($\hat{u} \cdot \hat{v} - \cos \tau_0$), *excluding* dihedrals (which are, by contrast, the ligand’s soft/free coordinates, exactly the ones Section 1.2 identified as free rotatable-bond degrees of freedom). $\mathcal{M}_c$ is sampled, in practice, by conformer-generation software (RDKit’s ETKDGV3 algorithm) rather than by literally simulating the Boltzmann distribution.

The question relevant to a fragment-based (rigid-template) docking model is whether a bound (experimentally observed) pose is well approximated by rigidly repositioning *some* sample from $\mathcal{M}_c$: writing $\psi(\phi)$ for the dihedral-to-Cartesian reconstruction map and $\Psi(p, R, \phi) = (p, R) \cdot \psi(\phi)$ for the composite (rigid motion applied to an internally-reconstructed conformer), the relevant quantity is

$$\min_{p \in \mathbb{R}^3, R \in SO(3), \phi \in \mathbb{T}^k} \text{RMSD}(x_b, \Psi(p, R, \phi)), \quad x_b \sim \pi_{\mathcal{M}_b}$$

(the bound-pose distribution). This document does not re-derive this empirical figure from scratch (it is a dataset-dependent, measured quantity, not a theorem); we record it, as reported in the thesis draft this monograph audits, and flag it as an empirical claim requiring the cited dataset (Astex Diverse Set) to independently reproduce: the best of five ETKDGV3 conformers aligns to the bound pose with mean RMSD 0.21 Å (and 0.39 Å for a single conformer), well under the 2 Å success threshold of Definition 1.5. The consequence for this document: the roughly 0.2 Å gap between a rigid-fragment reconstruction and the true bound pose is an **error floor** inherited by *any* model built on this fragmentation, including SigmaFlow, and is not attributable to the choice between diffusion and flow matching — it bounds what either method can, even in principle, achieve, and should not be mistaken for a deficiency of one method specifically when comparing the two (Chapter 13).

10.3 FR3D: reducing fragment count

Cutting *every* rotatable bond gives $\hat{m} = k + 1$ fragments and $6\hat{m} = 6k + 6$ degrees of freedom, exceeding the torsional parameterisation’s $k + 6$ — the naive fragment construction has bought Theorem 10.2’s decoupled measure at the cost of *more*, not fewer, dimensions than the alternative it replaces. FR3D (Fragment REduction) recovers some of this cost back by merging adjacent fragments: starting from the maximal $\hat{m}$ -fragment decomposition, a stochastic recursive procedure repeatedly merges two fragments across a shared torsional bond, terminating at an irreducible fragment set of size $m < \hat{m}$; empirically (per the audited thesis draft, not independently re-derived here) $m \approx \frac{2}{3}(k + 1)$. Because which bonds get merged is itself stochastic and (per the source description) equiprobable across valid cut-sets, re-running the fragmentation on the same molecule at a different training epoch can yield a genuinely different m -fragment decomposition, functioning simultaneously as a data augmentation scheme.

Caution 10.4. The *exact* merge heuristic and stopping criterion implemented by FR3D were not confirmed line-by-line against `fragmentation.py` in the code-research pass underlying this document (the file is over 1500 lines; only the rotatable-bond-detection and triangulation-mapping functions, described precisely below and in Section 10.4, were read to completion). What *was* confirmed directly is (i) rotatable-bond detection itself (Section 10.3.1 below), (ii) that the file’s own top-of-module documentation describes a “recursive merge” procedure, consistent with the description above, and (iii) the resulting dummy-atom bookkeeping (Section 10.3.2 below). The precise stochastic merge algorithm itself is therefore reported here as the audited thesis draft’s own description, not as independently re-derived or line-verified fact, and is flagged accordingly.

10.3.1 Rotatable-bond detection, as actually implemented

Code correspondence

Mathematical object: the set of rotatable bonds defining the cut points of Section 1.2

Implementation: `fragmentation.py::detect_torsional_bonds`

Interpretation: uses RDKit’s `TorsionFingerprints.CalculateTorsionLists` to enumerate torsion quadruples, then filters to bonds that are: not part of any ring, of bond type `SINGLE`, and (governed by an `ignore_conjugated` flag, default `False` per this project’s own training logs) optionally excluding conjugated bonds not touching a ring. This is a specific, code-verified criterion, more concrete than a generic “rotatable bond” description: not merely “any single, non-ring bond”, but specifically those bonds RDKit’s own torsion enumeration recognises as defining a well-formed dihedral quadruple.

10.3.2 Dummy atoms and fragment boundaries

Cutting a bond leaves each resulting fragment with a “dangling” bond at the cut point; SigmaDock represents this by introducing a *dummy atom* on either side, so that a fragment’s atom count can exceed the corresponding subset of the original ligand’s atom count ($|\mathcal{G}_{\text{ligand}}| \leq \sum_i |\mathcal{G}_{F_i}|$, one extra dummy per retained cut). A dummy atom is retained only if it remains *free* (marks a torsional bond that FR3D’s merge procedure did *not* collapse into a single fragment); if FR3D merges the two sides of a torsional bond back together, the corresponding dummy is *pruned*, since retaining it would spuriously freeze a dihedral that — per Section 10.2’s reachability argument — must remain free for the rigid-fragment reconstruction to stay close to the true bound-pose distribution.

10.4 Triangulation: an invariant, soft geometric prior at fragment boundaries

Fragmentation is lossy in one specific, quantifiable sense: once two fragments move independently, nothing in $SE(3)^m$ itself prevents the bond angles *across* the cut bond from taking chemically absurd values (Chapter 13’s empirical analysis of exactly this failure mode, in the flow-matching setting, is one of the central quantitative results of this project’s broader investigation, referenced there). SigmaDock’s answer is not to hard-constrain the manifold (which would reintroduce Theorem 10.2’s coupling problems) but to *condition the network* on an elementary, exact piece of Euclidean geometry.

Theorem 10.5 (Triangulation determines the boundary bond angles). *Let $\overline{BC}$ be a cut torsional bond, with B in fragment $\mathcal{A} \ni A$ and C in fragment $\mathcal{D} \ni D$ (A, D each bonded to B, C respectively, within their own fragment). Fixing the four intra-fragment/bond lengths $\|A - B\|$, $\|B - C\|$, $\|C - D\|$ and the two cross-fragment distances $\|A - C\|$, $\|B - D\|$ determines the two boundary bond angles $\angle(A, B, C)$, $\angle(B, C, D)$ uniquely, while leaving the dihedral ϕ_{ABCD} completely unconstrained.*

Proof. The triple $\{\|A - B\|, \|B - C\|, \|A - C\|\}$ determines the triangle (A, B, C) up to rigid motion (the classical side-side-side congruence criterion), and the law of cosines gives $\cos \angle(A, B, C) = \frac{\|A - B\|^2 + \|B - C\|^2 - \|A - C\|^2}{2\|A - B\|\|B - C\|}$, a function of the three fixed lengths alone; identically, $\cos \angle(B, C, D)$ is determined by $\{\|B - C\|, \|C - D\|, \|B - D\|\}$. Neither computation constrains the relative rotation of the two triangles’ planes about the shared edge $\overline{BC}$, which is exactly the dihedral ϕ_{ABCD} by Section 1.2’s definition. $\square$

The two distances the theorem needs, $\|A - C\|$ and $\|B - D\|$, are manifestly $SE(3)$ -invariant (ordinary Euclidean distances, Proposition 3.46), and hence admissible, per Section 8.5’s discussion, as ordinary type-0 (scalar) edge features consumable by an equivariant network without any special handling.

Code correspondence

Mathematical object: the triangulation conditioning feature

Implementation: the deviation formula $d_{ij}(t) = \left| \|x_i(t) - x_j(t)\| - \|x_i^{\text{ref}} - x_j^{\text{ref}}\| \right|$, computed at `net/model.py::compute_edge_embedding`’s “boundary” edge branch, using `pos_0` as the reference geometry and `pos_t` as the current (noised/interpolated) geometry; the *atom pairs* entering this formula are enumerated by `fragmentation.py::get_triangle_equality_mapping`

Interpretation: the network is given, at every step t , the absolute deviation of each triangulation distance from its equilibrium (reference-conformer) value, so $d_{ij}(t) \rightarrow 0$ as $t \rightarrow 1$ (approaching data, in this document’s generation-time convention) is exactly the signal that “this boundary is currently geometrically wrong, correct it”; no projection or hard equality is ever imposed, only this soft conditioning signal. **More general than Theorem 10.5’s minimal statement:** the actual code maps every anchor atom to *all* atoms directly bonded to each of its neighbouring anchors (not merely one representative atom A or D per side), so more than the minimal two cross-fragment distances of the Theorem are typically supplied per cut bond in practice — the Theorem establishes that two such distances *suffice* in principle, not that exactly two are used. An `ignore_triangulation` ablation flag exists (`data.py`), consistent with the audited thesis draft’s report of an ablation study in which removing this conditioning costs a docu-

mented 12.8 percentage points of PoseBusters validity (Def. 1.6) — the single largest effect size reported in that study, though we have not independently reproduced this specific number and report it as the audited source’s own claim.

Remark 10.6 (Degrees of freedom of the soft-constrained manifold). Were the triangulation distances imposed as *hard* equality constraints (which they are not — Theorem 10.5’s feature is a soft conditioning signal, never a projection), the constrained submanifold $\mathcal{M}_{\text{frag}} = \{z \in SE(3)^m : c(z) = 0\}$ would, for a tree-structured fragment graph ($e = m - 1$ cut bonds), have two non-degenerate distance constraints per joint removing five of the six relative degrees of freedom between adjacent fragments (leaving exactly the one free dihedral, as Theorem 10.5’s proof shows), so $\dim \mathcal{M}_{\text{frag}} = 6m - 5(m - 1) = m + 5$ — interpolating between the torsional parameterisation’s $k + 6$ and the unconstrained $SE(3)^m$ ’s $6m$. Since the actual constraints are soft, sampled trajectories only *concentrate* near $\mathcal{M}_{\text{frag}}$ as $t \rightarrow 1$; this dimension count is a useful sanity check on the geometry, not a claim about the trained network’s exact behaviour.

10.5 Symmetry: gauge invariance and stochastic $SE(3)$ -equivariance

Two distinct symmetry statements matter for SigmaDock, and conflating them (as the broader literature sometimes does) obscures which architectural property is responsible for which guarantee.

Gauge invariance. A rigid fragment’s *local* coordinate frame (the coordinate system its reference-conformer template is expressed in, before any rigid motion is applied) has no canonical choice: for any fixed $R_s \in SO(3)$, replacing the local template $\tilde{x}_F$ by $R_s \tilde{x}_F$ and simultaneously replacing the fragment’s rotation R_F by $R_F R_s^\top$ describes the *same* global pose, $R_F \tilde{x}_F + p_F = (R_F R_s^\top)(R_s \tilde{x}_F) + p_F$. A correctly constructed loss or sampler must not depend on this arbitrary choice.

Stochastic $SE(3)$ -equivariance. The *generated distribution* should satisfy $p_\theta(dz \mid R_0 \cdot \mathcal{G}_{\text{dock}}) = R_0 \cdot p_\theta(dz \mid \mathcal{G}_{\text{dock}})$ for every $R_0 \in SO(3)$: rotating the input pocket rotates the entire predicted pose *distribution* and nothing else about it.

Theorem 10.7 (SigmaDock’s symmetry). *SigmaDock’s training objective and sampling procedure are invariant to the (arbitrary) choice of fragment local-frame orientation, and the induced kernel $p_\theta(z \mid \mathcal{G}_{\text{dock}})$ is stochastically $SO(3)$ -equivariant.*

This is the stochastic-process analogue of Proposition 8.3: there, a deterministic equivariant ODE plus an invariant prior gave an invariant marginal law at every time t ; here, the corresponding statement for the *diffusion SDE* (Chapters 6 and 7) is that an $SE(3)$ -equivariant score network, together with the $SE(3)$ -invariant prior $p_{\text{init}} = \mathcal{N}(0, I) \otimes \mathcal{U}(SO(3)^m)$ established in Section 10.6 below, is sufficient (via the reverse-time SDE of Equations (6.8) and (7.3), whose drift and diffusion coefficients depend on the score in an equivariant way by construction) for stochastic equivariance of the generated law; we do not reprove this SDE-level statement in full technical generality (it requires checking that the reverse-time SDE’s law, not merely a deterministic flow’s pushforward, transforms correctly, a short but somewhat more technical argument than Proposition 8.3’s proof), and refer to the general treatment in De Bortoli et al. (2022, cited in Section 7.1) for the manifold-SDE case. *Gauge invariance* of the loss and sampler, by contrast, is a structural property directly checkable from Section 9.12’s observation that the network’s backbone consumes only world-frame Cartesian coordinates $\mathbf{x} = \varphi(\mathbf{T})$ and never the fragment-local decomposition $(\tilde{x}_F, p_F, R_F)$

separately — so its output is, by construction, manifestly independent of any particular choice of R_s , since R_s never appears anywhere in the network’s input.

10.6 The forward (noising) process

Remark 10.8 (SigmaDock’s own internal time convention, relabelled). SigmaDock’s own source code (`r3_diffuser.py`, `so3_diffuser.py`) runs its internal time variable in the *opposite* direction from this document’s global convention (Remark 0.2): its $t = 0$ is clean data ($\alpha_{t=0} = 1$, no noise) and its $t = 1$ is the fully-noised end (confirmed directly, e.g. by `S03Diffuser.sample_ref` literally evaluating the forward kernel at $t = 1$ to obtain an approximately-prior sample). Every formula in this chapter is stated in *this document’s* convention ($t = 0$ noise, $t = 1$ data, matching Remark 0.2 and, as Section 13.2 confirms, matching SigmaFlow’s own native convention exactly), obtained from SigmaDock’s native formulas by the substitution $s = 1 - t$ throughout — i.e. s below always denotes SigmaDock’s own noising-time variable, and every SigmaDock formula quoted in this chapter has already been relabelled via this substitution before being stated, rather than being transcribed directly in SigmaDock’s own time variable.

We can now assemble Chapters 6 and 7’s two constructions, componentwise, on the state space $\mathcal{M}_m = SE(3)^m$ fixed by Section 10.1, per Proposition 3.69’s licence to do so. Writing $s = 1 - t$ for noising time (Remark 10.8) and suppressing the fragment index:

Translation: a variance-preserving process (Example 6.6), $\beta(s) = \beta_{\min} + s(\beta_{\max} - \beta_{\min})$ (linear schedule), so the conditional law derived in Example 6.6 is $p_{s|0}(p^{(s)} \mid p^{\text{data}}) = \mathcal{N}(\alpha_s p^{\text{data}}, (1 - \alpha_s^2)I)$, $\alpha_s = \exp(-\frac{1}{2} \int_0^s \beta(r) dr)$.

Rotation: a variance-exploding process (Example 6.5), with the *logarithmic* schedule $\sigma(s) = \log(se^{\sigma_{\max}} + (1 - s)e^{\sigma_{\min}})$ (confirmed exactly against `so3_diffuser.py` in Section 7.2.2), so the conditional law is $p_{s|0}(R^{(s)} \mid R^{\text{data}}) = \text{IGSO}(3)(R^{\text{data}}, \sigma(s)^2)$ (Theorem 7.4).

By Proposition 3.69’s product structure, the forward kernel factorises exactly across the two components (and, trivially, across fragments, by Definition 3.70), and the stationary distribution — the model’s prior — is $p_{\text{init}} = \mathcal{N}(0, I) \otimes \mathcal{U}(SO(3)^m)$: an isotropic Gaussian for translations (Proposition 6.7’s VP schedule drives the marginal variance to 1 as $s \rightarrow 1$, for a suitably long/steep β schedule) and the Haar-uniform measure for rotations (Proposition 7.2), matching exactly the invariant prior Theorem 10.7 requires.

Remark 10.9 (Coordinate rescaling). Before any of the above is applied, ligand and protein coordinates are centred on the binding-pocket centre of mass and divided by a fixed constant, `dimensional_scale = 2.7` (`oracle.py`); this same constant additionally rescales *every* graph-construction cutoff radius used throughout Chapter 9 (protein–ligand interaction, Ca–Ca virtual-node, and every other edge type’s cutoff is divided by 2.7 before use, per `oracle.py::get_edge_specs`) — i.e. this is a single, global Ångstrom-to-model-units rescaling applied uniformly throughout the pipeline, *not* solely a device for matching the translational Gaussian prior’s unit variance to the data’s spread, as a narrower reading might suggest. Its specific numerical value (2.7) is reported, in the audited thesis draft, as the mean standard deviation (in Å) of fragment centres of mass relative to the pocket centre of mass across the training set — an empirical, dataset-derived constant, not a theoretically derived one.

Training minimises the Riemannian denoising score matching objective of Theorems 6.4 and 7.9, with per-component weights λ_s matching Section 6.3’s $\lambda(s) = 1/\mathbb{E}[\|\nabla \log p_{s|0}\|^2]$ recipe: for translation, this reduces to the closed form of Section 6.3’s discussion ($\lambda_s \propto$

$1/(1 - \alpha_s^2)$, matching `R3Diffuser.score_weight` exactly); for rotation, this expectation has no closed form and is computed numerically from the tabulated $\partial_\omega \log f_\sigma$ table (Section 7.2.2’s `score_scaling` code correspondence). This machinery — the IGSO(3) kernel with its truncated spectral series, the conditional score derived from it, and the tabulated loss weight — is the *entirety* of what is diffusion-specific in SigmaDock; Chapter 13 shows precisely which of these pieces flow matching removes and which it retains unchanged.

10.7 Input graph, featurisation, and backbone

The network never sees the abstract group element $z \in SE(3)^m$ directly; it sees the geometric graph of Definition 8.13, built from the current Cartesian coordinates $\mathbf{x}^{(t)} = \varphi(z^{(t)})$ (the rigid-fragment reconstruction of Section 10.7.1 below) and the (fixed) protein structure. Chapter 9 already derived this construction in full architectural detail — the categorical atom/edge features and their dimensions (Section 9.11’s code correspondence), the radial basis and smooth cutoffs (Section 8.4’s code correspondence), the hierarchical virtual-node structure (fragment-level and protein-C α -level, Section 9.11), and the triangulation edges just derived (Section 10.4) — and none of it is repeated here. We record only the one remaining, purely diffusion-side, distinction: the graph’s *topology* splits into a *global* part, immutable along the noising/generation trajectory (intra-fragment bonds, the protein structure, the triangulation and virtual-node edges), and a *transient* part, recomputed at every step from the current, moving positions $\mathbf{x}^{(t)}$ (the protein–ligand interaction radius graph, 4 Å cutoff per Section 8.4’s code correspondence): exactly the distinction that made the smooth radial-cutoff envelope χ of (8.9) necessary, since an edge in the transient part can appear or disappear as $\mathbf{x}^{(t)}$ evolves during either training’s noising or sampling’s ODE/SDE integration.

The backbone is EquiformerV2 exactly as reconstructed in Chapter 9, with the one modification already noted there (Section 9.5’s code correspondence): the radial MLPs generating $SO(2)$ -convolution weights on transient (protein–ligand interaction) edges have their bias terms removed, which is what makes the smooth-cutoff argument of Section 8.4 hold exactly rather than only approximately.

10.7.1 Fragment reconstruction: the map φ

We finally make precise the map $\varphi : \mathcal{M}_m \rightarrow \mathbb{R}^{n \times 3}$, used silently throughout this chapter, that turns an abstract group element $z = (R^{(1)}, p^{(1)}, \dots) \in SE(3)^m$ into the atomic Cartesian coordinates the network actually consumes.

Proposition 10.10 (Rigid per-fragment reconstruction). *Fix a reference conformer $x^{\text{ref}} \in \mathbb{R}^{n \times 3}$ (the rigid-fragment template of Section 10.2, one fixed geometry per training/sampling instance, never itself diffused or flowed), and let p_F^{ref} denote fragment F ’s centre of mass in that reference geometry. For a target pose $z = (R_F, p_F)_F$, the reconstructed atom position, for atom i in fragment F , is*

$$\varphi(z)_i = R_F(x_i^{\text{ref}} - p_F^{\text{ref}}) + p_F^{\text{ref}} + \Delta p_F, \quad \Delta p_F := p_F - p_F^{\text{ref}}, \quad (10.1)$$

i.e. every atom of fragment F is displaced from its reference position by exactly the same rigid motion: rotate about the fragment’s own reference centre of mass by R_F , then translate by Δp_F . Protein atoms, having no fragment structure, are passed through unchanged.

Why this is the right formula. This is, by construction, the unique map with two required properties: (i) *rigidity* — for two atoms i, j in the same fragment F , $\|\varphi(z)_i - \varphi(z)_j\| =$

$\|R_F(x_i^{\text{ref}} - x_j^{\text{ref}})\| = \|x_i^{\text{ref}} - x_j^{\text{ref}}\|$ by Proposition 3.46, i.e. every intra-fragment distance — in particular every bond length and bond angle — is preserved *exactly*, for *any* (R_F, p_F) , which is exactly the content of Section 10.2’s claim that rigidity is enforced structurally, not learned; and (ii) *correct fragment placement* — setting $R_F = I, \Delta p_F = 0$ recovers the reference conformer exactly, and in general the fragment’s centre of mass is placed at $p_F^{\text{ref}} + \Delta p_F = p_F$, matching the z -coordinate’s own translational component by construction. $\square$

Code correspondence

Mathematical object: φ of Proposition 10.10

Implementation: `denoiser.py::_apply_transformations` (SigmaDock) and, identically in form, `sigma_flow_generator.py::get_transformations_from_rototranslations` (SigmaFlow, Chapter 13)

Tensor shape: $[N, 3] \rightarrow [N, 3]$

Interpretation: both modules implement the identical formula $\text{new_pos}[i] = R_F \cdot (\text{pos_old}[i] - p_F^{\text{ref}}) + p_F^{\text{ref}} + \Delta p_F$, confirming, at the level of this specific, central piece of geometric plumbing, the claim (recorded again in Chapter 13) that SigmaFlow’s conversion from SigmaDock leaves this component completely unchanged: only the process that determines *which* $(R_F, \Delta p_F)$ to evaluate φ at, at a given step t , differs between the two methods (Chapters 6 and 7 vs. Chapter 11), never the reconstruction map itself.

10.8 The Newton–Euler prediction head

Section 9.12 established that the network’s raw output is a per-atom 3-vector in the world frame, read from the $\ell = 1$ channels. This section derives how that raw output becomes the two score components (8.1) requires — and, in doing so, explains precisely *why* the code names its intermediate quantities “force”, “torque”, “mass”, and “inertia”, since this is not decorative language but the literal reuse of classical rigid-body mechanics formulas as an architectural device.

10.8.1 From network output to a physical analogy

Code correspondence

Mathematical object: the raw per-atom network output, interpreted as a noise-prediction (an ϵ -parameterisation target, in the language of Section 6.2: recall $x = z + \sigma\epsilon$, so predicting ϵ and predicting the score $-\epsilon/\sigma$ are related by a fixed, known rescaling)

Implementation: `denoiser.py::_compute_forces`; the network call itself is `epsilon_theta = self.model(batch, t, **model_kwargs)`, followed by `lig_forces = -epsilon_theta.index_select(0, forces_idx)`

Tensor shape: $[N_{\text{ligand_atoms}}, 3]$ (the code’s own inline type annotation states $[B \times F \times A, 3, 3]$, i.e. a full 3×3 matrix per atom rather than a 3-vector; every downstream use — indexing along a single leading dimension, elementwise operations consistent with a 3-vector — is inconsistent with that annotation, and we record it, per this document’s stated policy, as a probable stale/incorrect docstring rather than a 3×3 -matrix-valued quantity actually being used)

Interpretation: the variable name `lig_forces`, and the sign flip (`-epsilon_theta`), signal the reinterpretation about to happen: **this per-atom vector is about to be treated as a literal force acting on that atom**, and everything from here to Equation (10.4) below is the consequence of taking that interpretation seriously and applying

textbook rigid-body mechanics.

10.8.2 Net force, torque, and inertia: exact classical mechanics

Definition 10.11 (Fragment mass, per this codebase’s specific convention). For fragment F with (non-virtual, non-dummy) atoms indexed by $i \in F$, the *mass* is $M_F := |F| = \sum_{i \in F} 1$ — the **atom count**, with every atom assigned unit mass regardless of its chemical identity. This is **not** physical/chemical atomic mass (e.g. carbon is not weighted $12\times$ more than hydrogen); it is a uniform, purely topological quantity.

Code correspondence

Mathematical object: M_F of Definition 10.11, and the fragment inertia tensor $I_F = \sum_{i \in F} (\|r_i\|^2 I_3 - r_i r_i^\top)$, $r_i = x_i - p_F$ (position relative to the fragment centre of mass) — the classical inertia tensor of a rigid body composed of point masses, specialised to unit mass per point

Implementation: `denoiser.py::get_fragment_mass_inertia`

Interpretation: the assertion `assert M.min() >= 2` (with message “Fragment mass below 2 detected. Check input data.”) is now precisely explained: it fires whenever a fragment has *fewer than two atoms*, a purely topological/fragmentation degeneracy (a fragment reduced, by FR3D’s merging or by the underlying molecule’s own structure, to a single heavy atom plus dummies), not a statement about any atom’s chemical mass being unusually low — the theory-document text of Chapter 10 must therefore never describe this condition as “fragment mass” in the chemical sense, only in the atom-count sense Definition 10.11 fixes. The inertia tensor’s eigenvalues are additionally floored (via an eigendecomposition, clamping to at least $\max(10^{-8} \text{tr}(I)/3, 10^{-12})$) before use, protecting the subsequent linear solve (Equation (10.3) below) against near-degenerate (e.g. near-linear, two-to-three-atom) fragments.

Given the per-atom pseudo-forces f_i (the negated network output) and the mass/inertia just defined, the exact classical-mechanics formulas for a rigid body’s net force, net torque about its centre of mass, are:

$$F_F = \sum_{i \in F} f_i, \quad \tau_F = \sum_{i \in F} (x_i - p_F) \times f_i. \quad (10.2)$$

Code correspondence

Mathematical object: (10.2)

Implementation: `denoiser.py::linear_mechanics` (a `scatter/index_add`-based per-fragment reduction of a per-atom force field, computed at the *current*, time- t fragment centres of mass, i.e. using `coms = trans_t`, not the data-time centres)

Interpretation: this is, literally, the textbook definition of net force ($\sum$ of per-particle forces) and net torque about a pivot ($\sum r_i \times f_i$) for a rigid body under a field of per-particle forces — the “pseudo-force”/“Newton–Euler” naming is earned by direct, unmodified reuse of these formulas, with the network’s noise prediction standing in for the physical force field.

10.8.3 Newton–Maruyama: one discretised step of rigid-body dynamics

$$dT = \frac{F_F}{M_F} \quad (\text{force over mass} = \text{acceleration-like increment}), \quad dW = I_F^{-1} \tau_F \quad (\text{Euler’s rigid-body equation}) \quad (10.3)$$

with $dW \in \mathbb{R}^3$ then lifted to $\widehat{dW}$ (Definition 3.49).

Code correspondence

Mathematical object: (10.3)

Implementation: `denoiser.py::newton_maruyama` (`dT = force_per_fragment / mass`; `dW_world = torch.linalg.solve(inertia, torque_per_fragment)`), with the inertia additionally Tikhonov-regularised by adding $10^{-8}I$ on top of the eigenvalue floor of Definition 10.11’s code correspondence; then `omega = hat(dW_world)`, clamped entrywise to $[-10^3, 10^3]$ as a loose numerical safety net)

Interpretation: the function name “Newton–Maruyama” fuses Newton’s second law (the $dT = F/M$ step) with the Euler–Maruyama discretisation scheme referenced throughout Chapters 6 and 7 (this specific function itself is purely deterministic given force/torque — the stochastic Maruyama noise increment is added *separately*, at the reverse-SDE-integration step of Section 10.10 below, not here). The code’s own inline comments assert this quantity is expressed in the “GLOBAL (left-invariant — WORLD) frame”, which we read, per Section 3.5’s resolved terminology, as the **world-frame (spatial)** angular velocity ω^{sp} of Definition 3.60 — note that the code’s word “left-invariant” carries the opposite attribution to this document’s, per Caution 3.61, while the object meant is unambiguous: no rotation by R_t is applied to dW before or after the linear solve, consistent with this reading, and consistent with I_F itself having been computed at the current *world-frame* positions x_i (not in any fragment-local frame). This is worth flagging explicitly against Section 3.5’s resolution for the *flow-matching* side, which found the *opposite* (left-trivialised/body-frame) convention in `so3_flow_matcher.py`: SigmaDock’s Newton–Euler head and SigmaFlow’s flow-matching update operate in different trivialisation conventions at this specific point, a genuine architectural difference (not necessarily an inconsistency, since each is internally self-consistent within its own module, by Section 3.5’s verification for the flow-matching side and the world-frame reading just given here for the diffusion side) that Chapter 13 discusses further when it reconstructs the corresponding flow-matching quantity.

10.8.4 Dressing the raw output into a score: the two branches

The pair $(dT, dW) \in \mathbb{R}^3 \times \mathbb{R}^3$ from (10.3) already has the correct *shape* of a tangent vector on $\mathcal{M}_m$ ((8.1)), but it is not yet a score: Proposition 7.8 and Remark 2.13 show the true conditional score carries a noise-level-dependent scalar prefactor that this raw mechanical quantity does not yet include. SigmaDock dresses the translational component in one fixed way, and the rotational component via a code-configurable choice of *two* distinct branches.

Translation (single formula, no branching):

$$s_\theta^p = r_3\text{-scaling}(t) \cdot dT, \quad r_3\text{-scaling}(t) = \text{R3Diffuser.score_scaling}(t), \quad (10.4)$$

matching Remark 2.13’s Gaussian-score form up to the fixed, schedule-determined scalar prefactor of Section 6.3.

Rotation, branch `rot_score_method="space"` (constructor default): treat dW as an *implied final-rotation update* rather than a score directly — form $\widehat{dW}$, rescale by the $SO(3)$ -analogue of the translational scaling (`so3_scaling`, from `S03Diffuser.score_scaling`), exponentiate ($\Delta R = \exp(\text{scaled } \widehat{dW})$, Section 3.4.3), compose $\widehat{R}_0 := \Delta R \cdot R_t$ (an implied estimate of the clean/data rotation), and *then* evaluate the exact closed-form conditional score (7.2) of Proposition 7.8 at this implied $\widehat{R}_0$:

$$s_\theta^R = \nabla_{R_t} \log p_{t|1}(R_t \mid \widehat{R}_0) \quad (\text{Equation (7.2), evaluated at } R_1 = \widehat{R}_0). \quad (10.5)$$

Rotation, branch `rot_score_method="score"` (raises a configuration error unless a scaling sub-choice is also specified): treat dW as *directly proportional to the score itself*,

$$s_{\theta}^R = -\alpha(t) \cdot \widehat{dW} \cdot R_t, \quad (10.6)$$

with the scalar $\alpha(t)$ chosen by a further sub-choice: either `"rms"` (reuse the same `so3_scaling` used by the `"space"` branch) or `"true"` (evaluate $\partial_{\omega} \log f_{\sigma(t)}(\omega)/\omega$ live, at the network's own implied angle $\omega = \|dW\|$, from Section 7.2.2's `d_log_f_d_omega` routine — i.e. the *exact* closed-form score-scaling of Proposition 7.8 evaluated at whatever angle the network happens to have produced, rather than the RMS-averaged $\sigma(t)$ -only scaling of the `"rms"` option).

Caution 10.12 (This document's confirmed configuration). Per the code-research pass underlying this document, the constructor default is `rot_score_method="space", rot_score_scaling="rms"`. Which branch the specific trained checkpoints referenced elsewhere in this project's broader development history actually used at training time was not independently cross-checked against those checkpoints' saved hyperparameters as part of producing this document, and should be verified separately before drawing any conclusion that depends on the specific branch in force.

10.9 The training loss

Code correspondence

Mathematical object: the four-component SigmaDock training loss

Implementation: `denoiser.py::compute_losses`

Interpretation: four terms, each a λ -weighted squared error, matching (6.7)'s continuous-time DSM objective pattern:

$$\begin{aligned} \text{(score-matching, translation)} \quad & \lambda_t^p \|s_{\theta}^p - s_{\text{true}}^p\|_2^2, & \lambda_t^p &= 1/(\text{T_score_scaling})^2, \\ \text{(score-matching, rotation)} \quad & \lambda_t^R \|s_{\theta}^R - s_{\text{true}}^R\|_{\text{Fro}}^2, & \lambda_t^R &= 1/(\text{R_score_scaling})^2, \\ \text{(data-space, translation)} \quad & \lambda_t^{p,0} \|p_0 - \widehat{p}_0\|_2^2, & \lambda_t^{p,0} &= (\alpha_t \cdot \text{T_score_scaling})^2, \\ \text{(data-space, rotation)} \quad & \lambda_t^{R,0} \|\log(\widehat{R}_0^{\top} R_0)\|_{\text{Fro}}^2, & \lambda_t^{R,0} &= (\text{R_score_scaling})^2, \end{aligned}$$

returned as a four-key dictionary `{T_score, R_score, T0, R0}`.

Code correspondence

Mathematical object: per-molecule reduction of the (per-fragment) losses above

Implementation: `denoiser.py::scaled_fragmented_loss: scatter(x, batch_idx, reduce="sum") / num_fragments^fragment_scaling`

Interpretation: `fragment_scaling=1` divides by the exact fragment count, i.e. averages per molecule regardless of how many fragments it has; `fragment_scaling=-1` (per one code comment site's own docstring) applies *no* normalisation, biasing the loss towards molecules with more fragments; intermediate values interpolate. **Note, per the code-research pass:** the default value of this hyperparameter differs between two source locations in the codebase (`1.0` in one configuration dataclass, `0.5` in the Lightning module's own constructor default) — almost certainly immaterial in practice if the training script's CLI parsing always threads an explicit value through, but this was not independently confirmed, and is recorded as an open item rather than resolved by assumption.

10.10 Sampling

Generation integrates the reverse-time SDEs of Equations (6.8) and (7.3), componentwise, from a near- $t = 0$ starting time to $t = 1$.

Code correspondence

Mathematical object: the timestep discretisation of the reverse integration

Implementation: `sampling.py::sampler`, matching `conf/sampling/base.yaml`'s `diffusion: block`

Interpretation: `default num_steps= 25, t_min= 0.005, rho= 3.0, default discretization="edm"` (the Karras et al. power-law-in- $1/\rho$ schedule, $t_k = (t_{\max}^{1/\rho} + \frac{k}{N-1}(t_{\min}^{1/\rho} - t_{\max}^{1/\rho}))^\rho$, rather than the simpler "power" schedule $t_k = \text{linspace}(t_{\max}, t_{\min}, N)^\rho$, which is available but not the default here); the timestep array itself walks *downward*, in this module's own internal (noising-time-aligned) convention, from $t_{\max} = 1$ to $t_{\min}$, consistent with Remark 0.2's warning that SigmaDock's native time convention is reversed relative to this document's global convention (Chapter 13 makes the exact reversal explicit and code-confirmed).

Code correspondence

Mathematical object: one reverse-time step (Euler–Maruyama for translation, geodesic random walk for rotation)

Implementation: `sampling.py::sampler`'s loop body, composing `denoiser._compute_forces` $\rightarrow$ `_compute_fragment_dynamics` $\rightarrow$ `_predict_fragment_updates` (Newton–Maruyama, Equation (10.3)) $\rightarrow$ `_compute_scores` (Section 10.8.4's branch) $\rightarrow$ `denoiser.diffuser.reverse` (dispatching componentwise to `R3Diffuser.reverse`, Example 6.6's code correspondence, and `S03Diffuser.reverse`, Section 7.3's code correspondence) $\rightarrow$ `_apply_transformations` (rigid-fragment position reconstruction, Section 10.7.1)

Interpretation: exactly one EquiformerV2 forward pass per integration step — the same cost structure Chapters 11 and 13 will show flow matching shares, so any difference in the number of network evaluations needed for comparable sample quality between the two methods is a genuine, comparable efficiency question, examined in this project's broader experimental work rather than in this theory document. **Heun's method is not implemented**, despite `solver: Literal["euler", "heun"]` appearing in the function signature: both this sampler and the flow-matching sampler of Chapter 13 contain only a dead `# TODO Heun's method` comment block after the (Euler-only) loop; selecting `solver="heun"` has, per the code-research pass underlying this document, no effect on the executed computation.

Code correspondence

Mathematical object: the stochastic noise-annealing schedule

Implementation: `quadratic decay noise_scales = linspace(noise_scale1/noise_decay, 0, num_steps)noise_decay`, default `noise_scale= 0.0, noise_decay= 2.0` (per `conf/sampling/base.yaml`)

Interpretation: this array rescales *only* the injected Brownian-increment term of the Euler–Maruyama/geodesic-random-walk update (the $\sqrt{dt} z$ term), independently of the score-drift term, and decays from `noise_scale` to exactly 0 over the course of sampling. **With the shipped default `noise_scale=0.0`, every injected-noise term is zero at every step:** SigmaDock's *out-of-the-box default sampling configuration is already fully deterministic*, i.e. it already runs, without any code change, the probability-flow-

ODE-equivalent trajectory of Theorem 6.9 rather than a genuinely stochastic SDE trajectory. This narrows the “diffusion is stochastic, flow matching is deterministic” framing considerably: the distinction that is architecturally forced for flow matching (Chapter 11 never constructs a noising process at all) is, for this specific configuration of SigmaDock, an *opt-in* choice (setting `noise_scale > 0`) rather than an always-active default behaviour.

10.11 Ranking generated poses

SigmaDock draws N_{seeds} (default 40, per the audited thesis draft) independent samples per complex and ranks them by a fixed, learned-model-free heuristic,

$$s_i = -b_i p_i^\beta, \quad \beta = 4,$$

where b_i is a Vinardo docking score (a classical, physics-based scoring function, not part of the generative model itself) and $p_i \in [0, 1]$ is the fraction of PoseBusters checks (Definition 1.6) sample i passes. SigmaDock deliberately dispenses with both a learned confidence/ranking model and any post-hoc coordinate-altering energy minimisation — both standard in the DiffDock lineage this document’s introduction references — specifically so that reported accuracy can be attributed to the generative model itself rather than to a downstream correction step; Section 13.9 in Chapter 13 returns to whether flow matching’s ODE structure offers a principled alternative to this heuristic.

10.12 What you should now be able to do

- explain the choice of state space $SE(3)^m$ and what rigidity costs in reachable conformations;
- follow a training step end to end: fragment the ligand, noise the pose, featurise, predict, and form the loss;
- explain what the Newton–Euler head does, and separate the exact classical mechanics in it from the two regularisations that are not;
- identify which design decisions follow from a theorem and which are choices with no derivation behind them.

The next two parts ask what happens when the generative process — and only the generative process — is replaced. Concretely: the state space, the fragmentation, the featurisation and the entire network of Chapters 8 and 9 are *kept*; the forward noising process, the score target, the output dressing and the sampler are *replaced*. Table 13.1 states the full component-by-component list, verified against both source trees.

Part VI

Flow Matching

Part VI at a glance

What you will learn. The second way to transport noise into data: choose a probability path directly, and learn the *velocity field* that carries samples along it. First in $\mathbb{R}^d$ — including the proof that regressing on a conditional target recovers the intractable marginal one — then on a Riemannian manifold, where exp and log replace + and −.

Why it is needed. Section 6.4 showed every diffusion model secretly contains a deterministic flow. This part develops that flow as a *starting point*, which removes the need for a noising process — decisive on $SO(3)$, where the noising process requires the truncated spectral series of Section 7.2.2 and the geodesic requires two lines.

What depends on it. Part VII: every construction here reappears there as a specific function in a specific file, verified line by line.

Chapter 11

Continuous Normalizing Flows and Flow Matching in $\mathbb{R}^d$

Section 6.4 showed, as a *consequence* of diffusion theory, that every score-based diffusion model secretly transports p_{init} to p_{data} along a deterministic ODE. This chapter develops the same object — a deterministic flow transporting one distribution to another — as a *starting point* rather than a derived fact, entirely independently of scores, noising processes, or SDEs. The resulting framework, flow matching, is what Chapter 13 substitutes for Chapters 6 and 7 to obtain SigmaFlow.

The chapter is self-contained: a reader who has not read Part III can start here, and a reader who has will find that roughly the last third of the chapter is spent showing that the two frameworks are two descriptions of one object. Nothing in the main development requires access to the SigmaDock or SigmaFlow source code; where a construction happens to be realised by that code, it is flagged in a short *implementation note* that can be skipped without loss.

Remark 11.1 (Sources used in this chapter). The didactic backbone of this chapter follows Holderrieth & Erives, *An Introduction to Flow Matching and Diffusion Models* (MIT 6.S184 lecture notes, 2026), §3–§4, which is available in this project’s `papers/` directory and which uses the same time convention as this document ($t = 0$ noise, $t = 1$ data). Every theorem taken from there has been re-derived here rather than quoted, and the algebra has additionally been checked numerically (Section 11.19). The diffusion bridge of Section 11.13 is checked against Song et al. (2021), also in `papers/`.

Not available locally: the original flow-matching paper (Lipman, Chen, Ben-Hamu, Nickel & Le, *Flow Matching for Generative Modeling*, ICLR 2023), and Salimans & Ho, *Progressive Distillation for Fast Sampling of Diffusion Models* (ICLR 2022), which is where the diffusion v -parameterisation of Section 11.14 originates. Where those works are referred to below, the reference is bibliographic only; the single place where a *definition* is imported from an unavailable source — the definition of v — is marked explicitly, and every statement *derived* from it is derived here in full.

11.1 The generative transport problem

Before any loss function, before any network, there is a problem statement. It is worth writing it down in its bare form, because everything that follows is a construction that solves it, and the constructions look arbitrary if one has forgotten what they are for.

We are given two distributions on $\mathbb{R}^d$:

- a **source** (or prior) distribution p_{init} that we can sample from cheaply and exactly — in practice almost always $\mathcal{N}(0, I_d)$, or, on a compact group, the uniform measure;
- a **target** distribution p_{data} that we *cannot* sample from, and whose density we do not know, but of which we possess a finite collection of samples (the training set).

We want an algorithm that produces fresh samples from p_{data} .

Flow matching solves this by transporting probability mass.

The idea is to build a machine that takes a sample from p_{init} and moves it, continuously and deterministically, until it has become a sample from p_{data} . If we can build such a machine, generation is: draw noise, run the machine, read off the result.

Two objects are needed, and the entire chapter is about the relationship between them. First, a *route*: a continuously varying family of distributions that begins at p_{init} and ends at p_{data} . Second, a *driver*: a rule saying, for each point of space and each instant, in which direction and how fast a particle sitting there should move. The route is a statement about populations; the driver is a statement about individuals.

Definition 11.2 (Probability path). A *probability path* is a family $(p_t)_{t \in [0,1]}$ of probability densities on $\mathbb{R}^d$, depending continuously on t . We say the path *connects* p_{init} to p_{data} if

$$p_0 = p_{\text{init}}, \quad p_1 = p_{\text{data}}.$$

Note what a probability path is *not*. It is not a curve in $\mathbb{R}^d$; it is a curve in the (infinite-dimensional) space of probability distributions on $\mathbb{R}^d$. At each fixed t the object p_t is an entire distribution — a whole cloud, not a point. As t runs from 0 to 1 that cloud deforms, continuously, from the source cloud into the data cloud.

Example 11.3 (A probability path in one dimension). Take $d = 1$, $p_{\text{init}} = \mathcal{N}(0, 1)$ and $p_{\text{data}} = \mathcal{N}(3, 1)$, and set $p_t = \mathcal{N}(3t, 1)$. At $t = 0$ this is $\mathcal{N}(0, 1) = p_{\text{init}}$; at $t = 1$ it is $\mathcal{N}(3, 1) = p_{\text{data}}$; in between it is a Gaussian bump of fixed width whose centre slides steadily to the right. That family of distributions is a probability path. Nothing has yet been said about how any individual sample moves.

Example 11.3 is worth pausing on, because it contains in miniature the distinction that the rest of the chapter depends on.

Notation 11.4 (X_t versus p_t : a random variable and its law). Throughout this chapter, and consistently with Section 5.7:

- X_t denotes a **random variable** — one particle, one concrete sample, at time t . Following it from $t = 0$ to $t = 1$ produces one *trajectory*, a curve $t \mapsto X_t$ in $\mathbb{R}^d$.
- p_t denotes a **distribution** — the law of X_t , written $X_t \sim p_t$. It is what one would see by drawing infinitely many independent particles, letting each evolve, and making a histogram at time t .

The two are related but carry very different amounts of information. From the trajectories one can always compute the marginals (take the histogram). From the marginals one *cannot* recover the trajectories: a histogram at each time says nothing about which particle went where.

This is exactly the asymmetry that Section 5.7 exploited for SDEs, and it reappears here for a completely different reason. In Example 11.3, the marginals $\mathcal{N}(3t, 1)$ are consistent with (at least) the following two entirely different stories:

1. every particle slides right at constant speed 3, i.e. $X_t = X_0 + 3t$; or
2. every particle jumps instantaneously, at a random time, from its start to a freshly drawn point of $\mathcal{N}(3t, 1)$ — an absurd but perfectly marginal-preserving prescription.

Both produce the histogram $\mathcal{N}(3t, 1)$ at every t . So:

A probability path prescribes where the mass is, not how it got there.

To generate samples, the histogram is not enough: an algorithm has to move a *particular* sample. We therefore need the second object.

11.2 Flows: how individual samples move

The device that moves individual samples is a *time-dependent vector field*. The picture to hold is a river. At every point x of space and every instant t there is a current $u_t(x) \in \mathbb{R}^d$: a direction and a speed. A leaf dropped into the river at x_0 is carried along, and its position at time t is determined by the current it has met at every instant so far. The field is not the trajectory — it is a rule that generates trajectories, one for each starting point.

Definition 11.5 (Time-dependent vector field, flow map). A *time-dependent vector field* is a map $u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$, written $u_t(x)$. Given u , the associated *flow map* $\phi_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is defined by solving, for each starting point x ,

$$\frac{d}{dt}\phi_t(x) = u_t(\phi_t(x)), \quad \phi_0(x) = x, \quad (11.1)$$

i.e. $\phi_t(x)$ is the position at time t of the particle that started at x and has been carried along by the velocity field u ever since. We assume throughout (as is standard, and guaranteed by, e.g., u_t globally Lipschitz in x uniformly in t) that this ODE has a unique solution for every x and every $t \in [0, 1]$, so that ϕ_t is well defined and, since distinct trajectories of an autonomous-in-space-at-each-fixed- t ODE cannot cross without violating uniqueness, a diffeomorphism of $\mathbb{R}^d$ for each fixed t .

Three readings of (11.1) are worth separating, because they are the same equation seen from three distances:

- **Pointwise.** $u_t(x)$ is the instantaneous velocity a particle has *if* it happens to be at x at time t . It is a local instruction, blind to where the particle came from.
- **Per trajectory.** Writing $X_t := \phi_t(X_0)$ for a particular starting point, (11.1) reads $\dot{X}_t = u_t(X_t)$: an ordinary initial value problem.

- **Globally.** For each fixed t , ϕ_t is a single map $\mathbb{R}^d \rightarrow \mathbb{R}^d$ that takes every starting point to its time- t position at once — a smooth deformation of the whole space, building up continuously from the identity $\phi_0 = \text{id}$.

Example 11.6 (A flow one can write down). Let $u_t(x) = -\theta x$ for a constant $\theta > 0$ (every particle is pulled towards the origin at a speed proportional to its distance from it). The ODE $\dot{X}_t = -\theta X_t$ is the standard linear one, with solution $X_t = e^{-\theta t} X_0$, so the flow map is

$$\phi_t(x) = e^{-\theta t} x.$$

One checks the definition directly: $\phi_0(x) = x$, and $\frac{d}{dt}\phi_t(x) = -\theta e^{-\theta t} x = -\theta \phi_t(x) = u_t(\phi_t(x))$. Geometrically, ϕ_t shrinks the whole space towards the origin by the factor $e^{-\theta t}$; every trajectory is a straight ray heading inwards, and no two of them ever meet, consistent with ϕ_t being a diffeomorphism for each t .

We now ask the population question: if the *starting points* are distributed according to p_{init} , how are the *time- t positions* distributed? The answer is the pushforward, and since it is the single piece of vocabulary this chapter cannot do without, we restate it concretely.

Notation 11.7 (Pushforward, concretely). $(\phi_t)_{\#} p_{\text{init}}$ (Definition 2.7) is defined by the following recipe, which *is* its meaning:

draw $X_0 \sim p_{\text{init}}$; compute $\phi_t(X_0)$; the distribution of the result is $(\phi_t)_{\#} p_{\text{init}}$.

Equivalently: take a very large sample from p_{init} , apply ϕ_t to every point of it, and make a histogram of what comes out. No integral has to be computed to *sample* from a pushforward; one only needs ϕ_t and the ability to sample the source. Computing its *density* is a different and much harder matter, and is exactly what Theorem 2.8 is for: the density gets divided by the local volume expansion factor $|\det D\phi_t|$, because pushing a fixed amount of mass into a larger volume lowers the density there.

Definition 11.8 (Continuous normalizing flow). Given u and a source density p_{init} , the *continuous normalizing flow (CNF)* associated with u is the probability path $p_t := (\phi_t)_{\#} p_{\text{init}}$, the pushforward (Definition 2.7) of p_{init} under the flow map.

Equivalently, and operationally: draw $X_0 \sim p_{\text{init}}$, solve $\dot{X}_t = u_t(X_t)$ forward to $t = 1$; then $X_1 \sim p_1$. This is precisely Definition 11.5's flow map restated with full definitional care, and it is worth being explicit about what has *not* yet been claimed: nothing so far says $p_1 = p_{\text{data}}$ for any *particular* choice of u , only that *some* probability path p_t is generated by *any* choice of (well-behaved) u_t . We have a machine that produces paths from fields; what generation needs is the opposite — a way to produce a field from a *desired* path. The next section supplies the bridge between the two directions.

11.3 The continuity equation

11.3.1 One dimension first: probability as a conserved fluid

The equation linking a path to a field is not a machine-learning construction. It is conservation of mass, and it is worth recovering it in one dimension by counting, before meeting it in its general form.

Section 5.8 already ran this argument once, on the way to Fokker–Planck (see (5.13)); we repeat it here in integral form, both because this chapter rests on it and because the integral version makes the conservation content unmistakable.

Fix an interval $[a, b] \subset \mathbb{R}$ and ask how the amount of probability inside it,

$$M(t) := \int_a^b p_t(x) dx,$$

changes with time. Probability is neither created nor destroyed: particles do not appear from nowhere, and each one has weight 1 forever. So M can change only because particles *cross the two endpoints*.

How fast does mass cross the point b ? In a short time Δt , the particles that cross b from the left are exactly those that were within distance $u_t(b)\Delta t$ of b , of which there are approximately $p_t(b) \cdot u_t(b)\Delta t$. Dividing by Δt : mass leaves through b at rate $p_t(b)u_t(b)$. The same count at a gives an inflow rate $p_t(a)u_t(a)$. Hence

$$\frac{d}{dt} \int_a^b p_t(x) dx = \underbrace{p_t(a)u_t(a)}_{\text{in at } a} - \underbrace{p_t(b)u_t(b)}_{\text{out at } b} = - \left[p_t(x)u_t(x) \right]_a^b. \quad (11.2)$$

The product $p_t u_t$ that appears here is the central quantity, and it has a name.

Definition 11.9 (Probability flux). The *probability flux* (or current) of the pair (p_t, u_t) is $j_t := p_t u_t$: the amount of probability crossing a given point, per unit time, in the direction of u_t . It is a product of “how much stuff is here” and “how fast that stuff is moving”.

To pass from (11.2) to a statement at a single point, write the right-hand side as an integral of a derivative, $[j_t]_a^b = \int_a^b \partial_x j_t(x) dx$, and move everything under one integral sign:

$$\int_a^b \left(\partial_t p_t(x) + \partial_x j_t(x) \right) dx = 0.$$

This holds for *every* interval $[a, b]$. A continuous function whose integral vanishes over every interval must vanish identically (if it were, say, positive at some x_0 , it would be positive on a small interval around x_0 by continuity, and its integral there would be positive). Hence, pointwise,

$$\partial_t p_t(x) + \partial_x (p_t(x)u_t(x)) = 0. \quad (11.3)$$

11.3.2 The general equation, term by term

In $\mathbb{R}^d$ the same argument is run on a small box instead of an interval, with mass crossing each of its $2d$ faces; the sum of the outflows through opposite faces is what the divergence measures, and one obtains (11.4) below. We now state the result properly, together with its converse — which is the direction generation actually needs.

Theorem 11.10 (Continuity equation). *Let u be a time-dependent vector field with flow ϕ_t (Definition 11.5), and let $p_t = (\phi_t)_\# p_{\text{init}}$ (Definition 11.8). Then, wherever p_t is differentiable,*

$$\partial_t p_t(x) + \nabla \cdot (u_t(x)p_t(x)) = 0, \quad \nabla \cdot v := \sum_{k=1}^d \partial_{x_k} v_k \text{ (divergence)}. \quad (11.4)$$

Conversely, if a smooth, compactly-supported-density path (p_t) and vector field u_t satisfy (11.4), then u_t generates (p_t) in the sense of Definition 11.8 (given standard regularity sufficient for existence/uniqueness of the flow, Definition 11.5).

Before the proof, the reading of (11.4). Each symbol has a physical job:

Term	Is	Says
$p_t(x)$	density	how much probability sits at x
$u_t(x)$	velocity	where the stuff at x is heading, and how fast
$p_t(x)u_t(x)$	flux	how much probability crosses x per unit time
$\nabla \cdot (p_t u_t)$	net outflow rate	how much more leaves x than arrives
$\partial_t p_t(x)$	rate of change	how fast the density at x is rising

so that (11.4) says, in words:

the density at a point rises exactly as fast as probability flows in, minus what flows out.

If more leaves than arrives ($\nabla \cdot j_t > 0$), the density falls. That is the whole content. There is no probabilistic input beyond the fact that total mass is 1 and stays there.

Proof. (Forward direction.) By Theorem 2.8 applied to the diffeomorphism ϕ_t ,

$$p_t(x) = p_{\text{init}}(\phi_t^{-1}(x)) |\det D\phi_t^{-1}(x)|.$$

Write $J_t(x) := \det D\phi_t(\phi_t^{-1}(x))$, so $|\det D\phi_t^{-1}(x)| = 1/J_t(x)$ (inverse function theorem, and $J_t > 0$ since ϕ_t is orientation-preserving as the flow of a continuous vector field starting at the identity, $\phi_0 = \text{id}$, $\det D\phi_0 = 1 > 0$, and J_t cannot cross zero continuously without the flow ceasing to be a diffeomorphism). We use *Liouville's formula* for the rate of change of this Jacobian along the flow,

$$\frac{d}{dt} \log J_t(\phi_t(x_0)) = (\nabla \cdot u_t)(\phi_t(x_0)), \quad (11.5)$$

which we justify as follows: differentiating $J_t(\phi_t(x_0)) = \det D\phi_t(x_0)$ in t , using the matrix identity $\frac{d}{dt} \det A(t) = \det A(t) \operatorname{tr}(A(t)^{-1} \dot{A}(t))$ (Jacobi's formula, a standard fact of matrix calculus we take as given) with $A(t) = D\phi_t(x_0)$, and $\dot{A}(t) = D(\partial_t \phi_t(x_0)) = D(u_t(\phi_t(x_0))) = Du_t(\phi_t(x_0)) \cdot D\phi_t(x_0) = Du_t(\phi_t(x_0)) A(t)$ (chain rule, differentiating the defining ODE of Definition 11.5 with respect to the initial condition x_0 , and using that spatial and initial-condition derivatives commute for a smooth flow), gives $\operatorname{tr}(A(t)^{-1} \dot{A}(t)) = \operatorname{tr}(Du_t(\phi_t(x_0))) = (\nabla \cdot u_t)(\phi_t(x_0))$ (the trace of the Jacobian matrix of a vector field is, by definition, its divergence), yielding (11.5) directly.

Now compute $\partial_t p_t(x)$ along a *fixed* x by relating it to the material (Lagrangian) derivative along the flow. Write $q(t) := p_t(\phi_t(x_0)) J_t(\phi_t(x_0))$ for fixed x_0 ; since $p_t(\phi_t(x_0)) = p_{\text{init}}(x_0)/J_t(\phi_t(x_0))$ (the change-of-variables formula above, evaluated at $x = \phi_t(x_0)$, using $\phi_t^{-1}(\phi_t(x_0)) = x_0$), we get $q(t) = p_{\text{init}}(x_0)$, constant in t . Differentiating $q(t) = p_t(\phi_t(x_0)) J_t(\phi_t(x_0))$ using the product rule and the chain rule (treating $p_t(\phi_t(x_0))$'s time-derivative as $\partial_t p_t(\phi_t(x_0)) + \nabla p_t(\phi_t(x_0)) \cdot u_t(\phi_t(x_0))$, the material derivative), and using (11.5) for $\dot{J}_t(\phi_t(x_0)) = J_t(\phi_t(x_0))(\nabla \cdot u_t)(\phi_t(x_0))$, gives, after dividing by $J_t(\phi_t(x_0)) \neq 0$ and evaluating at the (arbitrary) point $x = \phi_t(x_0)$,

$$0 = \partial_t p_t(x) + \nabla p_t(x) \cdot u_t(x) + p_t(x) (\nabla \cdot u_t)(x) = \partial_t p_t(x) + \nabla \cdot (p_t(x) u_t(x)),$$

using the product rule for divergence, $\nabla \cdot (pu) = \nabla p \cdot u + p \nabla \cdot u$, in the last step. This is (11.4).

(Converse direction.) We do not prove the converse in full generality here (it is a uniqueness statement for the linear transport PDE (11.4), following from the method of characteristics: any two solutions of (11.4) with the same initial condition p_0 agree, because (11.4) is exactly the statement that mass is conserved along the characteristics $\dot{x} = u_t(x)$, so the unique solution is obtained by transporting p_0 's mass along those characteristics, which is precisely $(\phi_t)_\# p_0$); see, e.g., Villani, *Optimal Transport: Old and New*, 2009, §4, or Chen et al., *Neural Ordinary Differential Equations*, NeurIPS 2018, §2 for the specific CNF framing. $\square$

Theorem 11.10 is the ODE analogue of a Fokker–Planck equation (indeed Theorem 6.9's proof sketch used exactly this fact, citing this theorem for the diffusionless special case that appears when the SDE's diffusion term is cancelled by the specific drift construction there), and it is the tool that converts “I want to transport p_{init} to p_{data} ” into a concrete, checkable condition on u_t : any (p_t, u_t) pair satisfying (11.4), with $p_0 = p_{\text{init}}$, generates a CNF ending, at $t = 1$, at whatever p_1 the pair specifies.

11.3.3 Two descriptions of one motion

It is worth setting the two levels side by side, because every subsequent argument in this chapter moves between them, and because the same pairing appeared in Part III for the stochastic case.

	Particle view	Density view
Object	one sample $X_t \in \mathbb{R}^d$	the whole law p_t
Equation	$\dot{X}_t = u_t(X_t)$	$\partial_t p_t + \nabla \cdot (p_t u_t) = 0$
Type	ODE, d unknowns	PDE, one unknown function
Randomness	only in X_0	none: p_t is deterministic
What you simulate	this one, numerically	never, in practice
Analogue in Part III	$\dot{X} = f + g\dot{W}$	Fokker–Planck (5.14)

The pairing is exact: the continuity equation is what the Fokker–Planck equation (5.14) reduces to when the diffusion coefficient g vanishes, since the Laplacian term $\frac{1}{2}g^2\Delta p_t$ disappears and only the transport term survives. Deterministic transport is diffusion with the noise turned off, *at the level of densities*. (It is *not* the same at the level of paths, and Section 6.4.1 is the discussion of why those two statements are compatible.)

We can now say precisely what it means for a field to implement a path.

Definition 11.11 (Generating a probability path). A vector field u_t *generates* the probability path (p_t) if the pair (p_t, u_t) satisfies the continuity equation (11.4) for all $t \in [0, 1]$ — equivalently, by Theorem 11.10, if pushing p_0 forward along u 's flow reproduces p_t at every time.

11.3.4 A path does not determine its field

A natural but false expectation, worth killing early, is that Definition 11.11 pins u_t down: that to each probability path there belongs *the* vector field generating it. It does not, and the counterexample is elementary.

Example 11.12 (Two fields, one path). Work in $\mathbb{R}^2$ and take the *constant* path $p_t \equiv \mathcal{N}(0, I_2)$ for all t — the distribution simply does not move. Two vector fields generate it.

The obvious one: $u_t \equiv 0$. Nothing moves at all, so certainly the law does not change, and $\partial_t p_t = 0 = \nabla \cdot (p_t \cdot 0)$.

A less obvious one: the rigid rotation $w(x) = (-x_2, x_1)$. Every particle circles the origin at constant angular speed. Since $\nabla \cdot w = \partial_{x_1}(-x_2) + \partial_{x_2}(x_1) = 0$ and $\nabla p_t(x) = -x p_t(x)$ for the standard Gaussian,

$$\begin{aligned} \nabla \cdot (p_t w) &= \underbrace{\nabla p_t \cdot w}_{=-p_t \langle x, (-x_2, x_1) \rangle} + \underbrace{p_t \nabla \cdot w}_{=0} = 0, \\ &= -p_t \langle x, (-x_2, x_1) \rangle = -p_t(-x_1 x_2 + x_2 x_1) = 0 \end{aligned}$$

so the continuity equation holds here too, with $\partial_t p_t = 0$. (The first bracket vanishes because a rotation field is everywhere orthogonal to the radial direction, and a rotationally symmetric density varies only radially.) This computation is confirmed numerically in Section 11.19, check 10.

The two fields could hardly be more different: under the first, every sample stays put forever; under the second, every sample travels a full circle. Yet they produce exactly the same p_t at every t .

The general statement behind Example 11.12 is immediate from the linearity of (11.4): if u_t generates (p_t) and w_t satisfies $\nabla \cdot (p_t w_t) = 0$ (a “ p_t -divergence-free” field, one that stirs the mass around without changing the density anywhere), then $u_t + w_t$ generates (p_t) as well. In $\mathbb{R}^d$ with $d \geq 2$ there are infinitely many such w_t .

Caution 11.13 (What is, and is not, determined). Do not read u_t as “the velocity field of the path p_t ”. The correct reading is: u_t is a field compatible with the path. Three consequences matter later.

First, this is not a defect. For generation, any compatible field works: all of them push p_{init} to p_{data} , and that is all we asked for. They differ in the trajectories they take, not in the endpoint distribution.

Second, it means that “choose a probability path” does *not* finish the design. What flow matching actually fixes, as Section 11.6 will show, is a *conditional construction*, and that construction singles out one particular field — the one obtained by posterior-averaging the conditional fields. Two different conditional constructions with the same marginal path can yield different marginal fields.

Third, in $d = 1$ the ambiguity largely disappears, which is why one-dimensional intuition can mislead here. Indeed $\partial_x(p_t w_t) = 0$ forces $p_t w_t \equiv c$ for a constant c ; if w_t is bounded then $p_t w_t \rightarrow 0$ at $\pm\infty$ (as p_t is integrable), so $c = 0$ and $w_t \equiv 0$ wherever $p_t > 0$. The escape is to allow $w_t = c/p_t$, which is unbounded — so the statement is “no bounded non-trivial ambiguity”, not “no ambiguity”. In $d \geq 2$ no such argument is available, and Example 11.12’s field is perfectly bounded on any compact set.

11.4 CNF likelihood: the instantaneous change of variables

A byproduct of Theorem 11.10’s proof is worth recording separately, since Section 13.9 in Chapter 13 needs it, and since it is what makes the word “normalizing” in “continuous normalizing flow” mean something.

Corollary 11.14 (Instantaneous change of variables). *Along a trajectory $X_t = \phi_t(X_0)$ of the flow, the log-density evolves by*

$$\frac{d}{dt} \log p_t(X_t) = -(\nabla \cdot u_t)(X_t).$$

Proof. This is exactly (11.5) combined with $p_t(\phi_t(x_0)) = p_{\text{init}}(x_0)/J_t(\phi_t(x_0))$ from the proof of Theorem 11.10: taking log of both sides, $\log p_t(\phi_t(x_0)) = \log p_{\text{init}}(x_0) - \log J_t(\phi_t(x_0))$, and differentiating in t using (11.5) for the second term gives the claim directly. $\square$

The minus sign is the whole intuition, and it is worth spelling out rather than checking. The divergence of a velocity field measures the rate at which a small blob of space *expands* as it is carried along:

- $\nabla \cdot u_t(x) > 0$: neighbouring particles are separating, so a blob around x is growing. It contains a fixed amount of probability — the same particles — spread over more volume, so the density it experiences *falls*. Hence $\frac{d}{dt} \log p_t < 0$.
- $\nabla \cdot u_t(x) < 0$: the blob is being compressed, the same mass occupies less room, and the density *rises*.
- $\nabla \cdot u_t \equiv 0$: an incompressible flow. Mass is stirred around, but the density each particle sits in never changes. This is exactly the rotation field of Example 11.12, which is why adding it changed nothing.

Corollary 11.14 says that, given the vector field u_t and the initial log-density $\log p_{\text{init}}(X_0)$, the log-density of the sample $X_1 = \phi_1(X_0)$ can be recovered *exactly* by integrating the (scalar!) divergence $-\nabla \cdot u_t$ along the same trajectory the sample itself follows — no separate, learned density network is needed, only the already-learned vector field and one additional scalar ODE run alongside the original one. Evaluating $\nabla \cdot u_t$ exactly requires the trace of a $d \times d$ Jacobian, $\mathcal{O}(d)$ backward passes if done naively; in practice this is estimated stochastically via the *Hutchinson trace estimator*,

$$\nabla \cdot u_t(x) = \text{tr}(Du_t(x)) = \mathbb{E}_\varepsilon [\varepsilon^\top Du_t(x) \varepsilon]$$

for any ε with $\mathbb{E}[\varepsilon \varepsilon^\top] = I$ (e.g. $\varepsilon \sim \mathcal{N}(0, I)$ or Rademacher), computable with a single Jacobian-vector product per sample of ε rather than a full Jacobian; we record this identity (a direct consequence of linearity of trace and expectation, $\mathbb{E}[\varepsilon^\top A \varepsilon] = \mathbb{E}[\text{tr}(A \varepsilon \varepsilon^\top)] = \text{tr}(A \mathbb{E}[\varepsilon \varepsilon^\top]) = \text{tr}(A)$) without further development, since Section 13.9 records only whether this machinery is implemented for SigmaFlow, not an independent numerical study of it.

11.4.1 Why this is not how CNFs are trained any more

Corollary 11.14 suggests an obvious training scheme, and it is the one CNFs were originally trained with: parameterise u_θ , and maximise the log-likelihood of the training data under the resulting model,

$$\log p_1^\theta(x) = \log p_{\text{init}}(\phi_1^{-1}(x)) + \int_1^0 (\nabla \cdot u_\theta)(X_\tau, \tau) d\tau,$$

obtained by integrating Corollary 11.14 backwards from the data point. This is mathematically clean and it is what makes CNFs *likelihood* models. It is also expensive in a specific way that matters:

- every gradient step requires *simulating the ODE* — a full trajectory, i.e. many network evaluations, forward and then differentiated through (or via an adjoint solve);
- and each step of that simulation requires a *divergence*, i.e. a trace of the network’s Jacobian, exactly or stochastically.

Flow matching removes both costs at once. It never simulates a trajectory during training and never touches a divergence: as Section 11.6 will show, one training example is a single point (t, x, target) that can be constructed in closed form, and the loss is an ordinary mean-squared error. This is what the literature means by calling flow matching *simulation-free*.

Caution 11.15 (What is given up). The trade is not free, and it is worth stating precisely rather than advertising flow matching as strictly better. Maximum-likelihood CNF training optimises exactly the quantity one usually wants to report (the data log-likelihood). Flow matching optimises a regression loss whose minimiser is the correct vector field, but whose *value* is not a likelihood and is not even monotonically related to one in any way this document establishes. Likelihoods remain *computable* after flow matching training — Corollary 11.14 does not care how u_θ was obtained — they are simply not what was optimised.

11.5 The marginal vector field, and why we cannot write it down

Suppose, for a moment, that somebody handed us a vector field u_t generating a path from p_{init} to p_{data} . Generation would be finished:

1. draw $X_0 \sim p_{\text{init}}$
2. solve $\dot{X}_t = u_t(X_t)$ from $t = 0$ to $t = 1$
3. return X_1 , which is distributed as p_{data}

so the entire problem of generative modelling has been reduced to *knowing one time-dependent vector field*. We do not know it, but we have a universal function approximator, so the obvious move is to learn it: introduce a network $u_\theta(x, t)$ and fit it by least squares,

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}(0,1), X_t \sim p_t} \left[\|u_\theta(X_t, t) - u_t(X_t)\|^2 \right]. \quad (11.6)$$

This is the objective we would *like* to minimise, and the reader should note that it is not yet an algorithm: it mentions $u_t(X_t)$, the very thing we do not have. Two separate obstructions sit in (11.6):

1. **We have no formula for u_t .** Even after choosing a probability path, Section 11.3.4 showed that the path does not hand us a field, and Section 11.6 will show that the natural candidate is an integral over all of p_{data} .
2. **We cannot even sample $X_t \sim p_t$,** for a general path, without already knowing how to sample intermediate distributions.

The structure of this difficulty — “the marginal object is the right target but is an intractable integral” — is not new. It is exactly the obstruction Section 6.2 met when trying to learn a score: the marginal score $\nabla_x \log p_\sigma(x)$ was unavailable, while the *conditional* score $\nabla_x \log p_\sigma(x | z)$ was a one-line formula. The resolution here will be the same resolution, and it rests on the same elementary tool, Proposition 2.3. A reader who found the denoising-score-matching argument convincing already knows the shape of what follows.

11.6 Conditional probability paths

The trick is to stop trying to build a path from the whole source cloud to the whole data cloud in one go, and instead build one path per data point.

Fix a single training example $z \in \mathbb{R}^d$ — one molecule, one image, one pose. Ask for a path that starts at the full source distribution and *collapses onto that one point*. Such a path is easy to write down (the next section gives the standard one in a line), because it involves no unknown distribution: p_{init} is known and z is a number we are holding in our hand.

Definition 11.16 (Conditional probability path, conditional vector field). A *conditional probability path* is a family $p_t(\cdot | z)$, $z \in \mathbb{R}^d$, with $p_0(\cdot | z) = p_{\text{init}}$ and $p_t(\cdot | z) \rightarrow \delta_z$ weakly as $t \rightarrow 1$ (a Dirac mass at z), for every z . A *conditional vector field* $u_t(\cdot | z)$ generates $p_t(\cdot | z)$ in the sense of Theorem 11.10, for each fixed z and each $t \in [0, 1)$.

Read the two boundary conditions as the two things we need: $p_0(\cdot | z) = p_{\text{init}}$ says every conditional path starts from the *same* place, the source we can sample; and $p_1(\cdot | z) = \delta_z$ says it ends on the one data point we conditioned on.

Remark 11.17 (The first boundary condition is an independence assumption). The requirement $p_0(\cdot | z) = p_{\text{init}}$ for every z is stronger than it looks: it says the starting distribution does not depend on which data point we are heading for, i.e. the source sample and the data sample are drawn *independently*. This is the *independent coupling*, and it is an assumption, not a necessity — Section 11.16 takes it apart and shows what else is possible. It is worth flagging now because it is easy to absorb Definition 11.16 without noticing that a modelling choice was made in it.

Now the key construction. Each conditional path is a route to one data point; *mixing* them, weighted by how likely each data point is, produces a route to the whole data distribution.

Definition 11.18 (The marginal path induced by a conditional path). Given a conditional path $p_t(\cdot | z)$, the induced *marginal probability path* is

$$p_t(x) := \int p_t(x | z) p_{\text{data}}(z) dz. \quad (11.7)$$

By Proposition 2.3, (11.7) is exactly the law of the following two-step sampling procedure:

draw a data point $z \sim p_{\text{data}}$; then draw $x \sim p_t(\cdot | z)$; then forget z .

That sentence is the operational meaning of the integral, and it is also the reason the construction is useful: *we can execute it*. Drawing z means picking a random element of the training set; drawing $x \sim p_t(\cdot | z)$ means evaluating a closed-form formula. So even though the *density* $p_t(x)$ in (11.7) is an intractable integral, *sampling* from p_t is trivial. That asymmetry — easy to sample, impossible to evaluate — is what the whole method is built on, and it is worth stating as a slogan:

We can draw from p_t ; we just cannot write it down.

The boundary behaviour is automatic, and this is where the construction pays off. At $t = 0$, every conditional path equals p_{init} , so their p_{data} -weighted mixture is p_{init} too:

$$p_0(x) = \int p_{\text{init}}(x) p_{\text{data}}(z) dz = p_{\text{init}}(x) \underbrace{\int p_{\text{data}}(z) dz}_{=1} = p_{\text{init}}(x).$$

At $t = 1$, each conditional path has collapsed onto its own z , so the mixture is the distribution of z itself:

$$p_1(x) = \int \delta_z(x) p_{\text{data}}(z) dz = p_{\text{data}}(x).$$

The marginal path therefore connects exactly what we wanted it to connect, and we obtained it without ever designing it directly.

Example 11.19 (What the mixture looks like). Take p_{data} to be two point masses, at a and at $-a$, with probability $\frac{1}{2}$ each, and let each conditional path be a Gaussian blob shrinking onto its target. Then p_t is, at every intermediate time, a *two-component Gaussian mixture*. At t near 0 the two blobs are wide and sit almost on top of each other, so the mixture looks like a single cloud; as t grows they narrow and separate, and near $t = 1$ the mixture has visibly split into two spikes. Nothing designed that splitting; it is what mixing collapse-to-a-point paths automatically does. This exact example is carried through in closed form in Section 11.10.3.

Caution 11.20 (The endpoint $t = 1$ is a limit, not a density). δ_z is a measure, not a density with respect to Lebesgue measure, so strictly speaking $p_1(\cdot | z)$ is outside the framework of Definition 2.1, and Theorem 11.10 — which requires p_t to be differentiable — does not apply at $t = 1$. Everything below is therefore stated on $[0, 1)$ and extended to $t = 1$ by continuity. Two remarks make this harmless, and one makes it practically important.

Harmless: the marginal path is still well defined, since $p_t(x) = \int p_t(x | z) p_{\text{data}}(z) dz \rightarrow p_{\text{data}}(x)$ as $t \rightarrow 1$ in the same weak sense; and the training loss $\mathcal{L}_{\text{CFM}}$ samples $t \sim \mathcal{U}(0, 1)$, so $t = 1$ has probability zero.

Practically important: the conditional field (11.13) has a $1/(1 - t)$ factor that blows up as $t \rightarrow 1$. Implementations therefore either use the equivalent endpoint form $z - x_0$ (which is finite everywhere — see the discussion after (11.13)), or replace the Dirac endpoint by a narrow Gaussian $\mathcal{N}(z, \sigma_{\min}^2 I)$ with $\sigma_{\min} > 0$, as in Lipman et al. (2023).

SigmaFlow takes the first route. Its `r3_flow_matcher.py` accepts a $\sigma_{\min}$ parameter, but the parameter is stored and never read (Section 13.3 verifies this against the source), and its default is 0; the endpoint really is a Dirac, and the $1/(1 - t)$ singularity is avoided purely by using the endpoint form of the target. This is legitimate, but it means the sampler must not be integrated to exactly $t = 1$ with the current-point form — a point taken up in Section 13.8.

11.7 The marginalisation theorem

We have a marginal path. What we need is a field that generates it. The conditional paths come with conditional fields (available in closed form, as the next section shows), so the question is how to assemble a marginal field out of conditional ones. The answer is not “average them” — it is “average them *with the right weights*”, and identifying the right weights is the content of the theorem.

Here is the reasoning that produces the formula. Suppose a particle of the marginal process is observed at position x at time t . It came from *some* conditional path, but we cannot see which; the data point z that path was heading for is hidden. What we can compute is how plausible each candidate z is, given the evidence that the particle is at x . That is Bayes’ rule:

$$\underbrace{\mathbb{P}(z | x, t)}_{\text{posterior over data points}} = \frac{\overbrace{p_t(x | z)}^{\text{likelihood}} \overbrace{p_{\text{data}}(z)}^{\text{prior}}}{\underbrace{p_t(x)}_{\text{evidence}}}. \quad (11.8)$$

If the particle is to behave correctly on average, it should move with the velocity that its possible origins would give it, weighted by how likely each origin is. That is a posterior expectation, and it is the formula below.

Theorem 11.21 (Marginalisation of vector fields). *Let $u_t(\cdot | z)$ generate the conditional path $p_t(\cdot | z)$ for every z . Then the marginal vector field*

$$u_t(x) := \int u_t(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz = \mathbb{E}[u_t(x | Z) | X_t = x] \quad (11.9)$$

(a $p_t(x)$ -weighted average of the conditional fields, i.e. the posterior expectation of $u_t(x | Z)$ given that the marginal path is at x at time t , treating $Z \sim p_{\text{data}}$ as a latent variable and $X_t \sim p_t(\cdot | Z)$) generates the marginal path (p_t) .

Proof. It suffices, by Theorem 11.10, to check that (p_t, u_t) satisfies the continuity equation (11.4). Each conditional pair satisfies it: $\partial_t p_t(x | z) + \nabla \cdot (u_t(x | z) p_t(x | z)) = 0$ for every z . Multiply by $p_{\text{data}}(z)$ and integrate over z :

$$\underbrace{\partial_t \int p_t(x | z) p_{\text{data}}(z) dz}_{=: p_t(x)} + \nabla \cdot \underbrace{\int u_t(x | z) p_t(x | z) p_{\text{data}}(z) dz}_{=: V(x)} = 0$$

(differentiation and the divergence operator both commute with integration over z , a standard application of dominated convergence, which we do not belabour). It remains to check $V(x) = u_t(x) p_t(x)$ with u_t as defined in (11.9): indeed, $V(x) = \int u_t(x | z) p_t(x | z) p_{\text{data}}(z) dz = p_t(x) \int u_t(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz = p_t(x) u_t(x)$, using the definition of $u_t(x)$ verbatim. Hence $\partial_t p_t(x) + \nabla \cdot (u_t(x) p_t(x)) = 0$, i.e. (p_t, u_t) satisfies (11.4), and Theorem 11.10's converse direction gives the claim. $\square$

Notice where the weighting came from in that proof. It was *not* chosen for interpretability; it is forced. The continuity equation is linear in the pair (p, pu) — in the density and the *flux*, not in the density and the velocity. Mixing conditional paths therefore mixes fluxes, $j_t = \int j_t(\cdot | z) p_{\text{data}}(z) dz$, and recovering a velocity from a flux means dividing by the density, $u_t = j_t / p_t$. The $p_t(x)$ in the denominator of (11.9) is that division, and the posterior interpretation is a consequence, not the motivation.

Caution 11.22 (Velocities do not mix; fluxes do). The naive guess $u_t(x) \stackrel{?}{=} \int u_t(x | z) p_{\text{data}}(z) dz$ — average the conditional velocities under the *prior* — is wrong. It ignores that some data points are far more plausible origins for a particle sitting at x than others. The correct weight is the posterior (11.8), which depends on x and t . As $t \rightarrow 1$ the posterior sharpens onto the nearest data point, and the marginal field converges to that point's conditional field; Section 11.10.3 exhibits this concretely.

The marginal field (11.9), like the marginal score of Section 6.2, is an intractable integral: it requires the unknown $p_t(x)$ in its own denominator and an integral over the whole data distribution in its numerator. We are, apparently, back where Section 11.5 left us. The next section shows that we are not.

11.8 Conditional expectation as an L^2 projection

This section contains no flow matching at all. It is a fact about least-squares regression, it is elementary, and it is the reason the entire method works. Every “the conditional loss has the

same minimiser as the marginal loss” argument in this document — denoising score matching in Theorem 6.4, conditional flow matching below, and its Riemannian version in Chapter 12 — is this one fact, wearing different clothes.

The setting: we have two random quantities, X (what the network sees) and Y (what it is asked to predict), and we fit a function f by minimising squared error. The question is what function we get.

Lemma 11.23 (Orthogonal decomposition of the squared error). *Let X and Y be random variables (with Y taking values in $\mathbb{R}^d$ and $\mathbb{E}\|Y\|^2 < \infty$), and let f be any measurable function of X with $\mathbb{E}\|f(X)\|^2 < \infty$. Then*

$$\mathbb{E}\|f(X) - Y\|^2 = \underbrace{\mathbb{E}\|f(X) - \mathbb{E}[Y | X]\|^2}_{\text{depends on } f} + \underbrace{\mathbb{E}\|Y - \mathbb{E}[Y | X]\|^2}_{\text{does not depend on } f}. \quad (11.10)$$

Consequently the minimiser over all measurable f is

$$f^*(X) = \mathbb{E}[Y | X],$$

and the minimum value is the second term, which is strictly positive unless Y is a deterministic function of X .

Proof. The entire proof is one addition and one subtraction. Write

$$Y - f(X) = \underbrace{(Y - \mathbb{E}[Y | X])}_{=:A} + \underbrace{(\mathbb{E}[Y | X] - f(X))}_{=:B}, \quad (11.11)$$

which is an identity, the term $\mathbb{E}[Y | X]$ having been added and subtracted. Expanding the square,

$$\mathbb{E}\|Y - f(X)\|^2 = \mathbb{E}\|A\|^2 + 2\mathbb{E}\langle A, B \rangle + \mathbb{E}\|B\|^2,$$

so everything hinges on the cross term $\mathbb{E}\langle A, B \rangle$, and the claim is that it vanishes.

To see this, condition on X and use the tower property $\mathbb{E}[\cdot] = \mathbb{E}[\mathbb{E}[\cdot | X]]$:

$$\mathbb{E}\langle A, B \rangle = \mathbb{E}\left[\mathbb{E}[\langle A, B \rangle | X]\right].$$

Inside the inner expectation, X is known, hence $B = \mathbb{E}[Y | X] - f(X)$ is a *fixed vector* — both of its terms are functions of X alone. A fixed vector can be pulled out of an expectation, so

$$\mathbb{E}[\langle A, B \rangle | X] = \langle \mathbb{E}[A | X], B \rangle.$$

And the remaining inner expectation is zero:

$$\mathbb{E}[A | X] = \mathbb{E}[Y - \mathbb{E}[Y | X] | X] = \mathbb{E}[Y | X] - \mathbb{E}[Y | X] = 0,$$

again because $\mathbb{E}[Y | X]$ is a function of X and therefore passes through the conditional expectation unchanged. Hence $\mathbb{E}\langle A, B \rangle = 0$, and (11.10) follows by substituting $\mathbb{E}\|A\|^2$ and $\mathbb{E}\|B\|^2$ back in.

For the consequence: in (11.10) only the first term involves f , and it is a squared norm, hence ≥ 0 , with equality exactly when $f(X) = \mathbb{E}[Y | X]$ almost surely. $\square$

Remark 11.24 (Why “projection”, and why “Pythagoras”). The three quantities in (11.11) form a right triangle in the space of square-integrable random variables, with inner product $\langle U, V \rangle_{L^2} = \mathbb{E}\langle U, V \rangle$. The functions of X form a linear subspace of that space. $\mathbb{E}[Y \mid X]$ is the point of that subspace closest to Y , i.e. the orthogonal projection of Y onto it, and $A = Y - \mathbb{E}[Y \mid X]$ is the residual, perpendicular to the subspace — which is precisely the statement $\mathbb{E}\langle A, B \rangle = 0$ for every B in the subspace. Equation (11.10) is then literally Pythagoras’ theorem for that triangle. Nothing about this is specific to regression: it is the same geometry as least-squares fitting in linear algebra, with “orthogonal to the column space” replaced by “uncorrelated with every function of X ”.

The lemma has an immediate and, at first, slightly startling reading.

A least-squares regression cannot help but return a conditional expectation. If you regress onto a noisy target, you get its mean, whether you wanted it or not.

That is normally regarded as a nuisance — it is why regression models blur. Flow matching turns it into the mechanism: we *want* a conditional expectation, namely (11.9), so we deliberately regress onto a target whose conditional mean is the thing we cannot compute.

11.9 The conditional flow matching loss

Everything is now in place. We regress the network onto the *conditional* field — which is available in closed form — and the lemma delivers the *marginal* field, which is not.

Theorem 11.25 (Conditional flow matching). *Define*

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}(0,1), x \sim p_t} [\|u_\theta(x, t) - u_t(x)\|^2], \quad \mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, z \sim p_{\text{data}}, x \sim p_t(\cdot \mid z)} [\|u_\theta(x, t) - u_t(x \mid z)\|^2].$$

Then

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathcal{L}_{\text{FM}}(\theta) + C, \quad C \geq 0 \text{ independent of } \theta;$$

the two losses have identical gradients and minimisers. In particular the minimiser of $\mathcal{L}_{\text{CFM}}$ over all measurable functions is $u_{\theta^}(x, t) = u_t(x)$, the marginal field of (11.9).*

Let us be completely explicit about what the three random quantities in $\mathcal{L}_{\text{CFM}}$ are, since the whole argument turns on their joint law:

Symbol	Drawn from	Meaning
t	$\mathcal{U}(0, 1)$	a time, uniformly at random
Z	p_{data}	a training example, i.e. a random row of the dataset
X_t	$p_t(\cdot \mid Z)$	a point on the conditional path towards that example

The network sees (X_t, t) and is scored against $u_t(X_t \mid Z)$. Crucially it does *not* see Z : the data point is used to build the target and then withheld. That is the whole asymmetry, and it is what makes the conditional expectation in Lemma 11.23 non-trivial.

Proof via Lemma 11.23. Apply Lemma 11.23 with

$$X := (X_t, t) \quad \text{and} \quad Y := u_t(X_t | Z),$$

the joint law being: $t \sim \mathcal{U}(0, 1)$, $Z \sim p_{\text{data}}$, $X_t \sim p_t(\cdot | Z)$. Note that under this joint law the marginal law of X_t given t is exactly p_t , by Proposition 2.3 and Definition 11.18 — this is the same identification used in Theorem 6.4’s proof.

The lemma gives, for any u_θ ,

$$\underbrace{\mathbb{E} \|u_\theta(X_t, t) - u_t(X_t | Z)\|^2}_{=\mathcal{L}_{\text{CFM}}(\theta)} = \underbrace{\mathbb{E} \|u_\theta(X_t, t) - \mathbb{E}[u_t(X_t | Z) | X_t, t]\|^2}_{(*)} + \underbrace{\mathbb{E} \|u_t(X_t | Z) - \mathbb{E}[u_t(X_t | Z) | X_t, t]\|^2}_{=:C},$$

and C manifestly does not involve θ . By Theorem 11.21, the inner conditional expectation is exactly the marginal field:

$$\mathbb{E}[u_t(X_t | Z) | X_t = x, t] = u_t(x),$$

so $(*) = \mathbb{E} \|u_\theta(X_t, t) - u_t(X_t)\|^2 = \mathcal{L}_{\text{FM}}(\theta)$. That is exactly $\mathcal{L}_{\text{CFM}} = \mathcal{L}_{\text{FM}} + C$, with $C \geq 0$ since it is an expected squared norm; the statement about the minimiser is the “consequently” clause of Lemma 11.23. $\square$

Remark 11.26 (The same theorem, proved by brute force). The identity can also be obtained without ever mentioning conditional expectations, by expanding both squares and comparing term by term — exactly as Theorem 6.4 was proved. The quadratic terms $\mathbb{E} \|u_\theta\|^2$ agree because $x \sim p_t$ and $(z \sim p_{\text{data}}, x \sim p_t(\cdot | z))$ are the same law for x ; the target-only terms are absorbed into C ; and the two cross terms agree because of the flux identity $p_t(x)u_t(x) = \int u_t(x | z)p_t(x | z)p_{\text{data}}(z) dz$ established inside Theorem 11.21’s proof. That route is shorter to write but explains less: it verifies the equality without exhibiting *why* the marginal field is what the regression finds. We have therefore given the projection proof as the primary one.

Caution 11.27 (The CFM loss has a floor, and it is not zero). The constant C in Theorem 11.25 is not a bookkeeping artefact, and it has a concrete consequence for reading training curves. By Lemma 11.23,

$$C = \mathbb{E} \|u_t(X_t | Z) - u_t(X_t)\|^2 = \mathbb{E} [\text{Var}(u_t(X_t | Z) | X_t, t)],$$

the average *conditional variance* of the target: how much the conditional velocity still varies across the data points z that could have produced the same (x, t) . This is strictly positive whenever two different data points can generate the same intermediate point — which is always, for any non-degenerate dataset.

Therefore: **a perfectly trained flow matching model has a strictly positive training loss.** $\mathcal{L}_{\text{FM}}(\theta^*) = 0$ but $\mathcal{L}_{\text{CFM}}(\theta^*) = C > 0$. A training curve that plateaus well above zero is the expected behaviour, not evidence of underfitting, and the plateau height is a property of the data and the chosen path, not of the network. Section 11.19, check 11, exhibits this numerically on a toy problem where both quantities can be computed exactly ($\mathcal{L}_{\text{CFM}}(\theta^*) = 1.58$ against $\mathcal{L}_{\text{FM}}(\theta^*) = 0$ to machine precision).

The same warning applies verbatim to denoising score matching, and for the same reason.

Theorem 11.25 is the entire flow-matching recipe: pick *any* conditional path $p_t(\cdot | z)$ whose conditional vector field $u_t(\cdot | z)$ can be written down in closed form, train u_θ by regressing against that closed-form target on samples (t, z, x) drawn exactly as in $\mathcal{L}_{\text{CFM}}$, and the trained u_θ approximates the marginal field that transports p_{init} to p_{data} — without ever needing to evaluate the intractable integral (11.9). Written out as a procedure:

One flow matching training step

1. draw a data point $z \sim p_{\text{data}}$ (a random element of the training set)
 2. draw a time $t \sim \mathcal{U}(0, 1)$
 3. draw a point on that data point's conditional path, $x \sim p_t(\cdot \mid z)$
 4. compute the target $u_t(x \mid z)$ in closed form
 5. take a gradient step on $\|u_\theta(x, t) - u_t(x \mid z)\|^2$
-

Two features of this procedure deserve to be named, because they are the practical reason the method displaced its predecessors. First, it is **simulation-free**: no ODE is ever solved during training, in contrast to the maximum-likelihood CNF training of Section 11.4.1. Second, it is **an ordinary supervised regression**: steps 1–4 manufacture an input–target pair, and step 5 is a mean-squared error. Nothing in the training loop knows that it is doing generative modelling.

11.9.1 The linear (conditional optimal transport) path

The recipe needs one concrete conditional path. The simplest possible choice is to move in a straight line from a noise sample to the data point, at constant speed.

Proposition 11.28 (The linear path and its vector field). *Let $p_t(\cdot \mid z)$ be the law of*

$$X_t = (1 - t)X_0 + tz, \quad X_0 \sim p_{\text{init}} \text{ drawn independently of } z,$$

i.e. $p_t(\cdot \mid z) = (x_0 \mapsto (1 - t)x_0 + tz)_{\#} p_{\text{init}}$ in the notation of Definition 2.7. For the Gaussian source $p_{\text{init}} = \mathcal{N}(0, I_d)$ this is explicitly

$$p_t(\cdot \mid z) = \mathcal{N}(tz, (1 - t)^2 I_d), \quad t \in [0, 1), \quad (11.12)$$

an ordinary linear interpolation between an independently drawn noise sample and the data point z . This path satisfies Definition 11.16, and its conditional vector field is

$$u_t(x \mid z) = \frac{z - x}{1 - t} = z - x_0 \quad (\text{the second equality valid exactly on the interpolant itself, } x = (1 - t)x_0 + tz). \quad (11.13)$$

Proof. The law (11.12). X_t is an affine function of the Gaussian $X_0 \sim \mathcal{N}(0, I_d)$, hence Gaussian, so it is determined by its mean and covariance. Taking expectations, $\mathbb{E}[X_t] = (1 - t)\mathbb{E}[X_0] + tz = tz$ since $\mathbb{E}[X_0] = 0$ and z is a constant. For the covariance, $X_t - \mathbb{E}[X_t] = (1 - t)X_0$, so

$$\text{Cov}(X_t) = \mathbb{E}[(1 - t)X_0 X_0^\top (1 - t)] = (1 - t)^2 \mathbb{E}[X_0 X_0^\top] = (1 - t)^2 I_d,$$

using that scalars pull out of the outer product twice. Hence $X_t \sim \mathcal{N}(tz, (1 - t)^2 I_d)$. Note the square: the *standard deviation* is $1 - t$, so the *variance* is $(1 - t)^2$, a place where a factor is easily dropped.

The boundary conditions. At $t = 0$, $X_0 \sim p_{\text{init}}$ by construction. As $t \rightarrow 1$, (11.12) has mean $\rightarrow z$ and variance $\rightarrow 0$, i.e. $p_t(\cdot \mid z) \rightarrow \delta_z$, and indeed at $t = 1$ we have $X_1 = z$ deterministically for *every* draw of X_0 .

The vector field. Differentiate the interpolant along a fixed sample path, i.e. holding X_0 and z fixed:

$$X_t = (1 - t)X_0 + tz \implies \dot{X}_t = -X_0 + z = z - X_0,$$

which is constant in t : the particle travels in a straight line at constant speed, as promised. To express this as a function of the current position alone — which is what Definition 11.16 demands, since a vector field is a function on $\mathbb{R}^d$, not a rule attached to a particular trajectory — invert the interpolation formula for X_0 ,

$$X_t = (1 - t)X_0 + tz \implies X_0 = \frac{X_t - tz}{1 - t} \quad (t < 1),$$

and substitute:

$$\dot{X}_t = z - \frac{X_t - tz}{1 - t} = \frac{(1 - t)z - X_t + tz}{1 - t} = \frac{z - tz + tz - X_t}{1 - t} = \frac{z - X_t}{1 - t},$$

which is (11.13)’s first form. $\square$

Equation (11.13)’s two forms are algebraically equal (exactly, not merely approximately) when evaluated on the interpolant, but computationally distinct: the first, $(z - x)/(1 - t)$, is a function of (x, t, z) alone and is what a network, given only the current state and time, is asked to match at training time; the second, $z - x_0$, is *constant in t* along a fixed pair (x_0, z) and is the more numerically convenient quantity to compute the regression *target* from during training (no division, no singularity as $t \rightarrow 1$), since x_0, z are both known exactly at the moment a training example is constructed. This exact distinction — “current-point form”, singular at $t \rightarrow 1$, versus “constant-in- t endpoint form”, used for the actual training target — reappears, verified directly against the code, as precisely the two forms `conditional_probability_path` and `calc_trans_vector_field` realise in SigmaFlow’s `r3_flow_matcher.py`, reconstructed exactly in Section 13.3.

Caution 11.29 (A singular formula for a perfectly finite quantity). The $1/(1 - t)$ in (11.13) causes recurring confusion, and the resolution is worth stating carefully, because the same pattern recurs on $SO(3)$ (Remark 12.27) and in the general Gaussian case (Caution 11.39).

As a function on $\mathbb{R}^d$, at fixed t , the field really does blow up. Fix $t = 0.999$ and any $x \neq z$. Then $\|u_t(x | z)\| = \|z - x\|/0.001 = 1000\|z - x\|$, which is enormous. There is nothing wrong with this: it says that a particle that finds itself far from z with almost no time left must move extremely fast to arrive on schedule.

Along an actual sample path, nothing blows up. A particle on the interpolant is *not* far from z at $t = 0.999$; it is at $x_t = (1 - t)x_0 + tz$, so $z - x_t = (1 - t)(z - x_0)$ and

$$u_t(x_t | z) = \frac{(1 - t)(z - x_0)}{1 - t} = z - x_0,$$

a fixed, finite vector, the same at every t . The factor $1 - t$ in the numerator cancels the one in the denominator *exactly*.

So the singularity is a property of *off-path* evaluation, and the two regimes must be kept apart:

- during **training**, only on-path points occur — x is constructed as $(1 - t)x_0 + tz$ — so the target is computed as $z - x_0$ and no division is performed at all;
- during **sampling**, the trajectory is generated by an *imperfect* network and by a discretised ODE, so the state drifts off the ideal path, and evaluating a $1/(1 - t)$ expression there is genuinely hazardous as $t \rightarrow 1$.

This is the reason implementations parameterise the network to predict the finite quantity and integrate with it, rather than predicting a current-point form.

Remark 11.30 (On the name “conditional optimal transport”). The linear path is frequently called the “conditional optimal transport” (CondOT) path, since for fixed x_0, z the straight-line interpolant $(1 - t)x_0 + tz$ is exactly the (constant-speed) geodesic of the flat Euclidean metric between them, and constant-speed geodesics are the building block of Euclidean optimal-transport (Wasserstein) interpolation. The word “conditional” is doing real work here and is easy to lose: each *conditional* path is an optimal-transport interpolation between two *points*; nothing is claimed about the induced *marginal* path being an optimal transport between the two *distributions*. It is generally not (Section 11.16). We do not develop optimal transport theory in this document; the name is recorded for consistency with the literature this monograph audits and with the terminology used when this path is generalised to a Riemannian manifold in Chapter 12, where “geodesic interpolant” (a notion this document *has* developed, Definition 3.8) is the more directly justified term and the one used throughout the rest of this document.

11.10 Three worked examples

The construction is short enough that it is easy to believe one has understood it before one has. The following three examples are fully computable by hand, and are arranged so that each adds exactly one ingredient: the first has no randomness at all, the second adds a random source, the third adds a non-trivial data distribution and thereby separates the conditional field from the marginal one.

11.10.1 Example 1: one source point, one data point

Take $d = 1$, and suppose for the moment that both endpoints are fixed: $x_0 = -1$ and $z = 3$. The linear interpolant is

$$x_t = (1 - t) \cdot (-1) + t \cdot 3 = -1 + 4t,$$

a point moving from -1 to 3 at constant speed. Differentiating,

$$u_t = \dot{x}_t = 4,$$

and indeed the endpoint form of (11.13) gives $z - x_0 = 3 - (-1) = 4$ directly. The current-point form agrees, as it must:

$$\frac{z - x_t}{1 - t} = \frac{3 - (-1 + 4t)}{1 - t} = \frac{4 - 4t}{1 - t} = \frac{4(1 - t)}{1 - t} = 4.$$

Both the numerator and the denominator vanish as $t \rightarrow 1$, and their ratio is a constant — the cancellation of Caution 11.29 made completely explicit, with no distributions in sight. Checking the endpoints: $x_0 = -1$ ✓, $x_1 = -1 + 4 = 3$ ✓.

11.10.2 Example 2: Gaussian source, one data point

Now let the source be random: $d = 1$, $X_0 \sim \mathcal{N}(0, 1)$, and $p_{\text{data}} = \delta_{x_1}$ for a fixed number x_1 . Since there is only one data point, the posterior in (11.8) is trivial (it puts all its mass on x_1), so the marginal field *equals* the conditional field; this example isolates the effect of a random source without the complication of averaging.

The path. By Proposition 11.28,

$$X_t = (1 - t)X_0 + tx_1 \sim \mathcal{N}(tx_1, (1 - t)^2).$$

At $t = 0$: $\mathcal{N}(0, 1)$, the source $\checkmark$. At $t = \frac{1}{2}$: $\mathcal{N}(x_1/2, 1/4)$ — the cloud has moved half-way and its standard deviation has halved. At $t \rightarrow 1$: mean x_1 , variance 0 $\checkmark$.

The velocity, three ways. Along a fixed sample path,

$$\dot{X}_t = x_1 - X_0,$$

which is random — it depends on which X_0 was drawn — but constant in t . Substituting $X_0 = (X_t - tx_1)/(1 - t)$ gives the field

$$u_t(x | x_1) = x_1 - \frac{x - tx_1}{1 - t} = \frac{x_1(1 - t) - x + tx_1}{1 - t} = \frac{x_1 - x}{1 - t}.$$

Solving the ODE, as a check. It is worth verifying, once, that this field really does move samples along the path we claimed, by solving $\dot{x} = (x_1 - x)/(1 - t)$ directly. Substitute $y := x_1 - x$, so $\dot{y} = -\dot{x} = -y/(1 - t)$, a separable equation:

$$\frac{dy}{y} = -\frac{dt}{1 - t} \implies \log |y| = \log |1 - t| + c \implies y_t = K(1 - t)$$

for a constant K fixed by the initial condition $y_0 = x_1 - x_0$, whence $K = x_1 - x_0$ and

$$x_1 - x_t = (1 - t)(x_1 - x_0) \implies x_t = x_1 - (1 - t)(x_1 - x_0) = (1 - t)x_0 + tx_1.$$

The flow of the field reproduces the interpolant exactly. Every sample lands on x_1 at $t = 1$, regardless of where it started, so $p_1 = \delta_{x_1} = p_{\text{data}}$ $\checkmark$.

What the network would learn. With a single data point the regression target $u_t(x | x_1) = (x_1 - x)/(1 - t)$ is a deterministic function of (x, t) , so the conditional variance in Caution 11.27 vanishes and $C = 0$: this is the degenerate case in which a perfectly trained model really does reach zero loss. Any dataset with more than one point breaks this.

11.10.3 Example 3: two data points in two dimensions

Now the interesting case. Let $d = 2$, $p_{\text{init}} = \mathcal{N}(0, I_2)$, and let the data distribution be two points of equal weight,

$$p_{\text{data}} = \frac{1}{2}\delta_a + \frac{1}{2}\delta_{-a}, \quad a = e_1 = (1, 0)^\top,$$

with the linear path throughout. This is the smallest example in which the marginal field genuinely differs from the conditional ones.

The marginal path. By Definition 11.18 and (11.12), p_t is a two-component Gaussian mixture,

$$p_t = \frac{1}{2}\mathcal{N}(ta, (1 - t)^2 I_2) + \frac{1}{2}\mathcal{N}(-ta, (1 - t)^2 I_2),$$

whose components separate (their centres move apart) and sharpen (their width shrinks) simultaneously as t grows.

The posterior weights. Write $w_+(x, t)$ for the posterior probability (11.8) that the particle at x is heading for $+a$. Because the two components have equal weight and equal variance, only the exponents differ, and

$$w_+ = \frac{e^{-\|x-ta\|^2/2(1-t)^2}}{e^{-\|x-ta\|^2/2(1-t)^2} + e^{-\|x+ta\|^2/2(1-t)^2}} = \frac{1}{1 + e^{-\Delta}}, \quad \Delta := \frac{\|x + ta\|^2 - \|x - ta\|^2}{2(1-t)^2}.$$

Expanding the two squares, the $\|x\|^2$ and $t^2\|a\|^2$ terms cancel and only the cross terms survive: $\|x + ta\|^2 - \|x - ta\|^2 = 4t\langle x, a \rangle = 4tx_1$, so

$$\Delta = \frac{2tx_1}{(1-t)^2}, \quad w_+ = \frac{1}{1 + \exp(-2tx_1/(1-t)^2)}.$$

(We deliberately do not name this the “ σ function”: in this document σ is a noise scale throughout, and reusing it for the logistic here would collide with σ_t two sections later.) The particle’s coordinate *along* the axis separating the data points, x_1 , is the only thing that matters — as it should be, by symmetry.

The marginal field. Averaging the two conditional fields with these weights (Theorem 11.21), and using $w_- = 1 - w_+$:

$$u_t(x) = w_+ \frac{a - x}{1 - t} + w_- \frac{-a - x}{1 - t} = \frac{(w_+ - w_-) a - x}{1 - t} = \frac{(2w_+ - 1) a - x}{1 - t}.$$

Finally, $\frac{2}{1 + e^{-2y}} - 1 = \tanh(y)$, so with $y = tx_1/(1-t)^2$,

$$\boxed{u_t(x) = \frac{1}{1-t} \left(\tanh\left(\frac{tx_1}{(1-t)^2}\right) e_1 - x \right).} \quad (11.14)$$

(Verified numerically against a direct posterior average in Section 11.19, check 6, to $7 \cdot 10^{-15}$.)

Reading the formula. Equation (11.14) shows the entire phenomenon in one line. The vector $\tanh(\cdot)e_1$ is a *soft target*: a point on the segment between $-a$ and $+a$ that the field currently aims at.

- **Early** (t small): the $\tanh$ argument is near 0, so the soft target is near the *origin* — the mean of the data distribution. The field does not commit to either mode. Indeed at $t = 0$, $u_0(x) = -x = \mathbb{E}[X_1] - x$ (the data mean is 0 here), which is exactly what Theorem 11.21 predicts when X_0 is independent of X_1 : with no information yet, aim at the average.
- **Late** ($t \rightarrow 1$): the argument $tx_1/(1-t)^2$ diverges, the $\tanh$ saturates to ± 1 , and the soft target snaps onto whichever data point lies on the particle’s side. From then on the marginal field is indistinguishable from that point’s conditional field.
- **The switch happens on the hyperplane** $x_1 = 0$, where the $\tanh$ is exactly 0 by symmetry. A trajectory that arrives there exactly is pulled towards neither mode — a measure-zero event, but the reason the field’s gradient becomes very steep near that plane at late times.

Numerically, at the point $x = (0.5, 0.3)$:

t	$\tanh(tx_1/(1-t)^2)$	marginal $u_t(x)$	conditional $u_t(x a)$	agree?
0.2	0.1550	(−0.4313, −0.3750)	(0.6250, −0.3750)	no, opposite sign
0.5	0.7616	(0.5232, −0.6000)	(1.0000, −0.6000)	no
0.8	1.0000	(2.5000, −1.5000)	(2.5000, −1.5000)	yes, to 4 d.p.
0.95	1.0000	(10.000, −6.0000)	(10.000, −6.0000)	yes

The second component agrees at every time (the data points differ only along e_1 , so the posterior cannot influence the orthogonal direction). The first component is the interesting one: at $t = 0.2$ the marginal field points *away* from a , back towards the data mean, even though this particular training example was heading for a . A network trained on this problem is *supposed* to disagree with the individual targets it is shown; it is supposed to reproduce their conditional average. That disagreement is precisely the constant C of Caution 11.27, made visible.

11.11 Gaussian probability paths

The linear path is one member of a family that covers essentially every path used in practice, and treating the family in one go costs no extra work and pays for itself twice over: it produces the general training target in closed form, and — in Section 11.13 — it turns out to contain the diffusion models of Part III as special cases.

Definition 11.31 (Gaussian conditional probability path). Let $\alpha_t, \sigma_t : [0, 1] \rightarrow \mathbb{R}$ be continuously differentiable, with

$$\alpha_0 = 0, \quad \alpha_1 = 1, \quad \sigma_0 = 1, \quad \sigma_1 = 0,$$

and $\sigma_t > 0$ for $t < 1$. The associated *Gaussian conditional probability path* is

$$p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \sigma_t^2 I_d), \quad \text{equivalently} \quad X_t = \alpha_t z + \sigma_t \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I_d). \quad (11.15)$$

The pair (α_t, σ_t) is called the *schedule* (or noise scheduler).

The two boundary conditions are exactly Definition 11.16's: at $t = 0$ the law is $\mathcal{N}(0, I_d) = p_{\text{init}}$, and at $t = 1$ it is $\mathcal{N}(z, 0) = \delta_z$. In between, α_t says how much *signal* (the data point) is present and σ_t how much *noise*; the schedule is the entire design freedom.

Caution 11.32 (Notation collision with Part III). Holderrieth & Erives write this schedule as (α_t, β_t) . This document writes σ_t for the second component, deliberately, because Example 6.6 already uses $\beta(s)$ for the VP-SDE's *noise-rate* schedule — a different object (a rate, not a standard deviation), living in the noising time s . Proposition 6.7 unfortunately also writes $\beta(s)$ for a standard deviation; that is an internal inconsistency of this document, noted here and to be read from context. Nothing below depends on either β .

Proposition 11.33 (Conditional vector field of a Gaussian path). *The Gaussian path (11.15) is generated by*

$$u_t(x | z) = \dot{\alpha}_t z + \frac{\dot{\sigma}_t}{\sigma_t} (x - \alpha_t z) = \underbrace{\left(\dot{\alpha}_t - \frac{\dot{\sigma}_t}{\sigma_t} \alpha_t \right)}_{\text{coefficient of } z} z + \underbrace{\frac{\dot{\sigma}_t}{\sigma_t}}_{\text{coefficient of } x} x, \quad t \in [0, 1), \quad (11.16)$$

where $\dot{\alpha}_t = \frac{d}{dt} \alpha_t$ and $\dot{\sigma}_t = \frac{d}{dt} \sigma_t$.

Proof. Fix z and a noise draw ε , and follow the resulting sample path $X_t = \alpha_t z + \sigma_t \varepsilon$. Differentiating in t with z, ε held fixed — the only t -dependence is in the two scalar schedule functions —

$$\dot{X}_t = \dot{\alpha}_t z + \dot{\sigma}_t \varepsilon. \quad (11.17)$$

This is the velocity, but expressed in terms of ε , which a vector field is not allowed to depend on: u_t must be a function of the current position x alone (for fixed z). So eliminate ε . From $x = \alpha_t z + \sigma_t \varepsilon$ and $\sigma_t > 0$,

$$\varepsilon = \frac{x - \alpha_t z}{\sigma_t}, \quad (11.18)$$

and substituting into (11.17) gives the first form of (11.16). Expanding the bracket, $\dot{\alpha}_t z + \frac{\dot{\sigma}_t}{\sigma_t} x - \frac{\dot{\sigma}_t}{\sigma_t} \alpha_t z$, and collecting the two z terms gives the second.

It remains to confirm that this field *generates* the path, i.e. that its flow started from p_{init} reproduces (11.15) rather than merely being consistent with one sample path. This is immediate from the construction: the map $\psi_t(x_0) := \alpha_t z + \sigma_t x_0$ satisfies $\psi_0 = \text{id}$ (since $\alpha_0 = 0, \sigma_0 = 1$) and, by the computation just performed, $\frac{d}{dt} \psi_t(x_0) = u_t(\psi_t(x_0) \mid z)$; so ψ_t is the flow map of $u_t(\cdot \mid z)$ by uniqueness of ODE solutions (Definition 11.5). Pushing $\mathcal{N}(0, I_d)$ forward through the affine map ψ_t gives $\mathcal{N}(\alpha_t z, \sigma_t^2 I_d)$ by the mean/covariance computation of Proposition 11.28's proof, which is (11.15). (Verified numerically by integrating the ODE for three different schedules in Section 11.19, check 1.) $\square$

Example 11.34 (The linear path is the schedule $\alpha_t = t, \sigma_t = 1 - t$). Then $\dot{\alpha}_t = 1$ and $\dot{\sigma}_t = -1$, and (11.16) reads

$$u_t(x \mid z) = z + \frac{-1}{1-t}(x - tz) = \frac{(1-t)z - x + tz}{1-t} = \frac{z - x}{1-t},$$

recovering (11.13) exactly. Every statement about the linear path in Sections 11.9.1 and 11.10 is therefore a special case of what follows.

Example 11.35 (The trigonometric, variance-preserving schedule). Take $\alpha_t = \cos \varphi_t$ and $\sigma_t = \sin \varphi_t$ with $\varphi_t = \frac{\pi}{2}(1 - t)$, so that φ runs from $\frac{\pi}{2}$ at $t = 0$ down to 0 at $t = 1$. The boundary conditions hold, and additionally

$$\alpha_t^2 + \sigma_t^2 = 1 \quad \text{for every } t,$$

so if the data are standardised ($\mathbb{E}\|z\|^2/d = 1$) the intermediate states X_t have unit variance at every time — the path is *variance-preserving*, in exactly the sense of Example 6.6. This is not the case for the linear path, whose intermediate variance $t^2 \text{Var}(z) + (1-t)^2$ dips in the middle. The trigonometric schedule reappears in Section 11.14 as the one on which the diffusion v -parameterisation is defined.

Remark 11.36 (What the schedule buys, and what it does not). Different schedules give different conditional paths, hence different marginal paths, hence different marginal fields and different trajectories. What they do *not* change is the endpoints: every schedule satisfying Definition 11.31's boundary conditions transports p_{init} to p_{data} exactly. The schedule is thus a pure efficiency-and-conditioning choice, not a correctness one — a point taken up in Section 11.16.

11.12 Score and velocity

The score, Definition 2.10, and the velocity field, Definition 11.5, are different objects with different types and different jobs:

	Score $\nabla_x \log p_t(x)$	Velocity $u_t(x)$
What it is	the gradient of a log-density	the prescribed motion of a particle
Determined by	p_t alone, at that single time	the whole path, and not uniquely (Section 11.3.4)
Points towards	higher probability, right now	the data, eventually
Needs a path?	no — any density has a score	yes

They are emphatically not the same thing, and the fact that they are routinely converted into one another in the Gaussian case is a special feature of that case, not a general identity. This section establishes exactly what the relation is.

11.12.1 The conditional score of a Gaussian path

For $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \sigma_t^2 I_d)$ we may read the score off Remark 2.13: taking the log of the Gaussian density kills the normalising constant (it does not depend on x), leaving $-\|x - \alpha_t z\|^2/2\sigma_t^2$, whose gradient is

$$\nabla_x \log p_t(x | z) = -\frac{x - \alpha_t z}{\sigma_t^2} \stackrel{(11.18)}{=} -\frac{\varepsilon}{\sigma_t}. \quad (11.19)$$

The second form is worth remembering: *the conditional score is the noise that was added, scaled by $-1/\sigma_t$* . This is the identity that makes score matching equivalent to noise prediction, and it is the reason the score blows up as $\sigma_t \rightarrow 0$ — an observation already made geometrically in Section 6.1.

11.12.2 The conversion formula

Proposition 11.37 (Score–velocity conversion for Gaussian paths). *Let $p_t(\cdot | z)$ be a Gaussian path (11.15) with $\alpha_t \neq 0$, and let p_t be the induced marginal path. Define*

$$a_t := \sigma_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\sigma}_t \sigma_t, \quad b_t := \frac{\dot{\alpha}_t}{\alpha_t}. \quad (11.20)$$

Then both the conditional and the marginal fields are the same affine function of their respective scores, with the same coefficients:

$$u_t(x | z) = a_t \nabla_x \log p_t(x | z) + b_t x, \quad (11.21)$$

$$u_t(x) = a_t \nabla_x \log p_t(x) + b_t x. \quad (11.22)$$

In particular, for a Gaussian path, knowing the marginal score is equivalent to knowing the marginal velocity: either determines the other pointwise, with no additional information.

Proof. Conditional case. Start from (11.16) and replace the bracket $x - \alpha_t z$ using (11.19), which says $x - \alpha_t z = -\sigma_t^2 \nabla_x \log p_t(x | z)$:

$$u_t(x | z) = \dot{\alpha}_t z + \frac{\dot{\sigma}_t}{\sigma_t} (-\sigma_t^2 \nabla_x \log p_t(x | z)) = \dot{\alpha}_t z - \dot{\sigma}_t \sigma_t \nabla_x \log p_t(x | z).$$

This already expresses $u_t(\cdot | z)$ through the score, but it still mentions z , which (11.21) does not. Eliminate z by rearranging (11.19) the other way,

$$\alpha_t z = x + \sigma_t^2 \nabla_x \log p_t(x | z) \implies z = \frac{x + \sigma_t^2 \nabla_x \log p_t(x | z)}{\alpha_t}$$

(this is where $\alpha_t \neq 0$ is needed), and substitute:

$$\begin{aligned} u_t(x | z) &= \frac{\dot{\alpha}_t}{\alpha_t} \left(x + \sigma_t^2 \nabla_x \log p_t(x | z) \right) - \dot{\sigma}_t \sigma_t \nabla_x \log p_t(x | z) \\ &= \underbrace{\left(\sigma_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\sigma}_t \sigma_t \right)}_{=a_t} \nabla_x \log p_t(x | z) + \underbrace{\frac{\dot{\alpha}_t}{\alpha_t}}_{=b_t} x, \end{aligned}$$

which is (11.21).

Marginal case. This is where the shared coefficients earn their keep. Both sides of (11.21) are being averaged against the *same* posterior weights: by Theorem 11.21 the marginal field is the posterior average of the conditional fields, and by the identical argument (or directly, differentiating $\log p_t$ under the integral sign, exactly as in Theorem 6.4's proof) the marginal score is the posterior average of the conditional scores,

$$\nabla_x \log p_t(x) = \int \nabla_x \log p_t(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz. \quad (11.23)$$

Write $\pi(z | x, t)$ for the posterior density (11.8). Then

$$u_t(x) = \int u_t(x | z) \pi(z | x, t) dz = \int \left(a_t \nabla_x \log p_t(x | z) + b_t x \right) \pi(z | x, t) dz.$$

The coefficients a_t, b_t do not depend on z , so they come out of the integral; the first term becomes a_t times (11.23); and the second term is $b_t x$ times $\int \pi(z | x, t) dz = 1$. This yields (11.22). (Both identities verified numerically for three schedules in Section 11.19, checks 2 and 3, to 10^{-14} .) $\square$

The proof's structure deserves a sentence, because it is a pattern worth recognising: *an identity that is affine in the conditioning variable, with coefficients not depending on it, survives marginalisation unchanged.* Had a_t or b_t depended on z , the marginal statement would simply be false.

Example 11.38 (The coefficients for the linear path). With $\alpha_t = t$, $\sigma_t = 1 - t$, $\dot{\alpha}_t = 1$, $\dot{\sigma}_t = -1$:

$$b_t = \frac{1}{t}, \quad a_t = \frac{(1-t)^2}{t} + (1-t) = (1-t) \left(\frac{1-t}{t} + 1 \right) = \frac{1-t}{t},$$

so

$$u_t(x) = \frac{x}{t} + \frac{1-t}{t} \nabla_x \log p_t(x).$$

Sanity check against Section 11.10.2, where $p_{\text{data}} = \delta_{x_1}$ and hence $p_t = \mathcal{N}(tx_1, (1-t)^2)$ with score $-(x - tx_1)/(1-t)^2$:

$$\frac{x}{t} + \frac{1-t}{t} \cdot \frac{-(x - tx_1)}{(1-t)^2} = \frac{x}{t} - \frac{x - tx_1}{t(1-t)} = \frac{x(1-t) - x + tx_1}{t(1-t)} = \frac{x_1 - x}{1-t} \checkmark$$

as Proposition 11.28 requires.

Caution 11.39 (A singular formula for a regular field, again). Both a_t and b_t carry a $1/\alpha_t$, which for the linear path means $1/t$: they diverge as $t \rightarrow 0$. The field u_t does not. At $t = 0$ the linear path's marginal field is the perfectly finite $u_0(x) = \mathbb{E}[X_1] - x$ (Section 11.10.3), and what happens in (11.22) is that the two divergent terms cancel, since the score itself carries

a compensating factor. This is the same phenomenon as Caution 11.29, at the other end of the interval, and it carries the same practical lesson: *convert between score and velocity symbolically, never by evaluating a_t, b_t numerically near a degenerate endpoint*. The $t \rightarrow 0$ blow-up is an artefact of choosing the score as the intermediate representation; the $t \rightarrow 1$ blow-up of (11.13) is an artefact of choosing the current point as the intermediate representation. Neither is a property of the generative model.

11.12.3 The four objects, and the denoiser that connects them

It is easy to lose track of which quantity is conditional and which is marginal, especially since the two obey formally identical equations. The following table is the one to keep.

	Conditional (knows z)	Marginal (does not)
Score	$\nabla_x \log p_t(x z) = -\frac{x - \alpha_t z}{\sigma_t^2}$ closed form, one line	$\nabla_x \log p_t(x)$ intractable integral; the target of score matching
Velocity	$u_t(x z) = \dot{\alpha}_t z + \frac{\dot{\sigma}_t}{\sigma_t}(x - \alpha_t z)$ closed form, Proposition 11.33	$u_t(x)$ intractable integral; the target of flow matching
Linked by	$u = a_t(\text{score}) + b_t x$ — the same a_t, b_t in both columns (Proposition 11.37)	
Linked by	posterior averaging $\mathbb{E}[\cdot X_t = x]$ — moves from the left column to the right one, for either row	

The table is a commuting square: one may convert score to velocity and then marginalise, or marginalise and then convert, and the result is the same. That is exactly what the second half of Proposition 11.37’s proof established.

There is a fifth object worth naming, because it is the most interpretable of all and because it explains the word “denoising”.

Definition 11.40 (Denoiser). The *denoiser* of a Gaussian path is the posterior mean of the clean data point given the noisy state,

$$D_t(x) := \mathbb{E}[Z | X_t = x].$$

Proposition 11.41 (Tweedie-type identities). For a Gaussian path (11.15),

$$D_t(x) = \frac{x + \sigma_t^2 \nabla_x \log p_t(x)}{\alpha_t} \quad (\alpha_t \neq 0), \quad (11.24)$$

and, equivalently, in terms of the velocity,

$$D_t(x) = \frac{\sigma_t u_t(x) - \dot{\sigma}_t x}{\dot{\alpha}_t \sigma_t - \alpha_t \dot{\sigma}_t}. \quad (11.25)$$

Proof. For (11.24), average the conditional score (11.19) against the posterior, using (11.23):

$$\nabla_x \log p_t(x) = \mathbb{E} \left[-\frac{x - \alpha_t Z}{\sigma_t^2} \middle| X_t = x \right] = \frac{\alpha_t \mathbb{E}[Z | X_t = x] - x}{\sigma_t^2} = \frac{\alpha_t D_t(x) - x}{\sigma_t^2},$$

(x and the schedule being constants inside the conditional expectation), and solve for $D_t(x)$.

For (11.25), substitute $\nabla_x \log p_t(x) = (u_t(x) - b_t x)/a_t$ from (11.22) into (11.24) and simplify: with $a_t = \sigma_t^2 \dot{\alpha}_t / \alpha_t - \dot{\sigma}_t \sigma_t$ and $b_t = \dot{\alpha}_t / \alpha_t$ one finds $a_t - \sigma_t^2 b_t = -\dot{\sigma}_t \sigma_t$ and $\alpha_t a_t = \sigma_t(\sigma_t \dot{\alpha}_t - \alpha_t \dot{\sigma}_t)$, so

$$D_t(x) = \frac{\sigma_t^2 u_t(x) + x(a_t - \sigma_t^2 b_t)}{\alpha_t a_t} = \frac{\sigma_t^2 u_t(x) - \dot{\sigma}_t \sigma_t x}{\sigma_t(\sigma_t \dot{\alpha}_t - \alpha_t \dot{\sigma}_t)},$$

and cancelling one factor of σ_t gives (11.25). $\square$

Proposition 11.41 is the cleanest statement of what a diffusion or flow-matching network is “really” doing on a Gaussian path: whatever it is nominally predicting, it is computing an estimate of *the clean data point that the noisy input most plausibly came from*, in a fixed, invertible, linear disguise. That observation is the subject of Section 11.14.

11.13 Diffusion models are flow matching with a particular path

This is the conceptual centre of the chapter. So far flow matching has been developed without a single reference to noising processes, SDEs or scores. We now show that the diffusion models of Part III are not an alternative framework but a *special case* of this one: a diffusion model chooses a particular Gaussian probability path and a particular parameterisation of the field that generates it.

11.13.1 First, the time convention

The two literatures run time in opposite directions, and merging them carelessly is the single most common source of sign errors. This document fixed the convention globally in Remark 0.2; we restate it here in the form needed for the comparison, because this is the section where it earns its keep.

	Noising time s	Generation time $t = 1 - s$
Used for	the forward/corruption process	everything else in this document
$= 0$ means	clean data	pure noise, p_{init}
$= 1$ means	pure noise	clean data, p_{data}
Process runs	data $\rightarrow$ noise	noise $\rightarrow$ data
Written	$Y_s, q_s, f(\cdot, s), g(s)$	X_t, p_t, u_t
Link	$X_t := Y_{1-t}, \quad p_t := q_{1-t}$	

Two consequences of $t = 1 - s$ are used constantly below and are worth isolating: any time derivative picks up a minus sign, $\frac{d}{dt} = -\frac{d}{ds}$; and any schedule expressed in s becomes, in generation time, the same function with its argument reversed.

Caution 11.42 (The probability flow ODE as it is usually written). Song et al. (2021), and most of the literature following them, state the probability flow ODE in *noising* time:

$$\frac{dY_s}{ds} = f(Y_s, s) - \frac{1}{2}g(s)^2 \nabla_x \log q_s(Y_s),$$

to be integrated *backwards* from $s = 1$ to $s = 0$. This document states it in generation time, (6.10):

$$\frac{dX_t}{dt} = -f(X_t, 1-t) + \frac{1}{2}g(1-t)^2 \nabla_x \log p_t(X_t),$$

integrated *forwards* from $t = 0$ to $t = 1$. These are the same equation. Substituting $X_t = Y_{1-t}$,

$$\frac{dX_t}{dt} = \frac{dY_s}{ds} \Big|_{s=1-t} \cdot \frac{ds}{dt} = - \left[f(X_t, 1-t) - \frac{1}{2}g(1-t)^2 \nabla_x \log q_{1-t}(X_t) \right],$$

and $q_{1-t} = p_t$; the overall minus sign from $ds/dt = -1$ flips *both* terms, turning f into $-f$ and $-\frac{1}{2}g^2$ into $+\frac{1}{2}g^2$. A reader who copies the first form and integrates it forwards in t will obtain a process that runs the wrong way and diverges. This is not a pedantic point: it is exactly the class of error that Section 13.2 documents having occurred in this project's own codebase.

11.13.2 A diffusion model's marginal path is a Gaussian path

Recall from Proposition 6.7 that the SDEs used in practice are linear, and therefore have Gaussian perturbation kernels: writing the schedule as $\alpha^{\text{dif}}(s)$ for the signal coefficient and $\sigma^{\text{dif}}(s)$ for the noise standard deviation,

$$Y_s = \alpha^{\text{dif}}(s) Y_0 + \sigma^{\text{dif}}(s) \varepsilon, \quad Y_0 \sim p_{\text{data}}, \quad \varepsilon \sim \mathcal{N}(0, I_d),$$

with $\alpha^{\text{dif}}(0) = 1$, $\sigma^{\text{dif}}(0) = 0$ (at $s = 0$ the state is the clean data point).

Now simply relabel. Setting $X_t = Y_{1-t}$ and $Z := Y_0$ (the clean data point, which under our convention sits at $t = 1$),

$$X_t = \underbrace{\alpha^{\text{dif}}(1-t)}_{=: \alpha_t} Z + \underbrace{\sigma^{\text{dif}}(1-t)}_{=: \sigma_t} \varepsilon. \quad (11.26)$$

This is *exactly* Definition 11.31. The boundary conditions match: $\alpha_1 = \alpha^{\text{dif}}(0) = 1$ and $\sigma_1 = \sigma^{\text{dif}}(0) = 0$ (data at $t = 1$), while α_0 and σ_0 are the fully-noised values at $s = 1$. So:

A diffusion model's family of noised marginals, read backwards, is a Gaussian conditional probability path.

The two standard SDE families instantiate it as follows.

	VP (Example 6.6)	VE (Example 6.5)
SDE	$f = -\frac{1}{2}\beta(s)x, g = \sqrt{\beta(s)}$	$f = 0, g(s)^2 = \frac{d}{ds}\sigma(s)^2$
α_t	$\exp(-\frac{1}{2}\int_0^{1-t}\beta)$	1
σ_t	$\sqrt{1 - \alpha_t^2}$	$\sigma(1-t)$
$\alpha_t^2 + \sigma_t^2$	= 1: variance preserving	grows without bound
α_0	≈ 0 : source $\approx \mathcal{N}(0, I)$	= 1: source is <i>data plus large noise</i>
Used by	SigmaDock's translations	SigmaDock's rotations

Caution 11.43 (VE does not have a data-independent source). The VE column contains a genuine asymmetry that is easy to miss and that matters for Chapter 13. Because $\alpha_t \equiv 1$, the VE path's $t = 0$ end is $\mathcal{N}(z, \sigma_{\max}^2 I)$ — it still remembers z . It is only *approximately* data-independent, and only because $\sigma_{\max}$ is large enough to swamp the data's spread. A VE diffusion model therefore does not strictly satisfy Definition 11.16's requirement $p_0(\cdot | z) = p_{\text{init}}$; it satisfies it in the limit. Flow matching, by contrast, imposes the correct source exactly, by construction. Section 15.6.1 shows this distinction is not academic in this project: it is one of the two measured confounders between SigmaDock and SigmaFlow.

11.13.3 The probability flow ODE is the Gaussian path's velocity

We now have two independent formulas for the same object. From the diffusion side, Theorem 6.9 says the deterministic process with the diffusion model's marginals has velocity

$$u_t^{\text{PF}}(x) = -f(x, 1-t) + \frac{1}{2}g(1-t)^2 \nabla_x \log p_t(x). \quad (11.27)$$

From the flow-matching side, Proposition 11.37 says the marginal field of a Gaussian path with schedule (α_t, σ_t) is

$$u_t(x) = a_t \nabla_x \log p_t(x) + b_t x, \quad a_t = \sigma_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\sigma}_t \sigma_t, \quad b_t = \frac{\dot{\alpha}_t}{\alpha_t}. \quad (11.28)$$

If the correspondence (11.26) is right, these must agree. They do, and checking it is a genuine test of both derivations rather than a formality.

Proposition 11.44 (The two velocities coincide). *Under the identification (11.26), $u_t^{\text{PF}} = u_t$ for both the VP and the VE families:*

$$\text{VP: } a_t = b_t = \frac{1}{2}\beta(1-t), \quad \text{VE: } b_t = 0, \quad a_t = \frac{1}{2}g(1-t)^2.$$

Proof. VP case. Here $\alpha^{\text{dif}}(s) = \exp(-\frac{1}{2} \int_0^s \beta)$, so $\frac{d}{ds} \alpha^{\text{dif}}(s) = -\frac{1}{2}\beta(s)\alpha^{\text{dif}}(s)$. Passing to generation time, $\alpha_t = \alpha^{\text{dif}}(1-t)$ and the chain rule contributes a factor -1 :

$$\dot{\alpha}_t = -\frac{d\alpha^{\text{dif}}}{ds} \Big|_{1-t} = +\frac{1}{2}\beta(1-t) \alpha^{\text{dif}}(1-t) = \frac{1}{2}\beta(1-t) \alpha_t.$$

Hence immediately

$$b_t = \frac{\dot{\alpha}_t}{\alpha_t} = \frac{1}{2}\beta(1-t),$$

which is exactly the coefficient of x in (11.27), since $-f(x, 1-t) = +\frac{1}{2}\beta(1-t)x$. ✓

For a_t , use the variance-preserving relation $\sigma_t^2 = 1 - \alpha_t^2$. Differentiating it, $2\sigma_t \dot{\sigma}_t = -2\alpha_t \dot{\alpha}_t$, i.e.

$$\dot{\sigma}_t \sigma_t = -\alpha_t \dot{\alpha}_t = -\frac{1}{2}\beta(1-t) \alpha_t^2.$$

Substituting this and $\dot{\alpha}_t/\alpha_t = \frac{1}{2}\beta(1-t)$ into (11.28),

$$a_t = \sigma_t^2 \cdot \frac{1}{2}\beta(1-t) - (-\frac{1}{2}\beta(1-t)\alpha_t^2) = \frac{1}{2}\beta(1-t) (\sigma_t^2 + \alpha_t^2) = \frac{1}{2}\beta(1-t),$$

the last step using $\alpha_t^2 + \sigma_t^2 = 1$ once more. And $\frac{1}{2}g(1-t)^2 = \frac{1}{2}\beta(1-t)$, so this matches (11.27)'s score coefficient. ✓

VE case. Now $\alpha_t \equiv 1$, so $\dot{\alpha}_t = 0$ and $b_t = 0$, matching $-f = 0$. ✓ For the score coefficient, $\sigma_t = \sigma(1-t)$ gives $\dot{\sigma}_t = -\sigma'(1-t)$, hence

$$a_t = 0 - \dot{\sigma}_t \sigma_t = \sigma'(1-t)\sigma(1-t) = \frac{1}{2} \frac{d}{ds} [\sigma(s)^2]_{s=1-t} = \frac{1}{2}g(1-t)^2,$$

by the VE definition of g . ✓

(Both cases verified numerically, with realistic schedules $\beta(s) = 0.1 + 19.9s$ and $\sigma(s) = 0.02 \cdot 50^s$, in Section 11.19, check 7; agreement to $8 \cdot 10^{-9}$, limited only by the finite-difference evaluation of $\dot{\alpha}_t$.) $\square$

Notice what happened in the VP computation: the factor $\frac{1}{2}$ that Section 6.4.1 had to extract from the Fokker–Planck equation, and that looked like a piece of stochastic-calculus book-keeping, here falls out of $\alpha_t^2 + \sigma_t^2 = 1$ — a statement about a schedule, with no Itô calculus anywhere in sight. The same number is reachable from either side, which is a useful sanity check on both derivations.

11.13.4 Same path, different learned object

We can now state the relationship in its sharpest form. Choose a Gaussian path — say the VP one. Then:

	Score-based diffusion	Flow matching
Probability path	Gaussian, from an SDE	Gaussian, chosen directly
Network learns	$s_\theta(x, t) \approx \nabla_x \log p_t(x)$	$u_\theta(x, t) \approx u_t(x)$
Training target	$\nabla_x \log p_t(x z) = -\varepsilon/\sigma_t$	$u_t(x z) = \dot{\alpha}_t z + \dot{\sigma}_t \varepsilon$
Why it works	Theorem 6.4	Theorem 11.25
Both are	L^2 projections onto a conditional expectation (Lemma 11.23)	
Generation	reverse SDE <i>or</i> PF-ODE	ODE
Converting	$u = a_t s + b_t x$ — either determines the other (Proposition 11.37)	

The two methods can therefore share a probability path and still be different algorithms, because they learn *different functions of the same path*. On a Gaussian path those functions are related by an explicit invertible map, so in that setting the difference is one of parameterisation and training-target scaling, not of expressive power. This is precisely the claim that Chapter 15 tests, at the level of source code, for SigmaDock versus SigmaFlow.

Caution 11.45 (Where the equivalence stops). Three qualifications, all of which matter for this project.

(i) *It is special to Gaussian paths.* Proposition 11.37 used $p_t(\cdot | z)$ Gaussian at every step. For a non-Gaussian conditional path there is no reason for score and velocity to be related by an affine map with z -independent coefficients, and in general they are not.

(ii) *Equal minimisers is not equal training.* Even where the map exists, it multiplies the target by schedule-dependent factors, so a fixed network capacity and a fixed optimiser will not distribute their errors the same way across t . The loss weighting discussion of (6.7) is exactly about this, and it does not transfer automatically.

(iii) *The source distributions may differ.* As Caution 11.43 noted, a VE diffusion model’s source is not exactly the flow-matching source. Two models can agree on everything in the table above and still be transporting from different starting distributions.

11.14 What does the network predict?

A recurring source of confusion in this literature is that several different quantities are all called “the model output”, and papers frequently switch between them without comment. On

a Gaussian path they are all linear functions of the same two hidden vectors, so we can lay them out once and be done.

Throughout this section, fix a Gaussian path and a training example: a data point z , a noise draw $\varepsilon \sim \mathcal{N}(0, I_d)$, a time t , and the resulting state

$$x = \alpha_t z + \sigma_t \varepsilon.$$

Everything the network could be asked to predict is a linear combination of z and ε .

Name	As a function of (z, ε)	What it means
clean sample z	z	the data point itself; “ x_0 -prediction” in the diffusion literature, “ x_1 -prediction” in ours (Caution 11.46)
noise ε	ε	the noise that was added
score s	$-\varepsilon/\sigma_t$	gradient of the log-density, (11.19)
flow velocity u	$\dot{\alpha}_t z + \dot{\sigma}_t \varepsilon$	the path’s velocity, (11.17)
diffusion v	$\alpha_t \varepsilon - \sigma_t z$	see Definition 11.49; <i>only</i> on the VP trigonometric path

Caution 11.46 (x_0 versus x_1). The clean data point is called x_0 in the diffusion literature (where $s = 0$ is data) and x_1 in the flow-matching literature (where $t = 1$ is data). This document uses z for it wherever possible precisely to avoid the collision, and x_1 where a subscript is unavoidable. When reading a paper, identify which end of the interval carries the data *before* reading any formula.

11.14.1 Interconversion, and exactly when it is possible

Since x is itself a linear combination of z and ε , knowing (x, t) and *any one* of the targets above generally determines the other two hidden vectors, hence all the remaining targets. “Generally” is not “always”, and the condition is worth having.

Proposition 11.47 (Interconvertibility). *Fix t and set*

$$\Delta_t := \dot{\alpha}_t \sigma_t - \alpha_t \dot{\sigma}_t = \sigma_t^2 \frac{d}{dt} \left(\frac{\alpha_t}{\sigma_t} \right). \quad (11.29)$$

Then the linear map $(z, \varepsilon) \mapsto (x, u)$ is invertible if and only if $\Delta_t \neq 0$, in which case

$$z = \frac{\sigma_t u - \dot{\sigma}_t x}{\Delta_t}, \quad \varepsilon = \frac{\dot{\alpha}_t x - \alpha_t u}{\Delta_t}. \quad (11.30)$$

Since α_t/σ_t is the (square root of the) signal-to-noise ratio, the condition $\Delta_t \neq 0$ says exactly that the signal-to-noise ratio is strictly monotone in t .

Proof. In matrix form,

$$\begin{pmatrix} x \\ u \end{pmatrix} = \underbrace{\begin{pmatrix} \alpha_t & \sigma_t \\ \dot{\alpha}_t & \dot{\sigma}_t \end{pmatrix}}_{=: M_t} \begin{pmatrix} z \\ \varepsilon \end{pmatrix},$$

each entry acting as a scalar multiple of I_d . Then $\det M_t = \alpha_t \dot{\sigma}_t - \sigma_t \dot{\alpha}_t = -\Delta_t$, so M_t is invertible iff $\Delta_t \neq 0$, and

$$M_t^{-1} = \frac{1}{-\Delta_t} \begin{pmatrix} \dot{\sigma}_t & -\sigma_t \\ -\dot{\alpha}_t & \alpha_t \end{pmatrix},$$

which gives (11.30) on multiplying out. For the second expression in (11.29), the quotient rule gives $\frac{d}{dt}(\alpha_t/\sigma_t) = (\dot{\alpha}_t\sigma_t - \alpha_t\dot{\sigma}_t)/\sigma_t^2 = \Delta_t/\sigma_t^2$. (Verified numerically for three schedules in Section 11.19, check 9.) $\square$

Example 11.48 (Which parameterisation degenerates where). For the linear path $\alpha_t = t$, $\sigma_t = 1 - t$:

$$\Delta_t = 1 \cdot (1 - t) - t \cdot (-1) = 1 \quad \text{for every } t.$$

The velocity parameterisation is therefore uniformly well conditioned on the whole interval. The others are not:

Target	Recovered from (x, t) via	Degenerates when
z	$z = (1 - t)u + x$	never (for this path)
ε	$\varepsilon = x - tu$	never (for this path)
$\text{score} = -\varepsilon/\sigma_t$	divide by $1 - t$	$t \rightarrow 1$ (no noise left to predict)
z from ε	$z = (x - \sigma_t\varepsilon)/\alpha_t$	$t \rightarrow 0$ (no signal present)
u	—	never (for this path)

This is a concrete, if modest, argument in favour of predicting the flow velocity: on the linear path it is the one target that neither vanishes nor explodes at either endpoint. The argument is about *numerical conditioning*, not about what the model can represent — all these parameterisations have the same optimum wherever they are all defined.

11.14.2 The name collision: diffusion v is not flow velocity

Definition 11.49 (Diffusion v -parameterisation). On the variance-preserving trigonometric path of Example 11.35, with $\alpha_t = \cos \varphi_t$ and $\sigma_t = \sin \varphi_t$, the *v -prediction target* is

$$v := \alpha_t\varepsilon - \sigma_t z. \quad (11.31)$$

Caution 11.50 (Provenance of Definition 11.49). Definition 11.49 originates with Salimans & Ho, *Progressive Distillation for Fast Sampling of Diffusion Models* (ICLR 2022), which is **not** available in this project’s **papers/** directory. The definition is therefore imported from the literature and could not be checked here against a primary source. Everything derived from it below is self-contained algebra performed in this document and does not depend on that source being quoted correctly — but a reader relying on the definition itself should verify it against the original.

The name is unfortunate: “ v ” is routinely read as “velocity”, and flow matching’s u_t is also a velocity. They are not the same quantity, and the following proposition says exactly how close they get.

Proposition 11.51 (Relation between v and u). *On the trigonometric path of Example 11.35,*

$$u_t(x \mid z) = \dot{\varphi}_t v, \quad (11.32)$$

a scalar multiple. With the standard choice $\varphi_t = \frac{\pi}{2}(1 - t)$ this scalar is the constant $-\frac{\pi}{2}$. On a general Gaussian path no such relation holds: u and v are then not even parallel.

Proof. Differentiating $\alpha_t = \cos \varphi_t$ and $\sigma_t = \sin \varphi_t$,

$$\dot{\alpha}_t = -\sin \varphi_t \dot{\varphi}_t = -\sigma_t \dot{\varphi}_t, \quad \dot{\sigma}_t = \cos \varphi_t \dot{\varphi}_t = \alpha_t \dot{\varphi}_t.$$

Substituting into $u_t = \dot{\alpha}_t z + \dot{\sigma}_t \varepsilon$ ((11.17)),

$$u_t = -\sigma_t \dot{\varphi}_t z + \alpha_t \dot{\varphi}_t \varepsilon = \dot{\varphi}_t (\alpha_t \varepsilon - \sigma_t z) = \dot{\varphi}_t v.$$

For the final claim, take the linear path instead ($\alpha_t = t$, $\sigma_t = 1 - t$): then $u_t = z - \varepsilon$ while $v = t\varepsilon - (1 - t)z$. These are proportional for generic z, ε exactly when their coefficient vectors $(1, -1)$ and $-(1 - t), t$ are parallel, i.e. when

$$\det \begin{pmatrix} 1 & -1 \\ -(1 - t) & t \end{pmatrix} = t - (1 - t) = 2t - 1 = 0,$$

that is at the single time $t = \frac{1}{2}$ — where indeed $v = \frac{1}{2}\varepsilon - \frac{1}{2}z = -\frac{1}{2}u_t$ — and at no other time. So on the linear path the relation (11.32) fails everywhere except at one instant, and even there the constant is not the trigonometric path's $\dot{\varphi}_t$. Numerically, over 300 random draws on the linear path the smallest observed $|\cos \angle(u, v)|$ was 0.05 (Section 11.19, check 8) — a scalar multiple at every t would force this to be 1 every time. $\square$

Caution 11.52 (Three ways this gets misread). Proposition 11.51 is a positive result, but a narrow one, and it is worth naming the three claims it does *not* support.

“v-prediction is flow matching.” No. The proportionality holds on one specific path. Change the schedule and the relation disappears entirely.

“The constant does not matter, it is absorbed by the network.” For the *minimiser* of an unweighted squared loss, a constant rescaling of the target does rescale the optimum by the same constant, so in that narrow sense it is harmless. But the constant here is *negative* ($-\frac{\pi}{2}$), which reverses direction, and it is negative because v is defined in the noising direction while u is defined in the generation direction. Feeding a v -prediction into an ODE integrator expecting u integrates the flow backwards.

“Flow matching is just diffusion with a different loss.” This is the broadest version of the error, and Section 11.13 is the precise answer: they coincide when (a) the path is Gaussian, (b) the schedules are matched, and (c) the source distributions agree. Drop any of the three and the claim fails — (c) is exactly what Caution 11.43 shows fails for VE.

11.15 Sampling: integrating the learned ODE

Given a trained $u_\theta \approx u_t$, generation solves $\dot{X}_t = u_\theta(X_t, t)$, $X_0 \sim p_{\text{init}}$, forward to $t = 1$, by any numerical ODE solver; the simplest, and the one this document's code correspondences confirm is what both SigmaDock's (nominally deterministic default, Section 10.10) and SigmaFlow's samplers actually execute despite a “heun” option appearing in each function's type signature, is the forward Euler step,

$$X_{t+\Delta t} = X_t + \Delta t u_\theta(X_t, t). \quad (11.33)$$

It is worth putting this next to the stochastic update it replaces, since the two differ in exactly one term and that term carries all the structural difference between the frameworks:

	Euler–Maruyama (SDE, (5.10))	Euler (ODE, (11.33))
Update	$X + f \Delta t + g\sqrt{\Delta t} \xi$	$X + u \Delta t$
Noise term	$g\sqrt{\Delta t} \xi, \xi \sim \mathcal{N}(0, I)$	absent
Step scaling	drift Δt , noise $\sqrt{\Delta t}$	everything Δt
Order	$O(\sqrt{\Delta t})$ strong	$O(\Delta t)$, improvable
Rerun from same X_0	different output	identical output

The $\sqrt{\Delta t}$ that Section 5.2 spent several pages deriving is simply not present here. That single absence is why the ODE admits high-order solvers and the SDE does not (a scheme cannot achieve order $p > 1/2$ against a term that scales as $\sqrt{\Delta t}$), and why repeated sampling from the same X_0 is exactly reproducible.

Unlike the diffusion sampler of (7.3), there is no injected noise anywhere in this recipe: the entire stochasticity of the generative model is confined to the single draw $X_0 \sim p_{\text{init}}$, after which the trajectory is completely deterministic given the trained network — the structural fact, not an implementation simplification, that makes the “deterministic vs. stochastic” framing of Section 10.10’s noise-annealing discussion an *architecturally forced* property for flow matching specifically, where for SigmaDock (as that section showed) it was, in the shipped default configuration, already true as well, just not forced by the method’s structure.

11.15.1 Higher-order solvers, and what a “step” costs

Since (11.33) is an ordinary initial value problem, the entire classical solver toolbox applies unchanged. We record the hierarchy conceptually only; no solver comparison is conducted in this document.

- **Euler**, (11.33): one evaluation per step, local error $O(\Delta t^2)$, global $O(\Delta t)$.
- **Midpoint** / **Heun**: two evaluations per step; take a trial step, re-evaluate the field there, and combine. Global $O(\Delta t^2)$.
- **Runge–Kutta 4**: four evaluations per step, global $O(\Delta t^4)$.
- **Adaptive schemes** (e.g. Dormand–Prince): estimate the local error from two embedded formulas and shrink or grow Δt automatically. The number of steps is then an output, not an input.

Caution 11.53 (Steps are not network evaluations). The quantity that determines sampling *cost* is the number of **network function evaluations** (NFE), not the number of solver steps. Heun with 50 steps costs 100 NFE, the same as Euler with 100 steps; RK4 with 25 steps also costs 100. Comparing “50-step Heun” against “50-step Euler” and concluding that the higher-order method is better is comparing two methods at 2:1 cost.

Any claim of the form “method A needs fewer steps” should therefore be restated in NFE before being believed. This matters here because “flow matching needs fewer sampling steps” is one of the two standard motivations for the SigmaDock-to-SigmaFlow substitution; it is an empirical claim about a particular trained pair of models, examined in this project’s experimental work and *not* established anywhere in this chapter.

11.16 The path is a design choice

Theorem 11.25 is remarkably permissive: *any* conditional path with a closed-form field will do, and all of them yield a correct generative model. Correctness is therefore not what distinguishes one choice from another. This section collects what does.

11.16.1 Two separate decisions

It is useful to see that “choosing the probability path” actually consists of two independent choices, which the literature often bundles.

	The coupling	The interpolant
What it fixes	which source point is paired with which data point	how a <i>given</i> pair is joined
Formally	a joint law $\pi(x_0, z)$ with the right marginals	a curve $t \mapsto x_t$ from x_0 to z
Default	independent: $\pi = p_{\text{init}} \otimes p_{\text{data}}$	straight line $(1 - t)x_0 + tz$
Where it entered	Remark 11.17	Proposition 11.28

Definition 11.54 (Coupling). A *coupling* of p_{init} and p_{data} is a joint distribution π on $\mathbb{R}^d \times \mathbb{R}^d$ whose first marginal is p_{init} and whose second marginal is p_{data} . The *independent coupling* is the product $\pi = p_{\text{init}} \otimes p_{\text{data}}$: draw the noise and the data point with no reference to each other.

Everything in this chapter so far used the independent coupling, and it is worth appreciating how odd it is when stated plainly: a randomly chosen noise vector is paired with a randomly chosen molecule, and the model is asked to learn the average of all the straight lines this produces. That average is a well-defined and correct vector field — but the individual conditional targets it averages over point in wildly different directions, which is precisely the loss floor of Caution 11.27. A coupling that pairs each noise sample with a *nearby* data point would produce conditional targets that disagree less, hence a smaller floor and a straighter marginal flow.

Caution 11.55 (Two-sided conditioning leaves the framework of Definition 11.16). It is tempting to implement a non-independent coupling by conditioning on the *pair* $z = (x_0, x_1)$ and setting $p_t(\cdot \mid z) = \delta_{(1-t)x_0 + tx_1}$. Note that this conditional “path” is a Dirac *at every* t , not only at $t = 1$, so it has no density and Theorem 11.10 — which we used to prove Theorem 11.21 — does not apply to it as stated. The construction is standard in the literature and can be made rigorous, either by smoothing (add a small $\sigma_{\text{min}} > 0$) or by working with the continuity equation in a weak/distributional sense. This document does not carry out either repair, and flags the gap rather than papering over it. The one-sided construction of Proposition 11.28 does *not* have this problem: there $p_t(\cdot \mid z)$ has a density for every $t < 1$, because X_0 is still random after conditioning on z .

11.16.2 Straight versus curved trajectories

An easily misread point: the linear path makes each *conditional* trajectory a straight line, but the *marginal* trajectories — the ones the sampler actually follows — are generally curved.

Section 11.10.3 shows why: early on, the marginal field aims at the data mean; later it swings towards one mode. A sample’s actual path therefore bends, even though every conditional path it was averaged from was straight.

This matters because curvature is what forces small ODE steps. A perfectly straight marginal trajectory could be integrated exactly in *one* Euler step; the more it bends, the more steps a fixed error tolerance demands. This — not correctness — is the real currency in which paths and couplings are compared, and it is the motivation behind optimal-transport couplings.

Remark 11.56 (Optimal-transport couplings, stated with care). The idea is to replace the independent coupling with one that pairs source and data points so as to minimise the expected squared distance $\mathbb{E}_\pi \|X_0 - Z\|^2$ — the (quadratic) optimal-transport coupling. The motivation is the one just given: shorter and more consistent conditional segments should produce a straighter marginal flow and hence cheaper sampling.

Three cautions. First, this document develops no optimal transport theory, so the motivation above is stated as motivation and not proved here. Second, the exact OT coupling between two continuous distributions is not computable; practical methods solve it approximately within each mini-batch, which introduces a bias that the informal argument does not account for. Third, and most importantly for this project, none of this is used by SigmaFlow: Chapter 13 confirms directly from the source that the implemented coupling is the independent one, on both the translational and the rotational component. The concept is recorded here so that the term is not mysterious if it arises later, not because it describes anything the code does.

11.17 Diffusion and flow matching in one picture

It is worth assembling the whole chapter, and Part III with it, into a single comparison.

	Score-based diffusion	Flow matching
1. The path	<i>derived</i> : run a corrupting SDE $dY_s = f ds + g dW_s$ from the data and take its marginals q_s .	<i>chosen</i> : pick $p_t(\cdot \mid z)$ directly, e.g. $\mathcal{N}(\alpha_t z, \sigma_t^2 I)$.
2. Density law	Fokker–Planck, (5.14)	continuity equation, (11.4)
3. Learn	the score $\nabla_x \log p_t$, by regressing onto the conditional score	the velocity u_t , by regressing onto the conditional velocity
4. Why it works	Theorem 6.4	Theorem 11.25
5. Generate	reverse SDE (6.8), or PF-ODE (6.10)	the ODE $\dot{X} = u_\theta(X, t)$
Needs	a tractable noising process; a heat kernel on the state space	only a closed-form interpolant

Rows 2–4 are the same mathematics twice: in both cases one has a population-level PDE, an intractable marginal object, a tractable conditional surrogate, and a proof — Lemma 11.23 in both cases — that regressing on the surrogate recovers the marginal. The frameworks differ in row 1 and in what row 3 names.

Both build a time-indexed probability path and learn a local quantity from which the route to the data can be reconstructed. Diffusion derives the path from a noising process and learns the score; flow matching prescribes the path and learns the velocity.

The last row is where the practical asymmetry sits, and it is the one that matters for this project. Diffusion needs a noising process whose transition kernel is available in closed form. In $\mathbb{R}^d$ that is a Gaussian and costs nothing. On $SO(3)$ it is the heat kernel, whose IGSO(3) series, truncation, tabulation and caching occupy the whole of Section 7.2.2. Flow matching needs no noising process at all — only a way to interpolate between two points — which on a Riemannian manifold is a geodesic, available in closed form on $SO(3)$. Chapter 12 makes this precise, and it is the first of the two arguments motivating the substitution studied in this project.

11.18 What flow matching is not

Each item below is a statement encountered in practice, followed by what is actually true. All of them have been established above; this section is a checklist, not new material.

1. “*Flow matching is diffusion without the noise.*” Misleading in both directions. It is not obtained by deleting a term from a diffusion model: it does not start from a noising process at all. And the noise is not absent — the source distribution is noise, and the Gaussian path adds noise at every intermediate time. What is absent is *injected noise during sampling* (Section 11.15).
2. “*Flow matching is training the probability flow ODE.*” True only for Gaussian paths with matched schedules and matched sources (Proposition 11.44, Caution 11.45). As a definition it is wrong: flow matching does not require any SDE to exist.
3. “*Flow matching is v -prediction.*” No. Proposition 11.51: the two coincide up to a scalar on one specific path, and the scalar is negative. On the linear path they are not even parallel.
4. “*Flow matching means linear interpolation.*” No. The linear path is one choice among the Gaussian family (Section 11.11), which is itself one family among all conditional paths admitted by Definition 11.16.
5. “*Flow matching requires a Gaussian source.*” No. Definition 11.16 requires only that p_{init} be sampleable and that a conditional path with a closed-form field exist. SigmaFlow’s rotational component uses the Haar measure on $SO(3)$ (Remark 12.29).
6. “*Flow matching is deterministic, therefore it has no randomness.*” The *sampler* is deterministic given X_0 ; the model is not. All the randomness sits in the single draw $X_0 \sim p_{\text{init}}$. And a deterministic sampler is a trade, not an improvement (Caution 6.10).
7. “*A probability path determines its vector field.*” No (Example 11.12). What determines the learned field is the *conditional construction*, via Theorem 11.21.
8. “*A non-zero training loss means the model is underfitting.*” No (Caution 11.27). The CFM loss has a strictly positive floor set by the data and the path.

11.19 Numerical verification of this chapter

Every non-trivial identity derived above was checked numerically, in double precision, by `audits/flow_matching_theory_audit.py` (pure `numpy/scipy`; no GPU, no checkpoint, no repository imports, so it is reproducible by any reader). Three schedules were used throughout: the linear path $(\alpha_t, \sigma_t) = (t, 1 - t)$, the trigonometric VP path of Example 11.35, and a VE path with $\alpha_t \equiv 1$.

#	Statement checked	Method	Residual
1	Proposition 11.33: the flow of $u_t(\cdot z)$ reproduces $\alpha_t z + \sigma_t \varepsilon$	integrate the ODE to $\text{rtol} = 10^{-11}$	$3 \cdot 10^{-13}$
2	Proposition 11.37, conditional (11.21)	random (x, z, t)	$3 \cdot 10^{-14}$
3	Proposition 11.37, marginal (11.22) (11.23): posterior-averaged score $= \nabla \log p_t$	discrete p_{data} , exact mixture central differences, $h = 10^{-6}$	$3 \cdot 10^{-14}$ $7 \cdot 10^{-8}$
4	Theorem 11.10 for the marginal pair (p_t, u_t)	finite differences in t and x	$3 \cdot 10^{-8}$ rel.
5	Theorem 11.21: $u_t(x) = \mathbb{E}[u_t(x Z) X_t = x]$	$8 \cdot 10^7$ draws, ball of radius 0.02, 32 331 hits	$0.72 \sigma_{\text{MC}}$
6	(11.14), the closed form of Section 11.10.3	against a direct posterior average	$7 \cdot 10^{-15}$
7	Proposition 11.44, VP and VE	realistic schedules, see proof	$8 \cdot 10^{-9}$
8	Proposition 11.51: $u = \dot{\varphi}_t v$ on the trigonometric path and <i>not</i> parallel on the linear path except at $t = \frac{1}{2}$, where $u = -2v$	random (z, ε, t) min $ \cos \angle(u, v) $ over 300 draws the exception identified in the proof	$3 \cdot 10^{-15}$ 0.05 $4 \cdot 10^{-16}$
9	(11.29): $\Delta_t = \sigma_t^2 \frac{d}{dt}(\alpha_t / \sigma_t)$	central differences	$1 \cdot 10^{-9}$ rel.
10	Example 11.12: $\nabla \cdot (p w) = 0$ for the rotation field	central differences	$2 \cdot 10^{-12}$
11	Lemma 11.23, and the loss floor of Caution 11.27	$2 \cdot 10^6$ Monte-Carlo draws	$4 \cdot 10^{-5}$ rel.

Check 5 is reported in units of the Monte-Carlo standard error rather than in absolute terms, because the correct statement about a stochastic estimate is that the discrepancy is small *relative to its own sampling noise*; 0.72 standard errors is consistent with the identity holding exactly. Check 11 additionally produced the numbers quoted in Caution 11.27: at the optimum, $\mathcal{L}_{\text{CFM}} = 1.5821$ while $\mathcal{L}_{\text{FM}} = 0$ exactly.

Remark 11.57 (A methodological note). Two of these checks were initially written in a form that could not have detected an error. The Monte-Carlo test of check 5 was first given an absolute tolerance tighter than its own sampling error, so it failed on a correct identity; and an earlier version of the posterior computation underflowed to 0/0 for small σ_t , silently producing NaNs that a max-residual test reports as “not greater than the tolerance”. Both were repaired (log-space posterior; tolerance stated in standard errors). The general lesson

— that a numerical check must be audited for its ability to fail before its passing is taken as evidence — is the same one recorded after the audits of Chapter 14.

11.20 Where $\mathbb{R}^d$ runs out

Everything above lives in $\mathbb{R}^d$, and it is worth being precise about where that was used — because the SigmaFlow state space is not $\mathbb{R}^d$, and the next chapter has to repair exactly these places.

Three expressions carried the whole construction:

Expression	Where it appeared	What it assumes
$(1 - t)x_0 + tz$	Proposition 11.28	points can be scaled and added
$z - x$	(11.13)	the difference of two points is a vector
$\ u_\theta - u\ ^2$	Theorem 11.25	one inner product for all points

On a rotation matrix, none of these is defined. The average of two rotation matrices is not a rotation matrix; the difference $R_1 - R_0$ is a matrix but not a rotation and not a meaningful “direction from R_0 to R_1 ”; and there is no single vector space in which all velocities live, since a velocity at R is constrained to be tangent to the group *at that point*.

Each therefore needs a geometric replacement, and Chapter 3 has already built all three:

Euclidean	Riemannian replacement	Developed in
$x + v$	$\exp_x(v)$	Definition 3.17
$y - x$	$\log_x(y)$	Definition 3.17
$(1 - t)x_0 + tz$	$\exp_{x_0}(t \log_{x_0}(z))$, the geodesic	Definition 12.9
$\ \cdot\ $	$\ \cdot\ _g$, the Riemannian norm at each point	Definition 3.8
$\nabla \cdot$	div_g , the Riemannian divergence	Section 12.2

The remarkable fact, and the subject of the next chapter, is that *only* these substitutions are needed. The marginalisation theorem and the L^2 projection argument used nothing about $\mathbb{R}^d$ beyond linearity of integration and the existence of the continuity equation, so they survive the move to a manifold verbatim. The one thing that does not come for free is a *concrete* conditional path with a closed-form velocity on the specific manifold at hand — and on $SO(3)$ that turns out to be two lines.

11.21 What you should now be able to do

- set up the generative transport problem and say what a probability path is, and what it is not (it does not determine trajectories, Example 11.12);
- derive the continuity equation from conservation of mass and read every term in it;
- explain why the marginal velocity field is unavailable, and why regressing on the conditional one nevertheless recovers it — the L^2 projection argument of Lemma 11.23, which you should be able to reproduce;
- state why a perfectly trained flow matching model still has a strictly positive loss (Caution 11.27);

- convert between the five things a network might predict — ε , score, clean sample, diffusion v , flow u — and say when the conversion is possible;
- show that a diffusion model’s marginals *are* a Gaussian probability path, and that its probability flow ODE *is* that path’s velocity (Proposition 11.44).

Everything so far was in $\mathbb{R}^d$. Section 11.20 lists the three expressions that break on rotations, and Chapter 12 repairs them.

Chapter 12

Riemannian Flow Matching

Chapter 11 built flow matching from scratch in $\mathbb{R}^d$ and ended, in Section 11.20, by listing the three Euclidean expressions the construction rested on. This chapter carries out the replacement those three expressions demand, and the outcome is worth stating before the work begins: *nothing in the logic of flow matching changes*. The probability path, the conditional construction, the marginalisation theorem, the L^2 projection argument and the training loss all survive verbatim. What changes is the geometry underneath them — and the whole difficulty is in noticing exactly where geometry was being used implicitly.

Remark 12.1 (Sources used in this chapter). The primary source is Chen & Lipman, *Flow Matching on General Geometries*, ICLR 2024, present in this project’s `papers/` directory and read in full for §2–§3.2; the construction below is the geodesic special case of their premetric framework, and every statement taken from there has been re-derived here rather than quoted. Their notation is matched to this document’s as follows: their x_1 is our data point (*same*), their p is our p_{init} , their q is our p_{data} , their scheduler $\kappa(t)$ with $\kappa(0) = 1$, $\kappa(1) = 0$ corresponds to our $1 - t$, and their time convention is ours ($t = 0$ noise, $t = 1$ data).

Geometry follows do Carmo, *Riemannian Geometry* (1992) and Lee, *Introduction to Riemannian Manifolds*, 2nd ed. (2018) for the standard facts imported in Chapter 3; the $SO(3)$ -specific conventions were fixed there, by derivation, in Section 3.5. Chapter 3 additionally now *proves* (Proposition 3.15) the one-parameter-subgroup fact that earlier drafts imported.

Not available locally: Lipman et al. (2023), the original Euclidean flow-matching paper, and Yim et al.’s $SE(3)$ flow-matching follow-up is present but is used only where Chapter 13 cites it. No statement below depends on an unavailable source.

12.1 What breaks, and what survives

Take the Euclidean construction and read it with a suspicious eye. The conditional path was

$$x_t = (1 - t)x_0 + tx_1, \quad u_t(x \mid x_1) = \frac{x_1 - x}{1 - t}, \quad \mathcal{L} = \mathbb{E} \|u_\theta(x_t, t) - u_t(x_t \mid x_1)\|_2^2.$$

Now suppose x_0 and x_1 are rotation matrices. Every symbol in those three formulas fails, and for three *different* reasons:

Expression	What it needs	Why it fails on $SO(3)$
$x_1 - x_0$	subtraction of points	$R_1 - R_0$ is a 3×3 matrix, but not a rotation and not a meaningful “direction from R_0 to R_1 ”
$(1 - t)x_0 + tx_1$	scaling and adding points	the average of two rotation matrices is not orthogonal; e.g. $\frac{1}{2}(I + R_z(\pi)) = \text{diag}(0, 0, 1)$, which is singular
$\ \cdot\ _2^2$	one inner product, valid everywhere	velocities at different rotations live in different tangent spaces (Section 3.1.1); there is no single $\mathbb{R}^d$ holding all of them

The middle row is worth checking, since it is the most concrete refutation of “just interpolate the matrices”. With $R_z(\pi) = \text{diag}(-1, -1, 1)$, the entrywise midpoint $\frac{1}{2}(I + R_z(\pi))$ is $\text{diag}(0, 0, 1)$, whose determinant is 0: it collapses two of the three axes entirely. It is not a rotation, not invertible, and not a small error — it is a complete loss of two dimensions of the pose.

12.1.1 Points are not vectors: the distinction that does all the work

Every one of those three failures is the same failure wearing a different hat, and it is worth naming once more, because in $\mathbb{R}^d$ it is invisible and here it is decisive.

	Point $x \in \mathcal{M}$	Tangent vector $v \in T_x \mathcal{M}$
What it is	a state of the system	a way the state can change
On $SO(3)$	a 3×3 matrix with $R^\top R = I$	a 3-vector ω (via $\dot{R} = R\hat{\omega}$)
Can be added?	no	yes, to other vectors <i>at the same point</i>
Lives in	a curved space	a vector space, one per point
In $\mathbb{R}^d$	$\mathbb{R}^d$	$\mathbb{R}^d$ — <i>the same space</i> , hence the confusion

In $\mathbb{R}^d$ the identification $T_x \mathbb{R}^d \cong \mathbb{R}^d$ holds for every x and all these copies are canonically the same, so “point” and “vector” become interchangeable and one stops distinguishing them. That habit is what breaks. On a manifold the two are different kinds of object with different type signatures, and the entire content of this chapter is a systematic substitution that respects the difference.

Euclidean operation	Riemannian replacement
$x + v$ (point + vector $\rightarrow$ point)	$\exp_x(v)$ (Definition 3.17)
$y - x$ (point – point $\rightarrow$ vector)	$\log_x(y)$
$(1 - t)x_0 + tx_1$	$\exp_{x_0}(t \log_{x_0}(x_1))$ (Corollary 3.22)
$\ v\ _2$	$\ v\ _{g_x}$ (Definition 3.8)
$\nabla \cdot$	div_g (Definition 3.35)
dx (Lebesgue)	$d\text{vol}_g$ (Definition 3.28)

Everything below is the consequence of taking that table seriously.

12.2 Tangent-vector-valued dynamics

The first casualty is the type of the network’s output.

Definition 12.2 (Vector field on a manifold, flow). A (time-dependent) *vector field* on $\mathcal{M}$ is $u : \mathcal{M} \times [0, 1] \rightarrow T\mathcal{M}$ with $u_t(x) \in T_x\mathcal{M}$ for every x — a section of the tangent bundle (Definition 3.4) at each time, in the sense of Definition 3.5. Its *flow* $\phi_t : \mathcal{M} \rightarrow \mathcal{M}$ solves $\frac{d}{dt}\phi_t(x) = u_t(\phi_t(x))$, $\phi_0(x) = x$, exactly as in Definition 11.5, now understood as a curve *on the manifold* whose velocity at each instant is the prescribed tangent vector.

Compare the two signatures side by side:

$$\text{Euclidean: } u_t : \mathbb{R}^d \rightarrow \mathbb{R}^d, \quad \text{Riemannian: } u_t(x) \in T_x\mathcal{M}.$$

On the left, the codomain is one fixed vector space and the output at any x could equally have been produced at any other x . On the right, the codomain *depends on the input*: an output that is legal at x is generally illegal at x' .

*A Riemannian flow matching model may not emit an arbitrary vector in some ambient $\mathbb{R}^N$. It must emit a vector tangent to the manifold **at the point it was given**.*

Example 12.3 (The radial field on S^2 is not a vector field on S^2). This is worth doing concretely, since it is the cleanest picture of the constraint. On $S^2 \subset \mathbb{R}^3$, $T_x S^2 = \{v \in \mathbb{R}^3 : x^\top v = 0\}$: a legal velocity is perpendicular to the position vector.

Now take the radial assignment $u(x) = x$ — “move outward”. It is smooth, it is defined at every point of the sphere, and it is a perfectly good map $S^2 \rightarrow \mathbb{R}^3$. It is not a vector field on S^2 , because $x^\top u(x) = \|x\|^2 = 1 \neq 0$. What goes wrong if we use it anyway? Solving $\dot{X} = X$ gives $X_t = e^t X_0$, so $\|X_t\| = e^t > 1$ for every $t > 0$: the sample leaves the sphere immediately and never returns. It is not a slightly wrong trajectory on S^2 ; it is not a trajectory on S^2 at all.

Now take $u(x) = e_3 - (e_3^\top x)x$, the tangential part of the constant field e_3 . Then $x^\top u(x) = e_3^\top x - (e_3^\top x)\|x\|^2 = 0$: legal. Its flow slides every point along its meridian towards the north pole, staying on the sphere forever.

The difference between the two is a single projection. That projection is what the tangency constraint costs, and Section 12.10.6 shows the $SO(3)$ version of the same statement.

Proposition 12.4 (Tangency is exactly what keeps the sample on $\mathcal{M}$). *If u_t is a vector field on $\mathcal{M}$ in the sense of Definition 12.2 and $X_0 \in \mathcal{M}$, then the solution of $\dot{X}_t = u_t(X_t)$ satisfies $X_t \in \mathcal{M}$ for all t .*

This is Proposition 3.7, proved in Chapter 3: the defining constraint of the manifold has zero derivative along the flow precisely because the velocity is annihilated by the constraint’s differential. Two consequences:

- *In exact arithmetic, no projection step is needed.* The generative ODE cannot produce an off-manifold sample.

- *After discretisation, it is needed — or avoided by construction.* A finite Euler step $X + \Delta t u$ taken in the ambient space *does* leave the manifold, by $O(\Delta t^2)$. This is why the $SO(3)$ integrator uses $R \exp(\Delta t \hat{v})$ (Proposition 3.63), whose output is exactly orthogonal for any step size, rather than $R + \Delta t R \hat{v}$, which is not orthogonal for any step size other than zero.

12.2.1 Curves, velocities, and what the ODE now means

Everything about the generative ODE is unchanged in form and changed in type. For a curve $\gamma : [0, 1] \rightarrow \mathcal{M}$, the velocity $\dot{\gamma}(t)$ is, by Definition 3.2, an element of $T_{\gamma(t)}\mathcal{M}$: the tangent space *at the point the curve is currently passing through*, which moves as the curve does. So the equation

$$\dot{X}_t = u_t(X_t)$$

is an equality between two elements of $T_{X_t}\mathcal{M}$, and it is type-correct exactly because u_t is a vector field in the sense of Definition 12.2. Writing the same equation with an untangential right-hand side is not a bad approximation; it is a type error.

12.3 Riemannian probability paths

Distributions on a manifold need one piece of care that $\mathbb{R}^d$ let us skip: a density is a density *with respect to something* (Section 3.2.1). We use the Riemannian volume measure throughout, per Notation 3.30, so that

$$\mathbb{P}(X \in A) = \int_A p(x) d\text{vol}_g(x), \quad \int_{\mathcal{M}} p d\text{vol}_g = 1.$$

Definition 12.5 (Probability path on a manifold). A *probability path* on $\mathcal{M}$ is a family $(p_t)_{t \in [0, 1]}$ of densities with respect to $d\text{vol}_g$, depending continuously on t . It *connects* p_{init} to p_{data} if $p_0 = p_{\text{init}}$ and $p_1 = p_{\text{data}}$. A *conditional* probability path is a family $p_t(\cdot | z)$, $z \in \mathcal{M}$, with $p_0(\cdot | z) = p_{\text{init}}$ and $p_t(\cdot | z) \rightarrow \delta_z$ as $t \rightarrow 1$, and the *marginal path* it induces is

$$p_t(x) = \int_{\mathcal{M}} p_t(x | z) p_{\text{data}}(z) d\text{vol}_g(z). \quad (12.1)$$

This is Definitions 11.2, 11.16 and 11.18 transcribed, with dz replaced by $d\text{vol}_g(z)$ and $\mathbb{R}^d$ by $\mathcal{M}$. The boundary computation of Section 11.6 goes through unchanged: $p_0 = p_{\text{init}}$ because every conditional starts there and $\int p_{\text{data}} d\text{vol}_g = 1$, and $p_1 = p_{\text{data}}$ because the conditionals collapse onto their own z .

Remark 12.6 (The source distribution is a free choice, here too). Nothing in Definition 12.5 constrains p_{init} beyond being a distribution on $\mathcal{M}$ that we can sample. On $\mathbb{R}^3$ the usual choice is a Gaussian; on a compact group the natural choice is the uniform (Haar) measure, which by Proposition 3.32 is the constant density with respect to $d\text{vol}_g$. But “natural” is not “required”: Section 11.16’s point that the path is a design decision carries over verbatim, and p_{init} is part of that design. Remark 12.29 records what SigmaFlow actually uses.

12.4 The Riemannian continuity equation

We now need the manifold version of Theorem 11.10: the equation connecting a probability path to the vector fields that generate it. Section 11.3.1 derived the Euclidean case twice —

once by counting flux across the ends of an interval, once by transporting the density along the flow. Both arguments generalise; we give the first, since it is the one that makes the conservation content visible, and then check the formula in coordinates.

12.4.1 Conservation, on a manifold

Fix a compact region $A \subseteq \mathcal{M}$ with smooth boundary, and ask how the probability inside it changes. As in $\mathbb{R}^d$, probability is neither created nor destroyed: it can only cross ∂A . The rate at which it crosses, per unit boundary area, is the component of the flux $p_t u_t$ normal to the boundary, so

$$\frac{d}{dt} \int_A p_t \, d\text{vol}_g = - \int_{\partial A} g(p_t u_t, \nu) \, dS, \quad (12.2)$$

with ν the outward unit normal (the minus sign because outward flux *reduces* the contents). Now apply the Riemannian divergence theorem (3.9) to the vector field $V = p_t u_t$, which converts the boundary integral into a volume integral:

$$\int_{\partial A} g(p_t u_t, \nu) \, dS = \int_A \text{div}_g(p_t u_t) \, d\text{vol}_g.$$

Substituting into (12.2) and moving the time derivative inside the integral (the region A is fixed, so nothing else depends on t),

$$\int_A \left(\partial_t p_t + \text{div}_g(p_t u_t) \right) d\text{vol}_g = 0.$$

This holds for *every* such region A , and a continuous function whose integral vanishes over every region vanishes identically. Hence:

Theorem 12.7 (Riemannian continuity equation). *Let u_t be a vector field on $\mathcal{M}$ with flow ϕ_t and let p_t be the density (with respect to $d\text{vol}_g$) of $(\phi_t)_\# p_{\text{init}}$. Then*

$$\boxed{\partial_t p_t(x) + \text{div}_g(p_t(x) u_t(x)) = 0} \quad (12.3)$$

wherever p_t is differentiable. Conversely, under the same regularity hypotheses as Theorem 11.10, a pair (p_t, u_t) satisfying (12.3) has u_t generating (p_t) . We say u_t generates (p_t) in this case.

12.4.2 The same equation, seen in coordinates

It is worth seeing (12.3) a second way, because it explains where the $1/\sqrt{|g|}$ inside div_g comes from and makes the Euclidean special case obvious.

Work in a chart. The probability contained in a coordinate box is $p_t \sqrt{|g|} dx^1 \cdots dx^n$ by Definition 3.28, so the quantity that behaves like an ordinary Euclidean density in these coordinates is not p_t but

$$\tilde{p}_t := p_t \sqrt{|g|}.$$

Conservation of mass in the coordinate chart is then the *flat* continuity equation for $\tilde{p}_t$ with the coordinate velocity components u^i ,

$$\partial_t \tilde{p}_t + \sum_i \partial_i (\tilde{p}_t u^i) = 0.$$

Since $\sqrt{|g|}$ does not depend on t , substituting $\tilde{p}_t = p_t \sqrt{|g|}$ and dividing through by $\sqrt{|g|}$ gives

$$\partial_t p_t + \frac{1}{\sqrt{|g|}} \sum_i \partial_i \left(\sqrt{|g|} p_t u^i \right) = 0,$$

and the second term is exactly $\operatorname{div}_g(p_t u_t)$ by (3.7). So:

The Riemannian continuity equation is the flat continuity equation applied to the coordinate density $p\sqrt{|g|}$; the two factors of $\sqrt{|g|}$ inside div_g are the conversion into coordinate volume and back.

Example 12.8 (Euclidean special case). With $g_{ij} = \delta_{ij}$ we have $\sqrt{|g|} = 1$, $\tilde{p} = p$, and $\operatorname{div}_g = \nabla \cdot$ (Example 3.36), so (12.3) becomes $\partial_t p_t + \nabla \cdot (p_t u_t) = 0$, which is (11.4) exactly. Theorem 12.7 generalises Theorem 11.10; it does not replace it.

12.5 Conditional geodesic paths and their velocity

We now need a concrete conditional path with a closed-form velocity. In $\mathbb{R}^d$ we used the straight line. On $\mathcal{M}$ the straight line's replacement is the geodesic, and Corollary 3.22 already built it.

Definition 12.9 (Geodesic interpolant). For $x_0, z \in \mathcal{M}$ with $z \notin \operatorname{Cut}(x_0)$ (Definition 3.23), the *geodesic interpolant* between x_0 and z is

$$\gamma_t(x_0, z) := \exp_{x_0}(t \log_{x_0}(z)), \quad t \in [0, 1]. \quad (12.4)$$

The associated *conditional path* $p_t(\cdot \mid z)$ is the law of $\gamma_t(X_0, z)$ for $X_0 \sim p_{\text{init}}$, drawn independently of z .

By Corollary 3.22 this is the constant-speed minimising geodesic from x_0 ($t = 0$) to z ($t = 1$), so Definition 12.5's boundary conditions hold: at $t = 0$ the law is p_{init} , and at $t = 1$ every sample has arrived at z , so the law is δ_z .

Example 12.10 (The flat case is the linear path). On $\mathcal{M} = \mathbb{R}^d$ with the flat metric, $\exp_x(v) = x + v$ and $\log_x(y) = y - x$ exactly, so

$$\gamma_t(x_0, z) = x_0 + t(z - x_0) = (1 - t)x_0 + tz,$$

recovering Proposition 11.28's linear path precisely.

12.5.1 The conditional vector field, in general

Here is the result that Chapter 11 needed and that earlier drafts of this document stated only for $SO(3)$. It holds on any Riemannian manifold, and its proof needs nothing beyond the two lemmas of Chapter 3.

Proposition 12.11 (Conditional vector field of the geodesic interpolant). *Let $x_0, z \in \mathcal{M}$ with $z \notin \operatorname{Cut}(x_0)$, and let $X_t = \gamma_t(x_0, z)$ be the geodesic interpolant (12.4). Then for every $t \in [0, 1)$ we have $z \notin \operatorname{Cut}(X_t)$, and the velocity of the interpolant is*

$$\dot{X}_t = \frac{\log_{X_t}(z)}{1 - t} \in T_{X_t} \mathcal{M}. \quad (12.5)$$

Moreover $\|\dot{X}_t\|_g = d_g(x_0, z)$ for every t : the speed is constant and equal to the distance to be covered.

Proof. Write $v := \log_{x_0}(z) \in T_{x_0}\mathcal{M}$, so that $X_t = \exp_{x_0}(tv) = \gamma_v(t)$ by Proposition 3.21, with γ_v the geodesic through x_0 with initial velocity v . Since $z \notin \text{Cut}(x_0)$, $\gamma_v|_{[0,1]}$ is the unique minimising geodesic from x_0 to z , and it remains minimising for a short while past $t = 1$ (this is what “ z is not a cut point of x_0 ” asserts, Definition 3.23).

Fix $t \in [0, 1)$ and consider the reparameterised restriction

$$\sigma(s) := \gamma_v(t + s(1 - t)), \quad s \in [0, 1].$$

Three facts about σ , each already available:

- (a) σ is a **geodesic**, by Lemma 3.20 with $a = 1 - t$, $b = t$ (an affine reparameterisation).
- (b) σ is **minimising**, by Lemma 3.26: it is the sub-arc $\gamma_v|_{[t,1]}$ of a minimising geodesic, up to that affine reparameterisation. Applying the same lemma to the slightly extended geodesic shows $\gamma_v|_{[t,1+\varepsilon]}$ is minimising too, so $z = \gamma_v(1)$ is *not* a cut point of $X_t = \gamma_v(t)$ — which is the first claim.
- (c) Its endpoints and initial velocity are

$$\sigma(0) = \gamma_v(t) = X_t, \quad \sigma(1) = \gamma_v(1) = z, \quad \dot{\sigma}(0) = (1 - t) \dot{\gamma}_v(t) = (1 - t) \dot{X}_t,$$

the last by the chain rule, as recorded in Lemma 3.20.

Now read σ through the definition of the exponential map (Definition 3.17): σ is *the* geodesic starting at X_t with initial velocity $\dot{\sigma}(0)$, so its value at parameter 1 is

$$z = \sigma(1) = \exp_{X_t}(\dot{\sigma}(0)) = \exp_{X_t}((1 - t)\dot{X}_t).$$

Because $z \notin \text{Cut}(X_t)$ by (b), $\log_{X_t}$ is defined at z and inverts $\exp_{X_t}$ on the relevant ball; moreover σ is minimising, so $(1 - t)\dot{X}_t$ is the *principal* preimage, i.e. the one $\log_{X_t}$ returns. Applying $\log_{X_t}$ to both sides,

$$\log_{X_t}(z) = (1 - t) \dot{X}_t,$$

and dividing by $1 - t > 0$ gives (12.5). □

Remark 12.12 (No Jacobi fields required). An earlier draft of this document deferred this proposition to the $SO(3)$ case, on the grounds that the general argument “requires facts about geodesics under reparameterisation and Jacobi fields”. The reparameterisation half of that was right; the Jacobi-field half was not. The proof above uses only Lemma 3.20 (affine reparameterisation preserves geodesics — two lines) and Lemma 3.26 (a sub-arc of a minimising curve is minimising — three lines). This is recorded because overstating a prerequisite is as much an error as understating one, and because the resulting proof is short enough to be worth reading.

Caution 12.13 (Two forms, one vector, and a fake singularity). Equation (12.5) has the same double character as its Euclidean ancestor (11.13), and for the same reason.

As a function on $\mathcal{M}$ at fixed t , the map $x \mapsto \log_x(z)/(1 - t)$ genuinely blows up as $t \rightarrow 1$: a point far from z with little time left must move fast.

Along the interpolant itself, nothing blows up. By (3.2) and constant speed,

$$\|\log_{X_t}(z)\|_g = d_g(X_t, z) = (1 - t) d_g(x_0, z),$$

so the numerator carries a factor $(1 - t)$ that cancels the denominator exactly, leaving $\|\dot{X}_t\|_g = d_g(x_0, z)$, constant — as Proposition 3.13 requires.

The practical consequence is identical to the Euclidean one (Caution 11.29): compute the training *target* from the endpoint form, which is finite everywhere, and never evaluate the current-point form near $t = 1$ on a trajectory that has drifted off the ideal path.

Example 12.14 (Euclidean consistency check). This check is worth doing explicitly, because it is the cheapest way to confirm that no sign or factor was lost in the generalisation. On $\mathcal{M} = \mathbb{R}^d$ with the flat metric, $\log_x(y) = y - x$, so (12.5) reads

$$\dot{X}_t = \frac{\log_{X_t}(x_1)}{1-t} = \frac{x_1 - X_t}{1-t},$$

which is precisely (11.13). And along the interpolant $X_t = (1-t)x_0 + tx_1$ we get $x_1 - X_t = (1-t)(x_1 - x_0)$, so $\dot{X}_t = x_1 - x_0$ — the endpoint form of Proposition 11.28. Both forms of the Euclidean conditional field are recovered, with the same signs. Section 12.16, check 5, verifies this numerically on three manifolds at once.

12.6 Marginalisation and the RCFM objective

We have a conditional path and a closed-form conditional field. What remains is to show that the Euclidean argument — regress on the conditional target, obtain the marginal field — survives. It does, and it is worth being precise about *why*, because the reason is a small observation that is easy to skip and impossible to do without.

12.6.1 Why averaging tangent vectors is legal here

In $\mathbb{R}^d$, Theorem 11.21 defined the marginal field as a conditional expectation, $u_t(x) = \mathbb{E}[u_t(x \mid Z) \mid X_t = x]$, an average of vectors. On a manifold this should give pause: tangent vectors at *different* points cannot be added (Section 3.1.4), so an expectation of tangent vectors is in general meaningless.

The resolution is that we are not averaging over different points.

*The conditioning event is $X_t = x$. Every conditional velocity being averaged is therefore $u_t(x \mid z)$ — evaluated at the **same** point x , for different data points z . All of them lie in the single vector space $T_x\mathcal{M}$.*

Once that is noticed, everything goes through: $T_x\mathcal{M}$ is a finite-dimensional real vector space equipped with an inner product g_x , so expectations, variances, and L^2 projections in it are ordinary linear algebra. What varies with z is the vector, not the space it lives in.

Caution 12.15 (What would go wrong otherwise). The corresponding statement for a *different* conditioning would be false. For instance $\mathbb{E}[X_t]$ — an unconditional expectation over the joint law of (X_0, Z) , with X_t varying — is a sum of vectors at different points and is not defined without extra structure (a parallel transport, or an embedding). Riemannian flow matching never needs it, and the reason it never needs it is exactly the conditioning on $X_t = x$. This is the single place where the Euclidean derivation was quietly using a property of $\mathbb{R}^d$, and the single place where checking is required.

12.6.2 The marginalisation theorem

Theorem 12.16 (Marginalisation of vector fields, Riemannian case). *Let $u_t(\cdot \mid z)$ generate the conditional path $p_t(\cdot \mid z)$ for every z , in the sense of Theorem 12.7. Then the marginal field*

$$u_t(x) := \int_{\mathcal{M}} u_t(x \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} d\text{vol}_g(z) = \mathbb{E}[u_t(x \mid Z) \mid X_t = x] \quad (12.6)$$

is a well-defined element of $T_x\mathcal{M}$ for each x , and it generates the marginal path (12.1).

Proof. Well-definedness. The integrand is, for each z , a vector in $T_x\mathcal{M}$ scaled by a non-negative number, and the weights $\pi(z | x, t) := p_t(x | z)p_{\text{data}}(z)/p_t(x)$ integrate to 1 over z (by (12.1)). So (12.6) is an average of elements of the fixed finite-dimensional vector space $T_x\mathcal{M}$, per Section 12.6.1, hence an element of $T_x\mathcal{M}$. In particular u_t is a legitimate vector field in the sense of Definition 12.2 — it is tangent at every point, automatically, because each term being averaged is.

It generates the marginal path. By Theorem 12.7 it suffices to verify (12.3) for the pair (p_t, u_t) . Each conditional pair satisfies it: $\partial_t p_t(x | z) + \text{div}_g(p_t(x | z)u_t(x | z)) = 0$. Multiply by $p_{\text{data}}(z)$ and integrate over z with respect to $d\text{vol}_g(z)$; both ∂_t and div_g act on the x -variable only, so they commute with $\int \cdot d\text{vol}_g(z)$ (dominated convergence, as in Theorem 11.21's proof). This gives

$$\underbrace{\partial_t \int p_t(x | z)p_{\text{data}}(z)d\text{vol}_g(z)}_{=p_t(x)} + \text{div}_g \underbrace{\int u_t(x | z)p_t(x | z)p_{\text{data}}(z)d\text{vol}_g(z)}_{=:V(x)} = 0.$$

The bracket $V(x)$ is again an average of tangent vectors at the single point x , hence well defined, and multiplying and dividing by $p_t(x)$ identifies it as $V(x) = p_t(x)u_t(x)$ with u_t exactly (12.6). Hence $\partial_t p_t + \text{div}_g(p_t u_t) = 0$, and the converse half of Theorem 12.7 completes the proof. $\square$

Note that the proof used the manifold structure in exactly two places: the reference measure in the integrals, and the tangency bookkeeping of Section 12.6.1. The algebra is identical to Theorem 11.21's.

12.6.3 The L^2 projection, at a fixed point

Lemma 12.17 (Orthogonal decomposition in a tangent space). *Fix $x \in \mathcal{M}$ and t . Let Y be a random element of the inner-product space $(T_x\mathcal{M}, g_x)$ with $\mathbb{E}\|Y\|_g^2 < \infty$, let X be any random variable, and let $f(X)$ take values in $T_x\mathcal{M}$. Then*

$$\mathbb{E}\|f(X) - Y\|_g^2 = \mathbb{E}\|f(X) - \mathbb{E}[Y | X]\|_g^2 + \mathbb{E}\|Y - \mathbb{E}[Y | X]\|_g^2,$$

so the minimiser is $f^*(X) = \mathbb{E}[Y | X]$.

Proof. Lemma 11.23's proof used only that the target values live in a real inner-product space: it added and subtracted $\mathbb{E}[Y | X]$, expanded the square with respect to that inner product, and killed the cross term with the tower property. Replacing $\langle \cdot, \cdot \rangle_{\mathbb{R}^d}$ by $g_x(\cdot, \cdot)$ throughout changes nothing — g_x is bilinear, symmetric and positive definite, which is all the argument used. $\square$

Theorem 12.18 (Riemannian conditional flow matching). *Let $p_t(\cdot | z)$ be a conditional probability path on $\mathcal{M}$ with conditional vector field $u_t(\cdot | z)$, and let u_t be the marginal field (12.6). Then*

$$\mathcal{L}_{\text{RCFM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}(0,1), \, z \sim p_{\text{data}} \\ x \sim p_t(\cdot | z)}} \left[\|u_\theta(x, t) - u_t(x | z)\|_{g_x}^2 \right] \quad (12.7)$$

satisfies

$$\mathcal{L}_{\text{RCFM}}(\theta) = \mathcal{L}_{\text{RFM}}(\theta) + C, \quad \mathcal{L}_{\text{RFM}}(\theta) := \mathbb{E}_{t, x \sim p_t} \|u_\theta(x, t) - u_t(x)\|_{g_x}^2,$$

with $C \geq 0$ independent of θ . The two therefore have the same gradients and the same minimisers, and the minimiser over all tangent-valued measurable functions is $u_{\theta^*}(x, t) = u_t(x)$.

Proof. Apply Lemma 12.17 with $X = (X_t, t)$ and $Y = u_t(X_t \mid Z)$, under the joint law $t \sim \mathcal{U}(0, 1)$, $Z \sim p_{\text{data}}$, $X_t \sim p_t(\cdot \mid Z)$; conditionally on $X_t = x$ both $f(X) = u_\theta(x, t)$ and Y take values in the single space $T_x \mathcal{M}$ (Section 12.6.1), so the lemma applies pointwise in x and then integrates over $x \sim p_t$. Its first term is $\mathcal{L}_{\text{RFM}}$, because $\mathbb{E}[u_t(X_t \mid Z) \mid X_t = x, t] = u_t(x)$ is exactly Theorem 12.16; its second term is the θ -free constant C , non-negative as an expected squared norm. $\square$

Caution 12.19 (The loss floor is still there). As in Caution 11.27, $C = \mathbb{E}[\text{Var}_g(u_t(X_t \mid Z) \mid X_t, t)]$ is strictly positive for any non-degenerate dataset, so a perfectly trained Riemannian flow matching model has a strictly positive training loss. On a compact manifold this floor can be large in absolute terms: the conditional targets $\log_{X_t}(z)/(1 - t)$ for different z can point in genuinely opposite directions, and their conditional variance does not shrink with better training.

12.7 What actually changed

Before working examples, it is worth pausing to see how little the previous two sections altered. Line by line against Chapter 11:

	Euclidean (Chapter 11)	Riemannian (this chapter)
Difference of points	$x_1 - x_0$	$\log_{x_0}(x_1)$
Interpolant	$(1 - t)x_0 + tx_1$, a straight line	$\exp_{x_0}(t \log_{x_0}(x_1))$, a geodesic
Conditional field	$\frac{x_1 - x}{1 - t}$	$\frac{\log_x(x_1)}{1 - t}$
Where the field lives	$\mathbb{R}^d$, the same for all x	$T_x \mathcal{M}$, one space per x
Loss norm	$\ \cdot\ _2^2$	$\ \cdot\ _{g_x}^2$
Density with respect to	Lebesgue measure dx	$d\text{vol}_g$
Divergence	$\nabla \cdot$	div_g
Integration step	$x + \Delta t u$	$\exp_x(\Delta t u)$
<i>Unchanged:</i>		
Marginalisation	$u_t(x) = \mathbb{E}[u_t(x \mid Z) \mid X_t = x]$ (Theorems 11.21 and 12.16)	
Why regression works	L^2 projection onto a conditional expectation (Lemmas 11.23 and 12.17)	
Training loop	sample z , sample t , build x_t , regress on the closed-form target	
Loss floor	$C = \mathbb{E}[\text{Var}(\text{target} \mid X_t, t)] > 0$ (Cautions 11.27 and 12.19)	
Path is a design choice	yes, in both (Section 11.16 and Remark 12.6)	

Every entry in the top block is a substitution from Section 12.1.1's table; every entry in the bottom block was proved twice and came out the same both times. That is the sense in which Riemannian flow matching is a generalisation rather than a new method.

12.8 Worked example: the circle S^1

The circle is the smallest manifold on which anything interesting happens, and every phenomenon of the $SO(3)$ case is already visible on it.

Setup. Identify S^1 with angles modulo 2π , so a point is $\theta \in \mathbb{R}/2\pi\mathbb{Z}$. The metric is the ordinary one ($\|\dot{\theta}\| = |\dot{\theta}|$), $T_\theta S^1 \cong \mathbb{R}$, and geodesics are constant-speed traversals. Then

$$\exp_{\theta_0}(v) = \theta_0 + v \pmod{2\pi}, \quad \log_{\theta_0}(\theta_1) = \text{wrap}(\theta_1 - \theta_0),$$

where $\text{wrap}(\alpha)$ is the representative of α in $(-\pi, \pi]$, i.e. $\text{wrap}(\alpha) = ((\alpha + \pi) \bmod 2\pi) - \pi$. The wrap is the principal branch of Caution 3.18(b): of the infinitely many v with $\theta_0 + v \equiv \theta_1$, it selects the one of smallest absolute value, i.e. the shortest way round.

Why the wrap is not a technicality. Take $\theta_0 = 170^\circ$ and $\theta_1 = -170^\circ$ (equivalently 190°). The naive difference is $\theta_1 - \theta_0 = -340^\circ$; the wrapped difference is $\text{wrap}(-340^\circ) = +20^\circ$. These are not two descriptions of the same motion: the first turns almost all the way round the circle clockwise, the second takes twenty degrees anticlockwise, passing through 180° . An implementation that subtracts angles without wrapping produces the first, and its training target is then seventeen times too large and points the wrong way.

The interpolant and its velocity. With $\Delta := \log_{\theta_0}(\theta_1) = 20^\circ$,

$$\theta_t = \theta_0 + t \Delta = 170^\circ + 20^\circ t, \quad \dot{\theta}_t = \Delta = 20^\circ,$$

so the endpoints check out ($\theta_0 = 170^\circ$, $\theta_1 = 190^\circ \equiv -170^\circ$) and the speed is constant, equal to $d_g(\theta_0, \theta_1) = 20^\circ$.

The current-point form, verified by hand. Evaluate (12.5) at $t = 0.75$. The current point is $\theta_{0.75} = 185^\circ$, and

$$\log_{\theta_{0.75}}(\theta_1) = \text{wrap}(190^\circ - 185^\circ) = 5^\circ = (1 - 0.75) \cdot 20^\circ,$$

so

$$\frac{\log_{\theta_t}(\theta_1)}{1 - t} = \frac{5^\circ}{0.25} = 20^\circ = \dot{\theta}_t \quad \checkmark$$

The factor $(1 - t)$ appears in the numerator on its own, exactly as Caution 12.13 predicts, and cancels.

The cut locus, concretely. Take $\theta_0 = 0^\circ$ and $\theta_1 = 180^\circ$. Now $\text{wrap}(180^\circ)$ is ambiguous: $+180^\circ$ and -180° are both of minimal absolute value, and both are legitimate values of $\log$. The two corresponding interpolants sweep the two opposite semicircles; they are different curves with the same endpoints and the same length. This is $\text{Cut}(\theta_0) = \{\theta_0 + \pi\}$ from Example 3.24, and the conditional path is genuinely non-unique there. It is a single point out of a whole circle — measure zero — but a training routine that hits it must break the tie somehow, and whichever way it breaks it, the target jumps discontinuously as θ_1 crosses the antipode.

12.9 Worked example: the sphere S^2

S^2 adds the feature S^1 cannot show: a tangent space of dimension greater than one, so that “which direction” becomes a real question.

Take $x_0 = (1, 0, 0)$ and $z = (0, 1, 0)$, and recall from Example 3.27 that

$$\log_{x_0}(z) = \left(0, \frac{\pi}{2}, 0\right), \quad \|\log_{x_0}(z)\| = \frac{\pi}{2} = d_g(x_0, z).$$

The interpolant. By (3.3),

$$X_t = \exp_{x_0}(t \log_{x_0}(z)) = \cos\left(\frac{\pi t}{2}\right)(1, 0, 0) + \sin\left(\frac{\pi t}{2}\right)(0, 1, 0) = \left(\cos \frac{\pi t}{2}, \sin \frac{\pi t}{2}, 0\right),$$

a quarter of the equator, from x_0 at $t = 0$ to z at $t = 1$ ✓.

The velocity. Differentiating,

$$\dot{X}_t = \frac{\pi}{2} \left(-\sin \frac{\pi t}{2}, \cos \frac{\pi t}{2}, 0\right), \quad \|\dot{X}_t\| = \frac{\pi}{2} \text{ (constant) } \checkmark$$

and the tangency check: $X_t^\top \dot{X}_t = \frac{\pi}{2}(-\cos \sin + \sin \cos) = 0$, so $\dot{X}_t \in T_{X_t}S^2$ ✓ — as Definition 12.2 demands and as the radial field of Example 12.3 conspicuously failed.

Confirming (12.5). The angle from X_t to z is $\arccos(X_t^\top z) = \arccos(\sin \frac{\pi t}{2}) = \frac{\pi}{2}(1 - t)$, so the remaining distance shrinks linearly, as it must. The tangential component of z at X_t is

$$w = z - (X_t^\top z)X_t = \cos\left(\frac{\pi t}{2}\right) \left(-\sin \frac{\pi t}{2}, \cos \frac{\pi t}{2}, 0\right),$$

so $w/\|w\| = (-\sin \frac{\pi t}{2}, \cos \frac{\pi t}{2}, 0)$ and, by (3.4),

$$\log_{X_t}(z) = \frac{\pi}{2}(1-t) \left(-\sin \frac{\pi t}{2}, \cos \frac{\pi t}{2}, 0\right), \quad \text{hence} \quad \frac{\log_{X_t}(z)}{1-t} = \frac{\pi}{2} \left(-\sin \frac{\pi t}{2}, \cos \frac{\pi t}{2}, 0\right) = \dot{X}_t \checkmark$$

Both halves of Caution 12.13 are visible in one line: the numerator carries the factor $(1 - t)$, and the quotient is the constant velocity.

What the marginal field would do. If p_{data} put mass on several points of the sphere rather than one, the marginal field at x would be the posterior average of the $\log_x(z_k)/(1 - t)$ over the candidate targets, exactly as in Section 11.10.3. Since all those vectors lie in the same plane $T_x S^2$, the average is an ordinary average of coplanar vectors — Section 12.6.1 in a picture. Note that the average of two unit tangent vectors is generally *not* a unit vector: the marginal field is shorter than the conditional ones where they disagree, which is the geometric content of the loss floor (Caution 12.19).

12.10 Flow matching on $SO(3)$

Now the manifold this document exists for. Everything below is Section 12.6 specialised, with $\exp$ and $\log$ supplied in closed form by Theorems 3.52 and 3.53.

12.10.1 The state space, recalled in one place

A point is a rotation matrix, $R \in SO(3) = \{R \in \mathbb{R}^{3 \times 3} : R^\top R = I, \det R = 1\}$, a three-dimensional manifold (Proposition 3.47). A tangent vector at R is $R\hat{\omega}$ for $\omega \in \mathbb{R}^3$ (Proposition 3.48 plus left translation), so the tangent space is three-dimensional and is identified with $\mathbb{R}^3$ by the body-frame (left-trivialised) convention $\widehat{\omega^{\text{bd}}} = R^\top \dot{R}$ of Definition 3.60. The metric is the bi-invariant one of Proposition 3.51, normalised so that $\|R\hat{\omega}\|_g = \|\omega\|_2$, and its geodesics through R_0 are $R_0 \exp(\tau \Xi)$ — proved, not imported, in Proposition 3.15.

12.10.2 The geodesic interpolant

Definition 12.20 (Geodesic interpolant on $SO(3)$, restated concretely). For $R_0, R_1 \in SO(3)$ with relative angle $\theta(R_0^\top R_1) < \pi$, and $t \in [0, 1]$,

$$R_t := \exp_{R_0}(t \log_{R_0}(R_1)) = R_0 \exp(t \log(R_0^\top R_1)), \quad (12.8)$$

using the body-frame (left-trivialised) exponential and logarithm of Theorem 3.52. We write $\Xi := \log(R_0^\top R_1) \in \mathfrak{so}(3)$ throughout, so that (12.8) reads $R_t = R_0 \exp(t\Xi)$.

The second equality in (12.8) is worth unpacking, since it is where the group structure enters. The Riemannian $\exp_{R_0}$ and $\log_{R_0}$ act on $T_{R_0}SO(3)$; the matrix $\exp$ and $\log$ act on $\mathfrak{so}(3) = T_I SO(3)$. Left translation by R_0 carries one to the other:

$$\log_{R_0}(R_1) = R_0 \log(R_0^\top R_1), \quad \exp_{R_0}(R_0 \Xi) = R_0 \exp(\Xi).$$

Both follow from bi-invariance: L_{R_0} is an isometry, and isometries map geodesics to geodesics, so the geodesic from R_0 towards R_1 is the L_{R_0} -image of the geodesic from I towards $R_0^\top R_1$.

Caution 12.21 (Matrix exponential versus Riemannian exponential). Two different maps are both written “exp” and they must not be conflated, even though on $SO(3)$ they are related by one multiplication.

	Matrix exponential	Riemannian exponential
Signature	$\exp : \mathfrak{so}(3) \rightarrow SO(3)$	$\exp_R : T_R SO(3) \rightarrow SO(3)$
Definition	power series $\sum_k A^k/k!$	follow the geodesic for unit time
Depends on	nothing but the algebra	the metric
Base point	always I	any R
Relation	$\exp_R(R\Xi) = R \exp(\Xi)$, valid <i>because</i> the metric is bi-invariant	

The relation in the last row is a theorem about $SO(3)$ with this metric, not a definition. On a manifold that is not a Lie group there is no matrix exponential at all, and only the right-hand column exists; on a Lie group with a non-bi-invariant metric the two exist but do not agree.

Example 12.22 (Endpoint check). At $t = 0$: $R_0 \exp(0) = R_0 I = R_0 \checkmark$. At $t = 1$: $R_0 \exp(\Xi) = R_0 \exp(\log(R_0^\top R_1)) = R_0 R_0^\top R_1 = R_1 \checkmark$, using that $\exp$ and $\log$ are mutually inverse away from the cut locus. In quaternion language (12.8) is exactly spherical linear interpolation (SLERP).

12.10.3 The velocity, derived

This is the central computation of the chapter’s $SO(3)$ half, and it is three lines.

Proposition 12.23 (Constant body-frame (left-trivialised) velocity of the geodesic). *Let $\Xi = \log(R_0^\top R_1) \in \mathfrak{so}(3)$ and $R_t = R_0 \exp(t\Xi)$ as in (12.8). Then*

$$\dot{R}_t = R_t \Xi, \quad \text{equivalently} \quad R_t^\top \dot{R}_t = \Xi,$$

*i.e. the curve has **constant left-trivialised (body-frame) angular velocity** $\omega^{\text{bd}}(t) \equiv \Xi^\vee$ (Definition 3.60), for every t .*

Proof. Differentiate (12.8) with respect to t , holding R_0 and Ξ fixed:

$$\dot{R}_t = \frac{d}{dt} \left(R_0 \exp(t\Xi) \right) = R_0 \exp(t\Xi) \Xi = R_t \Xi,$$

using $\frac{d}{dt} \exp(t\Xi) = \exp(t\Xi) \Xi$, which holds because Ξ commutes with every power of itself and hence with $\exp(t\Xi)$ (so the two possible orderings of the derivative agree). Multiplying on the left by $R_t^\top = R_t^{-1}$ gives $R_t^\top \dot{R}_t = \Xi$, which is Definition 3.60’s definition of ω^{bd} with $\widehat{\omega^{\text{bd}}} = \Xi$, constant in t . $\square$

Remark 12.24 (Why constancy is so convenient for training). Proposition 12.23 says the regression target does not depend on t . In practice this means a training example can be constructed as: draw R_0 and R_1 , compute $\Xi = \log(R_0^\top R_1)$ *once*, draw t , form $R_t = R_0 \exp(t\Xi)$, and use $\Xi^\vee$ as the target — with no second logarithm, no division by $1 - t$, and nothing that degrades as $t \rightarrow 1$. Contrast the current-point form $\log(R_t^\top R_1)^\vee / (1 - t)$, which needs a fresh matrix logarithm at every step and divides by a vanishing number. The two are equal (Proposition 12.26); only one is a good thing to compute.

12.10.4 Intrinsic vector versus trivialised coordinates

A point that is easy to slide past, and that matters for reading any implementation: Proposition 12.23 says ω^{bd} is constant, *not* that the tangent vector is constant.

	The tangent vector	Its body coordinates
Object	$\dot{R}_t = R_t \Xi \in T_{R_t} SO(3)$	$\omega^{\text{bd}} = \Xi^\vee \in \mathbb{R}^3$
Lives in	a space that changes with t	one fixed $\mathbb{R}^3$
Along the geodesic	<i>changes</i> (as a 3×3 matrix)	<i>constant</i>

Both statements are true simultaneously and there is no contradiction: the coordinates are taken with respect to a frame that itself rotates with R_t . Writing $\dot{R}_t$ as a matrix, its entries genuinely vary with t ; expressed in the body frame, the same vector has the same three numbers throughout. This is the entire content of “trivialisation”: a choice of moving frame that converts a section of the tangent bundle into an ordinary $\mathbb{R}^3$ -valued function.

Remark 12.25 (Why parallel transport is not needed here). On a general manifold, comparing tangent vectors at different points requires *parallel transport* — a rule for carrying a vector along a curve without turning it. Riemannian flow matching, as constructed above, never needs it, for two separate reasons.

In the theory: the only place vectors are compared is inside $\mathbb{E}[\cdot \mid X_t = x]$ and inside the loss $\|u_\theta(x, t) - u_t(x \mid z)\|_{g_x}$, and both fix the point x first (Section 12.6.1). Vectors at different points are never subtracted.

In the $SO(3)$ implementation: the body-frame trivialisation already provides a global identification $T_R SO(3) \cong \mathbb{R}^3$, so the network’s output and the target are both plain 3-vectors and are compared as such. That identification happens to *be* parallel transport along geodesics here — on a bi-invariant metric, left translation is an isometry and carries geodesics to geodesics — but nothing in the method depends on that coincidence, and no transport operator appears anywhere in the derivation or the code.

We therefore do not develop parallel transport in this document, per the principle of not adding theory the method does not use.

12.10.5 The current-point form

Proposition 12.26 (The current-point form: $SO(3)$ case of Proposition 12.11). *With R_t, Ξ as above and $t \in [0, 1)$,*

$$R_t^\top R_1 = \exp((1-t)\Xi), \quad \text{hence} \quad \log(R_t^\top R_1) = (1-t)\Xi,$$

and therefore

$$\frac{\log(R_t^\top R_1)^\vee}{1-t} = \Xi^\vee. \quad (12.9)$$

The “current-point” form (Proposition 12.11’s $\log_{X_t}(z)/(1-t)$, here with $X_t = R_t$, $z = R_1$) and the “constant-in- t endpoint” form (Proposition 12.23’s $\Xi^\vee$) are, exactly, the same tangent vector at every t where both are defined — not two competing formulas, but one quantity computed two different ways.

Proof. Compute the relative rotation from the current point to the target:

$$R_t^\top R_1 = (R_0 \exp(t\Xi))^\top R_1 = \exp(t\Xi)^\top R_0^\top R_1 = \exp(-t\Xi) \exp(\Xi) = \exp((1-t)\Xi),$$

using $\exp(A)^\top = \exp(A^\top) = \exp(-A)$ for $A \in \mathfrak{so}(3)$, then $R_0^\top R_1 = \exp(\Xi)$ (definition of Ξ), then the exponential addition law $\exp(A)\exp(B) = \exp(A+B)$ — which is valid here *specifically because* $-t\Xi$ and Ξ commute, both being multiples of the same matrix, and which is false in general for two arbitrary elements of $\mathfrak{so}(3)$.

Taking log of both sides is legitimate for $t \in [0, 1)$ provided $\|(1-t)\Xi\| \leq \|\Xi\| < \pi$, i.e. provided the original pair was not on the cut locus — the hypothesis of Definition 12.20. This gives $\log(R_t^\top R_1) = (1-t)\Xi$; applying $(\cdot)^\vee$ (linear) and dividing by $1-t$ gives (12.9). $\square$

Remark 12.27 (Numerical instability near $t = 1$). The current-point form (12.9) carries a $(1-t)^{-1}$ factor that is exactly cancelled, algebraically, by Proposition 12.26’s identity — but only when evaluated *exactly*, in exact arithmetic. In floating-point arithmetic, the numerator $\log(R_t^\top R_1)$ and the denominator $1-t$ both individually approach 0 as $t \rightarrow 1$, so this specific closed form is numerically fragile in exactly this limit, even though the underlying tangent vector it computes is perfectly well-behaved (constant, in fact, by Proposition 12.23). This is the $SO(3)$ instance of the general phenomenon of Caution 12.13 and of the Euclidean Caution 11.29: the same structure, on all three manifolds. It is a genuine, verified numerical consideration, examined empirically (via oracle-vector-field ablations that isolate this exact mechanism from network-quality effects) in this project’s broader experimental investigation, referenced but not re-conducted in this theory document.

12.10.6 A complete numerical case

Take $R_0 = I$ and $R_1 = R_z(\pi/2)$, the quarter-turn about the z -axis:

$$R_1 = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

The relative rotation and its logarithm. $R_0^\top R_1 = R_1$, whose trace is $0 + 0 + 1 = 1$, so by Theorem 3.53 the angle is $\theta = \arccos(\frac{1-1}{2}) = \arccos(0) = \frac{\pi}{2}$. The axis is e_z , so

$$\Xi = \log(R_0^\top R_1) = \frac{\pi}{2} \hat{e}_z = \frac{\pi}{2} \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad \Xi^\vee = \frac{\pi}{2} e_z \approx (0, 0, 1.5708)^\top.$$

The interpolant. $R_t = R_0 \exp(t\Xi) = R_z(\frac{\pi t}{2})$, i.e. the quarter-turn performed at uniform angular rate. At $t = \frac{1}{2}$ it is the 45° rotation; at $t = 0$ and $t = 1$ it is I and R_1 ✓.

The velocity, computed from scratch. With $\phi := \frac{\pi t}{2}$,

$$R_t = \begin{pmatrix} \cos \phi & -\sin \phi & 0 \\ \sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \dot{R}_t = \frac{\pi}{2} \begin{pmatrix} -\sin \phi & -\cos \phi & 0 \\ \cos \phi & -\sin \phi & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Note that $\dot{R}_t$ is *not* constant in t — its entries oscillate. Now form the body-frame velocity:

$$R_t^\top \dot{R}_t = \frac{\pi}{2} \begin{pmatrix} \cos \phi & \sin \phi & 0 \\ -\sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} -\sin \phi & -\cos \phi & 0 \\ \cos \phi & -\sin \phi & 0 \\ 0 & 0 & 0 \end{pmatrix} = \frac{\pi}{2} \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} = \frac{\pi}{2} \hat{e}_z = \Xi.$$

(Row 1: $(-\cos \phi \sin \phi + \sin \phi \cos \phi, -\cos^2 \phi - \sin^2 \phi, 0) = (0, -1, 0)$; row 2: $(\sin^2 \phi + \cos^2 \phi, \sin \phi \cos \phi - \cos \phi \sin \phi, 0) = (1, 0, 0)$.) So $\omega^{\text{bd}} = \frac{\pi}{2} e_z$ at every t , constant, exactly as Proposition 12.23 predicts — while the ambient matrix $\dot{R}_t$ that represents it is not. This one computation is Section 12.10.4 made numerical.

Speed and distance. $\|\omega^{\text{bd}}\|_2 = \pi/2$, which by Proposition 3.58 equals $d_g(R_0, R_1) = \theta = \pi/2$ ✓ — the geodesic covers its distance in unit time at unit-distance-per-unit-time speed.

12.10.7 The training objective on $SO(3)$

Theorem 12.28 (Riemannian CFM loss on $SO(3)$).

$$\mathcal{L}_{\text{RCFM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}(0,1), R_0 \sim p_{\text{init}}, R_1 \sim p_{\text{data}}} \left[\|u_\theta(R_t, t) - \Xi^\vee\|_2^2 \right], \quad R_t = R_0 \exp(t\Xi), \quad \Xi = \log(R_0^\top R_1),$$

has the same minimiser, $u_\theta(R, t) \rightarrow u_t(R)$ (the true marginal field), as the intractable marginal loss, by Theorem 12.18 specialised to $\mathcal{M} = SO(3)$.

The Riemannian norm has become the ordinary Euclidean norm of a 3-vector, which deserves one sentence rather than being passed over: by Proposition 3.58’s normalisation the identification $\mathfrak{so}(3) \cong \mathbb{R}^3$ is a metric isometry, $\|R\hat{\omega}\|_g = \|\omega\|_2$, so $\|\cdot\|_{g_{R_t}}$ on body-frame coordinates *is* $\|\cdot\|_2$. That is a property of the chosen normalisation (the factor $\frac{1}{2}$ in $g_I(\Omega_1, \Omega_2) = \frac{1}{2} \text{tr}(\Omega_1^\top \Omega_2)$), not a general fact, and it is the reason the implementation can use a plain mean-squared error on three numbers without any metric-dependent weighting.

Remark 12.29 (Prior distribution). The natural (and, per Chapter 13’s code correspondence, actually used) source distribution for $SO(3)$ flow matching is the exact Haar-uniform measure $\mathcal{U}(SO(3))$ of Proposition 3.59, sampled directly via Proposition 3.59’s code correspondence (`sample_uniform`) — *not* the diffusion side’s $t = 1$ -of-a-VE-schedule approximation to it (Section 7.2.2’s `sample_ref`). This is possible *because* flow matching never needs a noising process to define its prior at all: any fixed, sampleable distribution with the right invariance properties (Proposition 8.3) will do, and the exact Haar measure is simply the natural, no-approximation-needed choice, unlike diffusion’s necessarily-approximate large-noise limit.

By Proposition 3.32, “Haar” and “constant density with respect to $d\text{vol}_g$ ” are the same statement, so this prior is also the one for which $p_{\text{init}} \equiv 1/\text{vol}_g(SO(3))$ — the cleanest possible reference point for the likelihood computations of Section 12.12.

12.10.8 Sampling

Sampling integrates

$$R_{t+\Delta t} = \exp_{R_t}(\Delta t u_\theta(R_t, t)) = R_t \exp(\Delta t \widehat{u_\theta(R_t, t)}), \quad (12.10)$$

exactly Proposition 3.63’s body-frame Euler update, which — by construction, since $\exp$ always lands back on $SO(3)$ regardless of step size (Section 3.4.3) — keeps every iterate exactly on $SO(3)$, for any number of steps, not merely approximately for small steps. This is the concrete payoff of Proposition 12.4’s discretisation caveat: the constraint is not enforced by a projection after the fact, it is enforced by choosing an update map whose image is the manifold.

12.10.9 The cut locus, and what it does and does not cost

Definition 12.20 assumed $\theta(R_0^\top R_1) < \pi$. It is worth separating carefully what happens otherwise, because two quite different issues are routinely conflated.

The theoretical issue: non-uniqueness on a null set. If $\theta(R_0^\top R_1) = \pi$ exactly, then by Proposition 3.55 the logarithm is two-valued ($\pm\pi n$), so there are two minimising geodesics and the conditional path of Definition 12.20 is not unique. This is $\text{Cut}(R_0)$ from Example 3.24. Probabilistically it is harmless: $\text{Cut}(R_0)$ is a two-dimensional subset of a three-dimensional manifold, hence has vol_g -measure zero, so for any $p_{\text{init}}, p_{\text{data}}$ with densities the event $\{R_1 \in \text{Cut}(R_0)\}$ has probability zero and the training target is defined almost surely. The RCFM objective is an expectation, and an expectation does not see a null set.

The numerical issue: a whole neighbourhood, visited often. That argument covers $\theta = \pi$ exactly and says nothing about θ close to π , where the formulas remain well defined but badly conditioned: the $1/(2 \sin \theta)$ in Theorem 3.53 diverges, and the choice of branch becomes sensitive to round-off. And, by Caution 3.25, near- π is not a rare regime on $SO(3)$:

the density of the relative angle between independent Haar-random rotations, $p(\theta) \propto 1 - \cos \theta$, is *maximal* at π .

	Theoretical (at $\theta = \pi$)	Numerical (near $\theta = \pi$)
What happens	log is two-valued; two minimising geodesics	log is single-valued but ill-conditioned
Probability	zero	<i>high</i> : $\mathbb{P}(\theta > 172^\circ) \approx 9\%$ for Haar pairs
Fix	none needed; the objective is an expectation	careful branch handling, tight clamps
Where documented	Proposition 3.55	Caution 3.57

Keeping these apart matters, because the correct statement “the cut locus has measure zero” is often used to dismiss the second column, which it does not address at all.

12.11 Product manifolds: many rigid bodies at once

A docking state is not one rotation. It is, per fragment, a translation and a rotation, and there are N fragments. The geometry of that state space is a product, and products are where Riemannian flow matching is at its most convenient: everything decouples.

12.11.1 The state, its tangent space, and its metric

By Definition 3.70 the state space is

$$\mathcal{M} = (\mathbb{R}^3 \times SO(3))^N, \quad \mathbf{x} = (r^{(1)}, R^{(1)}, \dots, r^{(N)}, R^{(N)}).$$

A tangent vector at $\mathbf{x}$ is a tuple of tangent vectors, one per factor. Using the body-frame trivialisation of Definition 3.60 on each rotational factor,

$$T_{\mathbf{x}}\mathcal{M} \cong (\mathbb{R}^3 \times \mathbb{R}^3)^N, \quad \mathbf{v} = (v^{(1)}, \omega^{(1)}, \dots, v^{(N)}, \omega^{(N)}),$$

with $v^{(i)}$ the translational velocity of fragment i and $\omega^{(i)}$ its body-frame angular velocity. The tangent space is therefore an ordinary $6N$ -dimensional vector space — but only *after* the trivialisation, and the trivialisation depends on the current $R^{(i)}$ (Section 12.10.4).

The metric is the product metric of (3.10), extended fragmentwise:

$$\|\mathbf{v}\|_{g^\lambda}^2 = \sum_{i=1}^N \left(\|v^{(i)}\|_2^2 + \lambda \|\omega^{(i)}\|_2^2 \right). \quad (12.11)$$

Remark 12.30 (The weight is geometry, and it is still a hyperparameter). Proposition 3.67 established the two halves of this: a relative weight λ on the rotational term *is* the choice of product metric g^λ , and simultaneously it changes nothing about the exact minimiser, the interpolant, or the path — only the finite-capacity approximation. Numerically, Section 12.16 check 9 confirms that the same interpolant is a constant-speed geodesic for every $\lambda \in \{0.25, 1, 4, 9\}$, with speed equal to the corresponding d_{g^λ} .

Note what this does *not* settle: whether a particular λ is a good idea for a particular model. That is an empirical question about scales in a specific objective, and Chapter 15 treats it as one.

12.11.2 Everything factorises

Proposition 12.31 (Componentwise construction on a product). *For a Riemannian product $\mathcal{M} = \mathcal{M}_1 \times \cdots \times \mathcal{M}_k$ with the product metric, geodesics, $\exp$, $\log$, the geodesic interpolant (12.4) and the conditional vector field (12.5) are all computed factor by factor, with no cross terms.*

Proof. Geodesics factorise (Proposition 3.69), hence so do $\exp$ and $\log$; hence so does their composition, the interpolant (12.4); hence so does its derivative. There is nothing to couple: the metric (12.11) is a sum with no mixed terms, so the length functional splits and its stationary curves split with it. $\square$

Applied to the docking state, this gives exactly the construction the rest of this document uses:

$$\text{translation: } r_t^{(i)} = (1-t)r_0^{(i)} + tr_1^{(i)}, \quad \text{target } v^{(i)} = r_1^{(i)} - r_0^{(i)}, \quad (12.12)$$

$$\text{rotation: } R_t^{(i)} = R_0^{(i)} \exp(t\Xi^{(i)}), \quad \text{target } \omega^{(i)} = (\Xi^{(i)})^\vee, \quad (12.13)$$

with $\Xi^{(i)} := \log(R_0^{(i)\top} R_1^{(i)})$. The Euclidean rows are Proposition 11.28; the rotational rows are Proposition 12.23; and they are the same construction, since the flat case is the geodesic case (Example 12.10).

Caution 12.32 (Factorised geometry, coupled model). Proposition 12.31 says the *geometry* does not couple the fragments or the translation/rotation channels. It emphatically does not say the *model* does not. All coupling — between fragments, between a fragment’s position and its orientation, and with the protein — enters through the single neural network that predicts all $6N$ components jointly from the same input graph. The geometry supplies $2N$ independent interpolants; the network supplies every correlation. Losing track of this distinction makes the factorisation look like a modelling assumption about independence, which it is not.

Example 12.33 (One fragment, end to end). Take a single fragment with

$$r_0 = (0, 0, 0)^\top \text{ \AA}, \quad r_1 = (2, 1, 0)^\top \text{ \AA}, \quad R_0 = I, \quad R_1 = R_z(\pi/2).$$

By (12.12)–(12.13):

$$r_t = (2t, t, 0)^\top, \quad v = (2, 1, 0)^\top, \quad R_t = R_z\left(\frac{\pi t}{2}\right), \quad \omega = \left(0, 0, \frac{\pi}{2}\right)^\top.$$

At $t = \frac{1}{2}$ the fragment sits at $(1, 0.5, 0)$, rotated by 45° about z . Both velocities are constant in t , so the whole training target for this pair is the six numbers $(2, 1, 0, 0, 0, 1.5708)$, computed once.

The speed under $g^{\lambda=1}$ is

$$\|\mathbf{v}\|_g = \sqrt{\|v\|^2 + \|\omega\|^2} = \sqrt{5 + (\pi/2)^2} = \sqrt{7.4674} = 2.7327,$$

and the distance between the two poses is $d_g = \sqrt{\|r_1 - r_0\|^2 + \theta^2}$ with $\theta = \pi/2$, i.e. the same 2.7327 $\checkmark$ — constant speed over unit time covers exactly the distance (Section 12.16, check 9).

This is the state a reader should now recognise directly in Chapter 13: per fragment, three translational numbers and three rotational numbers, both constant along the conditional path.

12.12 Riemannian CNFs and likelihood

Corollary 11.14 gave the Euclidean instantaneous change of variables. Its manifold version is the same statement with $\nabla \cdot$ replaced by div_g — but the density is now with respect to $d\operatorname{vol}_g$, and that qualification is the whole reason this section exists separately.

Theorem 12.34 (Instantaneous change of variables on a manifold). *Let u_t be a vector field on $\mathcal{M}$ with flow ϕ_t , let p_t be the density of $(\phi_t)_\# p_{\text{init}}$ with respect to $d\operatorname{vol}_g$, and let $X_t = \phi_t(X_0)$. Then*

$$\frac{d}{dt} \log p_t(X_t) = -\operatorname{div}_g u_t(X_t). \quad (12.14)$$

Proof. Differentiate p_t along the trajectory, splitting into the explicit time dependence and the motion of the point (the material derivative):

$$\frac{d}{dt} p_t(X_t) = \partial_t p_t(X_t) + d(p_t)_{X_t}(\dot{X}_t) = \partial_t p_t(X_t) + g(\operatorname{grad}_g p_t, u_t)(X_t),$$

the second step by the defining property (3.5) of the Riemannian gradient. By Theorem 12.7, $\partial_t p_t = -\operatorname{div}_g(p_t u_t)$, and by the product rule (3.8),

$$\operatorname{div}_g(p_t u_t) = p_t \operatorname{div}_g u_t + g(\operatorname{grad}_g p_t, u_t).$$

Substituting, the two gradient terms cancel exactly:

$$\frac{d}{dt} p_t(X_t) = -p_t \operatorname{div}_g u_t - g(\operatorname{grad}_g p_t, u_t) + g(\operatorname{grad}_g p_t, u_t) = -p_t(X_t) \operatorname{div}_g u_t(X_t).$$

Dividing by $p_t(X_t) > 0$ gives (12.14). $\square$

Note that the cancellation is not a coincidence: it is the statement that the density seen by a *moving* particle changes only through the expansion of the volume around it, the transport part having been absorbed by following the flow. This is the same reading as Section 11.4’s Euclidean discussion of the minus sign, now with div_g measuring expansion of $d\operatorname{vol}_g$ ((3.6)).

Caution 12.35 (The likelihood is with respect to $d\operatorname{vol}_g$). Integrating (12.14) from 0 to 1 gives

$$\log p_1(X_1) = \log p_{\text{init}}(X_0) - \int_0^1 \operatorname{div}_g u_t(X_t) dt,$$

an exact expression for the model’s log-likelihood of a generated sample, computable alongside the sample itself. Two qualifications, both from Remark 3.31:

(i) The number produced is a log-density *with respect to* $d\operatorname{vol}_g$. Reporting it as “the log-likelihood” without naming the reference measure is meaningless on a manifold, and comparing it to a number computed under a different reference measure (say, a chart-based Lebesgue measure, or Haar normalised to total mass 1 rather than $\operatorname{vol}_g(\mathcal{M})$) is comparing two different quantities. On $SO(3)$ the two natural choices differ by the constant $\log \operatorname{vol}_g(SO(3))$, which is harmless for *ranking* samples and fatal for absolute statements.

(ii) On a product manifold the reference measure carries the λ of (12.11) through Proposition 3.67(ii), i.e. a global factor $\lambda^{3N/2}$. Again harmless for ranking, not for absolute values.

Remark 12.36 (Scope). Theorem 12.34 is stated here because it is the theory a likelihood-based confidence estimate would rest on, and because Section 13.9 records what SigmaFlow does and does not implement. Nothing in this document develops uncertainty quantification, confidence heads, or divergence estimators beyond the identity above and the Hutchinson remark of Section 11.4; that is deliberate and not an oversight.

12.13 Riemannian diffusion versus Riemannian flow matching

Chapter 7 built a generative model on $SO(3)$ the diffusion way. It is worth putting the two side by side, because the comparison is sharper on a manifold than it was in $\mathbb{R}^d$ — the asymmetry that was a matter of convenience there becomes a matter of what is computable at all.

	Riemannian diffusion (Chapter 7)	Riemannian flow matching (this chapter)
Path comes from	a noising process: Brownian motion on $\mathcal{M}$	a chosen interpolant: the geodesic
Requires	the heat kernel of $\mathcal{M}$	the geodesic of $\mathcal{M}$
On $SO(3)$ that is	IGSO(3): an infinite spectral series, truncated, tabulated and cached (Section 7.2.2)	$R_0 \exp(t \log(R_0^\top R_1))$: two closed-form matrix functions
Network learns	the Riemannian score $\text{grad}_g \log p_t$	the tangent velocity field u_t
Population equation	Fokker–Planck on $\mathcal{M}$	continuity equation (12.3)
Generation	reverse-time SDE, or its probability flow ODE	the ODE $\dot{X} = u_\theta(X, t)$
Both are	L^2 projections onto a conditional expectation, in $T_x \mathcal{M}$	

The relationship mirrors Chapter 11’s exactly: two ways of specifying a probability path, and two different local quantities learned from it. What changes on a manifold is the third row. In $\mathbb{R}^d$ the heat kernel is a Gaussian and costs nothing, so the choice between the two frameworks was about parameterisation. On $SO(3)$ the heat kernel is IGSO(3) — a series in Wigner characters, requiring truncation, numerical integration and a cached lookup table, all of which introduce approximation error before any network is trained. The geodesic, by contrast, is exact and two lines.

This is the first of the two arguments for the substitution studied in this project: on a manifold, flow matching removes an entire layer of numerical approximation that diffusion cannot avoid. The second argument — fewer network evaluations at sampling time — is empirical and is not established here (Caution 11.53).

12.14 The whole construction in one picture

Flow matching in $\mathbb{R}^d$ (Chapter 11)
interpolant $(1 - t)x_0 + tx_1$ straight line
field $u_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$ one global vector space
density with respect to Lebesgue dx
population law $\partial_t p_t + \nabla \cdot (p_t u_t) = 0$
loss $\mathbb{E} \ u_\theta - u_t(\cdot z)\ _2^2$

$\Downarrow$ replace every Euclidean operation by its geometric counterpart

Flow matching on $\mathcal{M}$	(this chapter)
interpolant $\exp_{x_0}(t \log_{x_0}(x_1))$	geodesic
field $u_t(x) \in T_x \mathcal{M}$	one vector space <i>per point</i>
density with respect to $d\text{vol}_g$	
population law $\partial_t p_t + \text{div}_g(p_t u_t) = 0$	
loss $\mathbb{E} \ u_\theta - u_t(\cdot z)\ _{g_x}^2$	

$\Downarrow$ *specialise to the docking state space*

$\mathcal{M} = (\mathbb{R}^3 \times SO(3))^N$	(Section 12.11)
translation $r_t = (1-t)r_0 + tr_1$,	target $v = r_1 - r_0$
rotation $R_t = R_0 \exp(t\Xi)$, $\Xi = \log(R_0^\top R_1)$,	target $\omega = \Xi^\vee$
both constant in t ; both computed once per training example	
geometry factorises completely; the network supplies all coupling	

Each arrow is a substitution, not a new theory. The theorems — marginalisation, L^2 projection, the equality of the conditional and marginal objectives — were proved once in $\mathbb{R}^d$ and once on $\mathcal{M}$, and came out identical both times, because their proofs used only linearity of integration inside a fixed tangent space.

12.15 What Riemannian flow matching is not

As in Section 11.18, a checklist of claims that sound right and are not. Each has been established above.

1. “*Riemannian flow matching means flow matching with rotations.*” Rotations are one instance. The construction needs only a Riemannian manifold with computable $\exp$ and $\log$; spheres, tori, hyperbolic space and products all qualify, and Chen & Lipman (2024) go further still, replacing the geodesic distance by a general premetric so that manifolds *without* closed-form geodesics can be handled.
2. “*Every trajectory is a globally shortest path.*” Two separate failures. The *conditional* paths are minimising geodesics only when the pair avoids the cut locus (Definition 12.20’s hypothesis), and the *marginal* trajectories that the sampler actually follows are not geodesics at all — they are integral curves of a posterior-averaged field, and they bend, exactly as in Section 11.16’s Euclidean discussion.
3. “*The source must be Haar / uniform.*” No. Remark 12.6: any sampleable distribution on $\mathcal{M}$ will do. Haar is the natural choice on a compact group and is what SigmaFlow uses, but that is a design decision, not a requirement of the method.
4. “*The vector field lives in some global $\mathbb{R}^n$.*” No — Definition 12.2, and Example 12.3 for what goes wrong. On $SO(3)$ a global $\mathbb{R}^3$ identification does exist via trivialisation, but that is a feature of Lie groups, not of manifolds, and it depends on the current point (Section 12.10.4).
5. “ *$\exp$ and $\log$ are inverse bijections.*” Only inside the injectivity radius. Outside it $\exp$ is not injective and $\log$ requires a branch choice (Definition 3.23 and Caution 3.18).

6. “*The cut locus is measure zero, so it can be ignored.*” Two different statements are being run together. The theoretical non-uniqueness is confined to a null set and genuinely can be ignored in an expectation; the numerical ill-conditioning occupies a whole neighbourhood which, on $SO(3)$, is visited with high probability. Section 12.10.9 separates them.
7. “*Product components are automatically weighted equally.*” No. The relative weight is a choice of product metric (Proposition 3.67), and translations and rotations are not even dimensionally comparable, so *some* choice must be made. Choosing $\lambda = 1$ is a decision, not the absence of one.
8. “*Riemannian flow matching needs parallel transport.*” Not as constructed here (Remark 12.25): every comparison of tangent vectors happens at a fixed point.

12.16 Numerical verification of this chapter

Every geometric identity above was checked in double precision, by the script

`audits/riemannian_fm_theory_audit.py`

(pure `numpy/scipy`; $SO(3)$ ’s `exp` and `log` implemented from Theorems 3.52 and 3.53 rather than imported, so the checks are independent of the project’s own code).

#	Statement checked	Method	Residual
1	$\exp(\log R) = R$ and $\log(\exp w) = w$ for $\ w\ < \pi$	400 Haar samples each	$5 \cdot 10^{-12}$
2	$\ R\hat{\omega}\ _g = \ \omega\ _2$: the $\frac{1}{2}$ in the metric	random R, ω	$2 \cdot 10^{-15}$
	metric is left- and right-invariant	random Q	$4 \cdot 10^{-15}$
	$\ \log(R_0^\top R_1)\ = \theta = d_g$ (Proposition 3.58)	Haar pairs	$1 \cdot 10^{-12}$
3	Proposition 3.15: $\ddot{R} \perp T_R SO(3)$	against 5 random tangents each	$9 \cdot 10^{-15}$
	Proposition 3.13: speed constant	7 times per curve	$2 \cdot 10^{-15}$
4	interpolant endpoints R_0, R_1	(12.8)	$3 \cdot 10^{-11}$
	Proposition 12.23: $R_t^\top \dot{R}_t = \Xi$	central differences, $h = 10^{-6}$	$4 \cdot 10^{-10}$
	Proposition 12.26: $\log(R_t^\top R_1)/(1-t) = \Xi^\vee$	random t , Haar pairs off the cut locus	$3 \cdot 10^{-10}$
5	Proposition 12.11 on S^2 : $\log_{X_t}(z)/(1-t) = \dot{X}_t$	central differences	$3 \cdot 10^{-10}$
	and on S^1 , and in the flat case	exact arithmetic	$5 \cdot 10^{-15}$
6	tangency: $x^\top \log_x(y) = 0$ and $X_t^\top \dot{X}_t = 0$ on S^2	random pairs	$2 \cdot 10^{-10}$
7	the 90° example of Section 12.10.6	$\Xi^\vee, R_t, R_t^\top \dot{R}_t$	$5 \cdot 10^{-11}$
8	Haar measure is bi-invariant (Proposition 3.32)	KS test, $N = 20\,000$, left and right	$p = 0.41$
	angle density $\propto 1 - \cos \theta$	$\mathbb{P}(\theta > 171.9^\circ)$: 0.0955 vs 0.0899	$6 \cdot 10^{-3}$
9	Proposition 3.67(i): same curve is a g^λ -geodesic	speed constant in t , $\lambda \in \{0.25, 1, 4, 9\}$	$6 \cdot 10^{-10}$
	and its speed equals d_{g^λ}	Example 12.33	$1 \cdot 10^{-11}$
10	flat case reduces to Proposition 11.28	$\log_x(y) = y - x$ substituted	$5 \cdot 10^{-15}$
11	Example 7.7: IGSO(3) $\rightarrow$ uniform as $\sigma \rightarrow \infty$	series summed to $\ell = 2000$	$1 \cdot 10^{-10}$
	and $\rightarrow$ the exp-pushforward Gaussian as $\sigma \rightarrow 0$	against the χ_3 angle density	$8 \cdot 10^{-3}$

All 35 checks pass. Check 8 is reported as the p -value of a two-sample Kolmogorov–Smirnov test rather than as a residual, since invariance of a *distribution* is not a numerical identity; $p = 0.41$ under both left and right translation is consistent with exact invariance.

Remark 12.37 (One check was initially unable to fail). The first version of check 9 compared an expression with itself — it recomputed $\|v\|^2 + \lambda\|\omega\|^2$ on both sides of the equality and would have passed for any λ , any manifold, and any error in the theory. It was replaced by the test above, which *can* fail: if λ changed the geodesic, the speed along the fixed interpolant would stop being constant in t and the check would catch it. This is the same lesson recorded in Remark 11.57 — a numerical check must be audited for its ability to fail before its passing counts as evidence — and it recurred here despite having been written down once already.

12.17 Bridge to SigmaFlow

Everything above is standard Riemannian flow matching on $(\mathbb{R}^3 \times SO(3))^N$, stated without reference to any implementation. Chapter 13 now checks a specific program against it. Two points of contact deserve flagging here, precisely so that they are *not* mistaken for parts of the general theory.

Caution 12.38 (SigmaFlow’s state is a delta rotation, and the group acts by conjugation). In the construction above, a state is a rotation $R \in SO(3)$ and a global rotation Q of the whole system acts by **left multiplication**, $R \mapsto QR$. That is the standard action, it is what Definition 3.44 formalises, and it is what makes the body-frame velocity behave the way Definition 3.60 describes.

SigmaFlow’s state variable is *not* an absolute orientation. It is a delta rotation measured against a reference conformer that is itself an input to the network and therefore rotates with the system. The consequence, established numerically rather than assumed in Chapter 14, is that the group acts on that state by **conjugation**,

$$R \mapsto QRQ^\top,$$

and correspondingly on the predicted velocity by $u \mapsto QuQ^\top$ rather than leaving it invariant. That is an implementation convention attached to a particular state parameterisation. It is *not* an alternative definition of Riemannian flow matching on $SO(3)$, and none of Section 12.10’s derivations depend on it.

The order matters for reading: understand the left action and the body/spatial distinction (Section 3.5) first; only then is the conjugation finding of Chapter 14 interpretable rather than merely surprising.

Remark 12.39 (The log map in theory and in the shipped code). Theorem 3.53 and Proposition 12.26 above describe the *exact* principal logarithm. What a given implementation computes may differ, and in this project’s case it does: Caution 3.57 documents an angle clamp that perturbs every angle slightly and truncates large ones outright, and Section 15.6.1 quantifies the resulting deviation of $\exp(\log(\cdot))$ from the identity. Those are implementation findings, recorded where implementations are audited. The theory of this chapter assumes the exact map, and every identity above was verified against an exact implementation (Section 12.16), independent of the project’s own code.

Beyond these two, the correspondence is direct: (12.12) and (12.13) are what `r3_flow_matcher.py` and `so3_flow_matcher.py` compute, (12.10) is what the sampler integrates, and Theorem 12.28 is the objective that is optimised. Chapter 13 verifies each of these line by line against the source.

12.18 What you should now be able to do

- name the three Euclidean operations that fail on a manifold and give the geometric replacement for each;
- explain why a network output must be tangent *at the point it was given* (Example 12.3);
- derive the Riemannian continuity equation, and recognise it as the flat one applied to the coordinate density $p\sqrt{|g|}$;
- prove that the geodesic interpolant’s velocity is $\log_{X_t}(x_1)/(1-t)$ — five lines, no Jacobi fields (Proposition 12.11);

- say why averaging conditional velocities is legal here and would not be under a different conditioning (Section 12.6.1);
- carry the whole construction out on $SO(3)$ by hand, and separate the measure-zero cut-locus issue from the high-probability numerical one.

Part VII now checks a program against all of it.

Part VII

SigmaFlow

Part VII at a glance

What you will learn. The converted model, and three independent audits of it: what the generator computes, what the network’s output heads actually emit and how they transform under a global rotation, and which layers of the system changed at all.

Why it is needed. The preceding six parts are theory. This part is the confrontation with a program that runs, and it is where claimed and implemented behaviour are separated.

What depends on it. Part [VIII](#) collects the numerical safeguards and the remaining open items; the experimental work outside this document builds on the findings recorded here.

Chapter 13

SigmaFlow: Full Reconstruction

We now perform, for SigmaFlow, exactly what Chapter 10 did for SigmaDock: assemble the already-derived theory (Chapters 11 and 12) with the specific network of Chapter 9 into a complete, code-verified reconstruction. Because Chapter 10 already derived the state space (Section 10.1), the fragmentation and triangulation machinery (Sections 10.3 and 10.4), the symmetry argument (Section 10.5), the input graph and backbone (Section 10.7), and the fragment-reconstruction map φ (Section 10.7.1) — all of which Table 13.1 below confirms are unchanged — this chapter focuses entirely on what *does* change: the probability path, the training target, the loss, and the sampler.

13.1 What changes, and what does not

Table 13.1: Component-wise comparison, verified against the source in both trees. Rows above the double rule are unchanged; rows below are what this chapter derives.

Component	SigmaDock (Chapter 10)	SigmaFlow
State space $\mathcal{M}_m = SE(3)^m$, product metric	identical (Section 10.1 and Proposition 3.69)	
Fragmentation (FR3D), triangulation	identical (Sections 10.3 and 10.4; code diff'd byte-identical, per this project's own audit)	
Input graph, featurisation, backbone	identical (Chapter 9 and section 10.7)	
Fragment reconstruction φ	identical formula (Proposition 10.10)	
Prior p_{init} , translation	$\mathcal{N}(0, I)$ (VP schedule's $s \rightarrow 1$ limit)	$\mathcal{N}(0, I)$ (exact, by construction)
Prior p_{init} , rotation	$\approx \mathcal{U}(SO(3))$ (VE schedule's $s \rightarrow 1$ limit, Section 7.2.2)	$\mathcal{U}(SO(3))$ (exact Haar sampler, Remark 12.29)
Probability path	VP Gaussian $\oplus$ IGSO(3) heat kernel	linear $\oplus$ geodesic (SLERP) interpolant
Regression target	conditional score $\nabla \log p_{t 1}$	conditional velocity $u_t(\cdot z)$

Component	SigmaDock	SigmaFlow
Numerical machinery	truncated IGSO(3) series, tabulation, caching (Section 7.2.2)	none — closed form throughout
Output dressing	Section 10.8.4’s t -dependent scalar prefactors	none (Section 13.5)
Loss terms	four ($T_score, R_score, T0, R0$, Section 10.9)	two ($loss_trans, loss_R$, Section 13.6)
Sampler	reverse-time SDE (deterministic by default configuration, Section 10.10)	deterministic ODE (architecturally, not by configuration)
Time convention (internal)	$t = 0$ data, $t = 1$ noise	$t = 0$ noise, $t = 1$ data (reversed , Section 13.2)

13.2 A time-convention warning, confirmed at the source level

Remark 0.2 fixed a global convention for this document ($t = 0$ noise, $t = 1$ data) and warned that SigmaDock’s own native convention runs the opposite way internally (Remark 10.8 in Chapter 10, and Section 10.10’s observation that the diffusion sampler’s own timestep array decreases from $t_{\max} = 1$ to $t_{\min}$). We record here, as a fact independently confirmed against *both* source trees during the preparation of this document (not merely inferred from exposition), that this is not a convention chosen for pedagogical convenience in this monograph but a genuine, load-bearing difference between the two codebases’ own internal time variables:

- **R3Diffuser/SO3Diffuser** (SigmaDock): $t = 0$ is clean data ($\alpha_{t=0} = 1$, no noise); $t = 1$ is the fully-noised end (`sample_ref` literally evaluates the forward kernel at $t = 1$).
- **R3_FlowMatcher/SO3_FlowMatcher** (SigmaFlow): $t = 0$ is the noise/source end ($X_0 \sim p_{\text{init}}$ drawn internally by `conditional_probability_path`); $t = 1$ is the data end ($X_1 = z$, the training target).

This document’s global convention (Remark 0.2) matches SigmaFlow’s own native convention exactly, and every formula attributed to SigmaDock throughout Chapter 10 was, correspondingly, already relabelled via $s = 1 - t$ before being stated (Remark 10.8 there). Any reader cross-referencing a formula directly against SigmaDock’s own source should apply this relabelling explicitly; failing to do so is, per this document’s own code-research process, a genuine and easy source of sign errors, exactly as this document’s introduction anticipated.

13.3 Translation flow matching, exactly as implemented

Code correspondence

Mathematical object: the source distribution, conditional path, and both forms of the conditional vector field of Proposition 11.28

Implementation: `r3_flow_matcher.py`, class `R3_FlowMatcher` (56 lines total)

Interpretation: `sample_init(n) = randn(n,3)` ($X_0 \sim \mathcal{N}(0, I_3)$ per fragment, no rescaling by any $\sigma_{\min}$ -type parameter); `conditional_probability_path(x1, t)` draws x_0 *internally* (unlike the diffusion side’s `forward_marginal`, which takes x_0 as an argument — a structural difference reflecting that flow matching’s source is drawn once per training ex-

ample and never needs to be revisited, whereas diffusion’s x_0 is the fixed data point being noised), computes $x_t = (1-t)x_0 + tx_1$ (Proposition 11.28’s interpolant) and returns $u_t = x_1 - x_0$, the *constant-in-t, endpoint* form — this is the quantity actually regressed against during training (Section 13.6). `calc_trans_vector_field( $x_t, x_1, t$ ) = (x_1 - x_t)/(1 - t)` implements the *current-point* form (11.13), singular as $t \rightarrow 1$ by construction (the docstring itself notes t must lie in $(0, 1)$); this form is **not** used during training (confirmed by tracing every call site in `sigma_flow_generator.py`), only at sampling time (Section 13.8) and for the `use_true_vector_field` oracle debugging path, where x_0 is not naturally available but the current point (x_t, t) is. `euler_step( $x_t, v_t, dt$ ) =  $x_t + v_t dt$` , the plain (non-geodesic, since $\mathbb{R}^3$ is flat) Euler step of Section 11.15, and the module’s own inline comment notes this step is *exact* (zero discretisation error, for any step size) specifically because u_t is constant along the true conditional path — a special property of the linear path, not shared by the rotational component’s geodesic (Section 13.4) once the network’s prediction, rather than the true field, is substituted in. The constructor parameter `sigma_min` is accepted but **provably unused**: it is stored (`self.sigma_min = sigma_min`) and never read anywhere else in the file, with an explicit docstring noting it is “reserved for future noised OT path, currently unused — the linear path below is deterministic”.

13.4 Rotation flow matching, exactly as implemented

Code correspondence

Mathematical object: the source distribution, conditional path, and both forms of the conditional vector field of Section 12.10

Implementation: `so3_flow_matcher.py`, class `S03_FlowMatcher` (63 lines total)

Interpretation: `sample_init( $n$ ) = so3_utils.sample_uniform( $n$ )`, the exact Haar sampler of Proposition 3.59’s code correspondence (Remark 12.29’s claim, confirmed directly). The class’s own module-level comment states the convention adopted is “right-trivialisation”. That *word* does not match this document’s attribution (Caution 3.61: what the code calls right-trivialisation is what we call left-trivialisation), but the *object* does, and the object is what matters: direct inspection of `euler_step( $R_t, v_t, dt$ ) =  $R_t \exp(v_t dt)$` shows it right-multiplies the exponential onto R_t , which is exactly Proposition 3.63’s **body-frame** update. The velocity the network supplies is therefore ω^{bd} , however either side chooses to label it. `conditional_probability_path( $R_1, t$ )` draws R_0 internally, computes $\Delta = R_0^\top R_1$, $\Xi = \log(\Delta) \in \mathfrak{so}(3)$ (Definition 12.20’s notation), sets $u_t = \Xi$ (a $[n, 3, 3]$ skew-symmetric-matrix-valued, i.e. $\mathfrak{so}(3)$ -valued, not yet vee-mapped-to- $\mathbb{R}^3$ quantity — the *constant-in-t* form of Proposition 12.23, and the training target, Section 13.6), and $R_t = R_0 \exp(t\Xi)$ (Equation (12.8), confirmed exactly). `calc_rot_vector_field( $R_t, R_1, t$ ) =  $\log(R_t^\top R_1)/(1 - t)$` implements the *current-point* form (12.9), algebraically equal to Ξ by Proposition 12.26 but singular in floating-point arithmetic as $t \rightarrow 1$ (Remark 12.27) — again not used during training, only at sampling time and inside the `use_true_vector_field` oracle path. The module additionally carries an explicit inline comment recording exactly Caution 3.57’s finding in this document’s own words: the original SigmaDock Omega clamp broke exact $\exp \circ \log = \text{id}$ round trips needed by (12.8), motivating the narrower clamp confirmed in Section 3.4.4 — i.e. the bugfix’s rationale is documented, first-party, directly in the file that needed it.

Remark 13.1 (The hard-coded $R_1 \equiv I$ simplification). A significant, easily-missed simplification, confirmed directly against `sigma_flow_generator.py::get_fragment_com_and_rot`:

the function that computes each fragment’s data-time state (trans_1, R_1) from the reference conformer sets the rotational component to the identity, *always*, for every fragment, with an explicit inline comment: “Rotation matrix is identity: it is (and should be) independent in the vector field.” Consequently, throughout the actual training and sampling code, $R_1 \equiv I$ is not a general fragment orientation but a fixed constant, and formulas that look like they involve a genuine R_1 — for instance the delta-rotation $\Delta R := R_t R_1^\top$ appearing inside Proposition 10.10’s reconstruction map when specialised to this codebase — collapse to $\Delta R = R_t$ exactly. This is consistent with, and is the concrete mechanism realising, Section 10.5’s gauge-invariance discussion: since the reference conformer’s own orientation is arbitrary (Theorem 10.7’s gauge freedom), fixing it at the identity is simply one particular, legitimate choice of gauge, not a loss of generality — but a reader auditing the code without this fact stated explicitly could easily mistake R_1 for a nontrivial, data-derived quantity, which it is not.

13.5 No output dressing: the entire diffusion-specific scalar machinery is gone

Code correspondence

Mathematical object: the network’s predicted vector field, ($\text{pred_u_t_trans}, \text{pred_u_t_R}$)

Implementation: `sigma_flow_generator.py::compute_vector_field`, quoted here in full (it is short enough to reproduce verbatim, and its brevity is itself the point):

```
def _compute_vector_field(self, sampled, updates, t_batch):
    """
    Interprets the pooled per-fragment force/torque directly as the
    predicted conditional vector field. No time dependent scaling
    needed, unlike the diffusion score.
    """
    pred_u_t_trans = updates["total_force"]
    pred_u_t_R = updates["omega"]
    return {"pred_u_t_trans": pred_u_t_trans, "pred_u_t_R": pred_u_t_R}
```

Interpretation: `updates["total_force"]` and `updates["omega"]` are exactly the raw Newton–Maruyama outputs $dT = F_F/M_F$ and $\widehat{dW} = \widehat{I_F^{-1}\tau_F}$ of (10.3) — *unchanged* from Section 10.8’s construction, since the physical Newton–Euler reduction (net force, torque, mass, inertia) is one of the components Table 13.1 lists as unmodified. What is different is that this raw mechanical output is returned *directly* as the predicted vector field, with **no** t -dependent rescaling of the kind Equations (10.4) to (10.6) apply: no `r3_scaling`, no `so3_scaling`, no $\partial_\omega \log f_\sigma/\omega$ dressing, no branch selection (`rot_score_method/rot_score_scaling`, whose corresponding config fields were removed entirely from `config.py` in an earlier development pass of this project, since they had become dead code once this function stopped consulting them). The comment’s own justification — “no time dependent scaling needed, unlike the diffusion score” — is a genuine mathematical observation, not an oversight: Propositions 11.28 and 12.23 both establish that the flow-matching conditional targets are *constant in t* (unlike the diffusion score, whose scale, per Section 6.3’s discussion, varies enormously across noise levels and specifically requires the $\lambda(s)$ /scalar-prefactor machinery to keep training well-conditioned) — **but** this observation concerns the *training target’s* scale, not necessarily whether the spe-

cific quantity a fixed-capacity network can most easily regress onto (raw Newton–Euler output, versus some other, still t -independent, reparameterisation) is optimally chosen; this project’s broader experimental investigation (the “time-weighting” variant referenced in this project’s development history) tested, and found no benefit from, reintroducing a t -dependent multiplicative reweighting on top of this already-unweighted target, discussed further in Section 13.10.

13.6 The loss function

Code correspondence

Mathematical object: $\mathcal{L}_{\text{trans}}$, $\mathcal{L}_R$ of Theorems 11.25 and 12.28, evaluated at their sampled-path targets

Implementation: `sigma_flow_generator.py::compute_losses`

Interpretation:

`loss_trans` = $\|\text{pred_u_t_trans} - u_t^{\text{trans}}\|_2^2$ (sum over the 3 coordinates, per fragment; $u_t^{\text{trans}} = x_1 - x_0$ of Definition 11.25)

`loss_R` = $\|\text{pred_u_t_R} - u_t^R\|_{\text{Fro}}^2$ (Frobenius norm² of the 3×3 difference; $u_t^R = \Xi$ of Definition 12.28)

Both are raw per-fragment sums with **no weighting and no clamping** inside `compute_losses` itself — confirming, precisely, Table 13.1’s claim that SigmaFlow’s loss has *two* terms where SigmaDock’s `compute_losses` (Section 10.9) has *four*: the two “data-space” ($\hat{x}_0$ -reconstruction) terms **T0/R0** that SigmaDock’s diffusion parameterisation computes alongside its score-matching terms have **no analogue at all** in SigmaFlow’s loss, confirmed by direct tracing of `trainer.py::SigmaLightningModule._shared_step`, whose total-loss combination `total_loss = trans_score_weight · loss_trans + rot_score_weight · loss_R` consumes exactly these two keys and no others (the CLI flag names `trans_score_weight`/`rot_score_weight` are themselves a naming holdover from the diffusion codebase this file is shared with; functionally, for the flow-matching path, they are ordinary fixed loss weights on (11.13)/(12.9)’s regression losses, not score-matching weights in Section 6.3’s $\lambda(s)$ sense). Per-molecule reduction uses `trainer.py::scaled_fragmented_loss`, structurally identical to Section 10.9’s code correspondence (shared infrastructure, per Table 13.1).

13.7 The full training step

We now trace `SigmaFlowGenerator.forward` end to end, with every intermediate tensor’s shape as commented in the source, assembling every piece derived above into the single computational pipeline a training batch actually executes.

1. **Prepare batch, sample time.** $t = \text{_sample_time}(\text{batch}) \in [\varepsilon_t, 1]^B$, one scalar *per molecule* in the batch (*not* per fragment), via $t = \varepsilon_t + (1 - \varepsilon_t) \cdot \text{Unif}(0, 1)$, $\varepsilon_t = \text{HPARAMS.general.epsilon_t} = 0.01$ — matching Section 6.3’s $t \sim \mathcal{U}(0, 1)$ up to the training-time floor ε_t , whose purpose (per an explicit code comment quoted verbatim, judged authoritative and reproduced here rather than re-derived) is distributional consistency between training and sampling, not any numerical-singularity avoidance: “training’s `sample_time` never samples t below this; sampling’s `t_min` should match it ... not related to the separate $1/(1 - t)$ singularity near $t = 1$ ” — i.e. the network

should never be *queried* at a t value it was never trained on, which is a distributional/generalisation concern entirely distinct from Remark 12.27’s numerical concern about the current-point vector-field formula near $t = 1$.

2. **Get initial (data-time) states.** $\text{pos}_0, \text{trans}_1, R_1, \text{num_fragments} = \text{_get_initial_states}(\text{batch})$: despite the subscript “0”, pos_0 is the (pocket-centred, `dimensional_scale`-rescaled, Remark 10.9) ground truth/reference atom geometry — the fixed per-fragment *template* x^{ref} of Proposition 10.10, reused at every t , not a draw from the source distribution — while trans_1, R_1 are the per-fragment data-time (centre-of-mass, and, per Remark 13.1, identically- I) rigid-motion state. t is broadcast from one value per molecule to one value per fragment, $t_{\text{batch}} \in [\varepsilon_t, 1]^{\Sigma F}$ (*all fragments of a given molecule share the same sampled t* , a design choice with no counterpart requirement in the theory of Chapters 11 and 12, which permit — but do not require — an independent t per component; the code’s own inline comment flags this exact choice as a documented, unresolved limitation: “better sampling. Uniform makes noises (sigmas) that are too redundant”).
3. **Sample the conditional path.** $(\text{trans}_t, R_t, u_t^{\text{trans}}, u_t^R) = \text{_sample_flow}(\text{trans}_1, R_1, t_{\text{batch}})$, i.e. Sections 13.3 and 13.4’s `conditional_probability_path`, dispatched component-wise per Proposition 3.69.
4. **Reconstruct atom positions at time t .** $\text{pos}_t = \varphi(\text{trans}_t, R_t)$ via Proposition 10.10’s formula (`\_apply\_transformations`), and the graph’s transient (interaction) edges are recomputed at these new positions (`\_update\_batch`, Section 10.7’s discussion of the global/transient topology split).
5. **Network forward pass.** $\text{lig_pseudoforces}, \text{forces_idxs} = \text{_compute_forces}(\text{batch}, t)$ — exactly one EquiformerV2 evaluation (Chapter 9 and section 10.8’s code correspondence), identical in cost and mechanism to SigmaDock’s own single-network-call Newton–Euler pipeline.
6. **Newton–Euler reduction and Newton–Maruyama update.** $(F_F, \tau_F, M_F, I_F) = \text{_compute_fragment_dynamics}(\dots) ((10.2)); (dT, d\widehat{W}) = \text{_predict_fragment_updates}(\dots) ((10.3))$ — structurally byte-for-byte the same construction as Section 10.8, confirming Table 13.1’s claim that this component is unchanged.
7. **Predicted vector field.** $(\text{pred_u_t_trans}, \text{pred_u_t_R}) = \text{_compute_vector_field}(\dots)$, Section 13.5’s undressed pass-through.
8. **Loss.** Section 13.6’s two-term `compute\_losses`, weighted and summed by `trainer.py::SigmaLightningModel` wrapped in the `try/except-AssertionError` and finite-value guards this project’s own development history documents in detail (catching Definition 10.11’s “fragment mass < 2 ” assertion specifically, among other numerical failure modes).

Every step above, except step 8’s loss formula and steps 2–3’s use of the flow-matching (rather than diffusion) conditional path, is verified, in Table 13.1, to be identical to SigmaDock’s corresponding pipeline stage — the precise, exhaustive sense in which this project’s conversion of SigmaDock into SigmaFlow is, as claimed throughout this document’s audited source material, a *surgical*, minimally-invasive substitution of the probability path and training target alone.

13.8 Sampling

Code correspondence

Mathematical object: the forward ODE integration loop of Sections 11.15 and 12.10
Implementation: `sampling.py::sampler` (in `SigmaFlow_Development/src/sigmadock/diff/`)
Interpretation: default `num_steps=25`, `t_min=εt=0.01` (matching training’s floor, per this chapter’s Step 1 discussion above), `rho=3.0`, default `discretization="power"` (*not* SigmaDock’s default `"edm"` — a genuine, code-confirmed default-configuration asymmetry between the two samplers, flagged here rather than assumed identical: any timing/quality comparison between the two must either override one to match the other or explicitly account for this difference), `solver="euler"` (Heun again unimplemented, per Section 10.10’s finding, verbatim-identical dead # TODO comment block in this file too). The timestep array walks *upward*, $t_{\min} \rightarrow t_{\max} = 1$, matching this document’s global convention directly (no relabelling needed, unlike SigmaDock’s sampler). Each loop iteration: (1) evaluate the closed-form current-point field $u_t(\cdot | z)$ (`_compute_true_vector_field`, i.e. Equations (11.13) and (12.9)’s singular-but-exact forms, using the *known* trans_1, R_1 — available here because sampling is being run, in this project’s own evaluation methodology, on complexes whose true pose is known, purely for diagnostic loss logging at each step, not because a real blind-docking deployment would have this available); (2) if `use_true_vector_field=False` (the normal, non-diagnostic case), run the network (Steps 5–7 of Section 13.7’s list) to get the predicted field instead; (3) Euler step (`SE3_FlowMatcher.euler_step`, dispatching to Sections 13.3 and 13.4’s componentwise `euler_step` implementations); (4) reconstruct positions via φ (Proposition 10.10) at the new $(\text{trans}_{t+dt}, R_{t+dt})$. Exactly one network evaluation per step, matching SigmaDock’s sampler exactly in this specific respect (Section 10.10).

Remark 13.2 (The `use_true_vector_field` oracle: what Test 1 established). Substituting the closed-form field for the network’s prediction at *every* step (`use_true_vector_field=True`) isolates a question this chapter’s derivations make precise: does the ODE integration/reconstruction machinery itself (Euler stepping, the φ reconstruction, batching and edge recomputation) introduce error, independent of network quality? By Proposition 12.26’s exact algebraic identity and Proposition 11.28’s exact linearity (both established rigorously in this chapter, not merely observed empirically), the answer, *in exact arithmetic*, must be no: the oracle field reconstructs the true trajectory exactly, for any number of steps, since both conditional vector fields are (by Section 11.15’s remark on the linear path, and Proposition 12.26 for the geodesic path) either exactly integrable by a single Euler step (translation) or exactly consistent with the constant-velocity geodesic the Euler step is discretising (rotation, up to floating-point round-off). This is the theoretical content underlying this project’s empirical “Test 1” finding (oracle-vector-field sampling reconstructs the true structure to within floating-point precision, $\leq 0.0002 \text{ \AA}$ deviation, across a sample of real generated structures) — the theorem, derived here, is what the experiment confirmed, not a separate, independent fact.

13.9 Likelihood and confidence: what is implemented, and what is not

Section 11.4 derived, from first principles, that a CNF’s log-density along any trajectory evolves by $\frac{d}{dt} \log p_t(X_t) = -(\nabla \cdot u_t)(X_t)$ (Corollary 11.14), estimable via the Hutchinson trace estimator without a separate density network. This is a genuine, well-established capa-

bility of the flow-matching framework in general (and one diffusion models do not share as directly, since the diffusion side’s analogous likelihood requires the full probability-flow-ODE construction of Theorem 6.9, an extra derived object rather than something immediately available from the sampler already being run).

Caution 13.3 (Not implemented in this codebase). Per the code-research pass underlying this document, **no code path in either `sampling.py` or `sigma_flow_generator.py` computes $\nabla \cdot u_\theta$, a Hutchinson trace estimate, or any accumulated log-density along a sampled trajectory**; no likelihood or confidence score derived from Corollary 11.14 is produced anywhere in the sampling pipeline reconstructed in Section 13.8. SigmaFlow’s actual ranking mechanism, where used, is inherited unchanged from SigmaDock’s non-learned heuristic (Section 10.11), per Table 13.1. Corollary 11.14’s construction is recorded in this document because it is a genuine, theoretically well-founded *capability* that the flow-matching formulation — unlike the diffusion formulation — would make relatively straightforward to add (a scalar ODE run alongside the existing vector field, using an already-trained u_θ with no retraining required), and because Chapter 1’s discussion of confidence estimation as a distinct task from pose generation (Section 1.3) specifically named this as an open, unimplemented direction; it should not be read as a description of current SigmaFlow behaviour.

13.10 A tested, and rejected, loss variant

We record, briefly and for completeness given Section 13.5’s discussion of the “no time-dependent scaling needed” design choice, one concrete alternative this project’s broader experimental work tested empirically, since a reader who has just followed Section 6.3’s discussion of diffusion’s $\lambda(s)$ weighting might reasonably ask whether an analogous reweighting should be added to SigmaFlow’s (currently unweighted, per Section 13.6) flow-matching loss.

Caution 13.4 (Empirically tested, no benefit found; not part of this document’s core theory). A variant multiplying Section 13.6’s per-fragment squared errors by a factor $1/(1-t)^2$ (capped near $t = 1$ for numerical stability) was implemented, trained, and evaluated as part of this project’s broader experimental investigation. Two findings from that investigation are relevant here, stated precisely rather than left vague: **(i)** this reweighting has *no* grounding in the flow-matching theory developed in Chapters 11 and 12 of this document — unlike diffusion’s $\lambda(s)$, which Section 6.3 derives as compensating for the conditional score’s genuinely t -varying scale, Section 13.5’s constant-in- t conditional targets have no analogous scale variation to compensate for, so this variant is, by this document’s own theory, an unmotivated modification rather than a theoretically indicated correction; **(ii)** consistent with point (i), it was found, empirically, to produce no measurable improvement (and if anything a marginal degradation) in the fragment-boundary bond-length metric this project’s broader work uses to quantify exactly the failure mode Section 10.4’s discussion anticipates. We record this only to close the loop on a natural question the theory of this chapter raises, not as part of SigmaFlow’s actual, shipped training recipe.

Chapter 14

The Output Heads, Audited

This chapter answers one question for each quantity the generative process consumes: *what is this object, in which space does it live, in which frame is it expressed, and how must it transform under a global rotation?*

The method is deliberately narrow. No frame label is taken from a comment, a variable name, or an earlier passage of this document — several of those have already been shown to disagree with each other. Every claim below is first derived from the operations the code actually performs, and then confirmed numerically by applying a random $Q \in SO(3)$ to a real input graph and comparing the two sides of the candidate transformation law. Where the derivation and the measurement disagree, the measurement decides what the code does, and the disagreement is recorded rather than smoothed over.

Caution 14.1 (Three heads were expected; the code has one). The natural mental model of a rigid-fragment docking network — one head for translation, one for rotation, one for torsion — does not describe SigmaDock or SigmaFlow. `EquiformerV2.forward` has exactly one geometric output:

$$\text{forces} \in \mathbb{R}^{N \times 3},$$

one 3-vector per *atom*. Translation and rotation are not predicted; they are *derived* from that single field by classical rigid-body mechanics, outside the network. Torsion is not predicted at all. The rest of this chapter is organised around that fact.

14.1 Locating the heads

Output	Where it is produced	Kind
per-atom pseudo-force	<code>net/model.py::EquiformerV2.forward</code>	network
translation	<code>diff/sigma_flow_generator.py::linear_mechanics</code>	derived
rotation	same, then <code>newton_maruyama</code> , then <code>_compute_vector_field</code>	derived
torsion	—	does not exist

A repository-wide search for `torsion` and `dihedral` returns 160 hits, all of them in `chem/` (fragmentation and preprocessing), `config.py`, `data.py`, and one comment in `net/__init__.py` labelling edge type 3. In `diff/` the count is zero, and in `net/` the only occurrence is that comment. There is no torsion head to audit.

Remark 14.2 (What replaced the torsion head). Torsional flexibility has not been dropped; it has been moved out of the model and into the state space. `detect_torsional_bonds` cuts the ligand at its rotatable bonds into F rigid fragments, and each fragment carries its own $SE(3)$ element. A torsion about a cut bond is then not an angle the network predicts, but the *relative* rigid motion of the two fragments on either side — already representable by the existing per-fragment outputs. This is why the state space is $SE(3)^F$ and not $SE(3) \times \mathbb{T}^k$: the torsional degrees of freedom were converted into rigid-body ones at the data-processing stage.

The price is that bond lengths and angles across a cut are no longer constrained by construction, which is exactly what the triangulation prior of Chapter 10 exists to repair.

14.2 The tensor entering the heads

Only one tensor leaves the backbone.

Property	Value
Object	<code>S03_Embedding</code> , attribute <code>.embedding</code>
Shape	$[N, (L+1)^2, C] = [N, 16, 128]$ at defaults
Level	node (per atom); edges are consumed, not returned
(ℓ, m) layout	flattened, ℓ -major: index $\ell^2 + \ell + m$
L	<code>l_max_list = (3,)</code>
m cap	<code>m_max_list = (2,)</code> , applied in the <i>edge</i> frame only
Frame	global (world); the edge frame exists only inside a block
Last op before readout	<code>self.norm</code> , then <code>self.force_block</code>

The index map $\ell^2 + \ell + m$ gives, for $L = 3$:

ℓ	m range	indices	count	running total	role
0	0	0	1	1	scalars
1	−1..1	1, 2, 3	3	4	the readout
2	−2..2	4..8	5	9	internal only
3	−3..3	9..15	7	16	internal only

`force_block` is a further `S02EquivariantGraphAttention` — a full attention block, not a linear layer — configured with one output channel. Its output is sliced by `narrow(1, 1, 3)`: axis 1, offset 1, length 3, i.e. exactly indices 1, 2, 3, exactly $\ell = 1$. Degrees $\ell = 0, 2, 3$ are computed and then discarded at the very last step.

14.3 Translation

Mathematical requirement. A translational velocity is a polar vector: under a global rotation of the scene it must satisfy $v(Qx) = Qv(x)$.

What the code does. `linear_mechanics` sums the per-atom field over each fragment, and `newton_maruyama` divides by the atom count:

$$F_f = \sum_{i \in f} f_i, \quad v_f^{\text{tr}} = \frac{F_f}{m_f}, \quad m_f = |f|.$$

That is the whole translation head. There is no projection layer, no magnitude/direction factorisation, no separate $\ell = 0$ scaling, and no learned pooling — an unweighted sum and a division by an integer. In particular the answer to “is v built as $su/\|u\|$ from an $\ell = 0$ magnitude and a normalised $\ell = 1$ direction?” is **no**, for translation and for rotation alike. Every head in this codebase is linear in the per-atom field.

The centre of mass enters only through `get_fragment_com_and_rot`, and only as the pivot for the torque; the net force does not use it.

Code correspondence

Where does it live?

$v_f^{\text{tr}} \in \mathbb{R}^3$, one per fragment.

Frame: world, the same Cartesian frame as the input coordinates (after pocket-COM subtraction and division by `dimensional_scale`).

Transformation law: $v^{\text{tr}}(Qx) = Q v^{\text{tr}}(x)$.

Measured: 4.5×10^{-15} relative, float64 (Table 14.1).

14.4 Rotation

This is the head that matters, and the one where the labels in the source have historically been wrong.

14.4.1 The two angular velocities, restated next to the code

For a curve $R(\tau)$ in $SO(3)$ there are two ways to write the tangent, and they are distinguished by the *equation*, never by a label:

$$\dot{R} = R \widehat{\omega^{\text{bd}}} \quad \text{body coordinates,} \quad (14.1)$$

$$\dot{R} = \widehat{\omega^{\text{sp}}} R \quad \text{spatial (world) coordinates,} \quad (14.2)$$

related by

$$\omega^{\text{sp}} = R \omega^{\text{bd}}, \quad \text{equivalently} \quad \widehat{\omega^{\text{sp}}} = R \widehat{\omega^{\text{bd}}} R^{\text{T}}. \quad (14.3)$$

The second form of (14.3) is the one the code uses, because it stores $\widehat{\omega}$, not ω . Conjugation by a rotation preserves skew-symmetry, so both sides are genuine $\mathfrak{so}(3)$ elements.

Two consumption sites in SigmaFlow fix which of (14.1)–(14.2) the loss and the sampler expect, and they agree:

Site	Code	Implies
target	<code>calc_rot_vector_field: $\log(R_t^{\text{T}} R_1)/(1-t)$</code>	body
integrator	<code>euler_step: $R_t @ \exp(v_t * dt)$</code>	body

$R_{\text{next}} = R_t \exp(v dt)$ is right multiplication, i.e. $\dot{R} = Rv$, i.e. (14.1). The docstring of `calc_rot_vector_field` calls this “right trivialised”; Section 3.5 of this document calls the same object left-trivialised. Both names are in circulation and neither is used here as evidence — (14.1) is the definition, the label is not.

14.4.2 The trace, operation by operation

#	Operation	Code	Shape
1	backbone features	<code>blocks, norm</code>	$[N, 16, 128]$
2	attention readout	<code>force_block</code>	$[N, 16, 1]$
3	select $\ell = 1$	<code>narrow(1, 1, 3)</code>	$[N, 3, 1]$
4	flatten	<code>view(-1, 3)</code>	$[N, 3]$
5	sign flip, ligand select	<code>_compute_forces: -eps[idx]</code>	$[N_L, 3]$
6	torque about COM	$\tau_f = \sum_{i \in f} (x_i - c_f) \times f_i$	$[F, 3]$
7	inertia solve	$\Delta W = I_f^{-1} \tau_f$	$[F, 3]$
8	hat map	$\widehat{\Delta W}$, then clamp to $\pm 10^3$	$[F, 3, 3]$
9	frame conversion	$R_t^T \widehat{\Delta W} R_t$	$[F, 3, 3]$

Step 9 is the only place a rotation matrix touches the rotational output. It lives in `_compute_vector_field` and it is the “frame fix”. It happens *after* aggregation, on the per-fragment quantity, and it uses R_t — the flow-matching path variable — not an edge frame and not a learned rotation.

Reading (14.3) right to left: steps 6–8 produce $\widehat{\omega^{\text{sp}}}$, and step 9 computes $R_t^T \widehat{\omega^{\text{sp}}} R_t = \widehat{\omega^{\text{bd}}}$. So the head does produce the body-frame quantity that steps “target” and “integrator” above require. The derivation and the code agree.

14.4.3 $\mathbb{R}^3$ or $\mathfrak{so}(3)$?

Both, at different points, and the code changes representation twice:

- Steps 6–7 hold $\tau_f, \Delta W \in \mathbb{R}^3$: three numbers, an ordinary Cartesian triple.
- Step 8 applies the hat map $\widehat{\cdot} : \mathbb{R}^3 \rightarrow \mathfrak{so}(3)$, a linear isomorphism onto the skew-symmetric 3×3 matrices, with $\widehat{v}w = v \times w$.
- From step 9 onward the object is a matrix in $\mathfrak{so}(3)$, and it stays a matrix: `compute_losses` takes a Frobenius norm over the last two axes, and `euler_step` feeds it to the matrix exponential.

So the *stored* object is a 3×3 matrix, and the loss is computed on matrices. Since $\|\widehat{a} - \widehat{b}\|_F^2 = 2\|a - b\|_2^2$, the matrix Frobenius loss equals twice the vector loss — a constant factor, absorbed into the loss weight, but worth knowing when comparing loss magnitudes across implementations.

14.4.4 Rotation or rotation rate?

The head does not predict R_{new} . Every quantity from step 6 onward lives in a tangent space: $\mathfrak{so}(3)$, the Lie algebra, not the group. The network output is a *rate*, integrated by

$R_{\text{next}} = R_t \exp(v dt)$. It is only a *velocity* rather than a score because of how the target is built — and that distinction, together with the units and scaling question, is the subject of the next round, not this one.

14.4.5 The group action is conjugation, not left multiplication

Here the standard textbook statement and this codebase part company, and the difference is not cosmetic.

Mathematical requirement. If a global rotation acts on fragment orientations by $R \mapsto QR$, then a body-frame angular velocity is *invariant*: from (14.1), $\dot{R} \mapsto Q\dot{R}$ and $R \mapsto QR$ give $\widehat{\omega^{\text{bd}}} = R^\top \dot{R} \mapsto R^\top Q^\top Q \dot{R} = R^\top \dot{R}$, unchanged. A spatial one transforms as $\omega^{\text{sp}} \mapsto Q\omega^{\text{sp}}$. That is the standard dichotomy, and it is what this document asserted before this audit.

What SigmaFlow actually implements. R_t is not a fragment’s absolute orientation. It is a *delta* rotation relative to the reference structure `pos_0`, applied about the fragment COM by `get_transformations_from_rototranslations`:

$$x_i^{(t)} = \Delta R_f (x_i^{(0)} - c_f) + c_f + \Delta T_f, \quad \Delta R_f = R_t R_1^\top. \quad (14.4)$$

Two facts about (14.4) settle the group action:

1. `pos_0` is an *input* to the network — it is read by `compute_edge_embedding` for the boundary edges — so a global rotation of the scene rotates it too: $x^{(0)} \mapsto Qx^{(0)}$, $c_f \mapsto Qc_f$.
2. `get_fragment_com_and_rot` returns $R_1 = I_3$ *unconditionally*, with the comment “Rotation matrix is identity: it is (and should be) independent in the vector field.”

Substituting $x^{(0)} \mapsto Qx^{(0)}$, $c_f \mapsto Qc_f$ into (14.4) and demanding $x^{(t)} \mapsto Qx^{(t)}$ forces

$$\boxed{R_t \mapsto Q R_t Q^\top} \quad (14.5)$$

and not $R_t \mapsto QR_t$. This is also the only action compatible with $R_1 \equiv I$: conjugation fixes the identity, $QIQ^\top = I$, whereas left multiplication does not, $QI = Q \neq I$.

Measured directly against `_apply_transformations`, asking which substitution reproduces $Qx^{(t)}$:

Substitution	relative residual
$R_t \mapsto QR_t$	1.06
$R_t \mapsto QR_t Q^\top$	3.7×10^{-16}

Proposition 14.3 (The body-frame output is *not* invariant here). *Under (14.5),*

$$\widehat{\omega^{\text{bd}}} = R_t^\top \widehat{\omega^{\text{sp}}} R_t \mapsto (QR_t Q^\top)^\top (Q \widehat{\omega^{\text{sp}}} Q^\top) (QR_t Q^\top) = Q (R_t^\top \widehat{\omega^{\text{sp}}} R_t) Q^\top,$$

*i.e. it transforms by conjugation, equivalently $\omega^{\text{bd}} \mapsto Q\omega^{\text{bd}}$ after the vee map. It is **not** left unchanged.*

Caution 14.4 (Correction to Section 8.2). An earlier passage of this document stated that “equivariance of the body-frame rotational output under a world-frame rotation of the whole scene means the body-frame output is left *unchanged*, since it is expressed relative to the fragment’s own, co-rotating frame”. That is correct for the action $R \mapsto QR$ on an *absolute* orientation. It is false for SigmaFlow, whose R_t is a delta rotation against a co-rotating reference and therefore conjugates. Measured: the invariance hypothesis fails at 1.20 relative, the conjugation hypothesis holds at 2.3×10^{-15} .

The lesson generalises: “body-frame” fixes the relation between the tangent vector and $\dot{R}$; it does *not* by itself determine the transformation law, because that also depends on how the group acts on R . Both must be stated.

Crucially, the target transforms the same way. With $u_t^R = \log(R_t^\top R_1)/(1-t)$ and $R_1 = I$ preserved by conjugation, $u_t^R \mapsto Qu_t^R Q^\top$ — measured 8.3×10^{-15} . Prediction and target therefore transform identically, the Frobenius norm of their difference is invariant, and the loss is a well-defined function on the quotient. **The frame fix is correct.**

Code correspondence

Where does it live?

$\omega^{\text{bd}} \in \mathfrak{so}(3)$, stored as `[F,3,3]`; equivalently $\omega^{\text{bd}} \in \mathbb{R}^3$ via the vee map.

Frame: body, in the sense $\dot{R} = R\hat{\omega}$ (14.1) — fixed by `euler_step`’s right multiplication, not by any comment.

Transformation law: $\hat{\omega} \mapsto Q\hat{\omega}Q^\top$ under $R_t \mapsto QR_tQ^\top$, *not* invariant.

Measured: 2.3×10^{-15} relative, float64.

14.5 Numerical verification

Setup. Two real complexes from the dummy set, $N = 887$ nodes, $E = 4564$ edges; randomly initialised weights with `zero_init_last=False` (the default `True` zeroes the final layer, so the network would output exactly 0 and every equivariance test would pass vacuously); `eval` mode, all dropout at 0; float64 throughout the geometric path; 12 Haar-random rotations per row; residuals are $\max |a - b| / \max(\|a\|_\infty, \|b\|_\infty)$.

Equivariance is a property of the architecture, not of the weights, so no trained checkpoint is needed — and a trained one would prove less, since a low loss is not evidence of a correct frame.

The permutation hypothesis, and why it had to be tested. In the `e3nn` convention the real $\ell = 1$ spherical harmonics are ordered $(m = -1, 0, +1) \propto (y, z, x)$, not (x, y, z) . Had the readout inherited that ordering while the mechanics downstream read the three numbers as Cartesian xyz , the composed map would transform as $PQP^\top$ and the bug would be nearly invisible: the output would still be a 3-vector of plausible magnitude, and the loss would still decrease. It is refuted at 1.49.

The reason it is refuted closes a loop left open earlier: `init_edge_rot_mat` builds `rot_inv` with columns $[n_z, n_x, n_y]$, aligning the edge to e_2 — the *second* axis. In the (y, z, x) ordering the second slot is exactly the $m = 0$ slot, which is what the `eSCN` reduction requires. The apparently odd choice of e_2 over e_1 and the apparently odd (y, z, x) ordering are the same convention, and they cancel. What last round recorded as a bare empirical fact now has its reason.

Quantity	Hypothesis tested	Residual
forces	$f(Qx) = Q f(x)$	5.3×10^{-15}
forces	$f(Qx) = f(x)$	1.47
forces	$f(Qx) = P Q P^\top f(x)$	1.49
total_force	$F \mapsto Q F$	4.5×10^{-15}
total_torque	$\tau \mapsto Q \tau$	1.3×10^{-15}
inertia	$I \mapsto Q I Q^\top$	8.6×10^{-16}
delta_W	$\Delta W \mapsto Q \Delta W$	2.4×10^{-15}
omega	$\hat{\omega} \mapsto Q \hat{\omega} Q^\top$	2.6×10^{-15}
pred_u_t_R	invariant	1.20
pred_u_t_R	$u \mapsto Q u Q^\top$	2.3×10^{-15}
u_t_R (target)	$u \mapsto Q u Q^\top$	8.3×10^{-15}

Table 14.1: Frame audit. Rows use the gate activation, which is exactly equivariant; see Section 14.6 for the production configuration. P is the permutation $xyz \mapsto yzx$.

14.6 A defect found on the way

The first pass of this audit produced residuals near 10^{-1} for *every* hypothesis, too noisy to separate “exact” from “approximate”. The cause turned out to be worth reporting in its own right.

`init_edge_rot_mat` chooses the axis perpendicular to each edge by `torch.rand_like`, freshly, on every forward pass. The eSCN construction is supposed to be indifferent to that choice: different choices differ by a rotation about the edge axis, and the $SO(2)$ convolution commutes with it. Measured on the production configuration (128 channels, 6 layers, $L = 3$, $m_{\max} = 2$, 14.98 M parameters), it is not indifferent:

Configuration	gauge dependence	equivariance error
S ² activation (production default)	1.06×10^{-1}	9.4×10^{-2}
gate activation	1.4×10^{-15}	5.3×10^{-15}

Swapping only the activation moves both numbers by fourteen orders of magnitude, which localises the defect precisely. In `S02EquivariantGraphAttention.forward` the S² activation sits at line 270, between `_rotate` (line 250) and `_rotate_inv` (line 324) — that is, *inside the edge frame*. It evaluates features on a fixed grid on the sphere, and that grid is not invariant under rotation about the edge axis. The arbitrary azimuthal choice therefore moves the features relative to the grid, and the discretisation error does not cancel.

Two consequences follow, and neither is a rounding issue:

- **The forward pass is not deterministic.** In `eval` mode, with all dropout disabled, two evaluations of the identical input differ by about 10% relative. During ODE sampling a fresh gauge is drawn at every step.
- **$SO(3)$ equivariance is broken at the same order.** About 10%, in the shipped configuration.

Caution 14.5 (Scope, and what is *not* claimed). These numbers are measured at **random initialisation**. Whether training suppresses the effect — by learning features that are smooth enough on the grid for aliasing to be small — is not established here, and cannot be: the only checkpoints available locally are 15-step archives. Measuring this on a trained checkpoint is marked PENDING ARC VALIDATION.

It is also not claimed that this explains the rotation-channel failure. It is a quantified, reproducible defect in the shipped configuration, on the path the rotational output travels, and it is a candidate — nothing stronger until it is measured on trained weights. Note also that it affects SigmaDock and SigmaFlow *identically*, since both use the same backbone, so it cannot by itself explain a *difference* between them.

14.7 Summary tables

Head	Backbone input	Irreps used	Intermediate	Final	Geometric type
translation	[N, 16, 128]	$\ell = 1$ only	[N, 3]	[F, 3]	polar vector
rotation	[N, 16, 128]	$\ell = 1$ only	[N, 3]	[F, 3, 3]	$\mathfrak{so}(3)$ element
torsion	—	—	—	—	does not exist

Table 14.2: Both heads read the same three numbers per atom. They differ only in how those numbers are pooled: sum for translation, cross-product moment for rotation.

Quantity	Coordinates	Under global Q	Residual
per-atom force	world	$f \mapsto Qf$	5.3×10^{-15}
translation	world	$v \mapsto Qv$	4.5×10^{-15}
torque	world	$\tau \mapsto Q\tau$	1.3×10^{-15}
$\widehat{\omega}^{\text{sp}}$ (omega)	world	$\widehat{\omega} \mapsto Q\widehat{\omega}Q^T$	2.6×10^{-15}
$\widehat{\omega}^{\text{bd}}$ (pred_u_t_R)	body	$\widehat{\omega} \mapsto Q\widehat{\omega}Q^T$	2.3×10^{-15}
torsion	—	—	—

Table 14.3: Frames. The last two rows carry the point of this chapter: the body-frame quantity transforms *by the same law* as the world-frame one, because the group acts on R_t by conjugation (14.5). Body-frame does not imply invariant.

14.8 Open items

Deliberately left for the next round, to keep this one narrow:

- S^2 aliasing on a trained checkpoint — PENDING ARC VALIDATION.
- Score-versus-velocity semantics, units, and the `dimensional_scale` bookkeeping.
- Whether the $\pm 10^3$ clamp on $\widehat{\Delta W}$ in `newton_maruyama` ever binds in practice, and what it does to the gradient when it does.
- The inertia eigenvalue clamp, likewise.

Chapter 15

Backbone, Mechanics, and Generative Semantics

The previous chapter established what the network emits. This one asks a different question: *which parts of the code belong to the geometry, which to the diffusion process, which were changed for flow matching — and are the changes mathematically coherent?*

The organising device is a strict three-layer split. Confusing the layers is what makes “score” and “velocity” look interchangeable when they are not.

	Layer	Content	Status
A	geometric backbone	graph, embeddings, EquiformerV2, $\ell = 1$ readout	byte-identical
B	mechanical reduction	force $\rightarrow$ torque $\rightarrow$ inertia $\rightarrow \omega$	formula-identical
C	generative semantics	path, target, loss, sampler	entirely replaced

15.1 Layer A is byte-identical

Not “similar”: identical. A file-by-file `diff` of the backbone between `SigmaDock/src_sigmadock_net/` and `SigmaFlow_Minimal/src_sigmadock_net/` returns zero differing lines for all seventeen files, `model.py` included — so the graph construction, the featurisation, the attention, the normalisation, the S^2 activation and the readout are the same code, and the trainable parameter count is identical by construction.

Outside `net/`, the only differences in the shared pipeline are non-semantic:

File	Differing lines	Nature
<code>net/*</code> (17 files)	0	—
<code>chem/processing.py</code>	0	—
<code>oracle.py</code>	5	one comment
<code>data.py</code>	13	error reporting, one added <code>None</code> check
<code>chem/parsing.py</code>	12	—
<code>diff/so3_utils.py</code>	2	one clamp, Section 15.6.3

For the minimal-change claim this is the strongest available evidence, and it is worth stating plainly: at layer A there is nothing to audit, because there is nothing different.

15.2 Layer B: the mechanical reduction

Both models pool the per-atom field identically. Writing f_i for the per-atom vector of Chapter 14 and f for a fragment:

$$F_f = \sum_{i \in f} f_i, \quad m_f = |f| \quad (\text{atom count, all masses 1}), \quad (15.1)$$

$$\tau_f = \sum_{i \in f} (x_i - c_f) \times f_i, \quad I_f = \sum_{i \in f} (\|r_i\|^2 I_3 - r_i r_i^\top), \quad (15.2)$$

$$\Delta T_f = F_f / m_f, \quad \Delta W_f = I_f^{-1} \tau_f. \quad (15.3)$$

These are textbook rigid-body formulae with two deviations, both present in both models:

- **Uniform unit masses.** $m_i \equiv 1$, so m_f is an atom count and I_f is a count-weighted inertia. Physically this is a body of uniform “atoms”, not of real atomic masses.
- **Two regularisations.** I_f is eigenvalue-clamped from below at $\max(10^{-8}, 10^{-8} \text{tr } I_f / 3)$ before use, then a further $10^{-8} I_3$ is added inside `newton_maruyama`; and $\widehat{\Delta W}$ is clamped to $\pm 10^3$. Whether the latter ever binds is listed as open in Section 14.8.

Nothing here is learned. The only trainable object in the entire chain is the per-atom field.

Remark 15.1 (Is `forces` a force?). No. There is no energy function, no conservativity constraint, and nothing in either loss that rewards f_i for being the gradient of anything. The correct description is: an **atom-wise $\ell = 1$ equivariant vector field**, which the code names `forces` and *consumes as if* it were a force field, because the Newton–Euler reduction is a convenient, exactly equivariant way to pool N atom vectors into 6 rigid-body numbers.

This document therefore uses “force-like” or “per-atom vector field”, and reserves “force” for the name in the source. The distinction matters once one asks what the network is being trained to represent — which is the subject of the rest of this chapter.

15.3 Layer C, SigmaDock: a score model

Time. $t \sim \text{Unif}[\varepsilon_t, 1]$ with $\varepsilon_t = 0.01$. $t = 0$ is *data*, $t = 1$ is *noise*.

Translation: VP-SDE. With $\beta(t) = \beta_{\min} + t(\beta_{\max} - \beta_{\min})$, $\beta_{\min} = 0.1$, $\beta_{\max} = 20$, and $\alpha_t = e^{-\frac{1}{2} \int_0^t \beta}$,

$$x_t = \alpha_t x_0 + \sqrt{1 - \alpha_t^2} z, \quad z \sim \mathcal{N}(0, I_3), \quad (15.4)$$

so the target is the *conditional* score

$$s^* = \nabla_{x_t} \log p_t(x_t | x_0) = -\frac{x_t - \alpha_t x_0}{1 - \alpha_t^2}. \quad (15.5)$$

Rotation: IGSO(3). $\sigma(t)$ follows a logarithmic schedule from 0.1 to 1.5, and

$$R_t = \Delta R R_0, \quad \Delta R \sim \text{IGSO}(3)(\sigma(t)), \quad (15.6)$$

i.e. left multiplication. The target is

$$s_R^* = R_t \log(R_0^\top R_t) \cdot \frac{\partial_\omega \log f_\sigma(\omega)}{\omega}, \quad \omega = \text{angle}(R_0^\top R_t), \quad (15.7)$$

with f_σ the truncated IGSO(3) angle density.

Scaling — the part that matters for the comparison.

$$\hat{s} = F_f \cdot \underbrace{\frac{1}{\sigma(t)}}_{\text{score_scaling}}, \quad \mathcal{L} = \underbrace{\frac{1}{\text{score_scaling}^2}}_{\lambda(t)=\sigma(t)^2} \|\hat{s} - s^\star\|^2 = \|F_f - \sigma(t) s^\star\|^2. \quad (15.8)$$

The two factors cancel exactly. So SigmaDock’s network does *not* learn the score: it learns the **variance-normalised score** $\sigma_t s^\star$, a quantity of roughly unit scale at every t . This is the standard device that stops the loss from being dominated by small- t samples where $\|s^\star\| \sim 1/\sigma$ diverges. The same construction, with $\lambda = 1/\text{score_scaling}^2$ from the IGSO(3) table, is used for rotation.

Two further losses, both switched off. `compute_losses` also returns data-space terms `T0` and `R0`, but the trainer’s defaults are `trans_data_weight = rot_data_weight = 0.0`. Both models therefore train exactly two active terms, weighted 1.0 and 0.5.

Sampling. Reverse SDE, with injected noise: $x_{t-1} = x_t - [(f_t - g_t^2 s_t) dt + \eta g_t \sqrt{dt} z]$.

15.4 Layer C, SigmaFlow: a velocity model

Time — reversed. Same sampler, same ε_t , but now $t = 0$ is the *source* and $t = 1$ is the *data*.

Meaning	SigmaDock	SigmaFlow
$t = 0$	data	source (noise)
$t = 1$	noise	data
data endpoint	<code>T0</code> , <code>R0</code>	<code>trans_1</code> , <code>R_1</code>
source endpoint	terminal law of the SDE	<code>sample_init</code>
training	corrupt forwards	interpolate
sampling	integrate $1 \rightarrow 0$	integrate $0 \rightarrow 1$

The renaming `T0`→`trans_1`, `R0`→`R_1` is not cosmetic: it tracks the flipped convention, and the variable holding the ground-truth pose changed its index. Both files feed the *same* sinusoidal encoder the raw t ; nothing diffusion-specific (no σ , no $\log \sigma$) is passed. Per the standing rule that a deterministic, invertible reparametrisation of time is not by itself a defect, this is clean — the encoder is a generic function of t , and each model is trained from scratch under its own convention.

Paths and targets.

$$x_t = (1 - t)x_0 + tx_1, \quad u_t = x_1 - x_0, \quad x_0 \sim \mathcal{N}(0, I_3), \quad (15.9)$$

$$R_t = R_0 \exp(t \log(R_0^\top R_1)), \quad u_t^R = \log(R_0^\top R_1), \quad R_0 \sim \text{Haar}. \quad (15.10)$$

Both targets are *constant in t* . For rotation this is a property of the geodesic, not an approximation: with $L = \log(R_0^\top R_1)$, $\dot{R}_t = R_0 \exp(tL)L = R_t L$, so $\widehat{\omega}^{\text{bd}} = R_t^\top \dot{R}_t = L$ for every t .

The code contains a second form, $\log(R_t^\top R_1)/(1-t)$ and $(x_1 - x_t)/(1-t)$. These are algebraically equal — $R_t^\top R_1 = \exp((1-t)L)$ — but the constant form has no $1/(1-t)$ and is used in training; the divided form appears only as a diagnostic in `diff/sampling.py`. Preferring the non-singular parametrisation is the right call.

No scaling. `_compute_vector_field` applies no factor to translation and only the frame conjugation to rotation. The loss is a plain MSE. This is correct for flow matching — u_t has no t -dependent divergence to tame — and it is the single largest semantic difference from SigmaDock.

Sampling. Explicit Euler, 1 network evaluation per step; `heun` is available at 2 evaluations per step.

15.5 Score versus velocity

Caution 15.2 (Same representation, different statistics). A translational score and a translational velocity are both $\ell = 1$ vectors. A rotational score and a rotational flow velocity are both tangent vectors on $SO(3)$. Representation theory constrains *how an object transforms*; it says nothing about *which statistical quantity it is*. The two questions are independent, and conflating them is what makes an engine swap look like a relabelling.

This is precisely why the swap is possible at all: layers A and B care only about the transformation law, which is unchanged, so they can be reused verbatim while layer C is replaced wholesale.

Property	Diffusion score	Flow velocity
Object	$\nabla_x \log p_t(x)$	$u_t(x)$, tangent to the transport
Represents	slope of a log-density	a displacement rate
Units (translation)	1/length	length/unit t
Behaviour as $t \rightarrow$ data	diverges like $1/\sigma_t$	finite, here constant
Needs $\lambda(t)$	yes	no
Target	(15.5)	$x_1 - x_0$
Sampling	reverse SDE, stochastic	ODE, deterministic
Transformation law	$v \mapsto Qv$	$v \mapsto Qv$

Table 15.1: The last row is the whole point: identical geometry, opposite statistics, even reciprocal units.

The units row deserves emphasis. Positions are divided by `dimensional_scale = 2.7 Å`, so both models work in the same dimensionless coordinates — but a score in those coordinates is an *inverse* length and a velocity is a length. The same network output tensor means reciprocal physical things in the two models. Since neither model converts to Å before the loss, and each is trained end-to-end in its own convention, this is consistent; it is a trap only if one compares raw loss values between the two, which would be meaningless.

15.6 Numerical validation

All float64, `audits/engine_swap_audit.py`, no GPU or checkpoint required.

Check	Method	Result
u_t is $\dot{x}_t$	central difference, $t \in [0.01, 0.99]$	$\leq 7 \times 10^{-11}$
u_t^R is $R_t^\top \dot{R}_t$	central difference	$\leq 3 \times 10^{-10}$
endpoints	x_t, R_t at $t = 0, 1$	exact
Euler exactness	full integration, $dt = 0.5, 0.25, 0.1$	$\leq 3 \times 10^{-16}$

Euler reproduces the endpoint exactly at *any* step size, because the paths are a straight line and a one-parameter subgroup. That confirms the integrator implements $x + dt v$ and $R \exp(dt v)$ and nothing else — no inherited σ or noise factor survived the port.

15.6.1 Source distributions are not quite matched

	SigmaDock terminal law	SigmaFlow source
translation	$\mathcal{N}(\alpha_1 x_0, (1 - \alpha_1^2)I)$, $\alpha_1 = 6.6 \times 10^{-3}$ std differs by 2×10^{-5} — effectively identical	$\mathcal{N}(0, I)$
rotation	IGSO(3)($\sigma=1.5$), mean angle 115.0° total variation 0.130 — not identical	Haar, mean angle 126.5°

IGSO(3) at $\sigma = 1.5$ has not yet reached the uniform distribution; it is short of mass at large angles. SigmaFlow starts from exactly Haar. The difference is modest but real, and it is a comparison confounder rather than a bug: SigmaFlow’s prior is the mathematically correct one for its construction, and SigmaDock’s is whatever its schedule reaches at $t = 1$. Recording it matters because a rotational metric that is already close to the random baseline is sensitive to exactly this.

15.6.2 An inherited weight that lost its justification

Both trainers combine $1.0 \cdot \mathcal{L}_{\text{trans}} + 0.5 \cdot \mathcal{L}_{\text{rot}}$. In SigmaDock those weights sit on top of $\lambda(t) = \sigma(t)^2$, which renders both terms dimensionless and $O(1)$; the 1:0.5 ratio is then a genuine preference between two comparable numbers. In SigmaFlow the λ is gone — correctly — but the 1:0.5 came along unchanged, and now multiplies two raw MSEs in different units.

Measured target magnitudes (Haar rotations, $R_1 = I$):

COM spread (normalised)	$\mathbb{E}\ u_{\text{trans}}\ ^2$	$\mathbb{E}\ u^R\ _F^2$	weighted ratio
0.5	3.77	10.59	0.71
1.0	6.04	10.62	1.14
2.0	15.12	10.55	2.87

$\mathbb{E}\|u^R\|_F^2 = 2 \mathbb{E}[\theta^2] \approx 10.6$ is essentially data-independent, since θ is a Haar angle. The translation term scales with `dimensional_scale`. So the effective balance is set by a normalisation constant chosen for a different purpose, and the 1:0.5 is inherited, not derived. It is not obviously wrong — the ratio lands within a factor of three of unity — but it is unjustified, and it should be recorded as a hyperparameter rather than a principled choice.

15.6.3 The log map is not exact

Because the rotational target is $\log(R_0^T R_1)$, the accuracy of the log map is the accuracy of the target. Over 40,000 Haar rotations in float64:

$$\|\exp(\log R) - R\|_\infty : \quad \text{median } 5 \times 10^{-7}, \quad 99\% \ 2.8 \times 10^{-3}, \quad \text{max } 2 \times 10^{-2}.$$

Nine orders of magnitude above machine precision. Two independent causes, both inherited from SigmaDock:

1. **Omega** computes $\arccos\left(\frac{(1-10^{-6}) \text{tr } R - 1}{2}\right)$. The factor $(1-10^{-6})$ is a guard against $\arccos$ leaving its domain, but it shrinks *every* trace, biasing every angle. At exactly 90° the error is 5.00×10^{-7} rad — the measured floor.
2. `rotation_vector_from_matrix` switches to its near- π branch only at $|\theta - \pi| < 10^{-2}$. Between roughly 178° and 179.4° it still uses $\theta/(2 \sin \theta)$, where $\sin \theta$ is already near zero. Measured error at 179° : 4.3×10^{-3} .

Caution 15.3 (Correcting a comment in `so3_flow_matcher.py`). A comment there states that the original clamp was “fixed”. SigmaFlow does narrow it from $[-0.99, 0.99]$ to $[-1+10^{-7}, 1-10^{-7}]$ — the only line by which `so3_utils.py` differs between the two repositories — and that removed a gross error affecting 9% of samples. But the map is still not exact, for the two reasons above. “Improved” is accurate; “fixed” is not.

Magnitude, honestly: relative to a target of norm ≈ 2 , this is 2×10^{-7} typically and $\approx 0.5\%$ at worst. It cannot explain a rotational failure measured at 138° against a 132° random baseline, which is an $O(1)$ effect. It is worth fixing for hygiene, not as a hypothesis.

15.6.4 sigma_min is dead

Setting `sigma_min` to 999 versus 0 changes neither x_t nor u_t (relative difference 10^0 only through the reseeded draw; the code path never reads the field). `S03_FlowMatcher` does not accept the argument at all. It is stored, threaded through `SigmaFlowGenerator.__init__` and the checkpoint, and never used. Keeping it invites the reader to believe the path is stochastic. It should either be removed or documented as reserved.

15.7 The minimal-change table

15.8 Findings, categorised

Finding	Category	Severity
S^2 / edge-gauge non-determinism, $\approx 10\%$	shared backbone	open, PENDING ARC
log map inexact (10^{-7} typ., 10^{-2} max)	shared backbone	low
unit masses in the inertia tensor	shared backbone	by design
rotational source Haar vs. IGSO(3)(1.5)	confounder	low-moderate
1:0.5 weights kept without λ	flow-specific	moderate
<code>sigma_min</code> stored, never read	flow-specific	cosmetic
“fixed” claim for Omega	documentation	corrected here
“forces” read as physical forces	documentation	corrected here

Component	SigmaDock	SigmaFlow	Same?	Should it differ?
graph construction	identical code		yes	no
EquiformerV2	byte-identical, 17 files		yes	no
$\ell = 1$ readout	<code>narrow(1,1,3)</code>		yes	no
parameter count	identical by construction		yes	no
mechanical reduction	same formulae		yes	no
time encoder	sinusoidal on t	sinusoidal on t	yes	no
time convention	0 data, 1 noise	0 source, 1 data	no	yes
corruption / path	VP-SDE, IGSO(3)	linear, geodesic	no	yes
target	normalised score	velocity	no	yes
$\lambda(t)$	σ_t^2	none	no	yes
sampler	reverse SDE	Euler ODE	no	yes
frame conversion	none needed	$R_t^\top \omega R_t$	no	yes
source (translation)	$\approx \mathcal{N}(0, I)$	$\mathcal{N}(0, I)$	yes	—
source (rotation)	IGSO(3)(1.5)	Haar	no	confounder
loss weights 1:0.5	with λ	without λ	yes	should have been revisited
Omega clamp	$[-0.99, 0.99]$	$[-1+10^{-7}, \cdot]$	no	yes, but incomplete
sigma_min	n/a	present, unused	—	remove

Table 15.2: Everything above the first rule is shared; everything between the rules is a necessary consequence of the engine swap; everything below is neither, and is where the scientific risk lives.

No *diffusion-specific* defect was found in SigmaDock on the paths examined, and — the question this round was really asked to answer — no diffusion-specific scaling factor survived into SigmaFlow. The σ -dependent machinery (`score_scaling`, `input_scaling`, the $\partial_\omega \log f$ table, $\lambda(t)$) is absent from `diff/` in SigmaFlow, not merely bypassed.

15.9 Verdict

Mostly a clean engine swap, with two confounders.

Layers A and B are shared to the byte. Every layer-C difference is a necessary consequence of replacing a score model by a velocity model, and the flow-matching construction is internally consistent: the targets are provably the derivatives of their own paths, the integrator implements exactly the derived ODE, prediction and target transform identically, and no inherited σ -scaling remains.

The two confounders are the rotational source distribution (Haar versus IGSO(3) at $\sigma = 1.5$, total variation 0.130) and the translation/rotation loss weights, retained from a setting in which a normalisation made them comparable and now applied where it does not. Neither invalidates the comparison; both must be stated when it is reported.

Part VIII

Implementation Reference and Synthesis

Part VIII at a glance

What you will learn. Two closing passes, in that order. First a practical one: an index of every clamp, epsilon, masking convention, precision choice and dimensional check that the preceding chapters introduced one at a time. Then a theoretical one: what, in the end, distinguishes a diffusion model from a flow matching model for this problem, once both are written on the same state space with the same backbone.

Why it is needed. The numerical fine print is worth having collected rather than scattered; and after nearly 280 pages of derivation the comparison the whole document was built to make is worth stating on one page.

What depends on it. Nothing — this is the end of the document. Neither chapter introduces new theory; both are organisational.

Chapter 16

Numerical and Implementation Reference

This chapter consolidates, in one place, the numerical-implementation details that Chapters 3, 6, 7, 9, 10 and 13 introduced individually, at the point each became relevant, and adds the handful that are more naturally stated only once every other piece of machinery is available. None of the content here is new mathematics; it is an index and a systematic check, in the spirit of Section 16.4’s closing consistency pass.

16.1 Every clamp, epsilon, and numerical safeguard in one table

Location	Guard	Purpose (cross-reference)
Omega (angle from trace)	<code>clamp(·, -1+10⁻⁷, 1-10⁻⁷)</code> (SigmaFlow); (-0.99, 0.99) (original SigmaDock, now recognised as a bug for the flow-matching use case)	are not domain safety; the wider original clamp additionally, and incorrectly, truncated true angles > 172° (Caution 3.57)
<code>rotation_vector_from_matrix</code>	<code>blend(isclose, atol=10⁻⁸)</code> near $\theta=0$; hard/loose blend (<code>atol=10⁻²</code>) near $\theta=\pi$	avoids $\sin \theta=0$ division in $\log(R) = \frac{\theta}{2 \sin \theta} (R - R^\top)$ (Section 3.4.4)

Location	Guard	Purpose
<code>d_log_f_d_omega</code> (IGSO(3) score derivative)	hard branch at $\sigma^2 < 0.3$: literal ratio vs. approximation $-\omega/\sigma^2$	literal $\partial_\omega f/f$ numerically ill-conditioned at small noise (Section 7.2.2); small-noise limit asserted by the code, not independently re-derived here
<code>get_fragment_mass_inertia</code>	inertia value floor $\max(10^{-8} \text{tr}(I)/3, 10^{-12})$ plus $+10^{-8}I$ Tikhonov term before the linear solve	protects <code>torch.linalg.solve</code> against near-degenerate (near-linear, 2–3-atom) fragment inertia tensors (Section 10.8)
<code>newton_maruyama</code>	$\omega = \text{hat}(dW)$ clamped entrywise to $[-10^3, 10^3]$	loose safety net against a diverging angular-velocity solve, not a tight physical bound
<code>d_f_igso3_d_omega</code> , series truncation	fixed truncation at $L=1000$ terms; angle grid excludes $\omega=0$ exactly	(7.1) is an infinite series (Remark 7.5); $\sin(\omega/2)=0$ singularity at $\omega=0$ avoided by grid construction, not a runtime guard
<code>BesselBasis</code> (radial embedding)	$+\varepsilon$ in denominator $d_{\text{scaled}} + \varepsilon$	avoids $0/0$ as an edge length $\rightarrow 0$ (Section 8.4)
<code>PolynomialCutoff</code>	indicator $\mathbf{1}[d < r_{\text{max}}]$, degree-8 envelope vanishing (with 2 derivatives) at r_{max}	continuity of messages as an edge crosses the cutoff radius (Section 8.4)
Assorted <code>expmap</code>	<code>vee</code> , <i>print-only</i> warnings (not raised exceptions) on a non-skew-symmetric input	a soft, non-blocking sanity check — callers can silently pass a malformed matrix without the check stopping execution (Definition 3.49’s code correspondence)
<code>assert M.min() ≥ 2</code>	hard assertion, caught upstream	fragment mass (atom count, Definition 10.11) degeneracy, not a chemical-mass concern

16.2 Masking, batching, and scatter conventions

Every per-fragment or per-molecule reduction encountered in Chapters 10 and 13 (net force/torque, mass/inertia, per-molecule loss aggregation) uses the same underlying mechanism, worth stating once explicitly since it is assumed silently throughout: a flat tensor over *all* atoms (or frag-

ments) in a batch, together with an integer index tensor recording which molecule/fragment each row belongs to, reduced via a `scatter/index_add` operation keyed on that index. This is the standard `torch_geometric` convention for representing a batch of variable-sized graphs as one large disjoint-union graph rather than as a padded, fixed-size tensor, and it is why, throughout Chapters 10 and 13, a formula written “per fragment” (e.g. (10.2)) is, in the actual tensor implementation, a single vectorised scatter-sum over the entire batch’s flattened atom list, not a Python-level loop over fragments. Masking (e.g. excluding virtual/dummy atoms from the physical mass/inertia computation, Section 10.8) is implemented via boolean indexing keyed on the categorical node-entity type (`HPARAMS.get_node_idx`), not via a separate, redundant tensor.

16.3 Floating-point precision

Several routines identified in earlier chapters explicitly upcast to `float64` (or, on Apple-Silicon/MPS backends where `float64` support is limited, round-trip through the CPU) before a numerically sensitive computation and cast back afterward: `Log`, `Omega` (Section 3.4’s code correspondences). This reflects that the angle/logarithm computation, involving `arccos` near its domain boundary and ratios like $\theta/\sin\theta$ near $\theta = 0$, is more sensitive to rounding error than the surrounding `float32`-precision network computation that dominates the rest of the pipeline; training and sampling otherwise run at `float32` (`conf/sampling/base.yaml`’s `hardware.precision:32` default).

16.4 Dimensional and consistency checks

As a final, explicit sanity pass — in the spirit this document has maintained throughout, of tying every formula to a concrete tensor shape — we record the shape/transformation-behaviour consistency of the central quantities of Chapter 13, gathered in one place.

Quantity	Shape	Transformation under global $Q \in SO(3)$
$\text{pos}_0, \text{pos}_t$ (atom coords)	$[N, 3]$	$x \mapsto Qx$ (world-frame, polar)
$\text{trans}_1, \text{trans}_t$ (fragment COM)	$[\Sigma F, 3]$	$x \mapsto Qx$ (world-frame, polar)
R_1, R_t (fragment orientation)	$[\Sigma F, 3, 3]$	$R \mapsto QR$ (world-frame)
$u_t^{\text{trans}}, \text{pred_u_t_trans}$	$[\Sigma F, 3]$	$v \mapsto Qv$ (polar, Equation (8.10))
$u_t^R, \text{pred_u_t_R}$	$[\Sigma F, 3, 3]$	world-frame $\mathfrak{so}(3)$ -valued (per Section 10.8’s reading); axial parity (Remark 3.74)
h_i (steerable node feature)	$[N, 16, 256]$ ($L=3$, <code>sphere_channels=256</code>)	$h^{\ell,c} \mapsto D^\ell(Q)h^{\ell,c}$, degree-wise
Network output (pre-slice)	$[N_{\text{atoms}}, 16]$	type-1 slice, indices 1–3, is the pseudo-force
Loss <code>loss_trans, loss_R</code>	$[\Sigma F]$	scalar (invariant), per fragment, before batch reduction

Every row is either a direct restatement of a code correspondence already given in full in an earlier chapter (cross-referenced there) or an immediate consequence of Proposition 4.25(iv)’s identification $D^1(Q) = Q$; we do not introduce any new claim in this table, only collect what has already been derived, as a single point of reference for verifying a specific tensor’s shape or transformation law without re-reading the chapter it first appeared in.

16.5 Summary of open items

In keeping with this document’s stated policy of flagging rather than guessing, we collect, in one place, every point flagged elsewhere in this document as not fully, independently verified from the code (each is discussed, with full context, at the cross-referenced location):

- The exact FR3D stochastic merge algorithm and stopping criterion (Section 10.3’s caution).
- The precise semantic identity of each of the ten categorical atom feature slots (Section 9.11’s code correspondence).
- The exact formula of the `rms_norm_sh` normalisation layer and the sinusoidal time-embedding (Section 9.9, Section 9.11).
- Whether the small-noise limit $-\omega/\sigma^2$ used by `d_log_f_d_omega` has been independently re-derived from (7.1) (it has not, in this document; it is reported as the code’s own documented claim, Section 7.2.2).
- Which specific value of `rot_score_method/rot_score_scaling` (Section 10.8.4) was used to train the specific checkpoints referenced in this project’s broader experimental work, as opposed to the constructor defaults recorded here.
- The `fragment_scaling` default-value discrepancy between two source locations (Section 10.9).

None of these open items affects the mathematical derivations of Chapters 3, 4, 6, 7, 11 and 12, which are self-contained proofs independent of any specific code detail; they affect only the precision with which a small number of secondary implementation facts are pinned down.

Chapter 17

Closing Synthesis

17.1 The two models, on one page

Everything in this document exists to make one comparison precise. It is worth having that comparison in a single place, with a pointer to where each row was established — and nowhere else, since each row is developed in full elsewhere and is not re-argued here.

	SigmaDock	SigmaFlow
State space	identical: $\mathcal{M}_m = SE(3)^m$, one rigid fragment per rotatable-bond-free piece (Definition 3.70 and section 10.1)	
Geometry	identical: exp/log on $SO(3)$ with the bi-invariant metric, product structure over fragments and over rotation/translation (Chapter 3, Proposition 3.69)	
Backbone	identical, and verified byte-for-byte: the same EquiformerV2, emitting one equivariant $\mathbb{R}^3$ vector per atom (Chapter 9, Section 15.1)	
Mechanics	identical: the same Newton–Euler reduction from atom-wise vectors to per-fragment translation and rotation (Section 10.8, Section 15.2)	
Probability path	<i>derived</i> from a noising SDE: VP on translations, VE plus the IGSO(3) heat kernel on rotations (Chapters 6 and 7)	<i>chosen</i> directly: the geodesic interpolant on each factor, closed-form (Definition 12.9)
What is learned	the score $\nabla \log p_t$, in a variance-normalised parameterisation (Section 15.3)	the velocity field u_t , tangent at the current state (Section 15.4)
Why the loss works	the same reason in both: an L^2 regression recovers a conditional expectation (Lemma 11.23); only the regressand differs	
Source distribution	the $\sigma_{\max}$ end of the noising process — approximately, not exactly, uniform on $SO(3)$ (Example 7.7)	exactly Haar on $SO(3)$, imposed by construction (Remark 12.29)
Generation	reverse-time SDE — which, in the shipped default configuration, already runs with zero injected noise (Section 10.10)	an ODE, deterministic by construction rather than by configuration (Section 13.8)

Read down the table and the shape of the substitution is visible: the top block is shared and the bottom block is not. That is what “minimally invasive” was supposed to mean, and Chapter 15 is where it was checked rather than assumed — including the two places where the substitution turned out to be less clean than intended (the mismatched rotational source distribution, and a loss weight inherited without its justification).

The deeper point is the row that is shared: *both* methods are L^2 projections onto a conditional expectation. Diffusion projects onto the conditional score, flow matching onto the conditional velocity. On a Gaussian path those two targets are related by an explicit invertible map (Proposition 11.37), so on $\mathbb{R}^d$ the choice is one of parameterisation. On $SO(3)$ it is not: the score route needs a heat kernel and the velocity route needs a geodesic, and only one of those is available in closed form (Section 12.13).

17.2 Closing remarks

This document set out to make one substitution precise: Table 13.1’s claim that converting SigmaDock into SigmaFlow replaces a probability path and a regression target while leaving the state space, the fragmentation and triangulation geometry, the input graph, and the entire EquiformerV2 backbone untouched. Making that claim *precise* required building, from elementary analysis and linear algebra upward, the full apparatus of Riemannian manifolds and Lie groups (Chapter 3), the representation theory of $SO(3)$ (Chapter 4), score-based diffusion in Euclidean space and on $SO(3)$ (Chapters 6 and 7), the equivariant message-passing architecture that both methods share (Chapters 8 and 9), and continuous-normalizing-flow theory in its Euclidean and Riemannian forms (Chapters 11 and 12) — at which point the substitution itself (Chapter 13) could be stated as a small, precisely delimited set of changes against the common baseline (Chapter 10) that the preceding chapters established.

A reader who has worked through this document in order should now be able to: state and prove why a fragment-based parameterisation, rather than a torsional one, gives a decoupled noising/flow measure (Theorem 10.2); derive the IGSO(3) heat kernel and its score from the Peter–Weyl decomposition of the Laplace–Beltrami operator on $SO(3)$ (Section 7.2); derive the geodesic flow-matching interpolant on $SO(3)$ and prove the exact algebraic identity between its two computational forms (Proposition 12.26); explain, from Schur’s lemma, exactly which operations an equivariant neural network layer is and is not permitted to perform (Propositions 4.46 and 4.48); and, for every one of these constructions, point to the specific file, function, tensor shape, and line of reasoning connecting the mathematics to the program that actually runs on real protein–ligand complexes.

Where this document’s own derivations and the codebase’s own comments disagreed, or where a detail could not be pinned down from a static reading of the source, this document has said so explicitly rather than resolving the disagreement by assumption — most notably the corrected $SO(3)$ -angle clamp (Caution 3.57), the reversed internal time conventions between the two methods (Section 13.2), the hybrid V1/V2 attention mechanism actually shipped (Caution 9.23), and the always-identity fragment reference rotation (Remark 13.1). These are exactly the kind of facts a purely paper-level account of either method would not surface, and locating them is, alongside the from-scratch mathematical development itself, the primary contribution of treating this material as a full monograph rather than a summary.